INSTITUTO POLITÉCNICO NACIONAL

ESCUELA SUPERIOR DE CÓMPUTO

ESCOM

Trabajo Terminal I

“Sistema de comparación de código en C/C++ mediante similitud semántica para cursos introductorios de programación”

2026-B032

Presentan:

Erick Daniel García Rodríguez
Claudia Emilia Sánchez García

Directores:

M. en C. Manuel Portillo Cedillo
Dra. Maribel Aragón García

13 de Mayo del 2026

2026 / 2

INSTITUTO POLITÉCNICO NACIONAL
ESCUELA SUPERIOR DE CÓMPUTO
SUBDIRECCIÓN ACADÉMICA

No. de TT: 2026-B032 -13 de Mayo del 2026

Documento Técnico

"Sistema de comparación de código en C/C++ mediante similitud semántica para cursos introductorios de programación"

Presentan:

Erick Daniel García Rodríguez
Claudia Emilia Sánchez García

Directores:

M. en C. Manuel Portillo Cedillo **Dra. Maribel Aragón García**

RESUMEN

Este Trabajo Terminal propone el desarrollo de un sistema de comparación de código en C/C++ diseñado como una herramienta de apoyo a la evaluación del docente en tareas de alumnos para cursos introductorios, como Fundamentos de Programación y Algoritmos. El sistema utiliza técnicas de Procesamiento de Lenguaje Natural (PLN) y representaciones vectoriales para realizar una comparación semántica de las entregas respecto al material de referencia. Además, se integra un análisis estilométrico para identificar patrones de escritura y detectar la similitud con código generado por Inteligencia Artificial (IA), permitiendo una revisión de tareas más objetiva y detallada.

Palabras clave: Procesamiento de Lenguaje Natural, Similitud semántica de código, Análisis estilométrico, Aprendizaje profundo, Herramienta de apoyo a la decisión docente.

Índice general

1 Introducción **1**

1.1 Planteamiento del problema 2
1.2 Justificación 5
1.3 Motivación 7
1.4 Objetivos 8
1.4.1 Objetivo General 8
1.4.2 Objetivos específicos 8

2 Estado del Arte **10**

2.1 Problema de la detección de similitud de código 10
2.2 Enfoques tradicionales basados en texto 10
2.3 Herramientas basadas en fingerprinting 11
2.3.1 MOSS 11
2.4 Herramientas basadas en análisis sintáctico y evaluación automática 12
2.4.1 JPlag 12
2.4.2 Gradescope 12
2.4.3 CodeGrade 12
2.5 Representaciones estructurales del código 13
2.5.1 Árboles de sintaxis abstracta (AST) 13
2.5.2 Grafos de flujo de control y datos 14
2.5.3 Embeddings de código 14
2.6 Modelos basados en aprendizaje profundo 14
2.7 Detección de código generado por inteligencia artificial 15
2.8 Análisis y brecha de investigación 16

3 Marco Teórico **19**

3.1 Introducción al análisis automático de código 19
3.2 Problema de la detección de similitud de código 19
3.3 Clones de código 20
3.4 Enfoques para la detección de similitud de código 20
3.4.1 Enfoques basados en análisis textual 21
3.4.2 Enfoques basados en estructura 21
3.4.3 Enfoques basados en semántica 21

3.4.4 Enfoques basados en aprendizaje automático . . . 22
3.4.5 Comparación de enfoques . . . 22
3.5 Representaciones del código fuente . . . 23
3.5.1 Representaciones textuales . . . 24
3.5.2 Representaciones sintácticas: Árboles de Sintaxis Abstracta . . . 24
3.5.3 Representaciones basadas en grafos . . . 25
3.5.4 Comparación de representaciones . . . 25
3.5.5 Representaciones distribuidas: embeddings . . . 26
3.6 Medidas de similitud en código fuente . . . 27
3.7 Modelos para análisis de código . . . 28
3.7.1 Evolución del análisis automático de código . . . 28
3.7.1.1 Modelos clásicos de aprendizaje automático . . . 28
3.7.1.2 Redes neuronales (RNN y CNN) . . . 28
3.7.1.3 Transformers . . . 29
3.7.2 Modelos basados en texto y tokens . . . 29
3.7.2.1 RoBERTa para código . . . 29
3.7.2.2 CodeBERT . . . 30
3.7.3 Modelos basados en Árboles de Sintaxis Abstracta . . . 30
3.7.3.1 Code2Vec . . . 31
3.7.3.2 ASTNN . . . 31
3.7.4 Modelos basados en grafos . . . 32
3.7.4.1 GGNN (Gated Graph Neural Networks) . . . 32
3.7.4.2 GraphCodeBERT . . . 32
3.7.5 Comparación de modelos . . . 33
3.8 Análisis estilométrico . . . 34
3.8.1 Taxonomía de rasgos estilométricos . . . 35
3.8.2 Enfoques de estilometría . . . 36
3.8.2.1 Métodos estadísticos . . . 36
3.8.2.2 Machine Learning clásico . . . 36
3.8.2.3 Deep Learning . . . 36
3.8.3 Modelos para el análisis estilométrico . . . 37
3.8.3.1 Modelos basados en n-gramas . . . 37
3.8.3.2 CharCNN . . . 38
3.8.3.3 RoBERTa . . . 38
3.8.3.4 Comparación de modelos . . . 39
3.8.4 Métricas de evaluación basadas en la matriz de confusión . . . 39
3.9 Integración y fusión de características (*Feature Fusion*) . . . 41
3.10 Conclusión del marco teórico . . . 41

4 Propuesta de solución **43**
4.1 Estructura Funcional del Sistema . . . 44

4.1.1 Entrada de datos y fuentes de referencia . . . 45
4.1.2 Módulo de Preprocesamiento y Normalización . . . 45
4.1.3 Canal A: Comparación Semántica . . . 46
4.1.3.1 Justificación y Selección del Modelo . . . 46
4.1.3.2 Extracción de Grafo de Flujo de Datos (DFG) . . . 47
4.1.3.3 Procesamiento en GraphCodeBERT . . . 47
4.1.3.4 Embedding Lógico . . . 48
4.1.4 Canal B: Comparación Estilométrica . . . 48
4.1.4.1 Justificación y Selección del Modelo . . . 48
4.1.4.2 Análisis de Autoría mediante CharCNN . . . 48
4.1.4.3 Flujo de procesamiento de la red CharCNN . . . 49
4.1.4.4 Codificación de entrada y capacidad computacional . . . 50
4.1.4.5 Cálculo de probabilidad de generación por IA . . . 50
4.1.5 Fusión de Características . . . 50
4.1.5.1 Consideraciones sobre la dimensionalidad y la integración de características . . . 51
4.1.5.2 Tratamiento del desbalance entre canales . . . 52
4.1.6 Similitud del Coseno . . . 53
4.1.6.1 Importancia de la métrica en la fusión de características . . . 53
4.1.7 Componentes funcionales, entradas y salidas . . . 54
4.2 Conclusión . . . 55

5 Análisis del Sistema **57**
5.1 Análisis de Requerimientos . . . 57
5.1.1 Requerimientos Funcionales . . . 57
5.1.2 Requerimientos No Funcionales . . . 58
5.1.3 Reglas de Negocio . . . 59
5.2 Identificación y descripción de actores . . . 60
5.2.1 Docente / Evaluador . . . 61
5.2.2 Motor de Inteligencia Artificial Externo (LLM) . . . 61
5.3 Trayectorias principales y alternativas . . . 62
5.3.1 Trayectoria principal (Flujo de Análisis Estándar) . . . 62
5.3.2 Trayectorias alternativas . . . 64
5.4 Diagrama de Casos de Uso . . . 66
5.4.1 Descripción de Actores . . . 66
5.4.2 Descripción de los Casos de Uso . . . 67
5.4.3 Especificación de Casos de Uso . . . 67
5.5 Modelado de Procesos de Negocio (BPMN) . . . 72
5.5.1 Descripción del Flujo de Trabajo . . . 73
5.6 Características semánticas del código . . . 74
5.7 Características estilométricas del código . . . 75

5.8 Definición de métricas de desempeño . . . 76
5.8.1 Métricas para el análisis de similitud semántica . . . 76
5.8.2 Métricas para el análisis estilométrico . . . 77
5.8.3 Resumen de métricas utilizadas . . . 77
5.8.4 Métricas de desempeño operativo . . . 79
5.8.5 Criterios de interpretación de resultados . . . 79
5.8.6 Responsabilidad docente e interpretación del reporte de Graphito . . . 80
5.8.7 Consideraciones para la experimentación . . . 81
5.8.8 Justificación de las métricas seleccionadas . . . 81
5.9 Análisis de riesgos . . . 82
5.9.1 Clasificación de riesgos . . . 82
5.9.2 Criterios de evaluación de riesgos . . . 82
5.9.3 Identificación y evaluación de riesgos . . . 83
5.9.4 Estrategias de mitigación . . . 83
5.9.5 Discusión del impacto de los riesgos . . . 84
5.9.6 Selección y justificación del conjunto de datos . . . 85
5.9.6.1 Código humano: IEEE Programming Homework Dataset . . . 85
5.9.6.2 Código generado por IA (Clase Sintética) . . . 85
5.9.6.3 Consideraciones de balance y partición . . . 86

6 Diseño del sistema **89**
6.1 Arquitectura (Modelo C4) . . . 89
6.1.1 Nivel 1: Contexto . . . 89
6.1.2 Nivel 2: Contenedores . . . 90
6.1.3 Nivel 3: Componentes . . . 91
6.1.4 Nivel 4: Código (Clases y Secuencia) . . . 92
6.1.4.1 Diagrama de Clases . . . 92
6.1.4.2 Diagrama de Secuencia . . . 93
6.1.5 Modelado Dinámico: Diagrama de Estados . . . 94
6.2 Metodología de Desarrollo: Ciclo de Vida Komorebi . . . 96
6.3 Diseño de Base de Datos . . . 100
6.3.1 Modelo Relacional (Diagrama Entidad-Relación) . . . 100
6.3.2 Esquema de Colección Vectorial (ChromaDB) . . . 101
6.4 Diseño de Interfaz de Usuario (UI/UX) . . . 102
6.4.1 Módulo de Acceso y Registro . . . 103
6.4.2 Gestión de Repositorio y Biblioteca . . . 103
6.4.3 Módulo de Carga y Configuración de Análisis . . . 104
6.4.4 Visualización de Reportes de Doble Canal . . . 105
6.5 Estimación de costos del proyecto . . . 106
6.5.1 Costos de recursos humanos . . . 107
6.5.2 Costos de infraestructura tecnológica . . . 109

6.5.3 Costos de herramientas de software y servicios externos 111
6.5.4 Costos indirectos 116
6.5.5 Concentrado general de costos 117
6.5.6 Análisis de viabilidad económica 119

7 Ingeniería de Datos **120**
7.1 Adquisición de Datos 121
7.1.1 Requerimientos del Conjunto de Datos 121
7.1.2 Selección del Dataset Base: IEEE Programming Homework Dataset . . 122
7.1.3 Limitaciones del Dataset 123
7.1.4 Reconstrucción de Enunciados 123
7.1.5 Generación de la Contraparte de IA 124
7.1.5.1 Control del Sesgo de Idioma 124
7.1.5.2 Perfiles de Estudiante y Cuota de Comentarios 125
7.1.5.3 Variedad de Generadores 125
7.1.6 Ejemplo Comparativo: Código Humano frente a Código de IA 125
7.1.7 ¿Cuántos Ejemplos de IA Bastan? Curva de Aprendizaje 128
7.2 Preparación y Preprocesamiento de Datos 131
7.2.1 Limpieza Simétrica y Aprendizaje de Atajos 132
7.2.2 Eliminación de Duplicados 132
7.2.3 Partición `split_v1` para Evitar Fuga de Información 132
7.2.4 Preparación para Cada Canal 133
7.2.4.1 Canal B: Normalización Simétrica para CharCNN 134
7.2.4.2 Canal A: Estructura y Flujo de Datos para GraphCodeBERT 135
7.2.5 Composición Final del Conjunto 136
7.3 Conclusión del capítulo 136

8 Modelización **138**
8.1 Entrenamiento de Modelos 139
8.1.1 Canal B: CharCNN 139
8.1.1.1 Idea General 139
8.1.1.2 Archivos Largos: Ventanas 140
8.1.1.3 Hiperparámetros y Entrenamiento 141
8.1.2 Canal A: GraphCodeBERT sin Entrenamiento Adicional 143
8.1.2.1 Idea General 143
8.1.2.2 Cómo se Obtiene el Vector de un Archivo 144
8.1.2.3 Adaptación a C y C++ 145
8.1.2.4 Por Qué no se Entrena (*Zero-Shot*) 146
8.1.3 Línea Base Léxica: Jaccard 146
8.1.4 Calibración y Umbrales 147
8.1.5 Evolución de los Modelos: de la Primera Versión a la Final 149

8.2 Evaluación de Modelos . . . 150
8.2.1 Protocolo de Evaluación . . . 150
8.2.2 Canal B: Resultados del CharCNN . . . 151
8.2.3 Canal A: ¿Qué tan Semántica es la Similitud? . . . 153
8.2.3.1 Pares de Plagio . . . 154
8.2.3.2 Robustez ante Cambios en el Código . . . 154
8.2.3.3 Agrupamiento por Lógica (E1) . . . 156
8.2.3.4 Similitud y Corrección (E2) . . . 158
8.2.3.5 Resumen de Hallazgos . . . 158
8.2.4 Explicabilidad . . . 159
8.2.5 Limitaciones e Ideas no Exploradas . . . 160
8.2.6 Discusión . . . 161
8.3 Conclusión del capítulo . . . 161

9 Despliegue **162**
9.1 Despliegue de Modelos . . . 162
9.1.1 Empaquetado y Verificación . . . 162
9.1.2 Cómo Decide la Plataforma . . . 163
9.1.3 Flujo de un Análisis . . . 164
9.1.4 Falla Segura . . . 164
9.1.5 Arquitectura e Infraestructura . . . 165
9.1.6 Presentación al Docente . . . 166
9.2 Monitorización de los Modelos . . . 166
9.2.1 Pruebas Automatizadas . . . 166
9.2.2 Prueba de Extremo a Extremo en Modo Real . . . 167
9.2.3 Seguimiento en Operación . . . 167
9.2.4 Validación con Docentes . . . 168
9.2.4.1 Objetivo . . . 168
9.2.4.2 Catálogo y tarea guiada . . . 168
9.2.4.3 Instrumento . . . 168
9.2.4.4 Participantes, ética y análisis . . . 169
9.2.4.5 Resultados . . . 169
9.2.4.6 Limitaciones . . . 170
9.3 Conclusión del Capítulo . . . 170

10 Conclusiones y Trabajo Futuro **171**
10.1 Cumplimiento de Objetivos y Resultados Técnicos . . . 171
10.2 Trabajo Futuro y Líneas de Investigación . . . 173

Referencias **174**

Índice de figuras

4.1 Diagrama de bloques funcional del sistema de comparación Graphito. 44

5.1 Diagrama de trayectoria principal del sistema Graphito correspondiente al flujo de análisis estándar. 63
5.2 Diagrama de casos de uso del sistema Graphito. 66
5.3 Diagrama BPMN de la comparación de código fuente. 74

6.1 Diagrama de Contexto (Nivel 1) del sistema Graphito. 90
6.2 Diagrama de Contenedores (Nivel 2) del sistema Graphito. 91
6.3 Diagrama de Componentes (Nivel 3) enfocado en el motor back-end en Python. 92
6.4 Diagrama de clases del sistema Graphito, ilustrando la arquitectura orientada a objetos correspondientes al Nivel 4 de abstracción. 93
6.5 Diagrama de secuencia transaccional de *Graphito* desde el cliente (Nivel 4). . . 94
6.6 Diagrama de estados del proceso de análisis de código en Graphito. 95
6.7 Flujo secuencial del pipeline de modelos basado en la metodología Komorebi. . 98
6.8 Modelo Entidad-Relación de la base de datos relacional (PostgreSQL). 101
6.9 Esquema conceptual de la colección vectorial en ChromaDB. 102
6.10 Interfaz de inicio de sesión (Cumplimiento de RF-01). 103
6.11 Interfaz de registro de nuevos usuarios. 103
6.12 Repositorio de problemas y gestión de ejercicios (Cumplimiento de RF-02). . . 104
6.13 Interfaz de carga de código de referencia (Cumplimiento de RF-03). 105
6.14 Configuración de parámetros para el inicio del análisis. 105
6.15 Dashboard de resultados híbridos (Cumplimiento de RF-10 y RF-11). 106

7.1 Flujo general de adquisición y preparación de los datos de Graphito, desde el código real de estudiantes hasta el conjunto particionado. 121
7.2 Reconstrucción de enunciados: de varias soluciones correctas de una misma tarea a un enunciado en dos idiomas. 123
7.3 Curva de aprendizaje de MultiScaleCharCNN según la cantidad de grupos únicos de IA de entrenamiento. La línea sólida es la media de 5 semillas (banda sombreada: $\pm 1\,\sigma$) y los puntos semitransparentes son corridas individuales. El 100 % equivale a 874 grupos únicos, tomados de los 897 archivos de IA de entrenamiento. 130

7.4 Mejora marginal de PR-AUC en validación entre fracciones consecutivas de datos de IA. Las tres primeras transiciones superan el ruido promedio entre semillas (línea punteada); las siguientes quedan por debajo de él. 131
7.5 Efecto de la limpieza y de la eliminación de duplicados que cruzarían particiones sobre el número de archivos. 131
7.6 Esquema de la partición `split_v1`: cada tarea, con sus dos años y sus dos clases, pertenece a una sola partición. 133
7.7 Bifurcación del preprocesamiento: un mismo archivo se prepara de forma distinta para el Canal B (texto de caracteres) y para el Canal A (estructura y flujo de datos). 134

8.1 Panorama de los modelos de Graphito: dos canales y una línea base léxica, cuyas salidas orientan la revisión del docente. 139
8.2 Arquitectura del CharCNN v2: del texto a una puntuación de estilo. Cada bloque indica en lenguaje llano qué hace. 140
8.3 Lectura por ventanas con traslape: cada ventana se puntúa por separado y los resultados se promedian. 141
8.4 Una época de entrenamiento del CharCNN y la regla de paro anticipado. 142
8.5 Curvas de entrenamiento del CharCNN v2: pérdida de entrenamiento (izquierda) y métricas de validación (derecha). La línea punteada marca la época seleccionada. 143
8.6 Intuición de la similitud del coseno: cuanto menor es el ángulo entre dos vectores, más parecidos son los códigos que representan. 143
8.7 Cómo el Canal A convierte un archivo en un vector y lo compara con la solución de referencia. 144
8.8 Grafo de flujo de datos de un programa mínimo: el valor 5 llega a `a`, de ahí a `b` y de ahí al `return`. 145
8.9 Línea base léxica. Arriba, una línea de código se abstrae y se divide en fragmentos de 5 tokens. Abajo, ejemplo con dos archivos de 5 y 4 fragmentos que comparten 3. 147
8.10 Línea de tiempo de la evolución de los modelos: de las primeras versiones al sistema final. 149
8.11 Protocolo de evaluación: cada partición tiene un único rol, y cada cifra se acompaña de su intervalo de confianza. 150
8.12 Curvas ROC (izquierda) y de precisión-recall (derecha) del CharCNN v2 en el conjunto de prueba (*TPR*: tasa de verdaderos positivos; *FPR*: tasa de falsos positivos; AP: precisión promedio). 152
8.13 Matriz de confusión del CharCNN v2 en la prueba con el umbral desplegado (`fpr_0.01`). 153
8.14 Degradación del ROC-AUC del Canal A en las condiciones del protocolo de robustez. 155
8.15 MAP@R de cada método con IC 95 % y línea base aleatoria (0.002). 157

8.16 Proyección t-SNE de los vectores de GraphCodeBERT: problemas distintos (a) y problemas similares que se superponen (b). La estrella marca la solución de referencia. 157
8.17 Matrices de similitud del coseno ordenadas por tarea. Los bloques densos sobre la diagonal indican que las entregas de una misma tarea son más parecidas entre sí. 158
8.18 Similitud con la solución de referencia según el resultado de las pruebas unitarias (pasa todas, parcial, falla todas), para tres métodos. 158

9.1 Del modelo entrenado al docente: cada paso se verifica y la decisión final es humana. 162
9.2 Empaquetado de los modelos: cada archivo lleva una huella que la plataforma revisa antes de funcionar. 163
9.3 Flujo de un análisis: tres cálculos en paralelo, una decisión y un reporte. 164
9.4 Dónde se ejecuta cada parte de Graphito. 165
9.5 Reporte de similitud mostrado al docente. 166
9.6 Catálogo de programas de la plataforma. 168

Índice de tablas

2.1 Comparación de herramientas de análisis de código 13
2.2 Comparación de representaciones de código 14
2.3 Modelos basados en Transformers para análisis de código 15

3.1 Tipos de clones de código según Bellon. 20
3.2 Comparación de enfoques para la detección de similitud de código 23
3.3 Comparación de representaciones del código fuente 26
3.4 Comparación de métricas de similitud en código fuente 27
3.5 Comparación de modelos preentrenados para análisis de código 33
3.6 Taxonomía de rasgos estilométricos en código fuente 35
3.7 Comparación de enfoques para el análisis estilométrico 37
3.8 Comparación de modelos para el análisis estilométrico de código fuente 39
3.9 Componentes fundamentales de la matriz de confusión binaria. 40
3.10 Métricas de rendimiento derivadas de la matriz de confusión. 40

4.1 Componentes funcionales de Graphito, entradas y salidas. 54

5.1 Especificación del Caso de Uso 1. 68

5.2 Especificación del Caso de Uso 2. 68
5.3 Especificación del Caso de Uso 3. 69
5.4 Especificación del Caso de Uso 4. 69
5.5 Especificación del Caso de Uso 5. 70
5.6 Especificación del Caso de Uso 6. 70
5.7 Especificación del Caso de Uso 7. 71
5.8 Especificación del Caso de Uso 8. 71
5.9 Especificación del Caso de Uso 9. 72
5.10 Especificación del Caso de Uso 10. 72
5.11 Clasificación sintetizada de características semánticas del código. 75
5.12 Clasificación de características estilométricas del código fuente. 76
5.13 Métricas de validación de modelos utilizadas en el sistema. 78
5.14 Criterios de interpretación para similitud semántica espacial. 80
5.15 Análisis de riesgos del sistema 83
5.16 Composición y origen del conjunto de datos dual 86
5.17 Distribución cuantitativa y partición del conjunto de datos 87

6.1 Estimación de costos de recursos humanos de Graphito. 107
6.2 Estimación de costos de infraestructura tecnológica. 110
6.3 Estimación de costos de herramientas de software y servicios externos. 112
6.4 Estimación de costos indirectos del proyecto. 116
6.5 Concentrado general de costos de Graphito. 118

7.1 Estructura del conjunto de datos IEEE Plagiarism. 122
7.2 Ejemplos de enunciados recuperados por reconstrucción. 124
7.3 Perfiles de simulación de estudiantes usados en la generación de código de IA. . 125
7.4 Contraste entre una implementación humana y una de IA para el mismo problema. 127
7.5 De los archivos de IA generados a los ejemplos usados en la curva de aprendizaje. 128
7.6 Tratamiento de cada componente del código en los dos canales. 134
7.7 Composición del conjunto de datos particionado por tarea (`split_v1`). 136

8.1 Organizador de hiperparámetros del CharCNN v2. 141
8.2 Organizador de la configuración de GraphCodeBERT (Canal A). 144
8.3 Ajustes del grafo de flujo de datos para C y C++. 145
8.4 Lo que el grafo registra en un ejemplo con apuntadores. 146
8.5 Organizador de calibración y umbrales, ajustados solo con datos de validación. 148
8.6 Evolución de los modelos. Las cifras del protocolo anterior no son comparables con las del actual. 149
8.7 Glosario de métricas: qué pregunta responde cada una. 151
8.8 Resultados del CharCNN en la prueba en los dos puntos de operación (IC 95 % entre corchetes). 152
8.9 Pares de plagio en la prueba: ROC-AUC (IC 95 %). 154

8.10 Transformaciones del protocolo de robustez, con un ejemplo de cada una. . . . 154
8.11 ROC-AUC [IC 95 %] en pares de plagio de la prueba bajo cada transformación. 155
8.12 Agrupamiento por tarea (E1) en el espacio original de los vectores. 156
8.13 Organizador de hallazgos: pregunta, resultado y significado para el docente. . . 159
8.14 Limitaciones de los resultados. 160
8.15 Ideas no exploradas (propuestas para trabajo futuro). 160

9.1 Componentes desplegados y su papel en la plataforma. 163
9.2 Etiqueta mostrada según las dos preguntas de la plataforma. 164
9.3 Situaciones anómalas y respuesta del sistema. 165
9.4 Contenido de cada reporte. 166
9.5 Tipos de prueba y qué verifican. 167
9.6 Prueba de extremo a extremo con los modelos reales. 167
9.7 Resumen del instrumento de la encuesta. 169
9.8 Calificación SUS (pendiente). 169
9.9 Utilidad percibida por ítem (pendiente). 170
9.10 Reducción estimada del tiempo de revisión (pendiente). 170

Capítulo 1

Introducción

La enseñanza de programación actualmente enfrenta un cambio de paradigma derivado del uso de herramientas de Inteligencia Artificial (IA) basadas en Modelos de Lenguaje de Gran Escala (LLMs), como ChatGPT, Claude o Gemini [1]. Estas herramientas han demostrado ser altamente eficientes para resolver tareas lógicas y algorítmicas, logrando un desempeño comparable al de un estudiante de programación destacado [1]. No obstante, el uso de estas tecnologías introduce nuevas dinámicas en los procesos de evaluación [1]. Algunas investigaciones han descrito este fenómeno como la "paradoja cognitiva de la inteligencia artificial", donde la IA puede actuar tanto como un facilitador del razonamiento como un mecanismo de delegación de procesos cognitivos [1]. En el ámbito educativo, esto se relaciona con el concepto de "descarga cognitiva", el cual modifica la forma en que se construyen y presentan las soluciones en programación [1].

Estudios recientes han señalado que el uso de asistentes de IA influye en la manera en que se abordan tareas técnicas, incluyendo actividades como la lectura, comprensión y depuración de código [2]. Este escenario no solo impacta la dinámica de aprendizaje, sino que introduce un reto adicional para los docentes, quienes deben interpretar el código como evidencia del proceso lógico seguido para resolver un problema [2]. En este contexto, la evaluación en cursos de programación se vuelve particularmente compleja. La revisión manual del código continúa siendo uno de los métodos más confiables para analizar el razonamiento detrás de una solución; sin embargo, en grupos numerosos o en contextos con alta carga académica, este proceso resulta poco escalable y difícil de sostener de manera sistemática [2]. Esta situación genera una sobrecarga operativa para el docente, limitando la profundidad con la que puede realizarse el análisis de cada programa [2].

Debido a estas limitaciones, el problema técnico puede analizarse desde dos dimensiones complementarias. La primera dimensión se relaciona con la dificultad para evaluar la equivalencia lógica entre programas que implementan una misma solución mediante estructuras sintácticas diferentes. Las herramientas tradicionales de detección de similitud, como aquellas basadas en comparaciones textuales o análisis léxico, presentan limitaciones para identificar relaciones se-

mánticas profundas entre programas, especialmente cuando existen modificaciones superficiales en el código o variaciones en la estructura de implementación [3]. Esta problemática se vuelve aún más relevante en escenarios donde las herramientas de inteligencia artificial generativa son capaces de producir soluciones funcionalmente equivalentes, pero sintácticamente distintas [3].

Para abordar esta problemática, el presente trabajo plantea un enfoque de comparación semántica basado en representaciones vectoriales del código fuente. Mediante el uso de modelos especializados en análisis de código, se busca representar las relaciones internas del programa, incluyendo dependencias y estructuras asociadas a su lógica de funcionamiento, con el objetivo de generar embeddings semánticos que permitan comparar programas desde una perspectiva lógica y no únicamente textual [3]. Bajo este enfoque, dos programas que resuelven un mismo problema mediante diferentes estructuras sintácticas pueden ser identificados como semánticamente similares al conservar una lógica de funcionamiento equivalente [4]. A partir de estas representaciones, es posible utilizar métricas de similitud para identificar programas con comportamiento equivalente aun cuando presenten diferencias sintácticas significativas [4].

La segunda dimensión del problema se relaciona con la identificación de patrones asociados al uso de inteligencia artificial generativa y el reto adicional para los docentes, quienes deben interpretar el código como evidencia del proceso lógico seguido para resolver un problema [2]. En este contexto, la evaluación en cursos de programación se vuelve particularmente compleja. La revisión manual del código continúa siendo uno de los métodos más confiables para analizar el razonamiento detrás de una solución; sin embargo, en grupos numerosos o en contextos con alta carga académica, este proceso resulta poco escalable y difícil de sostener de manera sistemática [2]. Esta situación genera una sobrecarga operativa para el docente, limitando la profundidad con la que puede realizarse el análisis de cada programa [2].

Frente a este escenario, el presente trabajo terminal tiene como objetivo desarrollar Graphito, un sistema de comparación de código en C/C++ mediante similitud semántica y análisis estilométrico para cursos introductorios de programación, concebido como una herramienta de apoyo a la decisión docente que facilite evaluaciones más integrales. Mediante este enfoque de doble canal, Graphito contribuye a la reducción de la carga operativa del docente.

1.1 Planteamiento del problema

La enseñanza de la programación en lenguajes como C/C++ se fundamenta en un ideal pedagógico orientado al desarrollo del pensamiento computacional, entendido como la capacidad de descomponer problemas, identificar patrones, diseñar algoritmos y construir soluciones lógicas de manera estructurada [4]. Este enfoque trasciende la simple escritura de código funcional, ya que busca que el estudiante comprenda, interprete y justifique la lógica de cada solución desarrollada, lo cual constituye una competencia esencial en la formación de ingenieros [4].

Sin embargo, este ideal pedagógico se ha visto históricamente comprometido por la dificultad

inherente al aprendizaje de la programación [5]. Diversos estudios han señalado que los estudiantes enfrentan barreras cognitivas significativas al intentar comprender conceptos abstractos como estructuras de control, manejo de memoria o diseño algorítmico [5]. Como consecuencia, se ha observado una tendencia recurrente a evadir el proceso de razonamiento mediante la reutilización de soluciones externas, ya sea a través de la copia directa de compañeros o mediante el uso de recursos disponibles en línea [5]. Este fenómeno pone en evidencia que el problema de entregar código sin comprenderlo no es reciente, sino una problemática estructural [5].

Para verificar que el estudiante ha comprendido la lógica de un programa, la revisión manual o la defensa oral se han considerado tradicionalmente como el método de referencia, ya que permiten evaluar directamente el razonamiento detrás de la solución [6]. Sin embargo, este tipo de evaluación se vuelve difícil de implementar en contextos con alta matrícula, donde la cantidad de estudiantes y las limitaciones de tiempo del docente generan un cuello de botella que impide su aplicación de forma sistemática [6].

Como respuesta a estas limitaciones, se han desarrollado diversas herramientas automatizadas de evaluación [7]. Entre ellas destacan los jueces en línea, los cuales operan bajo un modelo de evaluación estática centrado en la verificación de la salida del programa frente a un conjunto de casos de prueba [7]. Aunque estos sistemas permiten evaluar rápidamente grandes volúmenes de código, su principal limitación radica en que no consideran la lógica interna utilizada para resolver el problema, lo que puede derivar en la aprobación de soluciones funcionales sin una comprensión real por parte del estudiante [7]. De manera complementaria, las herramientas tradicionales de detección de similitud, como MOSS o JPlag, presentan limitaciones al basarse principalmente en comparaciones textuales y análisis léxico [7, 8]. Estos enfoques presentan vulnerabilidades ante modificaciones superficiales del código, tales como cambios en nombres de variables, reordenamiento de instrucciones o alteraciones en el formato [7]. Al no analizar la equivalencia semántica ni el flujo lógico del programa, se dificulta identificar soluciones que conservan la misma lógica algorítmica, pero presentan diferencias sintácticas [8].

En este contexto, la aparición de herramientas de inteligencia artificial generativa introduce una nueva dimensión al problema [9]. A diferencia de la copia tradicional, donde existía un código fuente original que podía ser rastreado y comparado, los modelos de lenguaje actuales son capaces de generar código de manera probabilística, produciendo soluciones funcionalmente equivalentes pero sintácticamente distintas en cada ejecución [9]. Esta característica difumina la noción de un punto de referencia fijo y reduce significativamente la efectividad de los métodos tradicionales de detección basados en similitud textual, ya que las soluciones generadas no presentan coincidencias directas que puedan ser identificadas convencionalmente [9]. Asimismo, los detectores de contenido generado por IA presentan resultados variables en código, ya que han sido diseñados principalmente para texto natural [8].

Como consecuencia directa de esta nueva dimensión, emerge el fenómeno del *outsourcing* cognitivo, en el cual el estudiante delega el proceso de resolución de problemas a herramientas de in-

teligencia artificial, reduciendo su participación activa en la construcción del conocimiento [10]. Este comportamiento impacta negativamente en el desarrollo de habilidades fundamentales, ya que el estudiante puede obtener resultados funcionales sin haber internalizado los conceptos necesarios para su construcción [10]. Estudios recientes han evidenciado que esta dependencia puede afectar el rendimiento real del estudiante, generando una discrepancia entre la calidad del código entregado y el nivel de comprensión del mismo [10].

Adicionalmente, esta situación genera una pérdida de trazabilidad, en la cual el profesor pierde la capacidad de distinguir de manera confiable entre código desarrollado por el estudiante y código generado automáticamente [11]. La combinación de evaluación estática, herramientas de similitud insuficientes y generación probabilística de código provoca que los mecanismos tradicionales resulten ineficaces para respaldar la integridad académica y la correcta medición del aprendizaje [11].

En este sentido, el problema no radica únicamente en el uso de herramientas de inteligencia artificial, sino en la incapacidad de los sistemas actuales para analizar el código desde una perspectiva integral [11]. Por lo tanto, la ausencia de herramientas capaces de integrar el análisis semántico del código con la identificación de patrones de autoría constituye el problema central de investigación [12]. Ante esta nueva dimensión, resulta necesario transitar hacia modelos de evaluación centrados en competencias y desarrollar herramientas híbridas que utilicen modelos avanzados para analizar la lógica del código a gran escala, permitiendo asistir al docente en la toma de decisiones para mitigar estos nuevos retos tecnológicos [12].

En este contexto, es importante distinguir entre la generación automatizada de código y la capacidad del estudiante para comprender, adaptar y justificar una solución. La utilización de herramientas de inteligencia artificial generativa no implica, por sí misma, una ausencia de aprendizaje, puesto que estas tecnologías también pueden emplearse como recursos de consulta, experimentación y depuración. La problemática surge cuando el producto generado sustituye el proceso de razonamiento que se pretende desarrollar y evaluar en los cursos introductorios de programación.

Por lo tanto, Graphito no busca restringir el uso de herramientas de inteligencia artificial ni establecer una distinción absoluta entre código humano y código generado artificialmente. Su propósito consiste en proporcionar al docente indicadores que permitan identificar semejanzas semánticas entre soluciones y patrones estilométricos que ameriten una revisión más detallada. Estos indicadores deben interpretarse considerando el contexto de la actividad, las condiciones de uso de herramientas externas y los criterios pedagógicos definidos por el docente.

Desde esta perspectiva, la herramienta se orienta a fortalecer la evaluación formativa y la integridad académica, promoviendo que el estudiante sea capaz de explicar, modificar y defender técnicamente sus soluciones. De esta manera, el análisis automatizado constituye un mecanismo de apoyo para identificar situaciones que requieren atención, mientras que la valoración del aprendizaje y la decisión final permanecen bajo la responsabilidad del docente.

1.2 Justificación

El panorama de la evaluación en la enseñanza de la programación se ha transformado profundamente debido a la adopción de la Inteligencia Artificial Generativa. La creciente integración de estos asistentes por parte del alumnado exige a los profesores enfrentarse al desafío de discernir si el código entregado es producto del razonamiento crítico del estudiante o si proviene de una asistencia automatizada [1, 2, 3, 13]. Frente a este escenario, los mecanismos de revisión convencionales se ven rebasados, lo que restringe la capacidad del educador para validar con certeza tanto la autoría como la estructura lógica de los trabajos presentados.

Ante esta situación, el presente trabajo propone el desarrollo de Graphito como un Sistema de Soporte a la Decisión Educativa, orientado a asistir al docente en el análisis del código fuente. Este sistema se propone como una herramienta para reducir la carga operativa en procesos complejos de toma de decisiones, al permitir procesar grandes volúmenes de información y generar indicadores interpretables que facilitan el análisis. En este sentido, Graphito no se plantea como un sistema de calificación automática, sino como una herramienta de análisis estructural y semántico que proporciona métricas objetivas para interpretar el proceso de razonamiento del estudiante, especialmente en escenarios con alta matrícula donde la revisión manual resulta limitada.

La ventaja de Graphito se materializa en la integración de dos perspectivas de análisis que pueden consultarse de manera conjunta. La similitud semántica permite priorizar entregas cuyas representaciones presentan una elevada proximidad respecto de las referencias del ejercicio, mientras que el análisis estilométrico aporta una señal complementaria relacionada con los patrones de escritura. Esta integración facilita organizar grandes conjuntos de entregas, identificar casos que requieren revisión y consultar los resultados en un reporte centralizado.

En términos operativos, el docente puede utilizar los indicadores para seleccionar las entregas que necesitan un examen más detallado, comparar casos dentro de un mismo ejercicio y dirigir su tiempo hacia las situaciones que requieran mayor atención. La utilidad del sistema deberá valorarse mediante pruebas que comparen el tiempo de revisión con y sin asistencia de Graphito, considerando también la calidad de la interpretación, los falsos positivos y la consistencia de las decisiones docentes (Sección 9.2.4).

Por tanto, la ventaja esperada no consiste en reemplazar el criterio pedagógico, sino en facilitar el acceso a evidencia organizada y reducir las tareas repetitivas de comparación. Su contribución dependerá de que los indicadores sean suficientemente confiables, comprensibles y útiles dentro del contexto real de evaluación.

Desde el punto de vista metodológico, la propuesta se sustenta en la integración de técnicas avanzadas de análisis de código que han mostrado resultados prometedores en la literatura reciente [14, 15]. Para evaluar la equivalencia semántica, se plantea el uso de un modelo Transformer preentrenado especializado en código, como GraphCodeBERT [15]. Este ha sido seleccionado

por su capacidad de incorporar información estructural mediante grafos de flujo de datos, lo que permite capturar relaciones semánticas profundas entre las variables internas del programa, superando las limitaciones de los enfoques basados únicamente en similitud textual [15]. De manera complementaria, se propone integrar un módulo de análisis estilométrico mediante una red neuronal convolucional a nivel de caracteres (CharCNN), entrenada para identificar patrones asociados a diferentes formas de construcción de código. Este componente permite extraer características estructurales y de formato, generando una representación que puede funcionar como un patrón o métrica del código analizado [16]. La combinación de estos enfoques proporciona una base sólida para desarrollar herramientas capaces de analizar el código desde una perspectiva más integral [16].

La diversidad estilística constituye una limitación relevante para la identificación del origen de un programa. Los modelos generativos pueden producir código con distintos formatos, convenciones de nomenclatura, comentarios y estructuras sintácticas cuando reciben instrucciones específicas. De forma análoga, los estudiantes pueden presentar estilos de programación semejantes como resultado de su formación académica, del uso de convenciones comunes o de la consulta de recursos educativos compartidos. En consecuencia, los rasgos estilométricos no constituyen evidencia suficiente para determinar la autoría de manera aislada.

En Graphito, el análisis estilométrico se plantea como una fuente complementaria de información y no como un mecanismo para restringir la creatividad del estudiante. La herramienta pretende contribuir a que el docente identifique posibles discrepancias entre la equivalencia lógica de una solución y sus patrones de escritura, con el propósito de orientar una revisión académica. Este enfoque busca favorecer un uso consciente de la inteligencia artificial y fortalecer las prácticas de *Vibe Engineering*, entendido en este trabajo como el proceso de revisar, comprender, validar, modificar y asumir responsabilidad técnica sobre el código producido con asistencia de herramientas generativas.

En el ámbito teórico, esta propuesta se alinea con las tendencias actuales en analítica del aprendizaje, las cuales buscan ir más allá de evaluaciones binarias centradas únicamente en la ejecución del programa [17]. Estudios recientes han señalado la necesidad de incorporar métricas que permitan analizar la lógica y el proceso de construcción del código, con el objetivo de obtener una evaluación más representativa del aprendizaje del estudiante [17]. El sistema propuesto contribuye a este enfoque al integrar análisis semántico y estilométrico, permitiendo aproximarse a la comprensión del razonamiento detrás del código.

Finalmente, la relevancia de este trabajo se enmarca en el contexto actual de la educación superior en México, donde la inteligencia artificial generativa ha pasado de ser una herramienta emergente a formar parte de las prácticas académicas cotidianas [18]. Estudios recientes evidencian que su uso se ha extendido de manera significativa entre estudiantes y docentes, lo que introduce nuevas dinámicas en el proceso de aprendizaje [19]. Esta adopción generalizada implica que los mecanismos tradicionales de evaluación, centrados principalmente en el resultado final, dejan

de ser suficientes para interpretar el nivel real de comprensión del estudiante [20].

En este sentido, el análisis del contexto educativo mexicano señala la necesidad de replantear los modelos de evaluación, orientándolos hacia el reconocimiento de procesos, competencias y formas de construcción del conocimiento, en lugar de limitarse únicamente a la verificación de productos terminados [21]. Este cambio responde a una transformación más amplia en la forma en que se concibe el aprendizaje en entornos mediados por inteligencia artificial, donde el énfasis se desplaza hacia la capacidad del estudiante para comprender, justificar y estructurar soluciones [21]. Bajo esta perspectiva, el desarrollo de herramientas como Graphito adquiere relevancia al alinearse con estas nuevas necesidades del entorno educativo, contribuyendo a optimizar el proceso de evaluación docente en el contexto mexicano y favoreciendo evaluaciones más consistentes y acordes con las dinámicas actuales de la educación superior.

1.3 Motivación

La motivación detrás de este trabajo terminal surge del interés por las fronteras actuales de la Inteligencia Artificial (IA), y el profundo interés por transformar y optimizar los procesos de evaluación docente en la educación superior, particularmente en disciplinas afines a la computación.

Desde una perspectiva técnica, el proyecto está impulsado por el desafío de aplicar conceptos avanzados de Procesamiento de Lenguaje Natural (PLN), como *embeddings* y mecanismos de atención en el código fuente. La posibilidad de tratar un programa no solo como texto plano, sino como una estructura con semántica profunda, nos permitió explorar cómo *GraphCodeBERT* puede capturar relaciones lógicas subyacentes mediante grafos de flujo de datos. En este sentido, el trabajo de Martínez Gil [6] ha sido una fuente de inspiración fundamental, demostrando que es posible construir puentes entre la algoritmia y el aprendizaje profundo para resolver problemas complejos de similitud y autoría.

En el ámbito académico, nuestra experiencia en las aulas de la ESCOM nos ha permitido observar una marcada discrepancia en cómo el cuerpo docente enfrenta los nuevos retos que presenta la IA generativa en cursos introductorios de programación. Mientras algunos profesores buscan nuevos recursos metodológicos para asegurar que el estudiante realmente haya asimilado el conocimiento, otros se ven obligados a ignorar esta variable debido a la gran carga de trabajo que implica revisar manualmente múltiples entregas o realizar evaluaciones uno a uno para que el estudiante demuestre su conocimiento del tema. Actualmente, realizar este tipo de evaluaciones consume una cantidad de tiempo insostenible, lo que a menudo reduce la calificación a una simple verificación estática de funcionalidad (entradas y salidas), dejando un punto ciego respecto a cómo se construyó la solución.

Resulta paradójico que, mientras los estudiantes adoptan rápidamente modelos de lenguaje para incrementar su productividad y automatizar la resolución de tareas, las metodologías de revisión

docente se mantengan, en gran medida, desprovistas de este mismo avance tecnológico. Nuestra motivación radica en equilibrar esta balanza: implementar el poder de la Inteligencia Artificial no solo como un asistente para quien escribe el código, sino como un respaldo robusto para quien lo evalúa.

Consideramos que el tiempo del docente es el recurso más valioso en el proceso educativo. Por lo tanto, el propósito fundamental de **Graphito** es proveer a los profesores de una herramienta que automatice el desgaste operativo del análisis de código. Al delegar la auditoría semántica y estilométrica a un sistema inteligente, buscamos que el docente optimice sus tiempos de revisión y pueda redirigir ese esfuerzo hacia actividades de mayor impacto pedagógico, tales como asesorías personalizadas, planeación de actividades pedagógicas en grupo y el fomento del pensamiento crítico, áreas donde el criterio humano (*Human-in-the-loop*) es verdaderamente irremplazable.

1.4 Objetivos

1.4.1 Objetivo General

Implementar un sistema de apoyo a la evaluación docente en cursos introductorios de programación en C/C++ basado en comparación semántica de código, que permita cuantificar la similitud lógica entre programas mediante el cálculo de la similitud del coseno e identificar patrones de generación por inteligencia artificial evaluados a través de f1-score y métricas de clasificación, con el fin de optimizar el proceso de evaluación.

1.4.2 Objetivos específicos

- Evaluar comparativamente modelos de representación semántica de código y arquitecturas de redes neuronales para análisis estilométrico, mediante la revisión de sus características, requerimientos computacionales y métricas de desempeño reportadas en la literatura, con el propósito de fundamentar la selección de GraphCodeBERT y CharCNN como componentes de la arquitectura propuesta.
- Diseñar la arquitectura de software del sistema mediante el modelado de diagramas estándar (UML/BPMN), definiendo la orquestación entre el cliente web, la API de servicios y la persistencia de datos.
- Desarrollar el módulo de análisis semántico integrando el modelo preentrenado GraphCodeBERT, automatizando la extracción de Grafos de Flujo de Datos (DFG) y el cálculo de la similitud del coseno entre representaciones vectoriales.
- Entrenar un modelo de clasificación estilométrica (CharCNN), conformando previamente un conjunto de datos conformado por un repositorio de código humano y código generado

automáticamente por IA, evaluando su desempeño en la detección de posibles patrones de inteligencia artificial generativa mediante f1-score.

- Integrar los motores de inteligencia artificial en una plataforma web funcional, permitiendo al docente configurar parámetros de evaluación y visualizar gráficamente los índices de similitud.
- Evaluar el desempeño del sistema en un entorno de pruebas controlado, ejecutando simulaciones con un repositorio estático, para medir sus tiempos de respuesta, precisión y exhaustividad (*recall*) previo a un escenario de producción.

La comparación de alternativas permite establecer criterios de selección técnicos y metodológicos antes de integrar los modelos al sistema. Se consideran aspectos como la capacidad de representación del código, el tipo de características que procesan, los recursos computacionales requeridos, la complejidad de implementación y la pertinencia de las métricas disponibles. El propósito no es incorporar simultáneamente todos los modelos estudiados, sino seleccionar aquellos que mejor respondan a los objetivos del proyecto.

Capítulo 2

Estado del Arte

2.1 Problema de la detección de similitud de código

La detección de similitud y plagio en código fuente es un problema ampliamente estudiado dentro de la ingeniería de software y la educación en ciencias de la computación. Este problema surge principalmente en cursos de programación donde múltiples estudiantes desarrollan soluciones para un mismo problema, lo que puede facilitar la copia parcial o total de código entre estudiantes. El plagio de código se define como el uso de un programa desarrollado por otra persona sin la debida atribución, lo cual dificulta evaluar si el estudiante realmente comprendió el problema o simplemente reutilizó una solución existente. De acuerdo con diversos estudios sobre análisis de similitud de código, la detección automática de plagio constituye un área fundamental dentro del análisis de software, ya que también se relaciona con tareas como detección de clones de código, reutilización de software y mantenimiento de sistemas [22].

2.2 Enfoques tradicionales basados en texto

Los primeros enfoques para detectar similitud entre programas se basaban principalmente en comparaciones textuales del código fuente. Estos métodos analizan el código como una secuencia de caracteres y buscan coincidencias exactas o casi exactas entre fragmentos de programas. Aunque este enfoque permite identificar copias directas, presenta limitaciones cuando los programas han sido modificados mediante cambios superficiales como renombramiento de variables, modificación de comentarios o alteraciones en el formato del código. En consecuencia, los investigadores comenzaron a explorar métodos más robustos que pudieran detectar similitudes incluso cuando el código había sido modificado para ocultar su origen [23].

2.3 Herramientas basadas en fingerprinting

2.3.1 MOSS

Uno de los sistemas más influyentes en la detección de plagio en código es MOSS (Measure of Software Similarity), desarrollado por Alex Aiken en la Universidad de Stanford. Este sistema utiliza técnicas de fingerprinting, un método que consiste en generar una representación compacta del código mediante huellas digitales o identificadores únicos derivados del contenido del programa. En términos generales, el proceso de fingerprinting consiste en dividir el código en fragmentos y aplicar una función hash a cada uno de ellos. Una función hash es un algoritmo matemático que transforma una secuencia de caracteres en un número de longitud fija. Estas huellas permiten representar el código de forma resumida, de modo que dos programas con fragmentos similares generarán hashes similares o idénticos [24].

Funcionamiento de MOSS

- División del código en fragmentos
- Aplicación de funciones hash
- Generación de huellas digitales
- Selección de fingerprints mediante winnowing

Para mejorar la precisión de este proceso, MOSS utiliza el algoritmo de winnowing, un método diseñado para seleccionar un subconjunto representativo de las huellas digitales generadas a partir del código. El algoritmo funciona generando múltiples hashes a partir de fragmentos consecutivos del programa y posteriormente seleccionando aquellos que representan la información más relevante dentro de una ventana de tamaño determinado. Esta técnica permite garantizar que cualquier coincidencia significativa entre dos programas sea detectada incluso cuando se realizan modificaciones menores en el código. Gracias a este procedimiento, el sistema puede identificar fragmentos similares de manera eficiente sin necesidad de comparar el código completo carácter por carácter [24].

Limitaciones

- Sensible a cambios estructurales del código
- No detecta similitud semántica profunda

A pesar de su amplia adopción en instituciones educativas, el sistema MOSS presenta ciertas limitaciones relacionadas con el tipo de similitud que es capaz de detectar. Debido a que su

funcionamiento se basa en la comparación de huellas digitales generadas a partir de fragmentos del código, el sistema puede verse afectado cuando los programas presentan modificaciones significativas en su estructura, incluso si implementan la misma lógica algorítmica. Por ejemplo, cambios como la reestructuración de bucles, la división de funciones o la reorganización del flujo del programa pueden alterar las huellas digitales generadas, reduciendo la probabilidad de detectar coincidencias. Estas limitaciones han motivado el desarrollo de enfoques más avanzados capaces de analizar la semántica del código en lugar de basarse únicamente en coincidencias textuales o estructurales [25].

2.4 Herramientas basadas en análisis sintáctico y evaluación automática

2.4.1 JPlag

Otra herramienta ampliamente utilizada en entornos educativos es JPlag, un sistema desarrollado en el Karlsruhe Institute of Technology que permite detectar similitudes entre múltiples programas dentro de un conjunto de entregas. A diferencia de MOSS, que se basa principalmente en fingerprinting, JPlag utiliza un enfoque basado en tokenización del código. La tokenización consiste en convertir el código fuente en una secuencia de unidades léxicas llamadas tokens, que representan elementos fundamentales del lenguaje de programación como palabras clave, operadores, identificadores y símbolos. Una vez que el código se transforma en tokens, el sistema compara las secuencias obtenidas utilizando algoritmos de coincidencia para identificar similitudes estructurales entre programas [26].

2.4.2 Gradescope

Otro sistema ampliamente utilizado en entornos educativos para la evaluación automática de programas es Gradescope, una plataforma que permite automatizar la revisión de tareas de programación mediante la ejecución de pruebas sobre el código entregado por los estudiantes. Este sistema se enfoca principalmente en la validación funcional del programa, evaluando si el código produce los resultados esperados para un conjunto de casos de prueba definidos por el instructor. Aunque este enfoque facilita la calificación automática de ejercicios de programación, su capacidad para detectar similitud entre programas es limitada, ya que se centra principalmente en verificar la funcionalidad del código y no necesariamente en analizar la lógica interna de la solución implementada. En consecuencia, dos programas que resuelven el mismo problema mediante estructuras algorítmicas equivalentes pueden obtener resultados idénticos sin que el sistema detecte posibles coincidencias estructurales entre ellos [27].

2.4.3 CodeGrade

De manera similar, herramientas como CodeGrade han sido desarrolladas para apoyar la evaluación automática en cursos de programación mediante la integración de sistemas de ejecución de

pruebas y detección de similitud entre programas. CodeGrade permite automatizar la revisión de tareas y proporcionar retroalimentación inmediata a los estudiantes, lo cual resulta particularmente útil en cursos con un gran número de participantes. Sin embargo, al igual que otras herramientas basadas en análisis sintáctico o textual, su capacidad para identificar similitud profunda entre programas puede verse limitada cuando los estudiantes aplican transformaciones estructurales al código, como el uso de diferentes estructuras de control o la reorganización de funciones dentro del programa [28].

Herramienta	**Enfoque**	**Ventaja**	**Limitación**
MOSS	Fingerprinting	Eficiente y escalable	No detecta semántica
JPlag	Tokenización	Detecta similitud estructural	Evadible
Gradescope	Pruebas	Automatización	No analiza lógica
CodeGrade	Evaluación automática	Retroalimentación	Limitado

Tabla 2.1: Comparación de herramientas de análisis de código

2.5 Representaciones estructurales del código

2.5.1 Árboles de sintaxis abstracta (AST)

Con el objetivo de mejorar la detección de similitud entre programas, los investigadores han explorado representaciones estructurales del código basadas en árboles de sintaxis abstracta (AST). Un árbol de sintaxis abstracta es una representación jerárquica del programa que describe su estructura gramatical de acuerdo con las reglas del lenguaje de programación. En esta representación, los nodos del árbol corresponden a construcciones del lenguaje como operaciones aritméticas, expresiones lógicas o estructuras de control. El uso de AST permite comparar la estructura del programa en lugar de su representación textual, lo que facilita detectar similitudes entre programas que presentan diferencias sintácticas superficiales [29].

El análisis basado en árboles de sintaxis abstracta ha sido ampliamente utilizado en sistemas de detección de clones de código debido a su capacidad para capturar la estructura gramatical de los programas. A diferencia de los métodos basados en texto, el análisis de AST permite ignorar detalles superficiales como espacios, comentarios o nombres de variables, enfocándose en la estructura lógica del programa. No obstante, este enfoque también presenta ciertas limitaciones. Dos programas que implementan el mismo algoritmo utilizando diferentes construcciones sintácticas pueden producir árboles sintácticos significativamente distintos, lo que dificulta la detección de similitudes semánticas profundas. Por esta razón, muchos trabajos recientes han propuesto combinar el análisis estructural con técnicas de aprendizaje automático que permitan capturar patrones más complejos en el código [29].

2.5.2 Grafos de flujo de control y datos

Sin embargo, el análisis estructural mediante árboles sintácticos puede resultar insuficiente para detectar similitudes más profundas relacionadas con la lógica del programa. Por esta razón, algunos enfoques utilizan representaciones basadas en grafos, como los grafos de flujo de control (CFG) y los grafos de dependencia de datos (DFG). Los grafos de flujo de control representan las posibles rutas de ejecución dentro de un programa, mientras que los grafos de dependencia de datos modelan las relaciones entre variables y operaciones. Estas representaciones permiten capturar información semántica más rica del programa, ya que describen cómo fluye la información dentro del código y cómo interactúan sus diferentes componentes [30].

2.5.3 Embeddings de código

En los últimos años, el avance del aprendizaje automático ha permitido desarrollar nuevas técnicas para analizar código fuente mediante representaciones vectoriales conocidas como embeddings. Un embedding es una representación numérica de alta dimensión que captura características semánticas de un objeto. En el contexto del código fuente, los embeddings permiten transformar fragmentos de código en vectores dentro de un espacio matemático donde programas similares se ubican cerca entre sí. Para medir la similitud entre estos vectores se utilizan métricas como la similitud del coseno [31].

Representación	Nivel	Ventaja	Limitación
Texto	Superficial	Simple	Fácil de evadir
AST	Sintáctico	Analiza estructura	No semántico
Grafos	Semántico	Captura flujo	Complejo
Embeddings	Semántico profundo	Detecta lógica	Requiere modelos

Tabla 2.2: Comparación de representaciones de código

2.6 Modelos basados en aprendizaje profundo

Una de las principales innovaciones en este campo ha sido la aplicación de modelos basados en la arquitectura Transformer, originalmente desarrollada para tareas de procesamiento de lenguaje natural. Estos modelos utilizan mecanismos de atención que permiten analizar relaciones entre diferentes partes de una secuencia de texto o código. Entre los modelos más relevantes se encuentra CodeBERT, un modelo preentrenado desarrollado por Microsoft que fue entrenado utilizando grandes repositorios de código fuente y documentación que permite generar representaciones semánticas del código que pueden utilizarse para tareas como búsqueda de código, generación de comentarios y detección de similitud entre programas [32].

Una evolución de este enfoque es GraphCodeBERT, que incorpora información estructural adicional mediante grafos de flujo de datos durante el proceso de entrenamiento. A diferencia de

modelos que analizan únicamente secuencias de tokens, este modelo integra relaciones entre variables y dependencias de datos dentro del programa, lo que permite capturar información semántica más profunda sobre el comportamiento del código. Investigaciones recientes han demostrado que este modelo puede mejorar significativamente la detección de similitud semántica entre programas que implementan la misma lógica mediante estructuras sintácticas diferentes [5].

Además de CodeBERT y GraphCodeBERT, investigaciones recientes han propuesto otros modelos basados en arquitecturas Transformer diseñados específicamente para el análisis de código fuente. Uno de estos modelos es UniXcoder, desarrollado por investigadores de Microsoft, el cual busca aprender representaciones unificadas del código mediante diferentes tareas de preentrenamiento. Este modelo combina información sintáctica y semántica del programa para generar representaciones que pueden utilizarse en tareas como búsqueda de código, generación automática de documentación y análisis de similitud entre programas. Estudios recientes han demostrado que este tipo de modelos puede capturar patrones complejos en el código y mejorar la capacidad de los sistemas para identificar relaciones semánticas entre programas escritos de manera diferente [33].

Características de los modelos basados en Transformers

- Capturan relaciones semánticas dentro del código mediante mecanismos de atención
- Permiten representar programas en espacios vectoriales de alta dimensión
- Integran información sintáctica y semántica en una sola representación

Modelo	Tipo	Característica	Aplicación
CodeBERT	Transformer	Representación semántica	Búsqueda de código
GraphCodeBERT	Transformer + grafos	Usa flujo de datos	Similitud semántica
UniXcoder	Multimodal	Representación unificada	Análisis avanzado

Tabla 2.3: Modelos basados en Transformers para análisis de código

2.7 Detección de código generado por inteligencia artificial

De manera complementaria, algunos trabajos recientes han propuesto el uso de técnicas de aprendizaje profundo para identificar patrones semánticos dentro del código utilizando redes neuronales y representaciones basadas en grafos. Por ejemplo, investigaciones sobre aprendizaje semántico de código mediante grafos han demostrado que los modelos que analizan relaciones

estructurales entre instrucciones pueden mejorar la detección de clones semánticos, es decir, programas que realizan la misma funcionalidad, aunque presenten diferencias en su estructura textual [30]. Estos enfoques permiten capturar información a múltiples niveles del programa, incluyendo tokens, sentencias y relaciones entre estructuras de control.

En los últimos años también se han realizado diversos estudios que exploran el uso de representaciones semánticas del código para detectar plagio o similitud entre programas. Por ejemplo, investigaciones recientes han propuesto sistemas de búsqueda semántica de código que utilizan modelos preentrenados para generar embeddings de programas y posteriormente comparar estos vectores utilizando métricas de similitud. Estos sistemas han demostrado que las representaciones semánticas pueden capturar relaciones entre programas incluso cuando presentan diferencias significativas en su estructura sintáctica. En particular, algunos trabajos han mostrado que los modelos basados en Transformers pueden correlacionar de manera significativa con evaluaciones humanas al medir la similitud entre programas, lo que sugiere que estas técnicas pueden convertirse en herramientas valiosas para apoyar la evaluación automática en cursos de programación [34].

Finalmente, estudios recientes han destacado que el crecimiento de herramientas de generación automática de código basadas en modelos de lenguaje introduce nuevos desafíos para la detección de plagio en entornos educativos. Los modelos generativos actuales son capaces de producir soluciones funcionalmente correctas para problemas de programación, lo que dificulta determinar si el código fue desarrollado por un estudiante o generado mediante inteligencia artificial. Ante este escenario, algunos trabajos han comenzado a explorar el uso de técnicas de análisis estilométrico y aprendizaje automático para identificar patrones característicos asociados a código generado automáticamente [35].

Retos en la detección de código generado por IA

- Los modelos generativos producen código funcionalmente correcto
- Difícil distinguir entre código humano y generado automáticamente
- Los patrones de generación pueden ser consistentes pero no evidentes

2.8 Análisis y brecha de investigación

En general, la literatura muestra que los sistemas existentes para la detección de similitud de código suelen enfocarse en uno de dos enfoques principales: el análisis estructural del programa o el análisis semántico mediante representaciones aprendidas por modelos de aprendizaje profundo. Sin embargo, cada uno de estos enfoques presenta limitaciones cuando se utiliza de manera aislada. Mientras que los métodos estructurales pueden verse afectados por modificaciones sintácticas significativas, los modelos basados en aprendizaje profundo pueden requerir grandes

cantidades de datos para su entrenamiento y presentar dificultades para explicar sus decisiones. Por esta razón, varios trabajos recientes sugieren que la combinación de diferentes enfoques de análisis puede mejorar la precisión y robustez de los sistemas de detección de similitud de código [22].

Limitaciones identificadas en la literatura

- Los enfoques estructurales no capturan completamente la semántica del código
- Los modelos de aprendizaje profundo requieren grandes volúmenes de datos
- Existe dificultad para interpretar los resultados de modelos complejos

En síntesis, la investigación en detección de similitud de código ha evolucionado desde enfoques basados en comparación textual hacia métodos más avanzados que incorporan representaciones estructurales y semánticas del programa. Mientras que herramientas tradicionales como MOSS y JPlag se enfocan principalmente en coincidencias sintácticas, los modelos modernos basados en aprendizaje profundo y arquitecturas Transformer permiten capturar relaciones semánticas más complejas dentro del código. No obstante, el desarrollo reciente de herramientas de generación automática de código plantea nuevos retos para la evaluación académica, lo que sugiere la necesidad de desarrollar sistemas híbridos que integren múltiples técnicas de análisis para mejorar la detección de similitud y la identificación de código generado mediante inteligencia artificial.

A pesar de los avances en la detección de similitud de código, las herramientas tradicionales como MOSS o JPlag se enfocan principalmente en coincidencias textuales o estructurales, lo que limita su capacidad para detectar equivalencias semánticas profundas entre programas que implementan la misma lógica mediante estructuras diferentes. Por otra parte, los modelos recientes basados en aprendizaje profundo y arquitecturas Transformer han demostrado ser capaces de capturar relaciones semánticas más complejas dentro del código, aunque su aplicación en entornos educativos aún presenta desafíos relacionados con la interpretación de resultados y la integración con herramientas de evaluación académica.

Asimismo, el crecimiento de herramientas de generación automática de código basadas en modelos de lenguaje ha introducido nuevos retos para la detección de plagio en programación. Los sistemas actuales no siempre permiten distinguir de manera confiable entre código escrito por estudiantes y código generado mediante inteligencia artificial, lo que dificulta evaluar adecuadamente el aprendizaje en cursos introductorios de programación.

Necesidad identificada

- Integrar análisis semántico y estilométrico en un mismo sistema

- Mejorar la interpretación de resultados para apoyo docente
- Desarrollar herramientas aplicables en contextos educativos reales

En este contexto, se identifica una brecha en la literatura relacionada con el desarrollo de sistemas que integren simultáneamente análisis semántico del código y técnicas adicionales que permitan identificar patrones asociados al uso de herramientas de inteligencia artificial. Por lo tanto, resulta pertinente explorar enfoques híbridos que combinen representaciones semánticas del código con técnicas complementarias de análisis, con el objetivo de mejorar la evaluación automática en entornos educativos.

Capítulo 3

Marco Teórico

3.1 Introducción al análisis automático de código

El análisis automático de código fuente es un área interdisciplinaria que integra conceptos de ingeniería de software, procesamiento de lenguaje natural y aprendizaje automático. En el ámbito educativo, particularmente en cursos introductorios de programación, se emplea para evaluar soluciones desarrolladas por estudiantes y detectar similitudes entre programas.

Sin embargo, debido a la complejidad estructural y semántica de los lenguajes de programación, identificar estas similitudes representa un desafío técnico. A lo largo del tiempo, se han propuesto diversas metodologías que abarcan desde comparaciones textuales hasta enfoques basados en grafos y modelos de aprendizaje profundo.

En este contexto, resulta necesario analizar los principales enfoques y técnicas utilizadas en el análisis de código, con el fin de comprender sus características, alcances y limitaciones.

3.2 Problema de la detección de similitud de código

Roy y Cordy describen los clones de código como fragmentos de programas que presentan similitudes en su estructura o funcionalidad, y destacan que su detección es una tarea fundamental para el mantenimiento y análisis de software [23]. No obstante, este proceso enfrenta múltiples desafíos técnicos debido a la ambigüedad estructural del código y a la existencia de múltiples implementaciones para un mismo algoritmo. Esto hace más complejo distinguir entre una similitud semántica real y coincidencias sintácticas por el uso de convenciones de programación estándar y buenas prácticas.

Para representar estos obstáculos, la literatura identifica los siguientes retos principales en la detección automática [23, 36]:

- **Variaciones sintácticas:** Cambios en el estilo de programación, nombres de variables o

estructuras de control que no alteran el resultado.

- **Ofuscación de código:** Transformaciones deliberadas para hacer el código ilegible manteniendo su funcionalidad original.
- **Diferentes implementaciones:** El uso de distintos algoritmos o estructuras de datos para resolver el mismo problema lógico.
- **Código generado por IA:** El auge de modelos de lenguaje (LLMs) permite generar múltiples versiones de un mismo programa, introduciendo patrones de similitud difíciles de rastrear mediante métodos tradicionales.

Estos desafíos evidencian la necesidad de una taxonomía que permita categorizar el grado de semejanza entre programas, lo cual se aborda a través de la clasificación técnica de clones.

3.3 Clones de código

La similitud entre programas no es un concepto binario, sino que puede presentarse en distintos niveles dependiendo del grado en que comparten estructura o funcionalidad. Para abordar esta variabilidad, la literatura propone clasificaciones que permiten categorizar los diferentes tipos de similitud. Una de las más utilizadas es la taxonomía de Bellon [37], la cual organiza los clones de código en cuatro niveles principales.

Tabla 3.1: Tipos de clones de código según Bellon.

Tipo	**Nivel de Similitud**	**Descripción**
Tipo I	Textual / Exacto	Copias idénticas, excepto por espacios y comentarios.
Tipo II	Sintáctico / Renombrado	Estructura idéntica con cambios en identificadores o tipos.
Tipo III	Sintáctico / Modificado	Código con sentencias añadidas, eliminadas o modificadas.
Tipo IV	Semántico / Funcional	Misma lógica implementada con distinta sintaxis.

Esta clasificación refleja un incremento progresivo en la complejidad de detección: mientras que los clones de Tipo I y II pueden identificarse mediante análisis superficiales, los clones de Tipo III requieren tolerancia a modificaciones estructurales, y los clones de Tipo IV implican comprender la lógica del programa. Por esta razón, los clones semánticos representan el principal desafío en la detección de similitud de código y motivan el uso de enfoques más avanzados.

3.4 Enfoques para la detección de similitud de código

Existen distintos enfoques para la detección de similitud de código, los cuales se diferencian en el nivel de abstracción con el que analizan los programas. Estos van desde comparaciones

textuales superficiales hasta métodos que capturan la estructura, la semántica o representaciones vectoriales mediante aprendizaje automático [23].

Cada enfoque presenta un compromiso entre precisión, capacidad para capturar similitud semántica y costo computacional, lo que determina su aplicabilidad en distintos contextos.

3.4.1 Enfoques basados en análisis textual

Los enfoques textuales tratan el programa como una secuencia de caracteres o cadenas, comparando fragmentos directamente tras aplicar filtros básicos como la eliminación de espacios y comentarios [23]. Su fortaleza radica en la eficiencia computacional y en la alta precisión para detectar copias exactas (clones Tipo I), sin requerir conocimiento gramatical del lenguaje [23, 38]. Sin embargo, al operar sobre la superficie del texto, son vulnerables a cambios menores como el renombrado de variables, la reubicación de instrucciones o la modificación del formato, los cuales alteran la representación textual sin afectar la lógica del programa [23].

3.4.2 Enfoques basados en estructura

Los enfoques estructurales analizan la organización interna del programa en lugar de su forma textual, modelando unidades como funciones, bloques de control y relaciones entre componentes. A partir de estas unidades se derivan métricas como la profundidad de anidamiento, la complejidad ciclomática o el número de llamadas a funciones, lo que permite identificar similitudes dentro de límites sintácticos específicos y a diferentes niveles de granularidad [23, 36].

A diferencia de los métodos textuales, estos enfoques capturan la organización lógica del código y las preferencias de diseño del programador como la utilización de funciones, modularidad, patrones recurrentes con menor sensibilidad a cambios de formato o nombres de variables [36]. No obstante, estas representaciones siguen siendo estáticas, puesto que no modelan el comportamiento dinámico del programa, por lo que cambios equivalentes como sustituir un bucle `for` por un `while` pueden producir estructuras distintas sin alterar la funcionalidad, limitando la detección de clones semánticos [23, 38].

3.4.3 Enfoques basados en semántica

Los enfoques semánticos analizan el código desde la perspectiva de cómo funcionan, ya que en lugar de examinar cómo está escrito, buscan comprender lo que hace, modelando su lógica de ejecución y las relaciones entre datos [23, 30]. Para ello, en lugar de comparar texto o estructura gramatical, abstraen el programa en representaciones que capturan las dependencias entre instrucciones y el flujo de información interno, independientemente de cómo esté implementado sintácticamente.

Esta capacidad de abstracción los convierte en los únicos enfoques capaces de detectar clones de

Tipo IV: programas funcionalmente equivalentes con una estructura sintáctica diferente [23, 30]. Por ejemplo, un bucle `for` y una recursión que cumplen con la misma tarea serían completamente diferentes para métodos textuales o estructurales, pero equivalentes bajo un análisis semántico.

Sin embargo, esta profundidad analítica tiene un costo, puesto que comparar programas a nivel semántico implica resolver problemas de alta complejidad computacional, lo que limita la escalabilidad de los métodos clásicos [30]. Investigaciones recientes han respondido a esta limitación con modelos de aprendizaje profundo que aproximan la semántica del código de manera más eficiente [30, 39]. Las representaciones concretas sobre las que operan estos enfoques como grafos de flujo de control, flujo de datos y dependencia de programas se describen en detalle más adelante.

3.4.4 Enfoques basados en aprendizaje automático

El análisis de código fuente ha evolucionado con la consolidación del área conocida como *Big Code*, que parte de la premisa de que el código puede interpretarse como un lenguaje altamente estructurado [40]. Bajo este enfoque, los modelos de aprendizaje profundo son capaces de analizar la sintaxis y la semántica de los programas sin necesidad de ejecutarlos, transformando el código en representaciones numéricas continuas denominadas *embeddings*, donde fragmentos con comportamientos similares se ubican en regiones cercanas del espacio vectorial [40, 41].

El paradigma dominante es la arquitectura codificador-decodificador (*encoder-decoder*) donde el codificador transforma representaciones de código como secuencia de tokens, árbol sintáctico o grafo en un vector de contexto, y el decodificador utiliza dicha representación para tareas como clasificación, búsqueda o detección de clones [40]. Los mecanismos de atención permiten al modelo ponderar las partes más relevantes del código durante la inferencia, facilitando la captura de relaciones entre elementos distantes dentro del programa [41]. Su principal limitación es estructural, ya que requieren grandes volúmenes de datos de entrenamiento, tienen alto costo computacional y operan como cajas negras, dificultando la interpretación de sus decisiones [40, 41].

3.4.5 Comparación de enfoques

Los enfoques descritos durante esta sección no son excluyentes entre sí, sino que representan niveles progresivos de abstracción sobre el mismo problema. Para facilitar su comparación, en la Tabla 3.2 se sintetizan las dimensiones más relevantes de cada enfoque.

Tabla 3.2: Comparación de enfoques para la detección de similitud de código

Enfoque	**Abstracción**	**Clones**	**Costo**	**Ventaja principal**	**Limitación principal**
Textual	Bajo (Texto)	I–II	Bajo	Alta eficiencia; independiente del lenguaje.	Vulnerable a cambios de nombres y formato.
Estructural	Medio (AST)	II–III	Medio	Captura lógica y diseño del programa.	Sensible a cambios en estructuras equivalentes.
Semántico	Alto (DFG/CFG)	IV	Alto	Detecta equivalencias funcionales reales.	Complejidad algorítmica y baja escalabilidad.
IA (ML/DL)	Variable (Vectores)	I–IV	Alto	Extracción automática de características complejas.	Requiere grandes datasets; modelo de caja negra.

Elaboración propia con base en [23, 38, 36, 30, 40, 41].

Esta limitación demuestra que, si se planea hacer una comparación compleja entre programas, no basta con utilizar un sólo enfoque para el análisis. Antes de determinar cómo afrontar el problema de realizar la comparación semántica mientras se tiene en cuenta los patrones de autoría, resulta necesario analizar las representaciones internas del código sobre las que operan estos enfoques, tema que se aborda en la siguiente sección.

3.5 Representaciones del código fuente

El análisis automático de código fuente requiere transformar los programas en estructuras que permitan su procesamiento y comparación. Estas transformaciones determinan el tipo de información que puede ser capturada y, por tanto, el nivel de similitud que el sistema es capaz de identificar. Las representaciones van desde la forma textual del programa hasta estructuras que modelan su comportamiento semántico, cada una con un equilibrio distinto entre expresividad y costo computacional.

3.5.1 Representaciones textuales

El nivel más básico consiste en tratar el código como una secuencia lineal de elementos. Existen tres variantes principales según el grado de abstracción aplicado [23]:

- **Nivel de caracteres:** el código se modela como una secuencia continua de caracteres y se compara mediante técnicas como *fingerprinting*. Es simple y eficiente, pero altamente sensible a cambios mínimos de formato como espacios o saltos de línea [23].
- **Nivel de tokens:** un analizador léxico (*lexer*) transforma el código en una secuencia de unidades como palabras clave, operadores e identificadores. Es común aplicar normalización sustituyendo nombres de variables por símbolos genéricos para reducir la variabilidad [23].
- ***N*-gramas:** se construyen subsecuencias contiguas de longitud n sobre caracteres o tokens. Valores entre 6 y 8 tokens ofrecen un buen balance entre precisión y generalización [42].

En todos los casos, el preprocesamiento incluye la eliminación de comentarios y espacios en blanco, y la comparación se realiza mediante estructuras eficientes como árboles de sufijos (*suffix trees*) [23]. Estas representaciones son portables entre lenguajes y computacionalmente económicas, pero su naturaleza lineal las hace incapaces de capturar la estructura interna o el comportamiento del programa, lo que motiva el uso de representaciones sintácticas más ricas, descritas en la siguiente sección.

3.5.2 Representaciones sintácticas: Árboles de Sintaxis Abstracta

El Árbol de Sintaxis Abstracta (AST) representa el programa como una estructura jerárquica en la que se eliminan elementos superficiales como espacios, comentarios, o formato; conservando únicamente la información gramaticalmente relevante. Los nodos internos corresponden a constructos del lenguaje como condicionales, bucles o declaraciones de funciones, mientras que los nodos hoja representan operandos como identificadores o literales [23, 43].

Su construcción se realiza mediante un analizador sintáctico (*parser*) que organiza la secuencia de tokens conforme a las reglas de una gramática libre de contexto [29]. Herramientas como *Tree-sitter* o *ANTLR* automatizan este proceso generando estructuras navegables que representan fielmente la sintaxis del programa. A diferencia de las representaciones textuales, los AST abstraen detalles como nombres de variables o formato y agrupan instrucciones en subárboles que corresponden a unidades sintácticas bien definidas, lo que los hace inmunes a modificaciones superficiales y adecuados para detectar clones de Tipo II y Tipo III [23, 44].

No obstante, presentan dos limitaciones importantes: requieren analizadores específicos para cada lenguaje, lo que reduce su portabilidad, y la comparación entre árboles mediante distancia de

edición o isomorfismo implica algoritmos de alto costo computacional [23, 44]. Más fundamentalmente, el AST captura la organización gramatical del programa pero no modela cómo fluye la información entre sus instrucciones, por lo que transformaciones equivalentes como sustituir un bucle `for` por un `while` pueden producir árboles distintos sin alterar la funcionalidad [23]. Esta limitación motiva el uso de representaciones basadas en grafos, descritas en la siguiente sección.

3.5.3 Representaciones basadas en grafos

Las representaciones basadas en grafos modelan el programa como una red de relaciones donde los nodos representan entidades del código como sentencias o variables y las aristas representan dependencias o relaciones de ejecución [30]. Existen tres variantes principales:

- **Grafo de Flujo de Control (CFG):** representa el orden de ejecución mediante aristas dirigidas que modelan los posibles caminos del programa, incluyendo condicionales, ciclos y bifurcaciones [30].
- **Grafo de Flujo de Datos (DFG):** modela cómo los valores se producen, transforman y consumen a lo largo del programa, representando dependencias entre variables [5].
- **Grafo de Dependencia de Programas (PDG):** integra el flujo de control y de datos en una única representación, ofreciendo una visión semántica completa del comportamiento del programa [30].

A diferencia del AST, que captura únicamente la jerarquía gramatical, estas representaciones modelan explícitamente las dependencias que determinan el comportamiento en tiempo de ejecución, incluyendo relaciones causales entre instrucciones y ciclos como bucles o recursividad [45]. Esto las convierte en las representaciones más adecuadas para detectar clones de Tipo IV, al abstraer completamente las diferencias superficiales de sintaxis o estilo [30, 5].

Su principal desventaja es el costo: la construcción y comparación de grafos implica algoritmos costosos en tiempo y memoria, y requieren herramientas avanzadas de análisis estático adaptadas a cada lenguaje [30]. Esta limitación ha impulsado el desarrollo de representaciones más compactas que proyectan estas estructuras complejas en espacios vectoriales continuos, descritas en la siguiente sección.

3.5.4 Comparación de representaciones

Las representaciones descritas en las secciones anteriores no son equivalentes entre sí ya que cada una captura un nivel distinto de información del programa. La Tabla 3.3 sintetiza estas diferencias considerando las dimensiones más relevantes para un sistema de comparación de código.

Tabla 3.3: Comparación de representaciones del código fuente

Representación	**Abstracción**	**Costo**	**Ventaja principal**	**Limitación principal**
Textual	Bajo (cadenas o tokens)	Bajo	Alta portabilidad; no requiere conocimiento gramatical [23]	Sensible a cambios de formato; no captura semántica [23]
AST	Medio (jerarquía sintáctica)	Medio	Abstrae formato; captura organización lógica [23, 44]	No modela flujo de datos; requiere *parser* por lenguaje [23]
Grafos (CFG/DFG)	Alto (flujo y datos)	Alto	Inmune a variaciones sintácticas; captura intención [30, 5]	Alta complejidad de extracción y comparación [30]

Elaboración propia con base en [23, 44, 30, 5].

Como se observa, las representaciones más simples priorizan la eficiencia a costa de profundidad semántica, mientras que las basadas en grafos logran capturar equivalencias funcionales complejas con un costo computacional mayor [23, 30]. Esta falta de equivalencia entre las representaciones ha motivado el desarrollo de un mecanismo que permite proyectar cualquiera de estas representaciones (textual, sintáctica o basada en grafos) en un espacio vectorial continuo donde la similitud puede cuantificarse mediante operaciones geométricas simples. Dicho mecanismo, conocido como *embedding*, no constituye una representación alternativa del código, sino una capa de procesamiento que puede aplicarse sobre las anteriores y que actúa como puente hacia los modelos de aprendizaje profundo descritos en la siguiente sección [40, 41].

3.5.5 Representaciones distribuidas: embeddings

Un elemento transversal a todas las representaciones descritas es la posibilidad de proyectarlas en un espacio vectorial continuo mediante modelos de aprendizaje profundo, generando lo que se conoce como *embeddings*. Un embedding no es una representación alternativa del código, sino una transformación numérica que puede aplicarse sobre cualquiera de las anteriores: una secuencia de tokens, un AST o un grafo de dependencias pueden convertirse en vectores densos de dimensión fija [40, 41].

La ventaja de esta proyección es que fragmentos de código con comportamientos similares tienden a ubicarse en regiones cercanas del espacio vectorial, lo que permite cuantificar la similitud mediante operaciones geométricas simples [41]. La siguiente sección se enfoca en presentar distintas medidas de similitud en código fuente, destacando sus usos, fortalezas y limitaciones.

3.6 Medidas de similitud en código fuente

Una vez definidas las representaciones del código, es necesario establecer cómo se cuantifica la semejanza entre dos fragmentos. La elección de la métrica está directamente condicionada por la representación utilizada: no es equivalente comparar cadenas de texto que árboles sintácticos, grafos o vectores continuos [23, 45]. La Tabla 3.4 resume las principales métricas empleadas en la literatura según su nivel de abstracción.

Tabla 3.4: Comparación de métricas de similitud en código fuente

Métrica	**Nivel**	**Clones**	**Ventaja**	**Limitación**
Similitud de Jaccard	Léxico	Tipo I–II	Simple y eficiente; mide solapamiento de tokens [23]	No captura orden ni estructura.
Distancia de Levenshtein	Léxico	Tipo I–II	Detecta copias cercanas con cambios mínimos [23]	Sensible a renombrado y reordenamiento.
Tree Edit Distance	Sintáctico	Tipo II–III	Captura similitud estructural sobre el AST [29]	Costo crece con la profundidad del árbol.
Graph Edit Distance	Semántico	Tipo III–IV	Captura diferencias en flujo de datos [30]	Complejidad NP-completo; baja escalabilidad.
Similitud del coseno	Vectorial	Tipo I–IV	Alta robustez semántica sobre *embeddings* [41, 45]	Depende de la calidad del modelo de IA.
Distancia euclidiana	Vectorial	Tipo I–IV	Intuitiva en espacios de baja dimensión [45]	Sensible a la magnitud de los vectores.

Como se observa, existe una relación directa entre la profundidad semántica de una métrica y su costo computacional [23]. Las métricas léxicas son eficientes pero frágiles ante modificaciones intencionales del código, mientras que las basadas en grafos ofrecen mayor capacidad de abstracción a costa de una complejidad computacional que las hace inviables a gran escala [30]. La similitud del coseno sobre embeddings representa el punto de equilibrio más adecuado para el contexto de Graphito: combinada con un modelo que incorpore información estructural del código, permite comparar programas con alta robustez semántica y bajo costo de inferencia [41, 45].

En la siguiente sección se exponen distintos modelos que siguen los enfoques y representaciones previamente planteados para evaluar sus fortalezas y limitaciones.

3.7 Modelos para análisis de código

3.7.1 Evolución del análisis automático de código

El análisis automatizado de código fuente ha evolucionado desde enfoques basados en características diseñadas manualmente hacia modelos capaces de aprender representaciones de manera automática a partir de los datos [41]. Esta transición responde a las limitaciones de los métodos tradicionales para capturar la estructura y semántica del código.

En general, los sistemas de análisis de código siguen un pipeline común: el código fuente se transforma en una representación procesable, un modelo analiza dicha representación y, finalmente, se emplea una métrica para cuantificar la similitud entre programas [41].

3.7.1.1 Modelos clásicos de aprendizaje automático

Los primeros enfoques utilizaron algoritmos como Máquinas de Vectores de Soporte (SVM), Bosques Aleatorios (Random Forest) y k-Vecinos Más Cercanos (kNN), operando sobre vectores de características extraídas manualmente, como frecuencias de tokens o métricas de software [46].

Estos modelos resultan eficientes y relativamente simples de implementar; sin embargo, su desempeño depende directamente de la calidad de las características diseñadas. Al centrarse en propiedades superficiales, presentan dificultades para capturar relaciones estructurales o semánticas del código, lo que los hace sensibles a transformaciones como renombrado de variables o cambios de estilo [47].

3.7.1.2 Redes neuronales (RNN y CNN)

Las redes neuronales introducen la capacidad de aprender representaciones automáticamente a partir del código, reduciendo la dependencia de la ingeniería manual de características [41].

Las Redes Neuronales Recurrentes (RNN) y sus variantes como LSTM modelan el código como una secuencia de tokens, procesando cada elemento en función del contexto previo mediante un estado interno [48]. Este enfoque permite capturar dependencias locales dentro del programa, aunque presenta dificultades para modelar relaciones a largo alcance debido a su naturaleza secuencial [40].

Por su parte, las Redes Neuronales Convolucionales (CNN), especialmente a nivel de caracteres (CharCNN), aplican filtros sobre fragmentos locales del código para identificar patrones recurrentes [7]. Esta aproximación resulta útil para capturar características estilísticas, pero su

capacidad para representar la estructura global del programa es limitada [40].

En conjunto, estos modelos mejoran la representación del código respecto a los métodos clásicos, aunque aún presentan limitaciones para capturar completamente su semántica.

3.7.1.3 Transformers

Los modelos basados en Transformers representan el estado del arte en el análisis de código, al utilizar mecanismos de atención que permiten relacionar directamente todos los elementos de una secuencia [49]. A través de este mecanismo, el modelo puede identificar qué partes del código son relevantes entre sí, capturando dependencias globales de manera más efectiva que los enfoques secuenciales.

Esta capacidad resulta especialmente relevante en código fuente, donde variables, funciones o estructuras pueden interactuar a lo largo de diferentes partes del programa. Además, la atención multicabezal permite modelar distintos tipos de relaciones en paralelo, facilitando la captura tanto de patrones sintácticos como semánticos.

Como resultado, los Transformers han demostrado una alta capacidad para aprender representaciones profundas del código, siendo la base de los modelos modernos en tareas como detección de similitud, búsqueda de código y análisis semántico [49]. No obstante, su uso implica un mayor costo computacional y una menor interpretabilidad en comparación con enfoques más simples.

3.7.2 Modelos basados en texto y tokens

Los modelos basados en texto y tokens constituyen una de las primeras aproximaciones modernas al análisis de código fuente mediante aprendizaje profundo. Su principio fundamental consiste en tratar el código como lenguaje natural, representándolo como una secuencia lineal de tokens sin considerar explícitamente su estructura sintáctica o semántica.

A pesar de esta simplificación, el uso de arquitecturas basadas en Transformers permite capturar dependencias contextuales entre los elementos del código, generando representaciones útiles para diversas tareas. No obstante, su capacidad está limitada por la ausencia de información estructural del programa.

3.7.2.1 RoBERTa para código

RoBERTa es una variante optimizada de BERT (Transformer) que mejora el proceso de preentrenamiento mediante el uso de mayores volúmenes de datos y la eliminación de ciertos objetivos auxiliares [50]. En el análisis de código, se emplea como modelo base para procesar el código fuente como texto plano.

El modelo utiliza tokenización basada en Codificación de Par de Bytes (BPE), lo que le per-

mite manejar eficientemente el vocabulario abierto característico del código fuente, donde los identificadores pueden variar ampliamente.

Su entrenamiento se basa en el objetivo de Modelado de Lenguaje Enmascarado, aprendiendo a predecir tokens a partir de su contexto. Esto le permite capturar patrones léxicos y ciertas dependencias contextuales dentro del código.

No obstante, al igual que otros modelos basados únicamente en texto, su principal limitación radica en la falta de representación explícita de la estructura del programa, lo que reduce su capacidad para distinguir entre similitudes superficiales y equivalencias funcionales.

3.7.2.2 CodeBERT

CodeBERT es un modelo bimodal basado en Transformers, diseñado para aprender representaciones conjuntas de código fuente y lenguaje natural [41]. Su arquitectura se basa en RoBERTa, lo que le permite procesar secuencias de tokens de manera bidireccional, capturando el contexto completo de cada elemento.

El modelo representa el código como una secuencia lineal de tokens, combinándolo opcionalmente con texto natural mediante tokens especiales como *[CLS]* y *[SEP]*. El token *[CLS]* se utiliza como un marcador inicial que permite al modelo generar una representación global de toda la secuencia, mientras que *[SEP]* funciona como un separador entre diferentes segmentos, como una descripción en lenguaje natural y el código correspondiente. Esta organización permite al modelo relacionar ambos tipos de información dentro de una misma entrada, facilitando tareas como la búsqueda de código o la generación de documentación.

En cuanto a su entrenamiento, CodeBERT utiliza una función híbrida que combina el Modelado de Lenguaje Enmascarado (MLM) y la Detección de Tokens Reemplazados (RTD), aprovechando grandes volúmenes de datos provenientes de repositorios públicos. Este enfoque le permite aprender patrones tanto léxicos como contextuales.

En conjunto, estos modelos demuestran que es posible obtener representaciones útiles del código a partir de su forma textual. Sin embargo, sus limitaciones evidencian la necesidad de incorporar información estructural más rica, lo que ha motivado el desarrollo de modelos basados en representaciones sintácticas y semánticas, como se aborda en la siguiente sección.

3.7.3 Modelos basados en Árboles de Sintaxis Abstracta

Los modelos basados en Árboles de Sintaxis Abstracta (AST) representan una evolución respecto a los enfoques puramente textuales, al incorporar explícitamente la estructura gramatical del programa. En lugar de tratar el código como una secuencia lineal de tokens, estos modelos lo organizan en una representación jerárquica que refleja las relaciones sintácticas entre sus componentes.

Esta representación permite capturar patrones estructurales como la anidación de bloques, la organización de expresiones y la descomposición funcional del código, proporcionando una visión más rica que los modelos basados únicamente en texto.

3.7.3.1 Code2Vec

Code2Vec es un modelo basado en una red neuronal diseñado para aprender representaciones vectoriales de fragmentos de código a partir de su estructura sintáctica [41]. Su arquitectura utiliza un mecanismo de atención suave (soft-attention) para combinar múltiples contextos estructurales en un único vector representativo.

El modelo representa el código como un conjunto de rutas extraídas del AST, donde cada ruta conecta pares de tokens a través de nodos intermedios. Esta estrategia permite capturar relaciones estructurales entre elementos del programa, como la interacción entre parámetros, variables y valores de retorno.

Code2Vec fue entrenado sobre grandes colecciones de código en Java, utilizando como objetivo la predicción del nombre de un método a partir de su implementación. Esto le permite aprender patrones semánticos asociados a la funcionalidad del código.

Sin embargo, su desempeño depende en gran medida de los identificadores presentes en el código, lo que lo hace sensible a la ofuscación o al uso de nombres no vistos durante el entrenamiento.

3.7.3.2 ASTNN

ASTNN es un modelo que combina redes neuronales recursivas para capturar tanto la información estructural como la secuencial del código, abordando las limitaciones de procesar árboles completos de gran profundidad [51].

Para ello, el modelo descompone el AST en una secuencia de subárboles correspondientes a sentencias individuales, codificando cada uno mediante redes neuronales y posteriormente modelando sus relaciones mediante una red recurrente bidireccional. Esta estrategia permite manejar programas de mayor tamaño sin perder información estructural relevante.

ASTNN ha demostrado buenos resultados en tareas como clasificación de código y detección de clones, al combinar información local (estructura de cada subárbol) con el contexto global del programa.

No obstante, su evaluación se ha centrado principalmente en conjuntos de datos académicos, lo que puede limitar su generalización a código real más diverso.

En conjunto, estos modelos evidencian que incorporar la estructura sintáctica del código permite mejorar significativamente la representación respecto a los enfoques basados únicamente en texto. Sin embargo, el AST describe la organización gramatical del programa, pero no modela

explícitamente el flujo de datos o el comportamiento en ejecución. Esta limitación ha motivado el desarrollo de modelos basados en grafos de dependencia, los cuales se abordan en la siguiente sección.

3.7.4 Modelos basados en grafos

Los modelos basados en grafos representan un nivel más avanzado en el análisis de código fuente, al enfocarse en capturar el comportamiento del programa en lugar de solo su forma textual o estructura sintáctica. En este enfoque, el código se modela como un grafo donde los nodos representan elementos del programa (como variables o instrucciones) y las aristas representan relaciones entre ellos, como el flujo de datos o de control.

Esta representación permite identificar similitudes funcionales entre programas, incluso cuando su implementación es completamente diferente, lo que resulta fundamental para la detección de clones semánticos.

3.7.4.1 GGNN (Gated Graph Neural Networks)

GGNN es un modelo basado en Redes Neuronales de Grafos (GNN) que utiliza unidades recurrentes tipo GRU para propagar información entre los nodos del grafo a través de múltiples iteraciones [52]. Esta arquitectura permite que cada nodo actualice su representación considerando el contexto de sus vecinos, capturando dependencias complejas dentro del programa.

Para representar el código, el modelo construye un grafo enriquecido a partir del AST, incorporando tanto relaciones sintácticas como semánticas, incluyendo flujo de datos y control. De este modo, no solo se modela la estructura del programa, sino también cómo se transmiten los valores entre sus componentes.

El modelo fue entrenado de manera supervisada en tareas específicas como predicción de nombres de variables y detección de errores. Su principal fortaleza es la capacidad de capturar relaciones semánticas explícitas; sin embargo, presenta limitaciones en su generalización a nuevos dominios y requiere que el código sea completamente válido para poder construir el grafo.

3.7.4.2 GraphCodeBERT

GraphCodeBERT es un modelo basado en la arquitectura Transformer que integra información estructural del código mediante el uso de grafos de flujo de datos [5]. A diferencia de los modelos puramente secuenciales, introduce restricciones en el mecanismo de atención para reflejar relaciones entre variables.

El modelo representa el código como una combinación de tokens y nodos de un grafo de flujo de datos, permitiendo que el mecanismo de atención considere únicamente aquellas conexiones que tienen sentido semántico. Esto le permite capturar dependencias relevantes sin necesidad de

procesar estructuras completas como el AST.

Su entrenamiento combina el Modelado de Lenguaje Enmascarado (MLM) con tareas adicionales como la predicción de relaciones entre nodos, lo que favorece el aprendizaje de representaciones más estructuradas.

GraphCodeBERT ha demostrado un alto desempeño en tareas como detección de clones, búsqueda de código y análisis semántico. No obstante, su complejidad computacional es elevada y su funcionamiento interno resulta menos interpretable.

En conjunto, los modelos basados en grafos permiten capturar el comportamiento funcional del código, siendo especialmente efectivos para detectar similitudes semánticas profundas. Sin embargo, al centrarse en la lógica del programa, tienden a ignorar características superficiales del código, como el estilo o la forma en que está escrito, lo que limita su utilidad en tareas donde estos aspectos son relevantes.

3.7.5 Comparación de modelos

La evolución de las arquitecturas para el análisis de código refleja un incremento progresivo en la capacidad de abstracción semántica. A continuación, se presenta una síntesis comparativa de los modelos analizados, categorizados por su tipo de representación, destacando sus fortalezas y las limitaciones técnicas que definen su alcance.

Tabla 3.5: Comparación de modelos preentrenados para análisis de código

Modelo	**Categoría**	**Representación**	**Ventaja principal**	**Limitación principal**
CodeBERT	Texto / tokens	Bimodal (Tokens)	Vincula código y lenguaje natural; eficaz en búsqueda [32].	Sin estructura jerárquica; requiere mucho *fine-tuning*.
RoBERTa	Texto / tokens	Tokens BPE	Robusto ante vocabulario abierto; distingue patrones léxicos [50].	Puramente textual; impreciso en lógica funcional equivalente.
Code2Vec	Estructural	Rutas del AST	Captura relaciones estructurales entre identificadores [41].	Vocabulario cerrado; sensible a ofuscación de nombres.

Continúa en la siguiente página...

Tabla 3.5 – Continuación

Modelo	Categoría	Representación	Ventaja principal	Limitación principal
ASTNN	Estructural	Subárboles	Maneja programas largos sin perder información local [51].	Validado principalmente en entornos académicos.
GGNN	Grafos	AST + Aristas semánticas	Modela dependencias de datos mediante grafos [52].	Requiere código compilable; difícil de generalizar.
GraphCodeBERT	Grafos	Tokens + DFG	Integra flujo de datos en el mecanismo de atención [5].	Opacidad interpretativa; alta demanda de cómputo.

Elaboración propia con base en [32, 50, 41, 51, 52, 5].

Una vez agotado el análisis de los modelos orientados a la captura de la semántica algorítmica, es imperativo reconocer que el código fuente no es solo una entidad funcional, sino también un producto de autoría humana. Mientras que el flujo de datos y control define la ejecución del programa, existen rasgos intrínsecos en la escritura que no alteran la lógica, pero que actúan como una huella digital del programador. En consecuencia, el análisis automático de código requiere de una perspectiva complementaria que no se centre en el 'qué' hace el algoritmo, sino en el 'cómo' se manifiesta el estilo individual del autor. Esta disciplina, conocida como estilometría de código, proporciona las herramientas necesarias para abordar la dimensión de identidad, la cual se explora en detalle en la siguiente sección.

3.8 Análisis estilométrico

La estilometría se define como el estudio cuantitativo del estilo de escritura individual [36]. En el contexto del código fuente, analiza cómo los hábitos de un desarrollador como la nomenclatura, el formato o la organización del código se manifiestan como patrones medibles.

Su objetivo principal es la atribución de autoría, es decir, determinar quién escribió un fragmento de código a partir de características estilísticas. Este problema es relevante en escenarios como detección de plagio, análisis forense de software y seguridad informática. A pesar de las restricciones de los lenguajes de programación, los desarrolladores conservan suficiente libertad

para dejar huellas distintivas en su código [36].

3.8.1 Taxonomía de rasgos estilométricos

Para caracterizar estos patrones, Horvath [36] clasifica los rasgos estilométricos en diferentes niveles de abstracción, desde aspectos superficiales hasta decisiones de diseño y comportamiento.

Tabla 3.6: Taxonomía de rasgos estilométricos en código fuente

Tipo de rasgo	**Qué analiza**	**Ejemplos**
Léxico	Elementos superficiales del código (tokens).	Convenciones de nombres (*camelCase* vs *snake_case*), uso de operadores, longitud de identificadores.
Sintáctico	Organización gramatical y reglas del lenguaje.	Uso de estructuras de control (*for* vs *while*), estilo de condiciones booleanas, posición de llaves.
Estructural	Decisiones de diseño y organización de alto nivel.	Funciones largas vs. modulares, profundidad de anidamiento, uso de recursividad o iteración.
Semántico	Intención y lógica detrás del programa.	Elección de estructuras de datos específicas, coherencia en nombres de dominio, algoritmos implementados.
Conductual	Dinámica de desarrollo y comportamiento del autor.	Frecuencia de *commits*, patrones de edición en el tiempo, dinámica de tecleo (*keystroke dynamics*).

Elaboración propia con base en [36].

En conjunto, estos rasgos permiten modelar el estilo de un programador desde múltiples niveles. Mientras que los rasgos superficiales son más fáciles de extraer, los niveles estructurales y semánticos ofrecen mayor capacidad discriminativa, aunque requieren representaciones más complejas. Por otro lado, los rasgos conductuales aportan una dimensión temporal que complementa el análisis estático del código.

3.8.2 Enfoques de estilometría

El análisis estilométrico puede abordarse mediante distintos enfoques metodológicos que varían en su nivel de complejidad, capacidad de generalización y profundidad semántica. De forma general, estos enfoques han evolucionado desde técnicas estadísticas simples hacia modelos avanzados basados en aprendizaje automático y profundo. Cada uno de estos paradigmas ofrece ventajas y limitaciones específicas, por lo que su elección depende del tipo de rasgos estilísticos que se deseen capturar y del nivel de abstracción requerido.

3.8.2.1 Métodos estadísticos

Los métodos estadísticos analizan el estilo mediante el cálculo de frecuencias sobre elementos superficiales del código: conteo de tokens, métricas de software simples y análisis de n-gramas [36]. Su principal ventaja es la interpretabilidad, ya que las decisiones de similitud se basan en recuentos directamente comprensibles para el analista. Sin embargo, al tratar el programa como una secuencia básica de símbolos, ignoran la intención funcional o lógica del mismo, lo que los hace útiles para capturar el vocabulario superficial de un programador pero insuficientes ante modificaciones estructurales.

3.8.2.2 Machine Learning clásico

El Machine Learning clásico emplea algoritmos como las Máquinas de Vectores de Soporte (SVM), los k-vecinos más cercanos (kNN) y árboles de decisión sobre vectores de características extraídas manualmente del código [36]. Representa una mejora respecto a los métodos estadísticos en capacidad de discriminación y generalización, pero mantiene su limitación fundamental: depende completamente de que expertos definan los atributos relevantes previamente, lo que restringe su alcance a similitudes léxicas o estructurales superficiales.

3.8.2.3 Deep Learning

Las arquitecturas de Deep Learning aprenden representaciones directamente desde los datos crudos, sin depender de características diseñadas manualmente [36]. Al mapear el código fuente en embeddings de alta dimensión, codifican conjuntamente propiedades sintácticas y semánticas, permitiendo que fragmentos con patrones de autoría similares converjan en el espacio vectorial. Esta capacidad de generalización los hace especialmente adecuados para escenarios complejos como la identificación de autoría en código ofuscado o la distinción entre código humano y generado por IA.

La Tabla 3.7 sintetiza las diferencias entre los tres paradigmas en las dimensiones más relevantes para el análisis estilométrico de código fuente.

Una vez conocidos los enfoques para abordar el problema, a continuación se expondrán distin-

Tabla 3.7: Comparación de enfoques para el análisis estilométrico

Enfoque	**Capacidad semántica**	**Costo computacional**	**Ventaja principal**	**Limitación principal**
Estadístico	Baja	Bajo	Alta interpretabilidad; no requiere entrenamiento [36]	Incapaz de capturar intención funcional o patrones estructurales complejos
Machine Learning clásico	Media	Medio	Mejor discriminación que métodos estadísticos; generalizable a múltiples autores [36]	Dependencia absoluta de ingeniería manual de características
Deep Learning	Alta	Alto	Aprende representaciones automáticamente; robusto ante variaciones imprevistas en el estilo [36]	Alto costo computacional de entrenamiento; comportamiento de caja negra

Elaboración propia con base en [36].

tos modelos de Inteligencia Artificial aplicados al análisis estilométrico siguiendo los enfoques previamente planteados, analizando sus fortalezas y debilidades.

3.8.3 Modelos para el análisis estilométrico

El análisis estilométrico de código fuente requiere modelos capaces de capturar las huellas personales que un programador deja en sus programas, más allá de la lógica del algoritmo. A diferencia de los modelos orientados a similitud semántica, estos enfoques se centran en patrones como convenciones de nomenclatura, formato e incluso hábitos de escritura.

En este contexto, los modelos estilométricos pueden entenderse como una evolución desde métodos estadísticos simples hasta arquitecturas profundas capaces de aprender representaciones complejas del estilo. A continuación, se presentan tres enfoques representativos que ilustran esta progresión.

3.8.3.1 Modelos basados en n-gramas

Los modelos de n-gramas constituyen un enfoque estadístico basado en frecuencia, donde el código se representa como secuencias contiguas de tokens o caracteres [42]. Su “arquitectura” no es neuronal, sino un modelo de perfilado: construyen vectores de frecuencia que describen el

estilo de un autor.

Esta representación es efectiva para estilometría porque captura patrones repetitivos de bajo nivel, como el uso de operadores, palabras clave o convenciones de escritura. Al comparar estos perfiles mediante métricas como similitud del coseno, es posible identificar similitudes entre autores.

Su principal fortaleza es la simplicidad e interpretabilidad. Sin embargo, al depender únicamente de frecuencias locales, no capturan relaciones de largo alcance ni contexto, lo que limita su capacidad cuando el estilo no es superficial o cuando hay pocos datos disponibles.

3.8.3.2 CharCNN

CharCNN es un modelo basado en Redes Neuronales Convolucionales (CNN) que opera directamente sobre secuencias de caracteres [7]. Su arquitectura utiliza convoluciones unidimensionales para detectar patrones locales en el texto, seguidas de operaciones de pooling que resumen la información relevante.

Esta arquitectura resulta adecuada para estilometría porque permite aprender automáticamente patrones de bajo nivel, como indentación, uso de caracteres especiales, convenciones de nombres o formato, sin necesidad de tokenizar el código ni conocer su sintaxis. En otras palabras, captura el estilo como una señal visual.

No obstante, su enfoque local limita la captura de dependencias globales dentro del código, y su desempeño depende fuertemente de grandes volúmenes de datos para aprender representaciones robustas.

3.8.3.3 RoBERTa

RoBERTa es un modelo basado en la arquitectura Transformer bidireccional, que utiliza mecanismos de autoatención para modelar relaciones entre todos los tokens de una secuencia [50]. Esta arquitectura permite capturar dependencias contextuales globales dentro del código.

En estilometría, esta capacidad resulta especialmente valiosa, ya que permite modelar no solo patrones léxicos, sino también decisiones de codificación que dependen del contexto completo del programa. Su tokenización basada en subpalabras (BPE) permite además manejar identificadores arbitrarios sin perder información relevante.

Gracias a su arquitectura, RoBERTa puede aprender representaciones profundas del estilo de programación, capturando patrones complejos que van más allá de lo superficial. Sin embargo, este nivel de abstracción implica un alto costo computacional y menor interpretabilidad, además de no incorporar explícitamente la estructura del código.

3.8.3.4 Comparación de modelos

Los modelos presentados reflejan distintos niveles de complejidad y capacidad para capturar el estilo de programación. Mientras que los enfoques estadísticos destacan por su simplicidad, los modelos neuronales permiten capturar patrones más complejos a costa de mayor costo computacional. La Tabla 3.8 sintetiza las diferencias entre los tres modelos en las dimensiones más relevantes para el análisis estilométrico.

Tabla 3.8: Comparación de modelos para el análisis estilométrico de código fuente

Modelo	**Arquitectura**	**Rasgos**	**Ventaja principal**	**Limitación principal**
N-gramas	Estadística (Frecuencias)	Léxicos superficiales	Alta interpretabilidad; independiente del lenguaje [42].	Degrada con múltiples autores; requiere muestras largas.
CharCNN	ConvNet 1D (Caracteres)	Léxicos y de formato	Detecta "biomarcadores" de estilo invisibles para modelos de tokens [7].	Requiere grandes volúmenes de datos para entrenamiento.
RoBERTa	Transformer (BPE)	Contexto semántico	Maneja identificadores arbitrarios sin perder contexto [50].	Sin comprensión estructural; alto costo computacional.

Elaboración propia con base en [42, 7, 50].

En conjunto, estos modelos ofrecen distintas formas de capturar el estilo de programación, desde patrones superficiales hasta representaciones profundas basadas en contexto. En la siguiente sección se presenta una síntesis que muestra que mediante la integración de distintos enfoques, se puede conseguir una arquitectura robusta capaz de superar las limitaciones de los modelos previamente presentados.

3.8.4 Métricas de evaluación basadas en la matriz de confusión

De acuerdo con [53], la matriz de confusión es una herramienta de evaluación detallada en la ciencia de datos que permite analizar el rendimiento de un clasificador mediante el desglose de predicciones acertadas y erróneas para cada clase. Esta estructura matricial de tamaño $N \times N$ ofrece una visión clara de cómo el modelo confunde o acierta las etiquetas reales frente a las predichas.

Para el caso de una clasificación binaria, los componentes fundamentales que integran esta ma-

triz se describen en la Tabla 3.9.

Tabla 3.9: Componentes fundamentales de la matriz de confusión binaria.

Componente	**Definición técnica**
Verdadero Positivo (TP)	Instancias donde tanto el valor real como la predicción del modelo son positivos.
Verdadero Negativo (TN)	Instancias donde el valor real es negativo y la predicción es igualmente negativa.
Falso Positivo (FP)	Casos donde la predicción es positiva pero el valor real es negativo (Error de Tipo I).
Falso Negativo (FN)	Casos donde la predicción es negativa pero el valor real es positivo (Error de Tipo II).

Elaboración propia basada en [53].

A partir de estos cuatro estados, se calculan indicadores de desempeño que permiten cuantificar la eficacia del modelo bajo distintas perspectivas (ver Tabla 3.10).

Tabla 3.10: Métricas de rendimiento derivadas de la matriz de confusión.

Métrica	**Fórmula**	**Propósito analítico**
Exactitud	$\frac{TP+TN}{Total}$	Mide la fracción total de predicciones correctas. Es útil en clases balanceadas.
Precisión	$\frac{TP}{TP+FP}$	Evalúa la exactitud de las predicciones positivas; busca minimizar los falsos positivos.
Recall	$\frac{TP}{TP+FN}$	Mide la capacidad del modelo para capturar la totalidad de los casos positivos reales.
Puntuación F_1	$\frac{2 \cdot P \cdot R}{P+R}$	Media armónica que equilibra precisión y exhaustividad, ideal para datos desbalanceados.
Especificidad	$\frac{TN}{TN+FP}$	Cuantifica qué tan bien el modelo identifica los resultados negativos reales.

Elaboración propia basada en [53].

Finalmente, el análisis se complementa con la curva ROC y el valor AUC (*Area Under the Curve*). La curva ROC permite visualizar el equilibrio entre la tasa de verdaderos positivos y falsos positivos a través de distintos umbrales, mientras que el AUC proporciona una medida agregada de la capacidad del modelo para distinguir entre clases, donde un valor de 1.0 representa un clasificador perfecto.

3.9 Integración y fusión de características (*Feature Fusion*)

La fusión de características es una técnica avanzada que permite mejorar el desempeño de modelos de análisis de código mediante la combinación de múltiples fuentes de información en una representación unificada [6]. A diferencia de los enfoques tradicionales, que dependen de una sola representación del código, este paradigma integra características aprendidas automáticamente por modelos profundos con atributos externos diseñados manualmente o derivados de otras fuentes.

Desde el punto de vista arquitectónico, este proceso se implementa mediante la concatenación del vector de salida global del modelo base (*pooled output*) con vectores adicionales de características, seguido de una capa de proyección que unifica ambas representaciones en un espacio latente común [6]. Esto permite que el modelo combine información heterogénea —como patrones sintácticos, métricas estructurales o datos de ejecución— en una única representación enriquecida.

Esta estrategia ofrece tres ventajas principales:

- **Compensación de limitaciones:** Permite complementar los modelos preentrenados con información específica del dominio que no es capturada durante el aprendizaje automático.
- **Complementariedad analítica:** Facilita la integración de distintos tipos de análisis, como el análisis estático del código y características externas que reflejan su comportamiento o contexto.
- **Representaciones más ricas:** Genera embeddings más completos al combinar múltiples niveles de información, mejorando la capacidad del modelo para capturar tanto la forma como la funcionalidad del código.

Diversos estudios han demostrado la efectividad de este enfoque híbrido, reportando mejoras significativas en tareas de análisis de código, con métricas que alcanzan valores cercanos a 0.98 en precisión y 0.99 en medida F [6].

En conjunto, la fusión de características se posiciona como una estrategia clave para superar las limitaciones individuales de los modelos existentes, al permitir la integración de múltiples perspectivas del código en un único sistema de análisis.

3.10 Conclusión del marco teórico

El análisis de la similitud de código fuente constituye un problema complejo que requiere la integración de múltiples perspectivas teóricas y técnicas. A lo largo de este marco teórico se ha evidenciado que no existe un enfoque único capaz de abordar de manera completa todas

las dimensiones del problema, desde la similitud superficial hasta la equivalencia semántica profunda.

En primer lugar, se estableció que la detección de clones de código presenta desafíos inherentes derivados de la diversidad de implementaciones, la ambigüedad estructural y la creciente influencia de herramientas automatizadas como los modelos de lenguaje. Asimismo, la clasificación de clones permitió comprender los distintos niveles de similitud, evidenciando que los enfoques más simples resultan insuficientes para capturar relaciones funcionales complejas.

Posteriormente, se analizaron los principales enfoques para la detección de similitud, observando una evolución progresiva desde métodos textuales y estructurales hasta modelos semánticos y basados en aprendizaje automático. Esta evolución refleja un compromiso constante entre precisión, interpretabilidad y costo computacional, donde cada paradigma aporta ventajas específicas pero también limitaciones significativas.

En este contexto, los modelos modernos basados en aprendizaje profundo, particularmente aquellos sustentados en arquitecturas Transformer y representaciones estructuradas como árboles y grafos, han demostrado una capacidad superior para capturar patrones complejos en el código. Sin embargo, su eficacia depende en gran medida de la representación utilizada y de la información disponible durante el proceso de aprendizaje.

De manera complementaria, el análisis estilométrico introduce una dimensión adicional al problema, al centrarse en la identificación de patrones de autoría más allá de la funcionalidad del código. Este enfoque permite capturar características individuales del programador, aunque enfrenta limitaciones cuando se consideran transformaciones que alteran la superficie del código sin modificar su comportamiento.

Finalmente, la integración de características (*feature fusion*) surge como una estrategia prometedora para superar las limitaciones de los enfoques individuales, al permitir la combinación de múltiples fuentes de información en una representación unificada. Este paradigma refuerza la idea de que las soluciones más efectivas no dependen de un único modelo, sino de la articulación de distintas técnicas de análisis.

En conjunto, el marco teórico desarrollado establece una base sólida para la comprensión del problema y de las herramientas disponibles, evidenciando la necesidad de diseñar soluciones híbridas que equilibren precisión, escalabilidad e interpretabilidad. A partir de este análisis, en las siguientes secciones se aborda el planteamiento del problema, así como la definición de los objetivos y la propuesta de solución, donde se seleccionarán y justificarán las técnicas más adecuadas en función de sus características y limitaciones.

Capítulo 4

Propuesta de solución

La propuesta de solución que se detalla a continuación constituye la respuesta técnica a la problemática en el proceso de evaluación analizada en los capítulos precedentes. A partir del planteamiento del problema y su justificación, se ha identificado la necesidad crítica de contar con herramientas de validación capaces de enfrentar los desafíos actuales en la enseñanza de la programación, especialmente ante el auge de soluciones generadas de forma automatizada.

Considerando las limitaciones detectadas en el estudio del estado del arte, donde los métodos de evaluación tradicionales han perdido efectividad frente a las nuevas técnicas de generación de código, y fundamentándose en los conceptos de análisis semántico y estilométrico explorados en el marco teórico, se propone el desarrollo de Graphito.

Graphito se define como un sistema de comparación de código diseñado para actuar como una herramienta de apoyo a la evaluación docente en cursos introductorios de programación en C/C++. Para lograr este objetivo, la solución se estructura mediante un pipeline de procesamiento híbrido de doble canal, el cual transforma el código fuente en representaciones vectoriales densas o embeddings. Por un lado, el análisis semántico es impulsado por el modelo GraphCodeBERT, que extrae el Grafo de Flujo de Datos (DFG) para capturar la esencia algorítmica del programa. Por otro lado, el análisis estilométrico emplea la red neuronal CharCNN, la cual procesa el código a nivel de caracteres para identificar patrones de escritura y detectar posibles indicios de generación mediante inteligencia artificial.

De esta manera, el sistema se constituye como un comparador integral que no solo realiza un contraste superficial, sino que coteja la estructura funcional y los patrones estilométricos de las entregas frente a soluciones de referencia, proporcionando métricas objetivas que fungen como herramienta de apoyo a la decisión, brindando al docente la evidencia técnica necesaria para optimizar y fundamentar el proceso de evaluación de cada código. En las secciones siguientes se describe la estructura operativa de la solución, iniciando con la representación esquemática de sus componentes funcionales y el flujo de procesamiento de la información, tal como se ilustra en el siguiente diagrama de bloques.

4.1 Estructura Funcional del Sistema

Para realizar la comparación de código fuente, Graphito se articula a través de una arquitectura de bloques funcionales que procesan la información de manera secuencial y paralela. En la Figura 4.1 se presenta el diagrama de bloques que describe el flujo de los datos, desde la ingesta del código fuente hasta la generación del reporte de similitud.

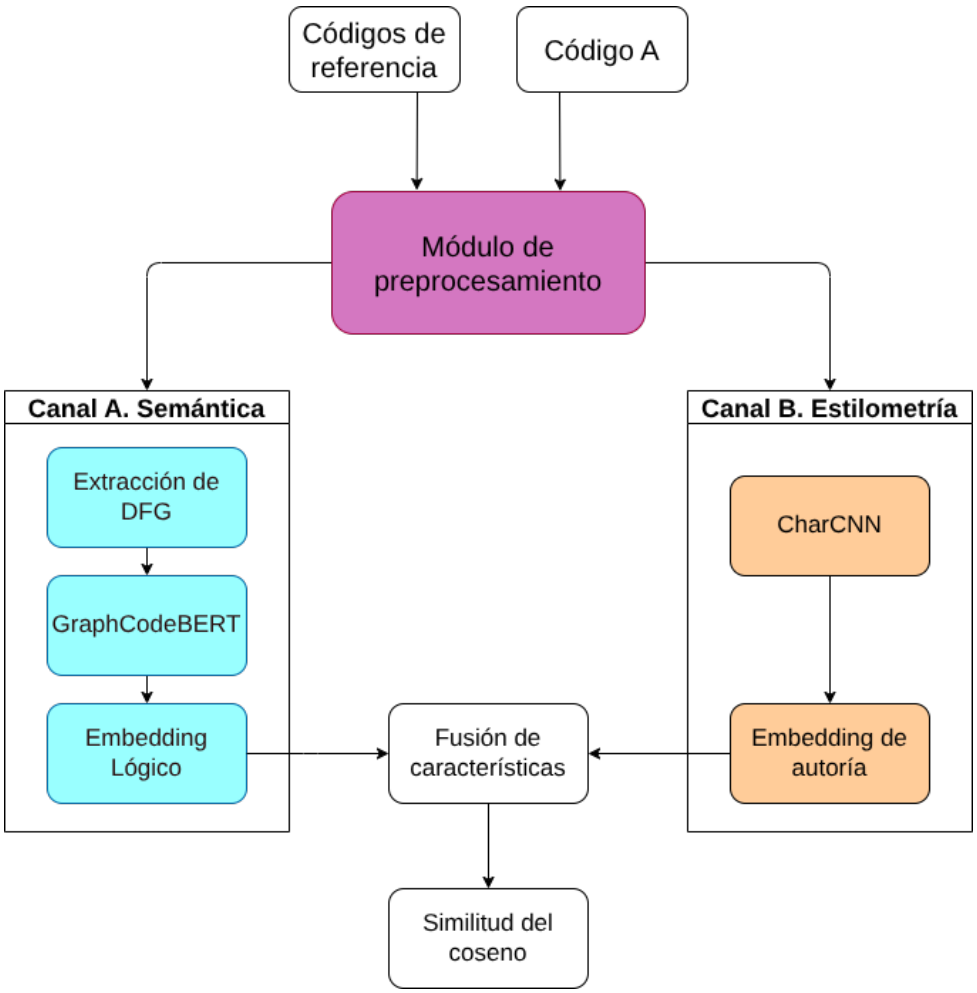


Figura 4.1: Diagrama de bloques funcional del sistema de comparación Graphito.

La Figura 4.1 representa la arquitectura funcional de Graphito, un sistema de apoyo a la evaluación docente que integra el procesamiento del código fuente, el análisis semántico, el análisis estilométrico y la generación de reportes interpretables. Graphito actúa como la capa de orquestación que recibe las solicitudes del docente, coordina la ejecución de los módulos de análisis y consolida sus resultados para su consulta posterior.

La arquitectura contempla dos canales de procesamiento complementarios. El primero utiliza GraphCodeBERT para obtener representaciones vectoriales relacionadas con la semántica del código, mientras que el segundo utiliza una red neuronal convolucional a nivel de caracteres (CharCNN) para identificar patrones estilométricos. Los resultados se integran en una etapa de fusión de características, cuya salida se utiliza para calcular indicadores de similitud y cons-

truir el reporte que se presenta al docente. Las bases de datos relacional y vectorial permiten, respectivamente, conservar los registros y recuperar las representaciones de referencia.

Las entradas principales del sistema son el código de referencia, el código entregado por el estudiante y los metadatos del ejercicio. Las salidas comprenden los indicadores numéricos de similitud, la estimación estilométrica, los datos de la comparación y un reporte de apoyo a la revisión docente. Ninguna de estas salidas constituye, de manera aislada, una determinación automática de plagio o de autoría.

A continuación, se describen los componentes principales que integran la solución:

4.1.1 Entrada de datos y fuentes de referencia

El proceso de comparación comienza con la recepción de dos flujos de información fundamentales:

- **Entrega del Alumno:** Archivos en formato fuente (.c o .cpp) que contienen la resolución propuesta por el estudiante a un problema en específico. El sistema está diseñado para procesar código basado en los estándares C11 y C++17, enfocándose en temas introductorios a la programación como fundamentos y algoritmos y estructuras de datos.
- **Códigos de Referencia:** Actúan como el parámetro de comparación principal, albergando la solución ideal al problema planteado por el docente. De manera opcional, se complementa este conjunto con variantes funcionalmente equivalentes obtenidas mediante modelos de lenguaje (LLM). Esta diversidad de referencias permite que el sistema ejecute un cotejo robusto contra múltiples implementaciones lógicas de un mismo ejercicio.

4.1.2 Módulo de Preprocesamiento y Normalización

Una vez ingresados los archivos, el **Módulo de Preprocesamiento** genera dos representaciones distintas a partir de un mismo archivo fuente para satisfacer los requerimientos de cada canal de análisis:

1. **Representación Normalizada (Canal A):** Se realiza una limpieza profunda de ambos conjuntos de código, eliminando comentarios, metadatos y espacios en blanco redundantes. Asimismo, se aplican técnicas de normalización de identificadores. Este proceso garantiza que el análisis semántico se enfoque exclusivamente en la topología del flujo de datos y la lógica algorítmica, ignorando cambios estéticos o de nomenclatura.
2. **Representación Estructural/Raw (Canal B):** Se aplica únicamente a la solución del alumno. El sistema preserva elementos como la indentación, el uso de comentarios, la

acentuación y caracteres especiales. Esta decisión técnica es clave para la detección de patrones de generación por IA, ya que permite identificar la perfección gramatical y hábitos de formato que constituyen la huella digital del autor o del modelo generativo utilizado.

Una vez terminados ambos procesos, el flujo sigue hacia los canales de comparación semántica y análisis estilométrico.

La normalización del código fuente consiste en aplicar transformaciones controladas para preparar el programa para su procesamiento automático. En Graphito, estas transformaciones se diferencian de acuerdo con el objetivo de cada canal y no implican modificar el significado del programa original ni sustituir su contenido por una solución generada.

Para el canal semántico, el preprocesamiento contempla la validación del archivo, la identificación de tokens y la preparación de la representación necesaria para GraphCodeBERT y la extracción del grafo de flujo de datos. Las transformaciones deberán conservar los elementos necesarios para representar las relaciones lógicas del programa. Por este motivo, cualquier eliminación o sustitución de tokens deberá justificarse y verificarse antes de su aplicación.

En el canal estilométrico, se parte del código original sin renombrar identificadores ni reorganizar su estructura. Las únicas transformaciones permitidas serán aquellas necesarias para codificar la secuencia, controlar su longitud y adaptarla a las dimensiones de entrada de CharCNN. Estas operaciones deberán documentarse y aplicarse con el mismo criterio a las muestras de ambas clases, para evitar que el preprocesamiento elimine sistemáticamente los rasgos que el modelo pretende analizar o introduzca atajos de clasificación (Sección 7.2.4.1).

La implementación conservará una copia inalterada del archivo original y mantendrá separadas las representaciones utilizadas por cada canal. Esta separación permitirá reproducir los análisis, rastrear las transformaciones realizadas y consultar el código original cuando el docente necesite revisar los resultados.

4.1.3 Canal A: Comparación Semántica

Este canal constituye el núcleo para la evaluación de equivalencias lógicas, cuyo objetivo es medir la similitud semántica entre soluciones que resuelven el mismo problema algorítmico, independientemente de sus variaciones sintácticas superficiales.

4.1.3.1 Justificación y Selección del Modelo

La selección de **GraphCodeBERT** [5] como motor de análisis semántico responde a la necesidad de superar las limitaciones de los modelos puramente secuenciales (como CodeBERT o Transformers estándar). La decisión se fundamenta en los siguientes pilares técnicos:

- **Integración Estructural:** A diferencia de otros modelos que tratan el código como una

simple secuencia de texto, GraphCodeBERT integra información estructural mediante el uso de grafos de flujo de datos (DFG). Esto permite que el mecanismo de atención se centre en las conexiones que tienen sentido semántico real.

- **Resistencia a la Ofuscación:** Debido a que el modelo captura la relación entre variables, es altamente efectivo detectando similitudes incluso cuando el código ha sido alterado mediante cambios de nombres de variables, reordenamiento de sentencias o cambios en las estructuras de control.
- **Especialización Funcional:** Como se detalla en el marco teórico, este modelo se especializa en el comportamiento lógico del programa. Si bien tiende a ignorar rasgos superficiales como el estilo, esta característica se alinea perfectamente con la arquitectura de doble canal de Graphito, delegando la responsabilidad del análisis de estilo al Canal B (CharCNN).

4.1.3.2 Extracción de Grafo de Flujo de Datos (DFG)

Un Grafo de Flujo de Datos (DFG) es una representación dirigida que captura las dependencias funcionales entre las variables de un programa mediante la identificación de sus relaciones de definición y uso. De acuerdo con la metodología propuesta por Guo et al. [5], los pasos para obtener esta representación son:

1. **Generación del AST (Análisis sintáctico):** El código fuente se procesa para construir su Árbol de Sintaxis Abstracta (AST), obteniendo la jerarquía gramatical del programa.
2. **Identificación de la secuencia de variables:** Se analizan los nodos terminales (hojas) del AST para extraer todas las variables presentes, preservando su orden de aparición.
3. **Definición de los nodos del grafo:** Cada ocurrencia de una variable se establece como un nodo individual dentro del grafo.
4. **Construcción de las aristas (Dependencias de valor):** Se trazan enlaces dirigidos entre nodos v_i y v_j si el valor de la variable v_j depende directamente de la definición o valor previo de v_i.

4.1.3.3 Procesamiento en GraphCodeBERT

Una vez construido el DFG, la secuencia de *tokens* del código y la estructura del grafo se integran en el modelo. La arquitectura utiliza tareas de entrenamiento como la predicción de relaciones entre nodos, lo que favorece el aprendizaje de representaciones mucho más estructuradas y fieles al comportamiento funcional del software.

4.1.3.4 Embedding Lógico

El resultado final de este canal es un **Embedding Semántico**: un vector denso de alta dimensionalidad que actúa como una firma lógica del programa. Este vector permite realizar comparaciones de proximidad matemática, identificando si dos soluciones son funcionalmente iguales sin importar la forma en que fueron escritas.

4.1.4 Canal B: Comparación Estilométrica

Mientras que el Canal A se encarga de la profundidad lógica, el Canal B se enfoca en la superficie del código. Para esta tarea, se seleccionó el modelo **CharCNN** [7] sobre otras arquitecturas de procesamiento de lenguaje natural más complejas por una razón de eficiencia y diseño: al contar ya con un modelo de alta capacidad para el análisis semántico, no es necesario un segundo modelo que intente comprender la sintaxis global. En su lugar, se requiere un extractor de rasgos que sea capaz de "ver" el código como una señal visual.

4.1.4.1 Justificación y Selección del Modelo

La elección de una Red Neuronal Convolucional a nivel de caracteres (CharCNN) responde a su capacidad innata para actuar como un escáner de patrones locales. Dado que las CNN son especialistas en visión computacional, al aplicarlas al código fuente en bruto (raw), el modelo puede identificar automáticamente rasgos que para otros sistemas serían invisibles:

- **Estilo como Señal Visual:** El modelo interpreta la indentación, el uso de espacios y la disposición de caracteres especiales como texturas visuales, permitiendo aprender el estilo del autor sin necesidad de conocer la gramática del lenguaje C++.
- **Detección de consistencia sintética:** A diferencia de un estudiante, cuya escritura presenta errores humanos y variaciones de formato, los modelos de lenguaje generan código con una estructura y gramática sumamente uniformes en sus comentarios. CharCNN resulta ideal para detectar esta estandarización algorítmica, convirtiéndola en una huella digital que evidencia el uso de inteligencia artificial.
- **Eficiencia Computacional:** Al operar localmente sobre secuencias de caracteres, el modelo aporta una capa de seguridad adicional sin incrementar drásticamente la carga computacional del sistema *backend*.

4.1.4.2 Análisis de Autoría mediante CharCNN

El procedimiento estructurado para transformar el código fuente en un identificador de estilo se detalla a continuación:

1. **Preservación de Código Raw:** Se utiliza la versión del código que conserva comentarios, sangrías y acentuación, elementos que suelen ser eliminados en análisis tradicionales pero que son críticos para la detección de la firma del autor.

2. **Mapeo de Caracteres:** El texto se transforma en una secuencia de índices numéricos basados en un alfabeto predefinido, permitiendo que la red procese el código como una señal discreta y medible.

3. **Extracción Convolucional de Rasgos:** Se aplican filtros convolucionales de distintos tamaños para detectar "n-gramas" de caracteres. Esto permite identificar hábitos microscópicos, como la preferencia por ciertos operadores o la pulcritud en la redacción de comentarios.

4. **Reducción de Dimensionalidad (Pooling):** Se seleccionan las características más relevantes detectadas por los filtros, consolidando los rasgos estilísticos más distintivos de la muestra analizada en una representación compacta.

4.1.4.3 Flujo de procesamiento de la red CharCNN

La red CharCNN procesa el código fuente a nivel de caracteres con el propósito de aprender patrones locales que puedan contribuir a la caracterización estilométrica de una solución. A diferencia del canal semántico, que busca representar la lógica del programa, este canal conserva características de escritura como la distribución de espacios, la indentación, la disposición de símbolos, los comentarios y las convenciones de nomenclatura.

El procesamiento inicia con la recepción del código fuente y la aplicación de las transformaciones necesarias para representarlo mediante una secuencia de caracteres. Cada carácter se asocia con un identificador dentro de un vocabulario definido durante la preparación del conjunto de datos. Posteriormente, la secuencia se ajusta a una longitud máxima establecida para el modelo, mediante truncamiento o relleno, de manera que las entradas puedan organizarse en lotes de tamaño compatible con la arquitectura.

La secuencia codificada se procesa mediante capas convolucionales unidimensionales, cuyos filtros identifican patrones locales de caracteres. Las activaciones resultantes pasan por funciones no lineales y operaciones de agrupamiento, que permiten reducir la longitud de las representaciones y conservar las características relevantes. La información obtenida se transforma en un vector de características y se proporciona a una capa de clasificación que genera una puntuación para las clases contempladas en el entrenamiento.

La salida debe interpretarse como una estimación condicionada por los datos, las etiquetas y el entrenamiento del modelo. Por tanto, una puntuación elevada para la clase asociada a generación artificial no demuestra que el código haya sido producido por una herramienta de inteligencia artificial, del mismo modo que una puntuación baja no demuestra autoría humana.

4.1.4.4 Codificación de entrada y capacidad computacional

La representación de entrada de CharCNN depende del esquema de codificación seleccionado para transformar los caracteres en datos numéricos. En una codificación mediante índices, cada carácter se representa mediante un identificador entero y posteriormente puede transformarse en un vector de características. Si se utiliza una representación *one-hot*, cada carácter se expresa mediante un vector cuya dimensionalidad corresponde al tamaño del vocabulario definido.

Esta decisión afecta las dimensiones del tensor de entrada y el consumo de memoria durante el procesamiento. Sin embargo, el número de filtros convolucionales y sus dimensiones constituyen parámetros independientes de la codificación inicial. Cada filtro aprende patrones locales sobre la secuencia y produce un mapa de activaciones; el conjunto de mapas generados determina la cantidad de canales de salida de la capa convolucional.

En consecuencia, la capacidad computacional requerida no puede determinarse únicamente a partir de la cantidad de caracteres del código o del número de dimensiones del embedding semántico. También depende de la longitud máxima de secuencia, el tamaño del vocabulario, la cantidad de filtros, el tamaño de los *kernels*, el número de capas, el tamaño de lote, la precisión numérica y el volumen de muestras procesadas. Estos parámetros se documentan y evalúan experimentalmente en la Sección 8.1.1 para establecer la viabilidad de ejecución de la arquitectura propuesta.

4.1.4.5 Cálculo de probabilidad de generación por IA

Al finalizar la extracción de patrones locales mediante los filtros convolucionales, la red CharCNN genera una métrica escalar que determina la **probabilidad de generación por inteligencia artificial**.

Esta métrica complementa directamente al Canal A: mientras que el módulo semántico analiza la lógica interna del código para compararla con la solución ideal del profesor, este escalar actúa como un indicador cuantitativo de los patrones estilísticos. Su función es proporcionar al docente evidencia objetiva sobre la homogeneidad sintética del texto y la ausencia de variaciones típicas de la escritura humana, dotándolo de los elementos técnicos necesarios para que tome la decisión de evaluación final bajo su propio criterio.

4.1.5 Fusión de Características

La fusión de características [6] representa el proceso de integración donde las salidas de los canales semántico y estilométrico convergen para formar una representación única de la entrega.

Este bloque técnico es fundamental para el sistema Graphito, ya que permite consolidar dos dimensiones de análisis distintas en un solo espacio vectorial:

- **Mecanismo de Integración:** El sistema toma el **Embedding Semántico** (que encapsula la lógica extraída por GraphCodeBERT) y la probabilidad de generación por IA (que representa los rasgos estructurales detectados por CharCNN) y los combina mediante una operación de concatenación. En la configuración planteada se obtiene una representación de 769 dimensiones (768 semánticas y una estilométrica), cuyas implicaciones se discuten a continuación.
- **Generación de la representación:** El resultado de esta fusión es un vector de características unificado que actúa como una representación del código. Esta no solo describe qué funciones realiza el programa, sino que también preserva los hábitos de escritura y formato del autor del código.

Al integrar los resultados de ambos canales, el sistema genera un perfil completo del código fuente. Esta unión permite cruzar dos datos clave: qué hace el programa (su lógica) y cómo fue escrito (su estilo). Esto facilita la identificación de casos inusuales; por ejemplo, un código que resuelve el problema de forma idéntica a la referencia, pero que tiene el formato uniforme típico de una inteligencia artificial. Así, el sistema entrega al docente información clara y medible para respaldar su decisión final de evaluación.

4.1.5.1 Consideraciones sobre la dimensionalidad y la integración de características

La representación híbrida propuesta para Graphito contempla la integración de dos fuentes de información: el embedding semántico producido por GraphCodeBERT y la puntuación escalar obtenida mediante CharCNN. En la configuración planteada, el embedding semántico posee 768 componentes, mientras que la salida estilométrica se representa mediante un único valor, dando lugar a una representación concatenada de 769 dimensiones.

Las 768 dimensiones del componente semántico constituyen una representación distribuida del código y no corresponden a 768 características interpretables de manera independiente. En conjunto, codifican información aprendida por el modelo sobre las relaciones presentes en la secuencia de código. La dimensión restante incorpora la puntuación estilométrica y permite conservar esta señal dentro de la representación combinada.

Sin embargo, la diferencia entre la dimensionalidad de ambos componentes introduce una consideración metodológica: la concatenación directa no asegura una contribución equilibrada de las señales. En particular, la similitud del coseno sobre el vector concatenado puede estar dominada por la orientación y magnitud del embedding semántico, mientras que la contribución del valor estilométrico depende de su escala y de la normalización aplicada.

Por este motivo, la arquitectura deberá evaluar una estrategia de integración que permita controlar la contribución de ambos canales. Entre las alternativas se encuentran la normalización independiente de las representaciones, la aplicación de ponderaciones definidas experimentalmente o la conservación de las puntuaciones semántica y estilométrica como indicadores separados.

La selección deberá sustentarse en pruebas que comparen la coherencia de los resultados con el juicio experto y el desempeño observado sobre un conjunto de validación.

En consecuencia, las 769 dimensiones constituyen una decisión de representación dentro de la arquitectura propuesta y no una garantía de que ambas señales tengan la misma relevancia. Su conveniencia deberá verificarse durante la experimentación, documentando el efecto de las decisiones de normalización y ponderación sobre los resultados de similitud. En el Capítulo 8 se reporta que una fusión con pesos elegidos manualmente obtuvo un desempeño inferior al del Canal B por separado (Tabla 8.6), por lo que el sistema final mantiene ambos canales como indicadores independientes con umbrales calibrados.

El empleo de relleno con ceros (*zero-padding*) en el componente estilométrico de las referencias deberá evaluarse de manera específica. Si la puntuación de CharCNN representa una probabilidad, asignar cero a todas las referencias no equivale necesariamente a representar su estilo real; además, puede introducir un efecto artificial en el cálculo de similitud. Por ello, se considerará la posibilidad de comparar la similitud semántica y la puntuación estilométrica de manera independiente, o de establecer una representación de referencia compatible con ambos canales cuando existan datos adecuados para hacerlo.

La decisión final se fundamentará en la evaluación de alternativas de integración y en la interpretación de los resultados obtenidos, evitando asumir que la concatenación y el relleno con ceros garantizan por sí mismos una comparación equilibrada.

4.1.5.2 Tratamiento del desbalance entre canales

La integración de los componentes semántico y estilométrico requiere distinguir entre la compatibilidad dimensional de las representaciones y la importancia relativa que cada señal aporta al análisis. Aunque ambos componentes pueden combinarse en una estructura de dimensiones compatibles, esto no implica que su contribución a la similitud resultante sea equivalente.

Para abordar esta situación, se propone evaluar alternativas de normalización y ponderación que permitan controlar la influencia de cada componente. Una posibilidad consiste en normalizar las representaciones semánticas y estandarizar la salida estilométrica con parámetros calculados exclusivamente a partir del conjunto de entrenamiento. Posteriormente, pueden compararse configuraciones en las que las señales se analicen por separado y otras en las que se integren mediante pesos seleccionados con el conjunto de validación.

La evaluación deberá determinar si la integración aporta información adicional respecto del análisis semántico aislado. Para ello, se compararán los resultados de ambas configuraciones frente a las valoraciones de un conjunto de referencia revisado por docentes. Esta estrategia permitirá evitar que la incorporación del canal estilométrico se justifique únicamente por su presencia en la arquitectura, sin comprobar su utilidad efectiva.

En el caso del desbalance entre clases del conjunto de datos, se aplicarán medidas diferentes, tales como la construcción de particiones representativas, la revisión de la distribución de las etiquetas y el uso de métricas como precisión, exhaustividad y F1-score. Si se utilizan técnicas de ponderación de clases o sobremuestreo, estas se aplicarán exclusivamente sobre los datos de entrenamiento para evitar fugas de información.

4.1.6 Similitud del Coseno

La Similitud del Coseno es una métrica matemática que mide el coseno del ángulo entre dos vectores en un espacio multidimensional. A diferencia de otras métricas de distancia, esta no se ve afectada por la magnitud de los vectores (en este caso, la longitud del código), sino por su orientación, lo que la hace ideal para comparar la esencia de dos fragmentos de software.

La similitud entre la entrega del alumno (A) y el código de referencia (R) se calcula mediante la siguiente fórmula:

$$\text{sim}(A, R) = \frac{A \cdot R}{\|A\|\|R\|} = \frac{\sum_{i=1}^{n} A_i R_i}{\sqrt{\sum_{i=1}^{n} A_i^2}\sqrt{\sum_{i=1}^{n} R_i^2}} \tag{4.1}$$

Donde:

- A y R representan los vectores de características (*embeddings* bimodales) de la entrega del alumno y la solución de referencia, respectivamente.
- A_i y R_i son los valores escalares de dichos vectores en la dimensión i.
- $\|A\|$ y $\|R\|$ denotan la norma euclidiana (o magnitud) de cada vector.
- n es la dimensionalidad total del espacio vectorial unificado.

El resultado de esta operación oscila en el intervalo $[0, 1]$, donde 1 representa una coincidencia total de orientación y 0 una independencia absoluta entre los vectores.

4.1.6.1 Importancia de la métrica en la fusión de características

El uso de la similitud del coseno es fundamental en Graphito debido a que permite comparar dos representaciones en un mismo espacio vectorial: las características lógicas multidimensionales (Canal A) y el escalar de probabilidad estilométrica (Canal B). Al combinarlos, esta métrica matemática proporciona un indicador objetivo que optimiza el proceso de revisión docente.

La pertinencia de utilizar este enfoque radica en su comportamiento geométrico frente a los vectores evaluados:

- **Impacto del indicador estilométrico:** En el sistema, el código de referencia proporcionado por el profesor se establece con una probabilidad nula de haber sido generado por IA (valor de 0 en la dimensión estilométrica). Si una entrega estudiantil posee una alta probabilidad de origen sintético, adquiere un valor escalar mayor en dicha dimensión, lo que altera matemáticamente su vector unificado.
- **Variación en la orientación espacial:** Aunque el código del alumno y el de referencia compartan una alta similitud semántica (lógica algorítmica), el valor adicional aportado por el modelo CharCNN modifica la orientación espacial del vector del estudiante. Este incremento en el ángulo entre ambos vectores se traduce directamente en una reducción del índice de similitud.
- **Ajuste cuantitativo de la similitud:** De acuerdo con la fórmula matemática, la magnitud del vector de la entrega ($\|A\|$) se encuentra en el denominador. Dado que la probabilidad sintética incrementa esta magnitud sin aportar al producto escalar del numerador (ya que se multiplica por el 0 de la referencia), el resultado global disminuye naturalmente. Esto permite que el sistema integre la detección de IA como una reducción proporcional en la similitud sin necesidad de programar penalizaciones arbitrarias.

En conclusión, la similitud del coseno permite a Graphito entregar un indicador numérico robusto. En lugar de limitarse a una verificación funcional o textual, el sistema evalúa simultáneamente la topología algorítmica y la huella estilométrica, brindando al docente evidencia cuantitativa clara para agilizar y fundamentar su proceso de evaluación.

4.1.7 Componentes funcionales, entradas y salidas

La Tabla 4.1 resume los componentes funcionales de Graphito, los datos que recibe cada uno, su función dentro del sistema y los datos que produce.

Tabla 4.1: Componentes funcionales de Graphito, entradas y salidas.

Componente o herramienta	**Entrada**	**Función dentro de Graphito**	**Salida**
Interfaz web	Archivos de código y parámetros del ejercicio	Permite al docente configurar y ejecutar comparaciones.	Solicitud de análisis y archivos cargados.
Backend de Graphito	Solicitud del docente, código y metadatos	Valida los archivos y coordina el flujo de procesamiento.	Datos validados y resultados consolidados.

Componente o herramienta	**Entrada**	**Función dentro de Graphito**	**Salida**
Preprocesamiento	Código fuente original	Verifica el formato y genera representaciones adecuadas para cada canal.	Código preparado para análisis semántico y estilométrico.
GraphCodeBERT	Código tokenizado y representación de flujo de datos cuando corresponda	Extrae representaciones semánticas del programa.	Embedding semántico.
CharCNN	Secuencia de caracteres codificada	Aprende patrones locales de escritura y produce una estimación estilométrica.	Probabilidad o puntuación de la clase definida por el modelo.
Módulo de fusión	Embedding semántico y salida estilométrica	Integra las señales conforme a una estrategia definida.	Representación combinada, si se conserva la arquitectura híbrida propuesta.
ChromaDB	Embeddings de referencia y consulta compatible	Recupera representaciones cercanas mediante búsqueda vectorial.	Referencias similares y puntuaciones de distancia o similitud.
PostgreSQL	Usuarios, ejercicios, archivos y metadatos de análisis	Mantiene los registros estructurados y el historial.	Registros persistidos y datos para los reportes.
Generador de reportes	Resultados de los dos canales y metadatos	Organiza los indicadores para facilitar su interpretación.	Reporte para el docente.
LLM externo, opcional	Instrucciones y código de referencia	Genera variantes candidatas para ampliar las referencias de comparación.	Variantes que deben validarse antes de incorporarse.

4.2 Conclusión

La propuesta de solución presentada en este capítulo, denominada **Graphito**, se aleja de los enfoques de comparación de texto tradicionales para adoptar una arquitectura de doble canal basada en aprendizaje profundo. A través de la integración del análisis semántico (Canal A) y el análisis estilométrico (Canal B), el sistema es capaz de evaluar el código fuente no solo por lo que hace, sino por el estilo de su escritura.

La principal fortaleza de esta propuesta reside en su capacidad de autorregulación mediante la similitud del coseno. Al tratar la lógica como el eje central y el estilo como un factor de penalización, Graphito ofrece una métrica de integridad académica robusta frente a las técnicas modernas de ofuscación y la generación sintética por modelos de lenguaje.

Con la definición funcional de estos bloques y la fundamentación de sus componentes principales, queda establecida la base teórica y metodológica de la solución. En el siguiente capítulo, se procederá con el Análisis, donde se detallará la arquitectura de software, la persistencia de datos y los diagramas de interacción necesarios para la implementación física de esta propuesta.

Capítulo 5

Análisis del Sistema

5.1 Análisis de Requerimientos

En esta sección se describen los requerimientos funcionales y no funcionales del sistema propuesto, los cuales establecen las capacidades, restricciones y criterios de desempeño que debe cumplir la solución desarrollada. Estos requerimientos sirven como base para el diseño, implementación y evaluación del sistema, asegurando la trazabilidad entre las necesidades del evaluador y la arquitectura final.

5.1.1 Requerimientos Funcionales

Los requerimientos funcionales definen las capacidades y procesos que el sistema debe implementar para cumplir con las funciones de análisis, gestión de información y auditoría de código en el contexto académico.

- **RF-01:** El sistema debe permitir el registro e inicio de sesión de docentes mediante credenciales cifradas para garantizar un espacio de trabajo privado.
- **RF-02:** El sistema debe permitir al docente la creación y gestión de un listado de ejercicios o problemas de programación (Biblioteca).
- **RF-03:** El sistema debe permitir la carga de código fuente en formato .c y .cpp asociados a un ejercicio específico.
- **RF-04:** El sistema debe realizar el preprocesamiento y normalización del código fuente, asegurando la correcta extracción de los grafos para el canal semántico, mientras se preserva la integridad de las estructuras de estilo originales para el detección de patrones generativos.
- **RF-05:** El sistema debe generar representaciones basadas en Grafos de Flujo de Datos (DFG) para el módulo semántico, y transformar el código fuente en secuencias matriciales

bidimensionales (carácter por vector) para su procesamiento mediante convoluciones 1D en el módulo estilométrico.

- **RF-06:** El sistema debe realizar el análisis de similitud semántica mediante el modelo GraphCodeBERT para la generación de vectores de características (embeddings), cuantificando la proximidad entre ellos utilizando la métrica de similitud del coseno.
- **RF-07:** El sistema debe realizar el análisis estilométrico mediante el modelo CharCNN para calcular la probabilidad de generación mediante inteligencia artificial sintética o humana.
- **RF-08:** El sistema debe solicitar y recibir múltiples versiones funcionalmente equivalentes del problema proporcionado por el docente a través de un modelo LLM externo.
- **RF-09:** El sistema debe realizar el almacenamiento persistente de los resultados del análisis y metadatos en una base de datos relacional (PostgreSQL), así como de los vectores de características en una base de datos vectorial (ChromaDB), asegurando la integridad referencial.
- **RF-10:** El sistema debe generar un reporte consolidado que integre el índice de similitud semántica y la probabilidad de generación mediante inteligencia artificial de forma visual.
- **RF-11:** El sistema debe realizar una fusión de características (Feature Fusion) que integre el vector semántico y la probabilidad estilométrica, consolidando las métricas de ambos módulos en una sola estructura para su análisis dual.
- **RF-12:** El sistema debe permitir la consulta histórica de reportes de análisis realizados previamente sin necesidad de re-procesar los archivos fuente.

5.1.2 Requerimientos No Funcionales

Los requerimientos no funcionales establecen las restricciones, atributos de calidad y condiciones de desempeño que debe cumplir el sistema propuesto durante su operación.

- **RNF-01:** Seguridad de datos: El sistema debe cifrar las contraseñas de los usuarios y utilizar tokens de sesión (JWT) para proteger la comunicación entre el cliente y el servidor.
- **RNF-02:** Persistencia híbrida: El sistema debe operar bajo un esquema de base de datos dual, optimizando las consultas relacionales en PostgreSQL y las búsquedas por similitud vectorial en ChromaDB.
- **RNF-03:** Los modelos de IA integrados deben estar calibrados buscando alcanzar un objetivo de rendimiento base de F1-Score ≥ 0.85 para GraphCodeBERT en tareas de detección de similitud semántica.

- **RNF-04:** El módulo estilométrico (CharCNN) debe estar optimizado para buscar una precisión de clasificación de autoría igual o superior a 0.8 bajo el corpus de pruebas establecido.
- **RNF-05:** El sistema debe soportar el análisis de código bajo los estándares C11 y C++17 para garantizar compatibilidad operativa con el currículo y las entregas esperadas.
- **RNF-06:** El orquestador de métricas debe ajustar los umbrales de decisión con el objetivo de mantener una tasa de falsos positivos menor al 10 % durante el análisis de doble canal.
- **RNF-07:** El tiempo de procesamiento de inferencia en el backend durante la fase de análisis de una entrega estudiantil no debe exceder los 20 segundos, dado que la indexación de referencias y la generación de variantes (LLM) se realizan de forma asíncrona en la fase de configuración previa.
- **RNF-08:** El sistema debe ser compatible para su ejecución en navegadores web basados en el motor Chromium versión 110 o superior.
- **RNF-09:** La interfaz web debe ser intuitiva, permitiendo ejecutar todas las tareas en 3 o menos acciones y proporcionar retroalimentación visual o indicadores de estado en tiempo real mientras el orquestador (LangGraph) procesa las tareas en segundo plano.
- **RNF-10:** El sistema debe asegurar alta disponibilidad operativa durante el procesamiento y análisis de código dentro del entorno de despliegue asignado para las pruebas.
- **RNF-11:** El sistema debe realizar validaciones preventivas en el cliente (tipo de archivo, tamaño e integridad) antes de iniciar la transmisión de la carga útil hacia la API REST.

5.1.3 Reglas de Negocio

Las reglas de negocio definen los criterios que guían el funcionamiento y uso ético del sistema dentro del entorno académico.

- **RN-01:** Confidencialidad de los datos: Un docente solo podrá visualizar, editar y consultar los ejercicios y reportes generados bajo su propia cuenta de usuario.
- **RN-02:** El sistema se limita a programas de cursos de programación de ESCOM-IPN que sigan los estándares C11 y C++17.
- **RN-03:** Para que el sistema pueda hacer una comparación, debe existir al menos un código de referencia previamente cargado. Sin ella, el sistema no tiene un punto de comparación y no puede generar ningún resultado.
- **RN-04:** La decisión final de calificación es responsabilidad exclusiva del docente; el sistema actúa únicamente como una herramienta objetiva de apoyo a la auditoría académica.

- **RN-05:** Se permite el uso de múltiples soluciones generadas por LLM para ampliar el espectro de comparación lógica, siempre que el docente las valide inicialmente.
- **RN-06:** Preservación de referencias: Los archivos cargados en calidad de entregas a analizar no podrán ser catalogados ni reutilizados por el sistema como códigos de referencia para futuras comparaciones, garantizando que el estándar de comparación se mantenga basado exclusivamente en soluciones validadas o generadas por el docente.
- **RN-07:** Límite de generación de variantes: Graphito deberá establecer un límite máximo configurable de variantes por solicitud. Para la versión inicial del prototipo se propone un máximo de cinco variantes por ejercicio y solicitud. El sistema deberá validar el parámetro antes de invocar al modelo externo y rechazar solicitudes que excedan el límite establecido. La modificación de este parámetro dependerá de la disponibilidad de recursos, los tiempos de respuesta y los resultados de las pruebas de validación.

El límite de la regla RN-07 se establece para controlar el costo de las consultas externas, el tiempo de procesamiento y el crecimiento del conjunto de referencias. No representa una garantía de cobertura de todas las soluciones posibles, ni implica que cada variante generada sea funcionalmente equivalente. Las variantes deberán conservar su estado de validación y no podrán considerarse referencias confirmadas únicamente por haber sido producidas por un modelo generativo.

5.2 Identificación y descripción de actores

En el contexto del sistema propuesto (Graphito), los actores representan las entidades externas que interactúan con la plataforma y que participan en el flujo de configuración, análisis y auditoría del código fuente. La correcta identificación de estos actores permite definir los casos de uso, delimitar responsabilidades y establecer las fronteras del sistema.

A partir de la arquitectura funcional, se identifican exclusivamente dos actores que interactúan de forma directa con la plataforma, descritos más adelante en esta sección.

Desde la perspectiva de uso, Graphito está dirigido principalmente a docentes e instructores responsables de cursos introductorios de programación en C/C++. Este grupo constituye el usuario principal, debido a que administra los ejercicios, establece las soluciones de referencia, carga las entregas y consulta los reportes generados por la plataforma.

Como usuarios beneficiarios indirectos se identifican los estudiantes, quienes pueden verse favorecidos por procesos de revisión más consistentes y por la posibilidad de recibir retroalimentación fundamentada en características observables de sus soluciones. No obstante, el prototipo se orienta principalmente al trabajo del docente y no contempla, como función central, la calificación autónoma de estudiantes ni la emisión de sanciones académicas.

Los modelos de inteligencia artificial y los sistemas de almacenamiento constituyen componentes técnicos de la solución. Aunque intervienen en el flujo de procesamiento, no sustituyen al usuario responsable de interpretar los resultados y determinar las acciones posteriores. El motor de IA externo (LLM) se considera un servicio externo con el que el sistema interactúa, no un usuario de la plataforma en el mismo sentido que el docente.

5.2.1 Docente / Evaluador

Es el actor principal y humano del sistema. Utiliza la plataforma como una herramienta de apoyo a la decisión en su proceso de auditoría académica (correspondiente a profesores o instructores de programación).

Entre sus principales responsabilidades de interacción se encuentran:

- Proporcionar la descripción técnica del problema o práctica.
- Configurar el marco de comparación cargando sus códigos de referencia.
- Cargar los archivos fuente (*.c, .cpp*) desarrollados por los estudiantes para su análisis.
- Interpretar los reportes visuales generados por el sistema, incluyendo los índices de similitud semántica y la probabilidad de generación mediante inteligencia artificial.
- Tomar la decisión final sobre la calificación del alumno, utilizando la evidencia estática proporcionada por la herramienta.

5.2.2 Motor de Inteligencia Artificial Externo (LLM)

Este actor corresponde a una API de un sistema externo basado en Modelos de Lenguaje Grande (LLM). Actúa como un proveedor de servicios de generación de código sintético bajo demanda.

Sus funciones en interacción con el orquestador de Graphito son:

- Recibir el código de referencia original y generar descripciones técnicas algorítmicas.
- Generar múltiples variantes de código que sean funcionalmente equivalentes a la referencia del docente.
- Proveer el insumo necesario para que el sistema indexe un espacio vectorial de comparación mucho más robusto.

5.3 Trayectorias principales y alternativas

Las trayectorias describen el flujo de interacción entre el docente, el orquestador backend y los modelos de IA, detallando las secuencias de acciones para llevar a cabo el análisis bimodal. Estas trayectorias reflejan el comportamiento esperado (Modelado BPMN) y los escenarios alternativos.

5.3.1 Trayectoria principal (Flujo de Análisis Estándar)

La Figura 5.1 ilustra la trayectoria principal del sistema Graphito, correspondiente al flujo de análisis estándar. Durante este proceso, el docente ingresa el código de referencia y las entregas estudiantiles, configura los parámetros de evaluación e inicia el procesamiento de doble canal. Posteriormente, el sistema ejecuta de manera concurrente el análisis semántico y el análisis estilométrico del código fuente, consolidando métricas e indicadores objetivos que respaldan la toma de decisiones del evaluador.

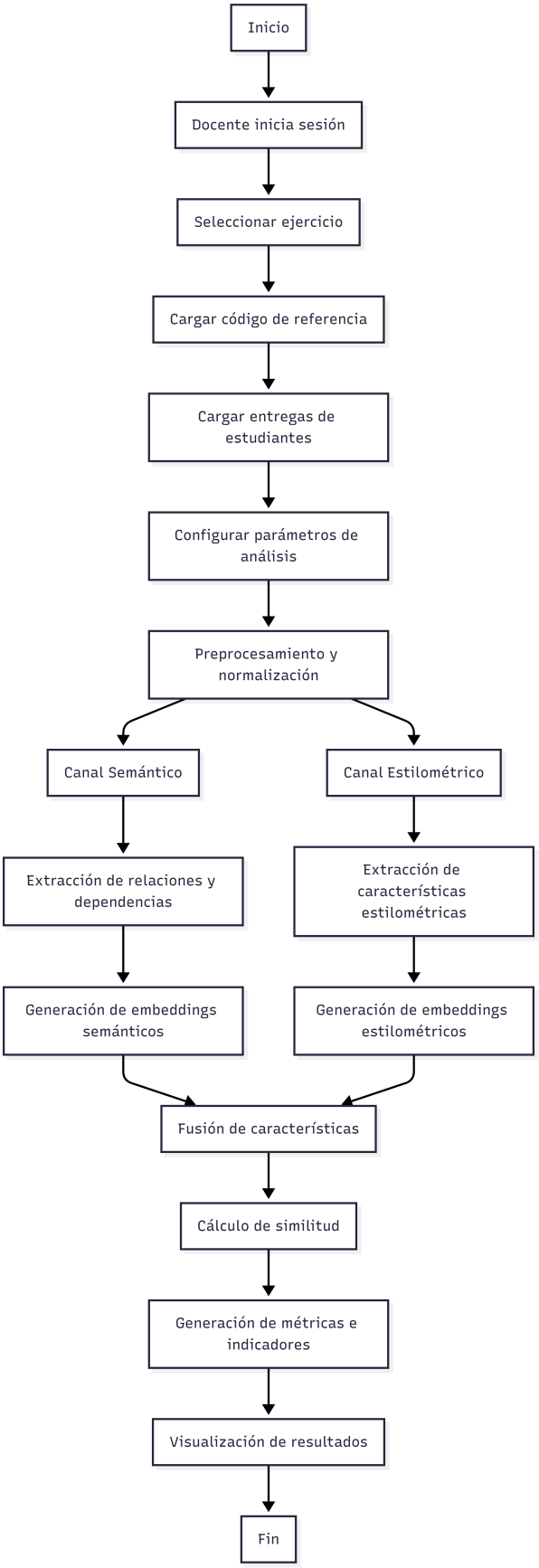


Figura 5.1: Diagrama de trayectoria principal del sistema Graphito correspondiente al flujo de análisis estándar.

1. El docente accede a su biblioteca en la plataforma web del sistema.

2. El docente carga el archivo fuente a analizar, seleccionando el problema correspondiente.
3. El backend recibe el código y ejecuta el preprocesamiento y normalización (limpieza para el canal semántico y preservación de formato para el estilométrico).
4. **Canal Semántico:** Se extrae el Grafo de Flujo de Datos (DFG) y el modelo *GraphCodeBERT* genera el embedding estructural.
5. **Canal Estilométrico:** El modelo *CharCNN* procesa la matriz de caracteres 1D para estimar la probabilidad de generación mediante inteligencia artificial.
6. El orquestador ejecuta la **Fusión de Características (Feature Fusion)** unificando ambos resultados.
7. El sistema calcula la similitud del coseno del vector unificado contra la base de datos vectorial (*ChromaDB*).
8. El sistema consolida los resultados y persiste los metadatos en la base de datos relacional (*PostgreSQL*).
9. El sistema renderiza el reporte de análisis dual.
10. El docente visualiza los indicadores numéricos y toma una decisión informada.

5.3.2 Trayectorias alternativas

Las trayectorias alternativas describen variaciones del flujo principal, orquestadas por *LangGraph* ante condiciones específicas.

A1. Ampliación controlada del conjunto de referencias mediante un LLM

1. Durante la configuración de un ejercicio, el docente podrá solicitar la generación de variantes de una solución de referencia.
2. Graphito enviará al modelo de lenguaje externo la descripción del problema y el código de referencia, junto con las restricciones necesarias para mantener el alcance funcional del ejercicio.
3. El modelo externo generará un número limitado de soluciones candidatas que podrán utilizar estructuras de control, descomposiciones o convenciones de escritura diferentes.
4. Las variantes recibidas serán sometidas a validaciones de formato y, cuando el entorno de ejecución lo permita, a pruebas funcionales diseñadas para el ejercicio. Estas comprobaciones permitirán descartar candidatos que incumplan las condiciones definidas.

5. Las soluciones que superen las validaciones se procesarán mediante el canal semántico y podrán incorporarse al conjunto de referencias de Graphito. Cuando una variante no pueda verificarse suficientemente, deberá identificarse como pendiente de validación y no utilizarse como referencia confirmada.

6. El sistema conservará la relación entre cada variante y el ejercicio de origen, junto con los metadatos necesarios para su trazabilidad.

La generación de variantes tiene como finalidad ampliar la diversidad de las referencias de comparación, no garantizar que se cubran todas las soluciones posibles del problema. La cantidad de variantes y las condiciones para aceptarlas deberán establecerse mediante parámetros configurables y restricciones de recursos computacionales (RN-07).

A2. Código con errores críticos de formato

1. El sistema detecta inconsistencias irreparables durante la normalización inicial del código.
2. El orquestador detiene el flujo antes de la inferencia de los modelos.
3. Se notifica al docente en la interfaz sobre la invalidez del archivo fuente.

A3. Análisis estilométrico de baja confianza

1. El sistema identifica que el código cargado carece de una extensión mínima para extraer patrones de caracteres viables.
2. El orquestador reduce la ponderación del canal estilométrico en la fusión de características.
3. El sistema prioriza el índice de similitud semántica y añade una nota técnica en el reporte final.

A4. Alta equivalencia lógica detectada

1. El cálculo de la similitud del coseno arroja un valor superior al umbral establecido.
2. El sistema clasifica la entrega como de alto interés de auditoría.
3. Se destaca visualmente el registro en el panel del docente para sugerir una revisión prioritaria.

A5. Discrepancia crítica en análisis de doble canal

1. El orquestador detecta una correlación anómala (ej. alta similitud lógica combinada con una huella de autoría clasificada como inteligencia artificial).
2. El sistema estructura un indicador visual de inconsistencia en el reporte final.
3. El docente analiza el reporte detallado para auditar la anomalía.

5.4 Diagrama de Casos de Uso

El análisis de casos de uso define las fronteras del sistema Graphito y describe las interacciones entre los actores externos y las funcionalidades internas. Este modelo permite visualizar cómo el evaluador gestiona el proceso de auditoría académica, desde la configuración inicial hasta la toma de decisiones basada en el reporte bimodal.

En la Figura 5.2 se presenta el diagrama de casos de uso general del sistema, estructurado de forma jerárquica para reflejar el flujo de trabajo del docente.

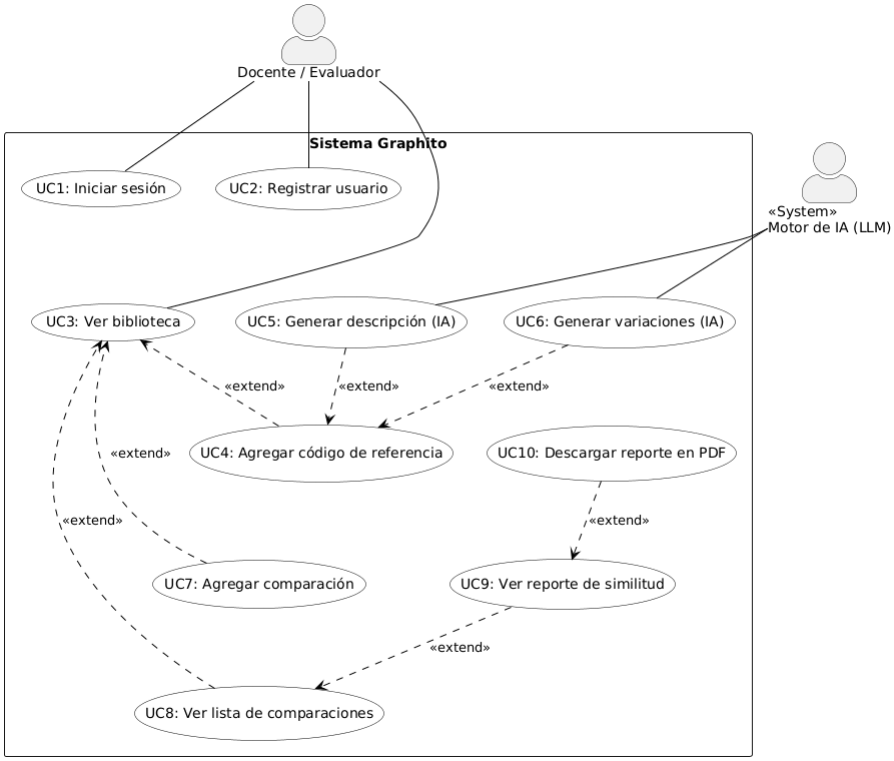


Figura 5.2: Diagrama de casos de uso del sistema Graphito.

5.4.1 Descripción de Actores

Para el funcionamiento del sistema se han identificado dos actores principales:

- **Docente / Evaluador:** Actor principal humano que interactúa con la plataforma para configurar problemas, gestionar entregas y visualizar los reportes de similitud.

- **Modelo LLM Externo:** Actor de sistema que actúa como proveedor de servicios de generación de código. Es consultado de forma opcional para robustecer el conjunto de referencias.

5.4.2 Descripción de los Casos de Uso

A continuación, se describen los módulos funcionales que integran la solución *Graphito*, estructurados de acuerdo con su rol en el flujo de auditoría académica:

- **Gestión de Acceso y Biblioteca (UC1, UC2, UC3):** Facilitan la administración de la sesión del docente y el acceso a la pagina principal (Biblioteca). Este módulo centraliza los ejercicios históricos y las comparaciones realizadas, permitiendo un entorno de trabajo persistente.
- **Gestión de Referencias y Pre-procesamiento (UC4, UC5, UC6, UC11):** Permiten al evaluador establecer el marco de comparación. Incluye la carga de soluciones base (UC4) y su enriquecimiento mediante un motor de IA externo para generar descripciones y variantes sintéticas (UC5, UC6). De forma obligatoria (*include*), el sistema realiza la generación de *embeddings* semánticos (UC11) sobre estas referencias.
- **Ejecución de Comparaciones (UC7, UC12):** Constituye el núcleo operativo del sistema. Al agregar una entrega para comparar (UC7), el sistema ejecuta automáticamente (*include*) el análisis de doble canal (UC12), el cual integra el procesamiento semántico de flujo de datos y la detección estilométrica de patrones generativos.
- **Resultados y Reporteo (UC8, UC9, UC10):** Proporcionan la interfaz de auditoría donde se listan las comparaciones (UC8) y se visualizan los reportes de similitud detallados (UC9). Finalmente, se permite la exportación de resultados en formato PDF (UC10) para su integración en actas de evaluación.

5.4.3 Especificación de Casos de Uso

A continuación se detallan los casos de uso que rigen el comportamiento del sistema Graphito. Estas especificaciones describen la interacción detallada entre los actores y los módulos de inteligencia artificial.

Caso de Uso:	UC1: Iniciar sesión
Actor:	Docente / Evaluador
Descripción:	El docente ingresa sus credenciales de acceso para entrar al sistema Graphito.
Precondiciones:	El docente debe estar registrado previamente en la plataforma (UC2).
Flujo Básico:	1. El docente ingresa su correo electrónico y contraseña en la interfaz. 2. El sistema valida las credenciales mediante un proceso de autenticación cifrada contra la base de datos relacional (PostgreSQL). 3. El sistema genera un token de sesión (JWT) y redirige al docente a su Biblioteca (UC3).
Trayectorias Alternativas:	Si las credenciales son incorrectas, el sistema muestra un mensaje de error y deniega el acceso, manteniendo la sesión cerrada.

Tabla 5.1: Especificación del Caso de Uso 1.

Caso de Uso:	UC2: Registrar usuario
Actor:	Docente / Evaluador
Descripción:	Un nuevo docente crea una cuenta en la plataforma para poder gestionar sus ejercicios, referencias y comparaciones.
Flujo Básico:	1. El docente proporciona su nombre, correo electrónico y define una contraseña. 2. El sistema cifra la contraseña y registra al nuevo usuario en PostgreSQL. 3. El sistema confirma el registro exitoso y redirige a la vista de inicio de sesión.
Postcondición:	Se inicializa un espacio aislado en la base de datos para el repositorio personal del docente.

Tabla 5.2: Especificación del Caso de Uso 2.

Caso de Uso:	UC3: Ver biblioteca
Actor:	Docente / Evaluador
Descripción:	Página principal del sistema donde el docente accede a su biblioteca personal, desde donde visualiza los problemas configurados previamente o inicia nuevas configuraciones.
Precondiciones:	El docente debe haber iniciado sesión de forma exitosa (UC1).
Flujo Básico:	1. El docente selecciona la opción de "Biblioteca" en la interfaz. 2. El sistema recupera de PostgreSQL la lista de ejercicios asociados al identificador del docente. 3. El sistema muestra la lista de ejercicios, habilitando las opciones de gestión (agregar referencia, ver comparaciones).
Postcondición:	El sistema queda a la espera de la ejecución de una acción extendida (UC4, UC7, UC8).

Tabla 5.3: Especificación del Caso de Uso 3.

Caso de Uso:	UC4: Agregar código de referencia
Actor:	Docente / Evaluador
Descripción:	El docente carga el código fuente que servirá como solución ideal, inicializando el proceso asíncrono de indexación semántica.
Precondiciones:	El docente debe encontrarse gestionando un ejercicio dentro de la Biblioteca (UC3).
Flujo Básico:	1. El docente selecciona agregar una nueva referencia y carga el archivo (.c o .cpp). 2. El sistema normaliza el código eliminando comentarios. 3. El orquestador extrae el Grafo de Flujo de Datos (DFG) y genera el embedding correspondiente mediante GraphCodeBERT. 4. El vector semántico resultante se almacena en ChromaDB. 5. El sistema habilita las opciones para enriquecer la referencia utilizando un modelo de IA externo (UC5, UC6).
Postcondición:	El código base queda indexado en la base vectorial, preparado para futuras comparaciones.

Tabla 5.4: Especificación del Caso de Uso 4.

Caso de Uso:	UC5: Generar descripción (IA)
Actor:	Motor de IA (LLM) / Docente
Descripción:	El sistema consume una API de LLM externo para redactar automáticamente un resumen o descripción técnica basada en la lógica del código de referencia.
Precondiciones:	Debe existir un código de referencia cargado en el sistema (UC4).
Flujo Básico:	1. El docente solicita la autogeneración de la descripción del problema. 2. El sistema envía el código crudo al LLM empaquetado en un prompt específico de resumen algorítmico. 3. El LLM devuelve la descripción técnica generada. 4. El docente revisa el texto, realiza ajustes si lo considera necesario y confirma su guardado en los metadatos de PostgreSQL.

Tabla 5.5: Especificación del Caso de Uso 5.

Caso de Uso:	UC6: Generar variaciones (IA)
Actor:	Motor de IA (LLM) / Docente
Descripción:	El sistema utiliza el LLM externo para generar variantes funcionalmente equivalentes al código base.
Precondiciones:	Se debe haber cargado exitosamente al menos un código de referencia manual (UC4).
Flujo Básico:	1. El docente solicita la generación de N variantes para robustecer las referencias, otorgando consideraciones para la generación de las mismas. 2. El sistema envía el código de referencia y las consideraciones al LLM externo. 3. El LLM retorna N versiones sintéticas del código con la lógica requerida. 4. El sistema preprocesa estas variantes, genera sus respectivos embeddings semánticos a través de GraphCodeBERT y las indexa en ChromaDB.
Trayectorias Alternativas:	Si la API del LLM no responde o excede el tiempo límite, el sistema notifica al docente y finaliza el proceso manteniendo activa únicamente la referencia original.

Tabla 5.6: Especificación del Caso de Uso 6.

Caso de Uso:	UC7: Agregar comparación
Actor:	Docente / Evaluador
Descripción:	El docente carga una entrega estudiantil para ser comparada con el repositorio de referencias indexadas, ejecutando el análisis de los modelos de Deep Learning.
Precondiciones:	El problema debe contar con al menos una referencia previamente indexada en ChromaDB.
Flujo Básico:	1. El docente carga el archivo (.c o .cpp) correspondiente a la entrega a evaluar. 2. El sistema inicia la inferencia paralela: GraphCodeBERT calcula el vector lógico (semántico) y CharCNN calcula la probabilidad de generación mediante inteligencia artificial sintética (estilométrico). 3. El orquestador ejecuta la Fusión de Características (Feature Fusion) para consolidar las métricas de doble canal. 4. Se ejecuta una búsqueda matemática de Similitud del Coseno contra los vectores indexados en ChromaDB. 5. Los resultados se persisten en PostgreSQL vinculados exclusivamente al nombre del archivo analizado.
Postcondición:	La entrega finaliza la comparación y queda disponible para su consulta detallada (UC8).

Tabla 5.7: Especificación del Caso de Uso 7.

Caso de Uso:	UC8: Ver lista de comparaciones
Actor:	Docente / Evaluador
Descripción:	El docente visualiza un panel con el listado general de todas las entregas que han completado su proceso de comparación para un ejercicio específico.
Precondiciones:	El docente debe haber seleccionado un ejercicio activo desde la Biblioteca (UC3).
Flujo Básico:	1. El docente accede a la pestaña de comparaciones del problema seleccionado. 2. El sistema consulta en PostgreSQL los metadatos de los análisis vinculados a dicho ejercicio. 3. El sistema despliega una tabla resumiendo los nombres de los archivos procesados y sus puntajes consolidados.
Postcondición:	El docente tiene la opción de seleccionar un archivo específico para abrir su reporte (UC9).

Tabla 5.8: Especificación del Caso de Uso 8.

Caso de Uso:	UC9: Ver reporte de similitud
Actor:	Docente / Evaluador
Descripción:	El docente abre la vista detallada de una comparación, donde el sistema renderiza la interpretación matemática de los modelos de IA.
Precondiciones:	El docente seleccionó una entrega desde el listado de comparaciones (UC8).
Flujo Básico:	1. El docente selecciona un archivo evaluado. 2. El sistema recupera el reporte de doble canal completo desde PostgreSQL. 3. El sistema renderiza la interfaz mostrando de forma gráfica el índice de similitud semántica y la probabilidad de generación mediante inteligencia artificial. del código.

Tabla 5.9: Especificación del Caso de Uso 9.

Caso de Uso:	UC10: Descargar reporte en PDF
Actor:	Docente / Evaluador
Descripción:	El docente exporta el reporte de doble canal de similitud a un documento estructurado, obteniendo una evidencia técnica estática e imprimible de la comparación.
Precondiciones:	El docente debe estar visualizando un reporte de similitud procesado (UC9).
Flujo Básico:	1. El docente ejecuta la acción "Descargar PDF" desde la interfaz del reporte. 2. El backend compila el nombre del archivo, el gráfico de similitud, el índice estilométrico y los indicadores visuales detectados. 3. El sistema genera un archivo en formato PDF aplicando una plantilla institucional. 4. El sistema transmite el archivo para su descarga directa en el dispositivo local del evaluador.
Postcondición:	El flujo de auditoría académica concluye satisfactoriamente con la generación de la evidencia estática.

Tabla 5.10: Especificación del Caso de Uso 10.

5.5 Modelado de Procesos de Negocio (BPMN)

Para la orquestación de las capacidades de inteligencia artificial y la gestión de la persistencia híbrida, se diseñó un modelo de procesos utilizando la notación BPMN 2.0. Este modelo, ilustrado en la Figura 5.3, detalla la interacción entre el docente, el motor de *backend* y los módulos

de inferencia de IA.

El proceso se divide en tres áreas de responsabilidad (*lanes*) claramente definidas:

- **Usuario (Docente):** Responsable de la carga de archivos de referencia y entrega, y la toma de decisiones basada en los reportes generados.
- **Backend:** Actúa como el orquestador del sistema, gestionando la validación de archivos, la comunicación con las bases de datos (relacional y vectorial) y la sincronización de los flujos de datos.
- **Módulo de IA:** Ejecuta las tareas de cómputo intensivo, incluyendo la generación de variantes sintéticas mediante LLMs, la extracción de características semánticas con *GraphCodeBERT* y el análisis estilométrico mediante convoluciones 1D (*CharCNN*).

5.5.1 Descripción del Flujo de Trabajo

El ciclo operativo de *Graphito* se desglosa en tres fases críticas:

1. **Fase de Gestión de Referencias:** El sistema ofrece caminos de ejecución alternativos para establecer la base de comparación. Si la configuración del problema es nueva, el *backend* verifica su existencia en la base de datos relacional; en caso de no existir, el módulo de IA genera variantes sintéticas para robustecer el espacio vectorial. Si la referencia ya existe, se recupera directamente su representación densa (*embedding*) desde la base de datos vectorial (*ChromaDB*), optimizando los tiempos de respuesta.
2. **Fase de Análisis de Doble Canal:** Una vez cargado el código a comparar, el sistema realiza un pre-procesamiento y normalización en el *backend*. Posteriormente, el flujo se bifurca mediante una compuerta paralela para realizar un análisis dual: una extracción de lógica estructural (semántica) y un análisis de patrones de escritura (estilometría). Esta fase garantiza que el sistema capture tanto la intención funcional como la huella de autoría sintética o humana.
3. **Fase de Fusión y Resultados:** Los vectores resultantes son integrados en un proceso de fusión de características (*Feature Fusion*). El sistema utiliza una compuerta de sincronización para asegurar que tanto el vector consolidado del archivo analizado como el vector de la referencia estén disponibles de forma concurrente antes de calcular la *Similitud del Coseno*. Finalmente, el módulo de IA interpreta los resultados numéricos para estructurar un reporte técnico que se persiste en el historial y se presenta al docente.

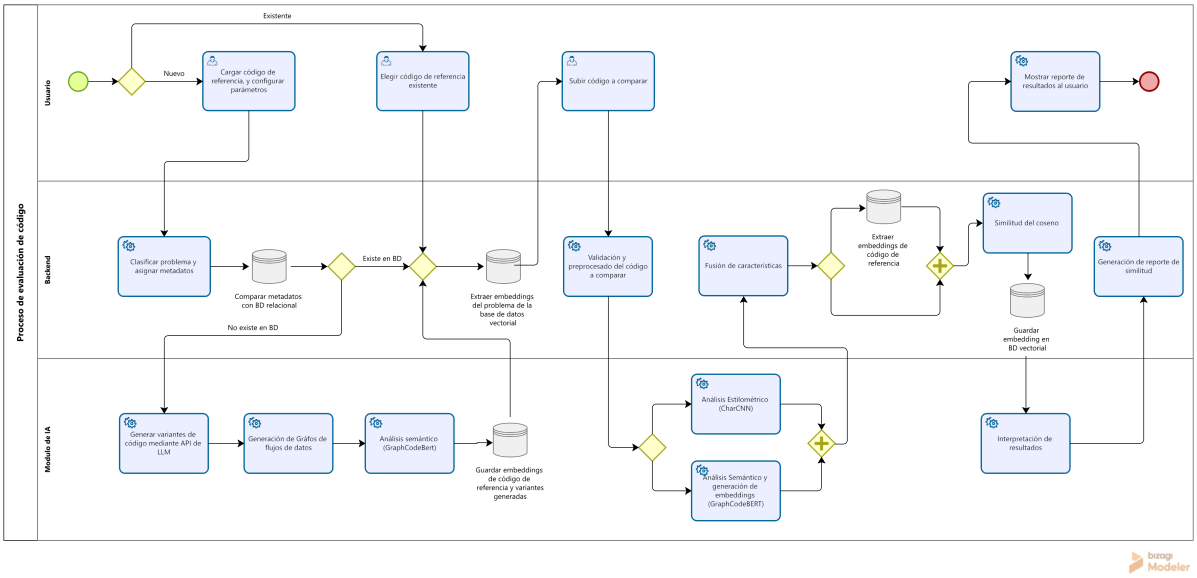


Figura 5.3: Diagrama BPMN de la comparación de código fuente.

En el flujo BPMN, la generación de variantes corresponde a una trayectoria alternativa y opcional del proceso de preparación de referencias. Esta trayectoria se activa cuando el docente solicita ampliar el conjunto de soluciones de comparación. El flujo contempla la generación de candidatos mediante un servicio externo, su validación y la incorporación de aquellos que cumplan las condiciones definidas.

La compuerta de decisión debe permitir distinguir entre variantes aceptadas, variantes que requieren revisión y variantes descartadas. Si ninguna solución generada supera las validaciones, el sistema conservará la referencia original y notificará que no se incorporaron variantes adicionales. De esta manera, la continuidad del proceso de análisis no dependerá de que el modelo externo produzca necesariamente soluciones válidas.

La representación BPMN describe los caminos de ejecución previstos en la aplicación y sus condiciones de control; no pretende enumerar todas las soluciones algorítmicas posibles para cada ejercicio. La diversidad de variantes obtenidas dependerá de la descripción del problema, las instrucciones utilizadas, el modelo generativo y los mecanismos de validación implementados.

5.6 Características semánticas del código

A diferencia del análisis estilométrico, el análisis semántico se centra en la lógica subyacente y la funcionalidad del programa, ignorando aspectos superficiales como el nombre de las variables o el formato del texto. Estas características permiten capturar la esencia algorítmica de una solución, lo que facilita la identificación de códigos que, aunque parezcan diferentes visualmente, ejecutan la misma lógica de resolución.

En la Tabla 5.11 se presenta una clasificación de las características semánticas identificadas para

la construcción de la firma lógica en este proyecto.

Categoría	Característica	Descripción Sintetizada
Flujo de Datos	Def-Uso	Rastreo del ciclo de vida y uso de variables.
	Dependencias	Vínculos lógicos entre variables y operaciones.
Lógica de Control	Caminos	Rutas lógicas y bifurcaciones del algoritmo.
	Estructuras	Patrones de repetición y decisiones funcionales.
Representación Estructural	AST / DFG	Jerarquía gramatical y grafo de flujo de datos.
	Scope	Contexto de validez de las entidades lógicas.
Contexto	Semántica	Significado de tokens según su entorno.
	Equivalencia	Funcionalidad idéntica con distinta sintaxis.

Tabla 5.11: Clasificación sintetizada de características semánticas del código.

La extracción de estas características permite generar una representación robusta ante técnicas de ofuscación comunes, como el renombrado de variables o el cambio de estructuras de control equivalentes (por ejemplo, sustituir un ciclo *for* por un *while*). En este sistema, el uso de *GraphCodeBERT* es fundamental, ya que este modelo no solo procesa el código como una secuencia de texto, sino que incorpora explícitamente el Grafo de Flujo de Datos (DFG). Esto permite que las representaciones vectoriales resultantes capturen de manera profunda las dependencias lógicas, facilitando un análisis semántico preciso incluso cuando la estructura superficial ha sido alterada intencionalmente.

5.7 Características estilométricas del código

Para el canal de análisis estilométrico, se identifican diferentes tipos de características que permiten capturar patrones asociados a la escritura del autor. Estas características no dependen de la funcionalidad del programa, sino de la huella digital sintáctica y de formato, lo que permite distinguir entre diferentes autores o identificar probables intervenciones de generación sintética (IA).

En la Tabla 5.12 se presenta una clasificación de las principales características estilométricas que la red neuronal procesa de forma implícita.

Categoría	Característica	Descripción
Léxicas	Frecuencia de caracteres	Distribución de caracteres y símbolos en el código.
	Uso de palabras clave	Frecuencia de keywords del lenguaje (C11/C++17).
	Longitud de tokens	Tamaño promedio de identificadores y expresiones.
Sintácticas	Estructuras de control	Preferencia de uso entre if, for, while, switch.
	Anidamiento	Profundidad promedio de los bloques de código.
	Modularidad	Tendencia en la organización en funciones o bloques.
Formato	Indentación	Uso predominante de espacios o tabulaciones.
	Saltos de línea	Patrones de densidad y distribución de líneas.
	Estilo de llaves	Ubicación de apertura y cierre de bloques { }.
Estructurales	Tamaño del código	Número total de líneas del archivo original.
	Uso de comentarios	Densidad y ubicación de comentarios respecto al código.

Tabla 5.12: Clasificación de características estilométricas del código fuente.

Estas características permiten construir una representación del estilo de codificación que puede ser utilizada por modelos de aprendizaje automático para identificar anomalías de autoría. En particular, los modelos basados en redes neuronales convolucionales a nivel de caracteres (Char-CNN) son capaces de aprender automáticamente patrones locales y distribuciones espaciales en la secuencia de caracteres cruda (preservando el estilo y formato original), lo que resulta especialmente útil para capturar diferencias sutiles entre código escrito por un estudiante humano y código generado de forma automatizada.

5.8 Definición de métricas de desempeño

La validación del sistema propuesto requiere la definición de métricas cuantitativas que permitan medir el desempeño algorítmico en los distintos módulos que lo componen. En particular, se consideran los dos componentes de la arquitectura dual: el canal de similitud semántica y el canal de análisis estilométrico.

Cada uno de estos módulos requiere métricas específicas que permitan evaluar su precisión, confiabilidad y utilidad computacional, permitiendo así validar la efectividad de la herramienta como apoyo objetivo a la auditoría docente.

5.8.1 Métricas para el análisis de similitud semántica

El canal de análisis semántico tiene como objetivo cuantificar el grado de equivalencia lógica entre dos programas. Para ello, se utilizan representaciones vectoriales (embeddings) generadas mediante el modelo *GraphCodeBERT*.

La principal métrica utilizada es la similitud del coseno, la cual mide el ángulo entre dos vectores en un espacio de alta dimensión. Su valor se encuentra en el rango de [-1, 1], donde valores cercanos a 1 indican una alta proximidad entre los grafos lógicos de los programas analizados.

Esta métrica resulta especialmente adecuada para comparar embeddings, ya que permite determinar la cercanía semántica entre soluciones de software independientemente de su estructura sintáctica superficial.

Adicionalmente, se considera la correlación con el juicio experto como un mecanismo de validación externa, con el fin de determinar si los índices matemáticos generados por el sistema son consistentes con la auditoría manual del evaluador.

5.8.2 Métricas para el análisis estilométrico

El canal estilométrico, impulsado por la red neuronal *CharCNN*, se plantea como un problema de clasificación binaria, en el cual se infiere si un fragmento de código presenta patrones de escritura correspondientes a un humano o a generación sintética mediante inteligencia artificial.

Para evaluar el desempeño de este modelo de Deep Learning, se utilizan métricas estándar de clasificación supervisada:

- **Accuracy (Exactitud)**: mide la proporción de predicciones correctas respecto al total de muestras analizadas.
- **Precision (Precisión)**: indica la proporción de predicciones positivas correctas respecto al total de predicciones positivas realizadas.
- **Recall (Exhaustividad)**: mide la capacidad del modelo para identificar correctamente todos los casos positivos reales.
- **F1-score**: representa la media armónica entre precisión y exhaustividad, proporcionando una medida equilibrada del desempeño del modelo ante posibles desbalances de clases.

Estas métricas permiten calibrar el comportamiento del modelo en distintos escenarios, particularmente para minimizar la tasa de falsos positivos al detectar anomalías de autoría.

5.8.3 Resumen de métricas utilizadas

En la Tabla 5.13 se presenta un resumen de las métricas empleadas en cada uno de los módulos de la arquitectura.

Módulo	Métrica	Propósito
Similitud semántica	Similitud del coseno	Medir cercanía espacial entre embeddings.
	Correlación humana	Validar coherencia con la auditoría docente.
Análisis estilométrico	Accuracy	Evaluar desempeño general de la red Char-CNN.
	Precision	Medir exacti-

5.8.4 Métricas de desempeño operativo

Además del desempeño de los modelos, la evaluación de Graphito requiere medir el tiempo y los recursos empleados durante la ejecución de los análisis. Estas métricas permiten determinar si el sistema proporciona resultados en un tiempo adecuado para el contexto educativo y ayudan a identificar posibles cuellos de botella en el procesamiento de los archivos.

El tiempo de carga corresponde al intervalo transcurrido entre el inicio de la transferencia de un archivo desde la interfaz y la confirmación de que se encuentra disponible para su procesamiento. Esta medida deberá distinguirse del tiempo de respuesta de la interfaz, que comprende el intervalo entre una acción del usuario y la actualización visible de la aplicación.

El tiempo de preprocesamiento mide la duración de las operaciones de validación, limpieza, tokenización y codificación necesarias para preparar la entrada de cada modelo. El tiempo de inferencia semántica corresponde a la ejecución de GraphCodeBERT, mientras que el tiempo de inferencia estilométrica registra la ejecución de CharCNN. El tiempo total de análisis comprende el intervalo desde el inicio de una solicitud válida hasta la disponibilidad del reporte final.

También se registrará el consumo máximo de memoria durante las pruebas, debido a que los modelos y los lotes de procesamiento pueden incrementar los requerimientos de recursos conforme aumenta el volumen de datos. Para analizar el comportamiento bajo carga, se realizarán experimentos con diferentes cantidades de archivos y tamaños de lote, documentando los tiempos obtenidos y las posibles fallas.

Los resultados se utilizarán para caracterizar el desempeño del prototipo y establecer sus límites operativos. Las mediciones deberán realizarse en un entorno documentado, manteniendo constantes, cuando sea posible, el hardware, las versiones de software y las condiciones de prueba. Se reportarán tanto los resultados individuales como estadísticas agregadas que permitan comparar escenarios.

5.8.5 Criterios de interpretación de resultados

Para facilitar el análisis de los resultados arrojados por los modelos, se definen criterios de interpretación que permiten contextualizar los valores numéricos obtenidos.

En el caso del canal semántico, los valores de similitud del coseno se interpretan bajo los siguientes umbrales preliminares:

Rango de similitud	Interpretación algorítmica
0.80 – 1.00	Alta equivalencia lógica.
0.60 – 0.79	Equivalencia moderada (posible inspiración estructural).
0.40 – 0.59	Baja equivalencia.
0.00 – 0.39	Disimilitud algorítmica.

Tabla 5.14: Criterios de interpretación para similitud semántica espacial.

Estos rangos permiten identificar casos atípicos y resaltarlos visualmente para revisión detallada del docente, especialmente aquellos que cruzan el umbral de alta equivalencia lógica.

Para el análisis estilométrico, la interpretación técnica se basa en el comportamiento conjunto de las métricas de clasificación:

- Valores altos de precisión y exhaustividad indican una sólida capacidad de la CharCNN para diferenciar características de autoría humana versus sintética.
- Un F1-score elevado refleja un modelo confiable que no sesga sus clasificaciones hacia la clase mayoritaria.
- Diferencias significativas entre precisión y exhaustividad requerirán ajustes en el orquestador para evitar falsos positivos que afecten la transparencia de la auditoría.

5.8.6 Responsabilidad docente e interpretación del reporte de Graphito

Los indicadores generados por Graphito tienen como finalidad facilitar la identificación de entregas que requieren una revisión académica más detallada. El sistema proporciona evidencia cuantitativa relacionada con la proximidad semántica de las soluciones y con los patrones estilométricos identificados por el modelo entrenado. La interpretación de estos indicadores debe considerar el ejercicio evaluado, las características del código de referencia y las limitaciones de los modelos utilizados.

La similitud semántica del coseno expresa el grado de proximidad entre las representaciones vectoriales comparadas. Un valor elevado indica que los embeddings presentan una orientación semejante dentro del espacio vectorial del modelo. Sin embargo, este resultado no demuestra por sí mismo que exista copia, dependencia indebida o identidad de autoría. En ejercicios introductorios, diferentes estudiantes pueden producir soluciones similares debido a las restricciones del problema, al uso de algoritmos conocidos o a las convenciones de enseñanza.

Por su parte, la puntuación estilométrica representa la salida del clasificador CharCNN para la clase correspondiente. Su interpretación depende de la codificación de las etiquetas, del procedimiento de entrenamiento y de la calibración del modelo. La puntuación no debe considerarse

una probabilidad fiable de autoría sin una evaluación de calibración y de desempeño sobre datos independientes.

Para apoyar una revisión responsable, el reporte deberá mostrar los indicadores de ambos canales, el ejercicio asociado, las referencias utilizadas y las advertencias necesarias cuando existan limitaciones en los datos de entrada. Cuando se identifique una similitud elevada o una discrepancia entre los canales, Graphito podrá destacar la entrega para priorizar su revisión.

La responsabilidad de interpretar el reporte y determinar las acciones académicas corresponde al docente. Ante un resultado que amerite revisión, podrán utilizarse mecanismos complementarios, como solicitar al estudiante que explique la lógica implementada, modifique una parte del programa, justifique las decisiones algorítmicas o resuelva una actividad breve relacionada con el ejercicio original. Estas acciones permiten contrastar los indicadores automatizados con evidencia directa de comprensión.

Por tanto, Graphito no constituye un sistema de calificación automática ni un mecanismo definitivo de detección de plagio. Su resultado final es un reporte de apoyo a la decisión que organiza indicadores técnicos para facilitar la revisión, reducir la carga operativa y promover una evaluación sustentada en evidencia.

5.8.7 Consideraciones para la experimentación

Para garantizar una validación rigurosa de los modelos, es necesario considerar los siguientes factores durante el diseño del corpus experimental:

- **Diversidad algorítmica**: incluir soluciones con diferentes lógicas de control para el mismo problema.
- **Presencia de variantes IA**: incorporar entregas generadas específicamente mediante la Trayectoria A1 (Modelos de Lenguaje Externos).
- **Balance de clases**: asegurar una distribución representativa entre código escrito por estudiantes y código sintético.
- **Validación cruzada**: aplicar técnicas de partición de datos para evitar el sobreajuste (overfitting) de la red estilométrica.

5.8.8 Justificación de las métricas seleccionadas

La combinación de estas métricas permite validar el sistema desde dos perspectivas técnicas complementarias. Por un lado, la similitud del coseno aborda el problema de la detección de clones estructurales, cuantificando la equivalencia lógica a través de los grafos. Por otro lado, las métricas de clasificación evalúan la capacidad de la red convolucional para identificar patrones anómalos asociados a la Inteligencia Artificial.

Este enfoque dual permite certificar el desempeño del backend, proporcionando evidencia algorítmica sobre su viabilidad como herramienta de auditoría de código en entornos académicos.

5.9 Análisis de riesgos

El desarrollo e implementación del sistema propuesto implica la consideración de diversos riesgos que pueden afectar su funcionamiento, desempeño y adopción en el contexto educativo. Estos riesgos pueden surgir tanto a nivel técnico como en el uso práctico del sistema por parte de los evaluadores.

El análisis de riesgos permite identificar posibles escenarios adversos, evaluar su impacto y definir estrategias de mitigación que contribuyan a la robustez, confiabilidad e interpretabilidad del sistema.

5.9.1 Clasificación de riesgos

Los riesgos identificados se agrupan en las siguientes categorías:

- **Riesgos técnicos**: relacionados con el funcionamiento de los modelos de inteligencia artificial y la infraestructura del sistema.
- **Riesgos de datos**: asociados a la calidad, disponibilidad y representatividad de los datos utilizados.
- **Riesgos de interpretación**: vinculados a la forma en que los resultados son comprendidos por el evaluador.
- **Riesgos académicos**: relacionados con el impacto del sistema en el proceso de evaluación y aprendizaje.

5.9.2 Criterios de evaluación de riesgos

Para evaluar los riesgos identificados, se utilizan dos criterios principales: la probabilidad de ocurrencia y el impacto que pueden generar en el sistema.

La **probabilidad** se refiere a la posibilidad de que un riesgo ocurra durante la operación del sistema, mientras que el **impacto** describe el nivel de afectación que tendría dicho riesgo en el desempeño, confiabilidad o uso del sistema.

Se consideran tres niveles para cada criterio:

- **Alta**: el riesgo es frecuente o puede afectar significativamente el funcionamiento del sistema.

- **Media**: el riesgo puede ocurrir bajo ciertas condiciones y su impacto es moderado.
- **Baja**: el riesgo es poco probable o su impacto es limitado.

Estos niveles permiten priorizar los riesgos y enfocar los esfuerzos de mitigación en aquellos que representan una mayor amenaza para el sistema.

5.9.3 Identificación y evaluación de riesgos

En la Tabla 5.15 se presentan los principales riesgos identificados, junto con su probabilidad de ocurrencia, impacto y estrategia de mitigación.

Riesgo	**Prob.**	**Impacto**	**Mitigación**
Baja precisión del modelo semántico	Media	Alta	Ajuste de hiperparámetros y validación con datos reales
Clasificación incorrecta en estilometría	Media	Alta	Uso de múltiples métricas y validación cruzada
Datos insuficientes o no representativos	Alta	Alta	Ampliación y diversificación del dataset
Dependencia de modelos externos (LLM)	Alta	Alta	Uso de rutas de respaldo con modelos alternativos y locales
Interpretación incorrecta por el docente	Media	Alta	Visualización clara y explicaciones dentro del sistema
Falsos positivos en detección de similitud	Media	Alta	Uso combinado de análisis semántico y estilométrico
Alto costo computacional	Media	Media	Optimización del procesamiento y uso de almacenamiento en caché
Sesgo en el modelo de clasificación	Media	Alta	Balanceo de clases y evaluación continua del modelo

Tabla 5.15: Análisis de riesgos del sistema

5.9.4 Estrategias de mitigación

Para reducir el impacto de los riesgos identificados, se plantean diversas estrategias orientadas a mejorar la robustez del sistema:

- Validación continua de los modelos mediante conjuntos de datos diversos y representativos.
- Integración de múltiples enfoques de análisis (semántico y estilométrico) para reducir errores individuales.
- Implementación de mecanismos de interpretación de resultados que faciliten la comprensión por parte del evaluador.
- Optimización del rendimiento del sistema para garantizar tiempos de respuesta adecuados.

5.9.5 Discusión del impacto de los riesgos

El análisis realizado permite identificar que los riesgos con mayor impacto están asociados a la precisión de los modelos de inteligencia artificial y a la interpretación de los resultados en el contexto educativo. Estos factores son críticos, ya que pueden influir directamente en la toma de decisiones del evaluador.

Asimismo, los riesgos relacionados con los datos, como la falta de representatividad o el sesgo, pueden afectar la capacidad del sistema para generalizar correctamente en escenarios reales.

No obstante, mediante la implementación de estrategias de mitigación adecuadas, es posible reducir significativamente estos riesgos, permitiendo que el sistema funcione como una herramienta confiable de apoyo a la evaluación docente.

El análisis de riesgos realizado permite identificar que los factores más críticos del sistema se concentran en la precisión de los modelos de inteligencia artificial y en la correcta interpretación de los resultados por parte del evaluador. Estos elementos son fundamentales, ya que influyen directamente en la confiabilidad del sistema dentro del proceso de evaluación académica.

Asimismo, se observa que los riesgos asociados a la calidad y representatividad de los datos pueden impactar de manera significativa en el desempeño del sistema, particularmente en la capacidad de generalización de los modelos en escenarios reales.

No obstante, la integración de estrategias de mitigación, como el uso combinado de análisis semántico y estilométrico, la validación continua y la incorporación de mecanismos de interpretación, permite reducir estos riesgos y fortalecer la robustez del sistema.

En este sentido, el análisis realizado respalda la viabilidad del sistema propuesto como una herramienta de apoyo a la decisión docente, siempre que se considere como un complemento al criterio humano y no como un sustituto del mismo.

5.9.6 Selección y justificación del conjunto de datos

Para el entrenamiento del clasificador estilométrico propuesto se requieren dos tipos de datos: código fuente escrito por humanos en un contexto educativo introductorio, y código equivalente generado por modelos de inteligencia artificial. A continuación se describe y justifica cada fuente utilizada, así como la estructura cuantitativa del dataset resultante.

5.9.6.1 Código humano: IEEE Programming Homework Dataset

El componente de código humano proviene del *Programming Homework Dataset* publicado por Ljubovic y Pajic (2020) en [54]. Este dataset contiene envíos reales de estudiantes en dos cursos introductorios de programación universitaria: el curso A en lenguaje C y el curso B en C++, con entre 16 y 22 asignaciones por curso. Su selección se justifica por tres razones principales.

En primer lugar, el origen de los datos es directamente relevante al dominio del problema. Los envíos provienen de estudiantes en cursos de fundamentos de programación, que es exactamente la población objetivo del sistema propuesto. Esto garantiza que el modelo aprenda patrones estilométricos propios del código estudiantil introductorio y no de programadores expertos o código de producción.

En segundo lugar, el dataset cubre los tipos de problemas de interés: estructuras de control, manejo de arreglos, funciones, estructuras de datos básicas y manejo de archivos, todos representativos del currículo de un curso introductorio de C/C++ (acorde a los estándares C11 y C++17).

En tercer lugar, respecto al idioma de los identificadores y comentarios, el análisis del código fuente contenido en el dataset confirmó que este se encuentra escrito en serbocroata con alfabeto latino. Dado que los comentarios constituyen un rasgo estilométrico relevante —puesto que codifican información sobre la frecuencia, extensión y estilo de documentación del programador—, estos se conservan íntegramente durante el preprocesamiento como parte del *input* de la Char-CNN. La compatibilidad general con código escrito en otros idiomas de alfabeto latino, como el español o el inglés, se garantiza a nivel de distribución de caracteres y topología estructural.

5.9.6.2 Código generado por IA (Clase Sintética)

Para construir la clase positiva del clasificador (código generado por IA), se opta por una estrategia de generación propia sobre los mismos enunciados de problemas presentes en el dataset humano. Esta decisión se fundamenta en la necesidad de que ambas clases respondan al mismo dominio del problema, evitando que el modelo aprenda diferencias asociadas a la lógica de la tarea y no a la naturaleza de su origen (humano vs. máquina).

Con el objetivo de maximizar la generalización de la red convolucional ante distintos modelos de lenguaje, se emplea un conjunto diverso de generadores organizado en dos niveles, como

se resume en la Tabla 5.16. En el primer nivel se incluyen modelos de referencia comerciales (SOTA), mientras que el segundo nivel abarca modelos de código abierto.

Tabla 5.16: Composición y origen del conjunto de datos dual

Categoría	**Fuente / Modelo**	**Lenguaje**	**Origen de Escritura**	**Propósito en el sistema**
Código Humano	IEEE Programming Homework [54]	C / C++	Estudiante (Humano)	Base de referencia de escritura manual y entrenamiento de CharCNN.
Código IA (Comercial)	GPT-4o, Claude 3.5, Gemini 1.5	C / C++	Generativo (SOTA)	Identificación de patrones de IAs de alta disponibilidad para estudiantes.
Código IA (Local)	CodeLlama, DeepSeek, Qwen2.5	C / C++	Generativo (Open-Source)	Evaluación de robustez ante modelos locales y diversidad estilística.

Elaboración propia con base en [54].

Para garantizar la consistencia entre ambas clases, todos los *prompts* de generación se formulan indicando explícitamente al modelo que utilice nombres de variables y comentarios en serbocroata. Esto asegura que el idioma no constituya una señal discriminante (sesgo) durante el entrenamiento, forzando a la red a aprender diferencias estrictamente de naturaleza estilométrica espacial. Adicionalmente, se generan variantes de *prompt* para cada problema con el fin de reducir la dependencia a un estilo de generación único:

- Un *prompt* directo con el enunciado del problema.
- Un *prompt* con instrucción de simular el estilo de un estudiante principiante.
- Un *prompt* con enfoque en optimización sin instrucciones de estilo adicionales.

5.9.6.3 Consideraciones de balance y partición

El corpus final se construye con clases estrictamente balanceadas (50 % humano, 50 % IA) para evitar sesgos de clase mayoritaria en las predicciones de la CharCNN. La partición de los datos se realiza a nivel de *problema*; es decir, todas las soluciones (humanas o sintéticas) de un problema específico aparecen exclusivamente en una de las fases (entrenamiento, validación o prueba). Esta restricción arquitectónica previene la fuga de información (*data leakage*), obligando al modelo a clasificar basándose en el estilo y no en la lógica del algoritmo. La Tabla 5.17 presenta la distribución cuantitativa establecida.

Tabla 5.17: Distribución cuantitativa y partición del conjunto de datos

Subconjunto	**Porcentaje**	**Cantidad (n)**	**Finalidad en el modelo**
Entrenamiento (*Train*)	70 %	14,000	Ajuste de pesos y aprendizaje de características estilométricas 1D.
Validación (*Val*)	15 %	3,000	Ajuste de hiperparámetros y prevención de *overfitting* durante el entrenamiento.
Prueba (*Test*)	15 %	3,000	Evaluación cuantitativa del rendimiento y matriz de confusión en datos no vistos.
Total (N)	**100 %**	**20,000**	**Dataset consolidado e híbrido.**

Elaboración propia.

En conclusión, la estructuración de este corpus híbrido permite calibrar con precisión el canal estilométrico del sistema **Graphito**. El diseño metodológico empleado fuerza a la arquitectura CharCNN a desarrollar una alta sensibilidad a las micro-estructuras de estilo que diferencian el código orgánico del sintético. El uso de $N = 20,000$ muestras asegura una base estadística robusta para generalizar características más allá de las palabras clave, capturando la huella digital del código fuente en entornos académicos iniciales.

Para reducir los efectos del desbalance de clases en el entrenamiento de CharCNN, se propone construir un conjunto de datos que contenga muestras de ambas clases y documentar su distribución antes y después del preprocesamiento. La clase humana estará integrada por soluciones provenientes de fuentes académicas identificadas, mientras que la clase sintética estará constituida por programas generados mediante herramientas de inteligencia artificial bajo condiciones de generación documentadas.

El balance cuantitativo entre clases constituye una medida inicial, pero no garantiza por sí mismo que el conjunto de datos sea representativo. También es necesario considerar la diversidad de ejercicios, la complejidad de las soluciones, los lenguajes, las convenciones de escritura y los modelos generativos utilizados. De lo contrario, el clasificador podría aprender diferencias asociadas con una fuente, un ejercicio o una herramienta específica, en lugar de identificar patrones que generalicen a nuevos programas.

Para evitar la fuga de información, las particiones de entrenamiento, validación y prueba deberán establecerse antes de realizar las transformaciones que impliquen aprendizaje de parámetros. Cuando existan múltiples variantes de una misma solución, estas deberán agruparse de manera

que las soluciones relacionadas no aparezcan simultáneamente en entrenamiento y prueba. De esta forma, se reduce el riesgo de obtener métricas artificialmente elevadas debido a la presencia de ejemplos muy similares en distintos subconjuntos.

El desempeño se evaluará mediante precisión, *recall* y F1-score, complementados por la matriz de confusión y el análisis de falsos positivos y falsos negativos. Si persisten diferencias relevantes en la distribución de las clases, podrán evaluarse estrategias como ponderación de la función de pérdida o sobremuestreo exclusivo del conjunto de entrenamiento. La selección de estas medidas se realizará con base en los resultados de validación, sin modificar el conjunto de prueba para mejorar artificialmente el desempeño.

Finalmente, deberá reconocerse que el modelo aprende a distinguir las clases definidas en el conjunto experimental, no la autoría real de cualquier programa. La generalización a código escrito con asistencia de IA, código humano posteriormente refactorizado y soluciones producidas por modelos no incluidos en el entrenamiento constituye una limitación que deberá analizarse mediante experimentos adicionales. La composición final del conjunto y su partición se detallan en el Capítulo 7.

Capítulo 6

Diseño del sistema

6.1 Arquitectura (Modelo C4)

El diseño arquitectónico de *Graphito* se estructura utilizando el Modelo C4, el cual permite describir el software a través de múltiples niveles de abstracción. Esto proporciona diferentes perspectivas técnicas, desde la visión contextual de las interacciones externas hasta el nivel más profundo de componentes.

6.1.1 Nivel 1: Contexto

El diagrama de contexto ilustra cómo el sistema **GRAPHITO** se relaciona con sus actores externos. En el centro de la arquitectura, el sistema provee las capacidades de procesamiento y análisis. A su alrededor se ubican: el **Docente**, quien interactúa para gestionar ejercicios, cargar el código de los archivos a analizar y visualizar los reportes; y la **API de LLM Externo**, que es consultada por la plataforma para generar automáticamente soluciones sintéticas que formarán parte del marco de referencias.

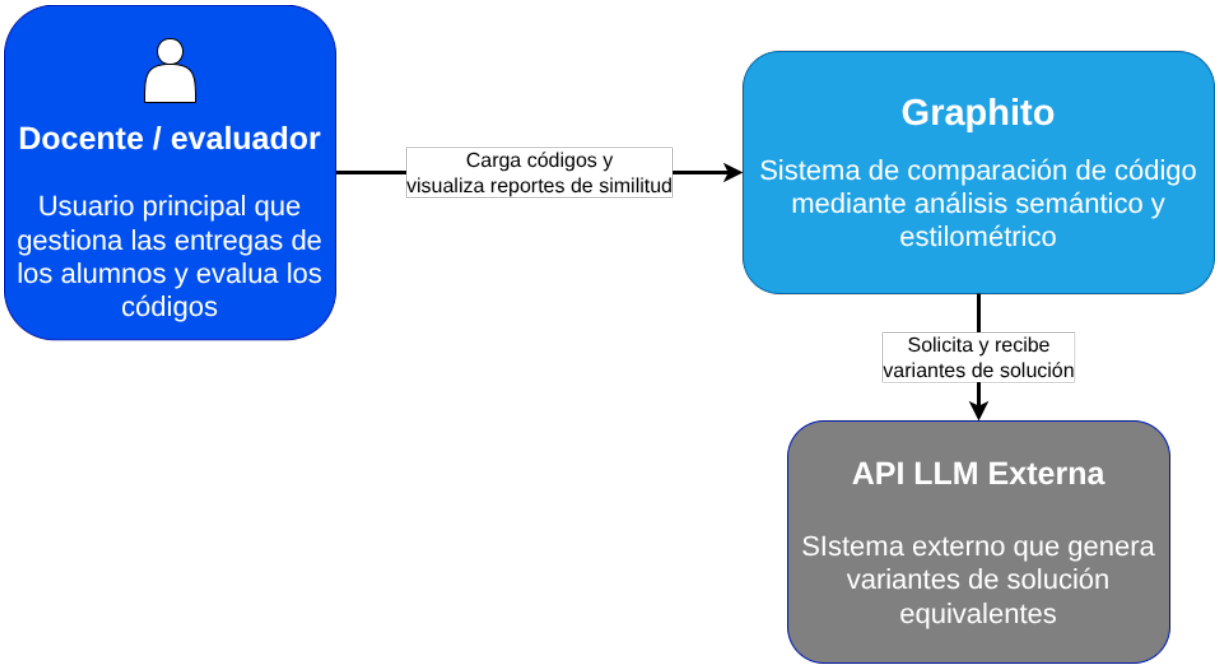


Figura 6.1: Diagrama de Contexto (Nivel 1) del sistema Graphito.

6.1.2 Nivel 2: Contenedores

En este nivel de abstracción se detalla la arquitectura de micro-servicios y bases de datos que componen el sistema. *Graphito* se separa funcionalmente en los siguientes contenedores principales:

- **Capa de Cliente (Front-end):** Interfaz web interactiva en *React* que consume servicios REST, facilitando el trabajo del docente.
- **Núcleo de Aplicación (Back-end):** Servicio principal implementado en *FastAPI* (Python) encargado de orquestar el sistema, gestionar las peticiones a la IA y procesar la lógica general.
- **DB Relacional (PostgreSQL):** Persistencia estructurada para metadatos del usuario, problemas y registro histórico integral del análisis.
- **Vector DB (ChromaDB):** Persistencia hiperdimensional de los *embeddings* que agilizan las búsquedas por proximidad semántica espacial en las referencias.

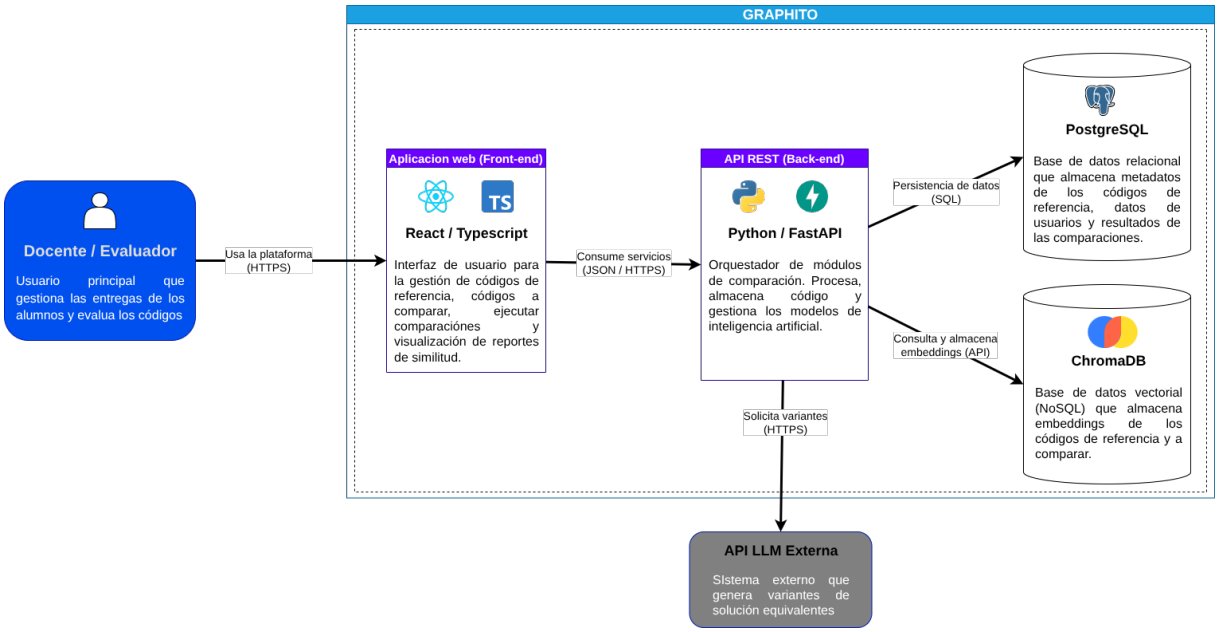


Figura 6.2: Diagrama de Contenedores (Nivel 2) del sistema Graphito.

6.1.3 Nivel 3: Componentes

Haciendo un acercamiento al motor central de analítica del sistema (el contenedor Back-end de FastAPI), se ilustran los componentes internos que materializan la funcionalidad de doble canal para Graphito:

- **Orquestador Inteligente (LangGraph):** Componente maestro de orquestación que evalúa iteraciones, fusiona características híbridas e interactúa cíclicamente para extender las referencias funcionales.
- **Módulo de Análisis Semántico:** Utiliza de forma profunda el modelo pre-entrenado *GraphCodeBERT* para interpretar características lógicas estructurales a través del grafo de flujo de datos (DFG).
- **Módulo de Estilometría:** Implementa un clasificador *CharCNN* mediante convoluciones 1D a nivel de caracteres, capaz de clasificar el origen (humano o sintético) al aprender la textura de redacción original y capturar singularidades de formato (ej. sangrías o estilo de llaves).

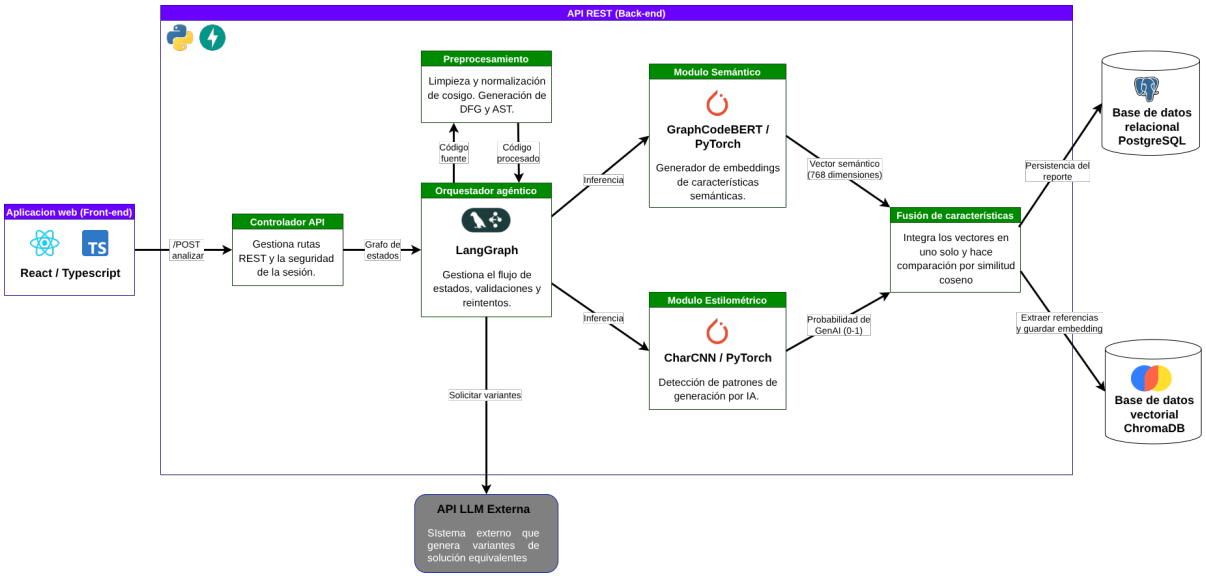


Figura 6.3: Diagrama de Componentes (Nivel 3) enfocado en el motor back-end en Python.

6.1.4 Nivel 4: Código (Clases y Secuencia)

El nivel de código ofrece el máximo detalle referencial. Se subdivide en la vista estática, representada por las clases y patrones, y la vista dinámica, que muestra el paso a paso transaccional.

6.1.4.1 Diagrama de Clases

A nivel estático, el corazón del diseño obedece al patrón de diseño *Strategy*.

Como es posible constatar en la Figura 6.4, la ejecución transaccional principal se centraliza en **OrquestadorGraphito**, que coordina los módulos. El patrón *Strategy* se aplica a la interfaz de inyección **ModeloIA**, separando los modelos predictivos como **ModeloGraphCodeBERT** y **ModeloCharCNN** del núcleo aplicativo.

El componente **Preprocesador** bifurca el flujo para proveer grafos limpios (análisis semántico) o matrices crudas (estilometría). Finalizada la ejecución iterativa, se concreta la **FirmaDual**. Por su parte, la persistencia se expone con la interfaz **AlmacenamientoDatos** la cual rige las lógicas independientes de **GestorPostgreSQL** y **GestorChromaDB**.

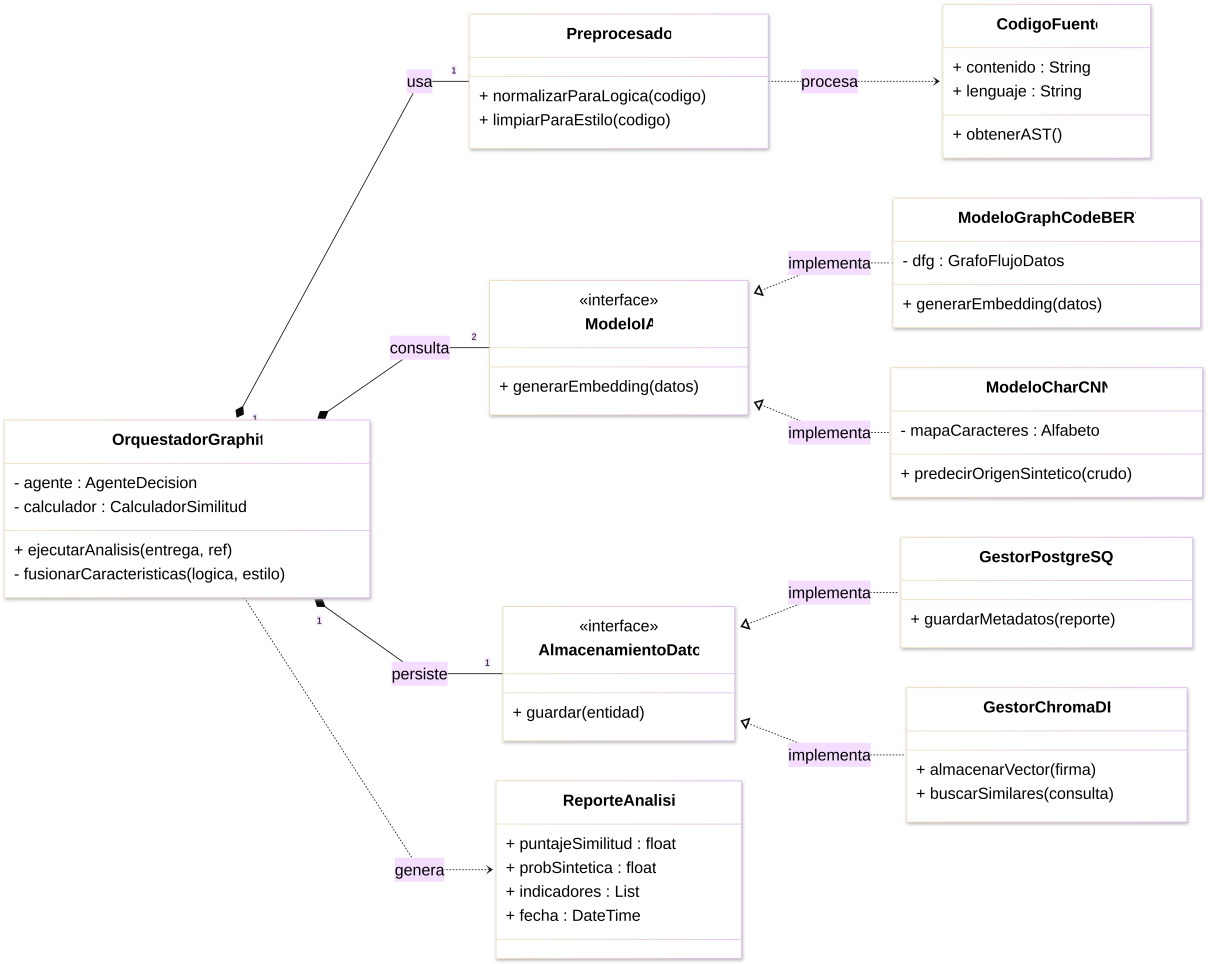


Figura 6.4: Diagrama de clases del sistema Graphito, ilustrando la arquitectura orientada a objetos correspondientes al Nivel 4 de abstracción.

6.1.4.2 Diagrama de Secuencia

Para el flujo de eventos, el diagrama de secuencia dicta el actuar de los módulos durante un análisis estándar:

- **Ejecución Paralela:** Tanto *GraphCodeBERT* (para interpretar la lógica estructural a través de grafos) como *CharCNN* (para captar la huella estilométrica) se ejecutan de manera concurrente con la finalidad de optimizar los tiempos de inferencia en el servidor.
- **Consolidación e Indicadores:** Una vez concatenado el vector unificado mediante la Fusión de Características (*Feature Fusion*), se calcula su similitud del coseno contra las referencias indexadas en ChromaDB y se retornan los indicadores visuales a la capa de abstracción del *frontend* para la revisión del docente.

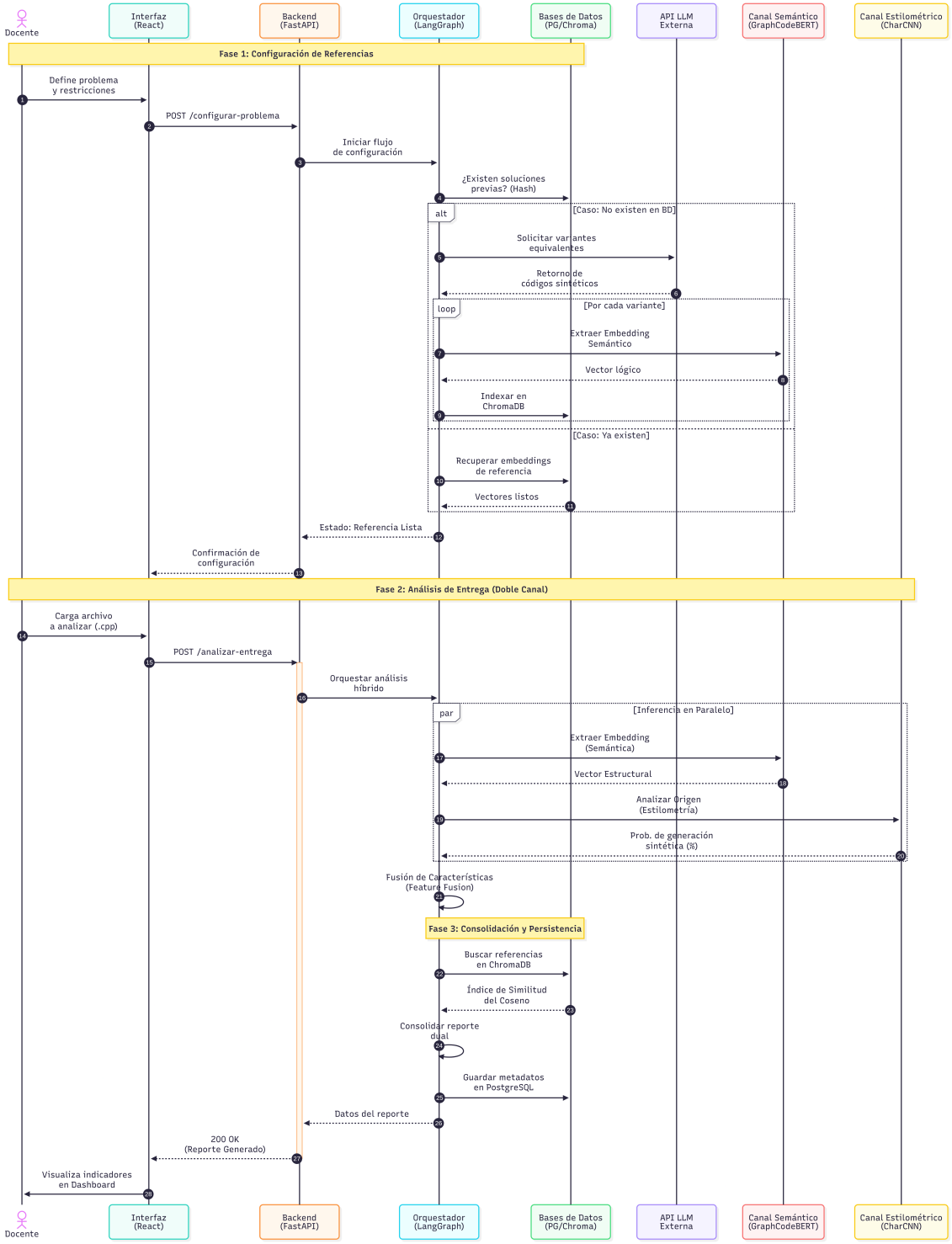


Figura 6.5: Diagrama de secuencia transaccional de *Graphito* desde el cliente (Nivel 4).

6.1.5 Modelado Dinámico: Diagrama de Estados

El modelado dinámico permite representar el comportamiento interno del sistema ante diversos estímulos y eventos. Para el sistema **Graphito**, el diagrama de estados describe el ciclo de vida de una tarea de análisis, desde la interacción inicial del docente hasta la consolidación de los resultados.

Este diagrama es fundamental para comprender cómo el sistema transita entre las fases de procesamiento paralelo y la fusión de características finales. En la Figura 6.6 se ilustra el flujo de estados transaccionales.

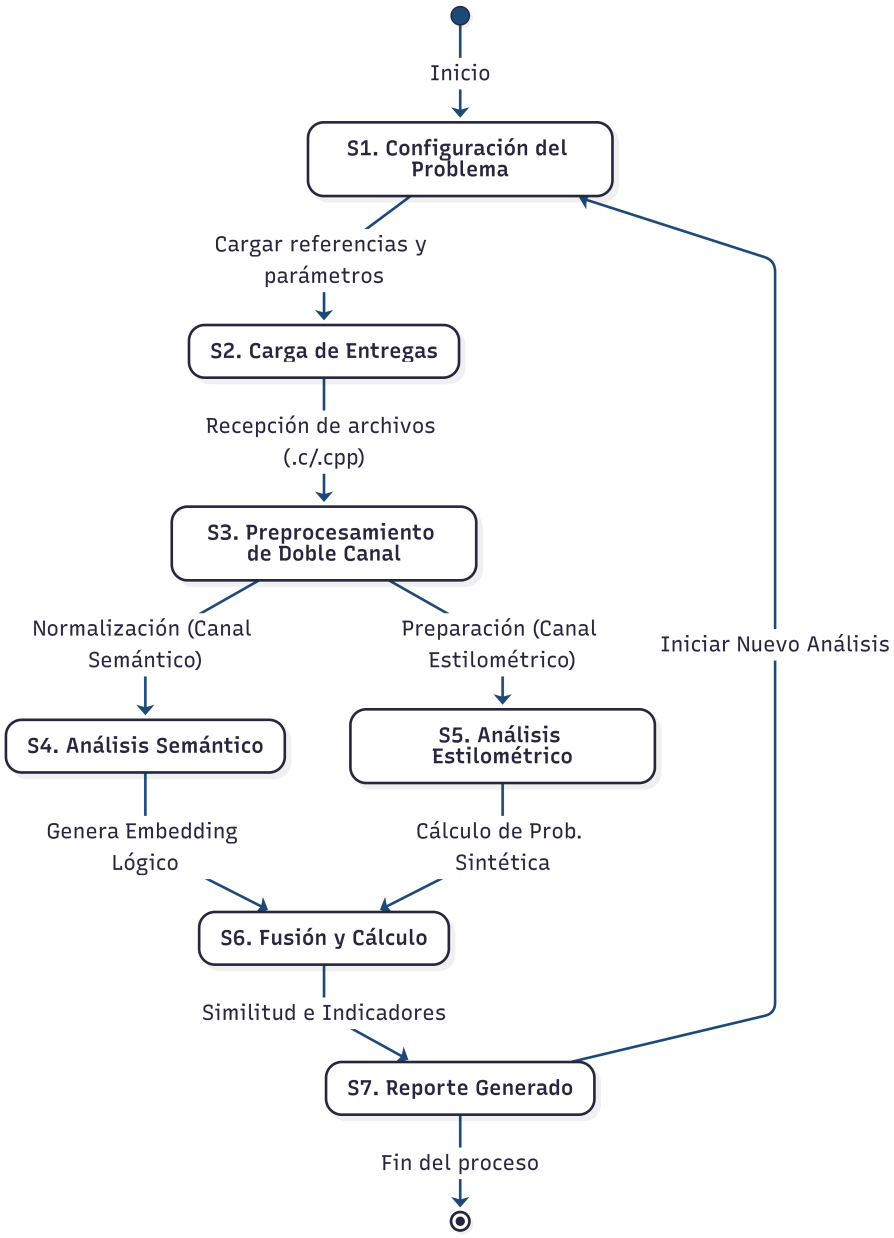


Figura 6.6: Diagrama de estados del proceso de análisis de código en Graphito.

A continuación, se describen las etapas clave representadas en el modelo:

- **S1. Configuración del Problema:** Estado inicial donde el sistema se encuentra a la espera de los parámetros base, como el lenguaje de programación, las restricciones del ejercicio y la carga de soluciones de referencia.
- **S2. Carga de Entregas:** Transición en la que el sistema recibe los archivos fuente en formato `.c` o `.cpp`. En este punto, se valida la integridad de los archivos antes de proceder al motor de analítica.

- **S3. Preprocesamiento de Doble Canal:** Estado crítico donde el código es bifurcado. Aquí se ejecutan simultáneamente las tareas de limpieza para el análisis de lógica y la preservación de formato para el análisis de redacción.
- **S4. Análisis Semántico:** El sistema transita a este estado para la extracción de características lógicas mediante grafos. El resultado es un vector denso (*embedding*) que encapsula la funcionalidad del código.
- **S5. Análisis Estilométrico:** En paralelo al estado anterior, el sistema procesa la textura del código para identificar rasgos de origen humano o sintético, generando una probabilidad porcentual de generación por IA.
- **S6. Fusión y Cálculo:** Una vez obtenidos ambos vectores, el sistema entra en una fase de consolidación. Se realiza el cálculo de la similitud del coseno contra las referencias y se unifican las métricas obtenidas.
- **S7. Reporte Generado:** Estado final donde los resultados se persisten en las bases de datos y se presentan al docente mediante indicadores visuales. Desde aquí, el sistema puede finalizar el proceso o retornar al estado S1 para una nueva evaluación.

6.2 Metodología de Desarrollo: Ciclo de Vida Komorebi

Para estructurar el ciclo de vida de los datos y los modelos de inteligencia artificial dentro del sistema **Graphito**, se adoptó la metodología **Komorebi** propuesta por Díaz Vergés [55]. En la ingeniería de software tradicional, los proyectos suelen gestionarse mediante ciclos secuenciales o iterativos convencionales (definir, diseñar, desarrollar, probar y desplegar) donde los requerimientos son deterministas y los datos sobre los que opera el sistema son estables y predecibles.

Sin embargo, como argumenta Díaz Vergés [55], los proyectos de Inteligencia Artificial y Aprendizaje Automático poseen una naturaleza cualitativamente distinta: combinan desarrollo de software con investigación empírica e innovación científica. Esto introduce un elevado grado de incertidumbre respecto a la convergencia de los modelos, la calidad intrínseca de las fuentes de información y la representatividad de las características extraídas. Bajo este paradigma, la metodología Komorebi plantea una gestión ágil adaptada específicamente a proyectos de *Data & Machine Learning*, estructurando el flujo técnico en etapas iterativas que garantizan la trazabilidad, reproducibilidad y gobernanza de los experimentos.

En el sistema **Graphito**, la metodología Komorebi actúa como el marco metodológico rector que gobierna el pipeline de procesamiento bimodal (semántico y estilométrico). En la Figura 6.7 se ilustra la arquitectura global de este flujo operativo.

El pipeline de modelos representa el flujo de procesamiento de datos y la secuencia de actividades necesarias para obtener los indicadores de Graphito. A diferencia del diagrama de bloques funcional, que presenta los componentes principales del sistema y sus relaciones, el pipeline

se centra en la preparación de los datos, la generación de representaciones, el entrenamiento o configuración de los modelos, su evaluación y su integración en el entorno de ejecución.

Graphito coordina estas actividades mediante su backend, que recibe las solicitudes, valida las entradas, administra la ejecución de los modelos y consolida los resultados. GraphCodeBERT se utiliza como extractor de características semánticas y CharCNN como clasificador estilométrico. Las representaciones de referencia pueden almacenarse en ChromaDB para su recuperación, mientras que PostgreSQL conserva la información estructurada necesaria para relacionar las comparaciones con los ejercicios y sus resultados.

La separación entre el pipeline experimental y el flujo operativo permite distinguir las actividades de preparación y evaluación de modelos de las operaciones que se ejecutan cuando un docente solicita analizar una entrega. Esta distinción facilita la reproducibilidad de los experimentos y la identificación de las responsabilidades de cada componente.

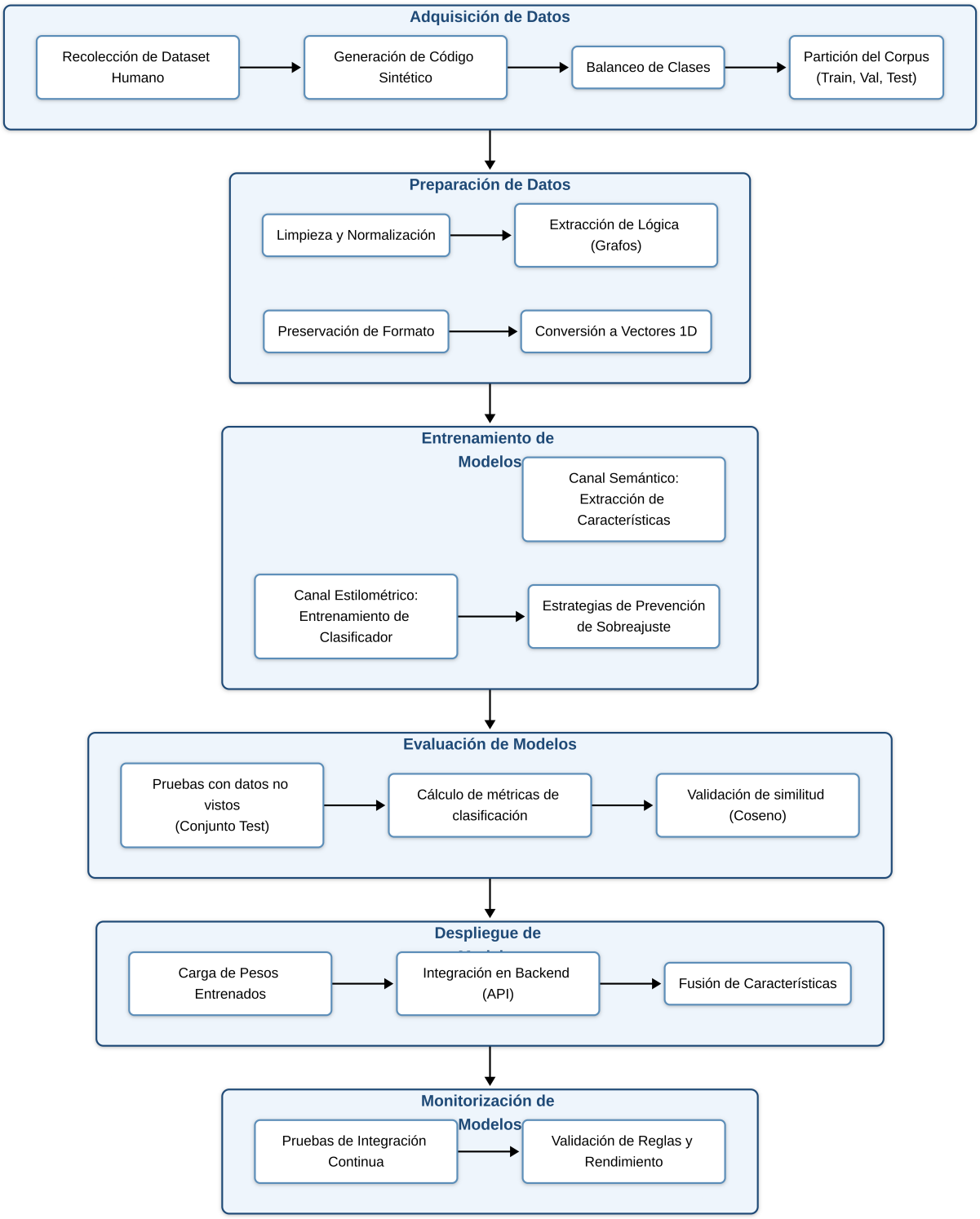


Figura 6.7: Flujo secuencial del pipeline de modelos basado en la metodología Komorebi.

A nivel documental y de ingeniería, las fases del ciclo de vida de Komorebi se articulan y desarrollan a lo largo de este Trabajo Terminal de la siguiente manera:

1. **Fase I. Planificación y Comprensión del Problema:** Consiste en el levantamiento de requerimientos técnicos, delimitación del contexto educativo (la paradoja cognitiva de la IA en cursos de programación) y el establecimiento de métricas de éxito (RNF-03 a RNF-07), aspectos detallados en los Capítulos 1, 4 y 5.

2. **Fase II. Adquisición de Datos (*Data Ingestion*):** Comprende la recolección del corpus humano representativo (*IEEE Programming Homework Dataset*), la ingeniería inversa asistida por LLMs para recuperar los enunciados algorítmicos (`extraer_enunciados.py`) y la síntesis del conjunto de código generado por IA con generadores comerciales y abiertos (`generar_referencias.py`). Esta etapa se formaliza detalladamente en el Capítulo 7 (Sección 7.1).

3. **Fase III. Preparación y Normalización de Datos:** Introduce la bifurcación arquitectónica divergente del código fuente para alimentar de forma especializada al canal semántico (depuración léxica, AST y Grafo de Flujo de Datos) y al canal estilométrico (código *raw* y cuantización matricial 1D de caracteres), aplicando una partición agrupada por problema para prevenir fuga de información (*data leakage*). Su desarrollo exhaustivo se expone en el Capítulo 7 (Sección 7.2).

4. **Fase IV. Entrenamiento y Modelado de IA:** Abarca la configuración de las redes neuronales: el uso de *GraphCodeBERT* como extractor de características semánticas mediante grafos de dependencias enriquecidos con adaptadores LoRA, el entrenamiento de la red convolucional *CharCNN* para clasificación de autoría mediante convoluciones 1D sobre caracteres, y el algoritmo de **Fusión de Características (*Feature Fusion*)** que consolida el vector multimodal de 769 dimensiones.

5. **Fase V. Evaluación y Validación Experimental:** Comprende la evaluación estadística rigurosa de los clasificadores (matriz de confusión, F_1-Score, curvas ROC-AUC), el análisis de invariancia de la similitud del coseno ante la taxonomía de clones de Bellon, y la contrastación empírica con arquetipos de entregas estudiantiles.

6. **Fase VI. Despliegue de Modelos:** Corresponde a la integración de los modelos empaquetados dentro de los servicios backend en FastAPI (bajo principios de *Clean Architecture*), la orquestación del almacenamiento híbrido (PostgreSQL y ChromaDB) y la interfaz interactiva para el docente en React.

7. **Fase VII. Monitorización y Calibración Continua:** Fase de estabilización operativa enfocada en la ejecución de pruebas unitarias y de estrés automatizadas, la calibración empírica de umbrales para mantener la tasa de falsos positivos por debajo del 10 %, y la integración del principio *Human-in-the-loop*, donde la herramienta asiste al docente sin sustituir su juicio pedagógico.

Esta articulación metodológica garantiza que las decisiones de diseño expuestas en las siguientes secciones (modelo relacional, almacenamiento vectorial y arquitectura de interfaz) se encuentren perfectamente integradas con las fases operativas de ingeniería de datos y modelado.

6.3 Diseño de Base de Datos

Se ha implementado un sistema de persistencia híbrido que combina el almacenamiento relacional para datos estructurados y el almacenamiento vectorial para el manejo de representaciones densas (embeddings).

6.3.1 Modelo Relacional (Diagrama Entidad-Relación)

El repositorio relacional gestiona la persistencia de metadatos y registros históricos mediante el motor **PostgreSQL**. La arquitectura del modelo permite que cada **DOCENTE** administre de forma independiente sus entidades de **PROBLEMA**. A su vez, cada problema se vincula con múltiples instancias de **CODIGO_FUENTE**, permitiendo diferenciar entre las entregas de los alumnos y las soluciones de referencia generadas o recolectadas.

La lógica central de la auditoría se consolida en la tabla **REPORTE_ANALISIS**. Esta entidad posee una relación dual con el código fuente para contrastar una entrega específica frente a una referencia seleccionada, almacenando los índices de similitud lógica y probabilidad sintética resultantes. Finalmente, el sistema registra hallazgos técnicos en la entidad **INDICADOR_INTEGRIDAD** cuando se detectan anomalías en la estructura lógica o patrones estilométricos que requieren la atención del docente.

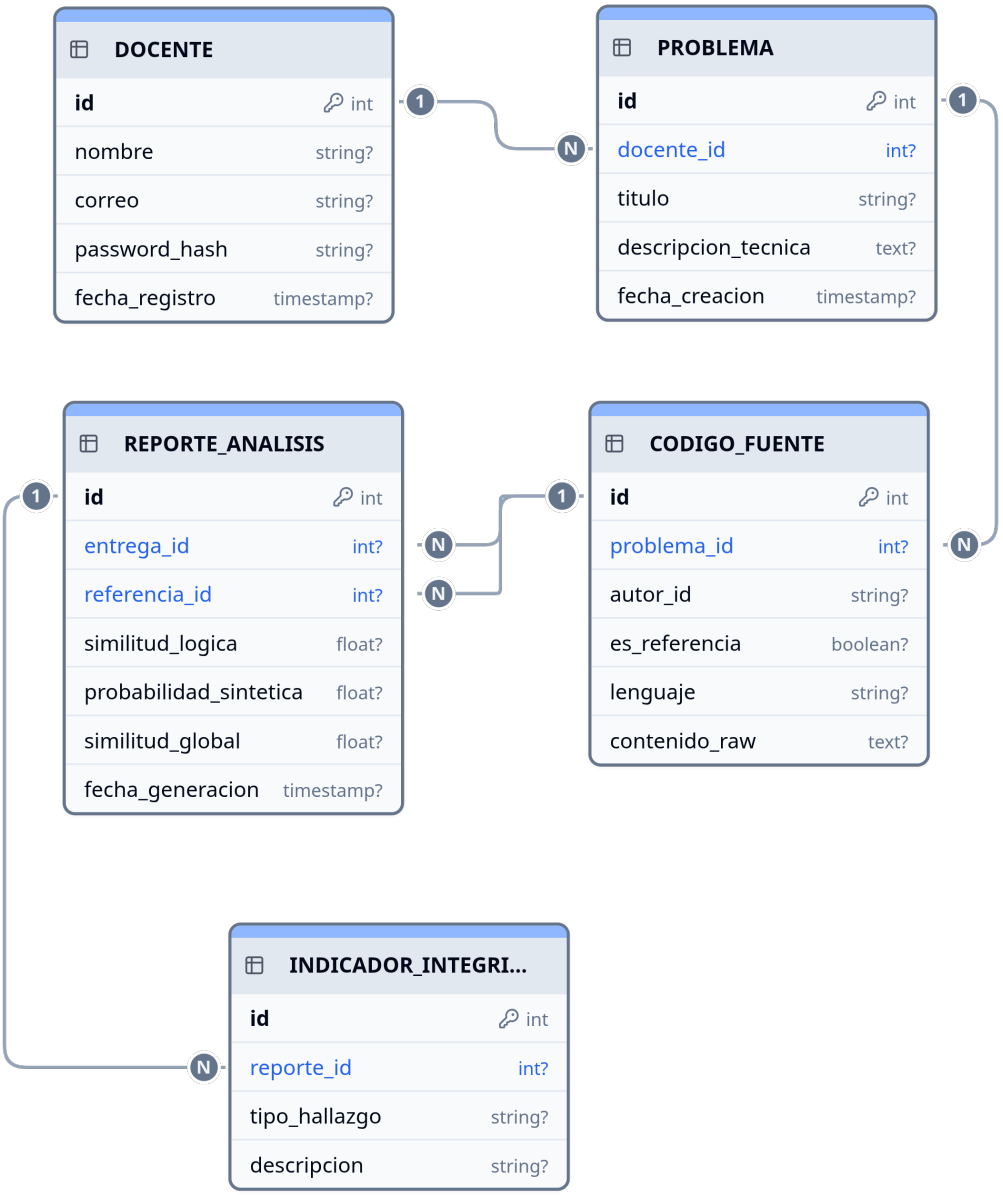


Figura 6.8: Modelo Entidad-Relación de la base de datos relacional (PostgreSQL).

6.3.2 Esquema de Colección Vectorial (ChromaDB)

Para la gestión de las representaciones densas y la ejecución de búsquedas por proximidad semántica, el sistema implementa una capa de persistencia heurística mediante **ChromaDB**. A diferencia del modelo relacional, esta base de datos está optimizada para el almacenamiento de tensores y el cálculo de distancias en espacios de alta dimensionalidad.

Como se muestra en la Figura 6.9, el diseño de la colección se mantiene bajo una estructura plana (*flat schema*) para minimizar la latencia durante la fase de recuperación de información.

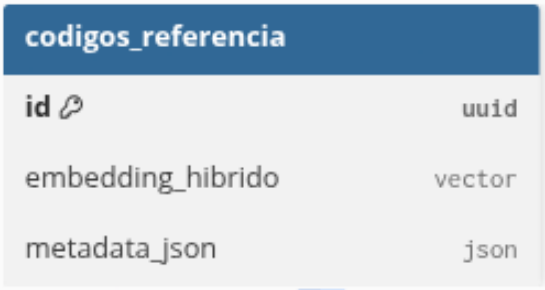


Figura 6.9: Esquema conceptual de la colección vectorial en ChromaDB.

La colección `codigos_referencia` se articula bajo tres pilares fundamentales que garantizan la integridad del análisis:

1. **Identificador Único (UUID):** Se utiliza el mismo identificador generado en la base de datos relacional (`CODIGO_FUENTE.id`) para actuar como puente lógico, permitiendo al orquestador reconstruir la información del alumno tras una coincidencia vectorial.
2. **Embedding Híbrido (769 Dimensiones):** Es el componente central del sistema. Este vector concatena dos señales distintas:
   - **Canal Semántico (0-767):** Dimensiones que capturan la lógica y el flujo de datos del código fuente.
   - **Canal Estilométrico (768):** Un valor escalar que representa la probabilidad de origen sintético, permitiendo que la búsqueda por similitud considere tanto la funcionalidad como la textura de la redacción.
3. **Metadatos JSON:** Almacenan atributos clave como el `problema_id` y el `lenguaje`. Esto permite que el sistema realice un pre-filtrado lógico, asegurando que la búsqueda de vecinos más cercanos se ejecute únicamente sobre el conjunto de soluciones de referencia que pertenecen al mismo ejercicio académico.

Cabe precisar que las 768 dimensiones semánticas constituyen una representación distribuida y no características interpretables de forma independiente, y que la concatenación con un único valor estilométrico no garantiza una contribución equilibrada de ambas señales. Estas consideraciones, junto con el efecto del relleno con ceros en las referencias, se discuten en la Sección 4.1.5.

Esta arquitectura híbrida asegura que **Graphito** no solo sea preciso en la identificación de similitudes, sino también eficiente en términos computacionales al segmentar el espacio de búsqueda.

6.4 Diseño de Interfaz de Usuario (UI/UX)

El diseño de la interfaz de **Graphito** se fundamenta en principios de usabilidad para entornos técnicos, buscando que la interpretación de métricas de IA sea inmediata para el docente. A continuación, se documenta la maquetación completa del sistema.

6.4.1 Módulo de Acceso y Registro

Este módulo garantiza la privacidad de los datos y el aislamiento de los ejercicios por docente.

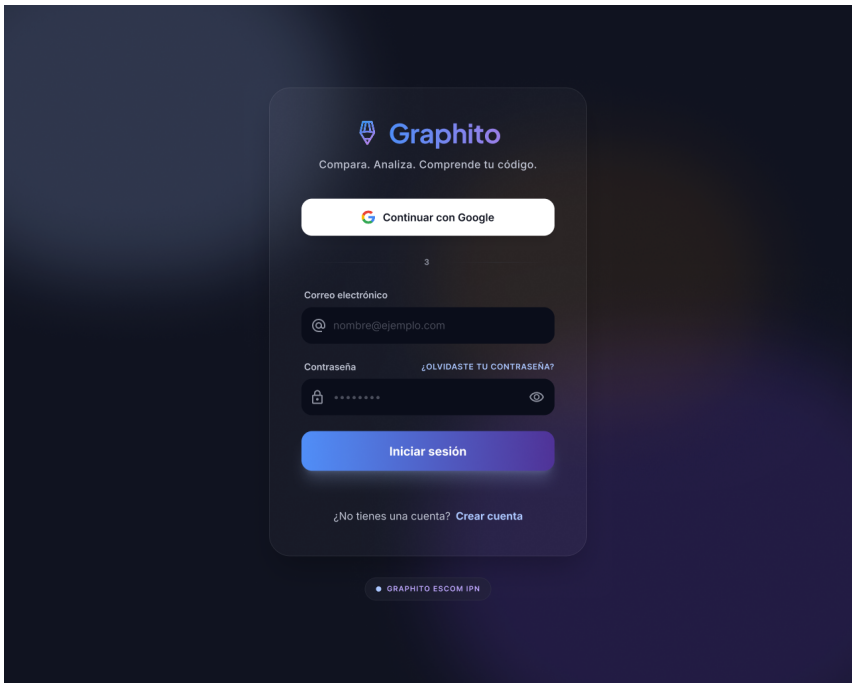


Figura 6.10: Interfaz de inicio de sesión (Cumplimiento de RF-01).

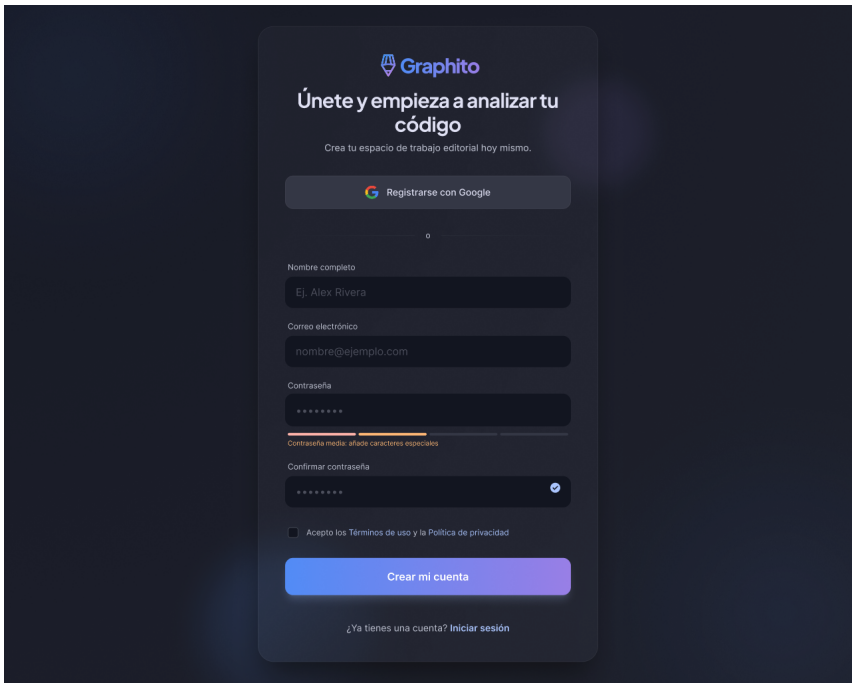


Figura 6.11: Interfaz de registro de nuevos usuarios.

6.4.2 Gestión de Repositorio y Biblioteca

El sistema permite organizar el material didáctico y las soluciones de referencia de forma jerárquica.

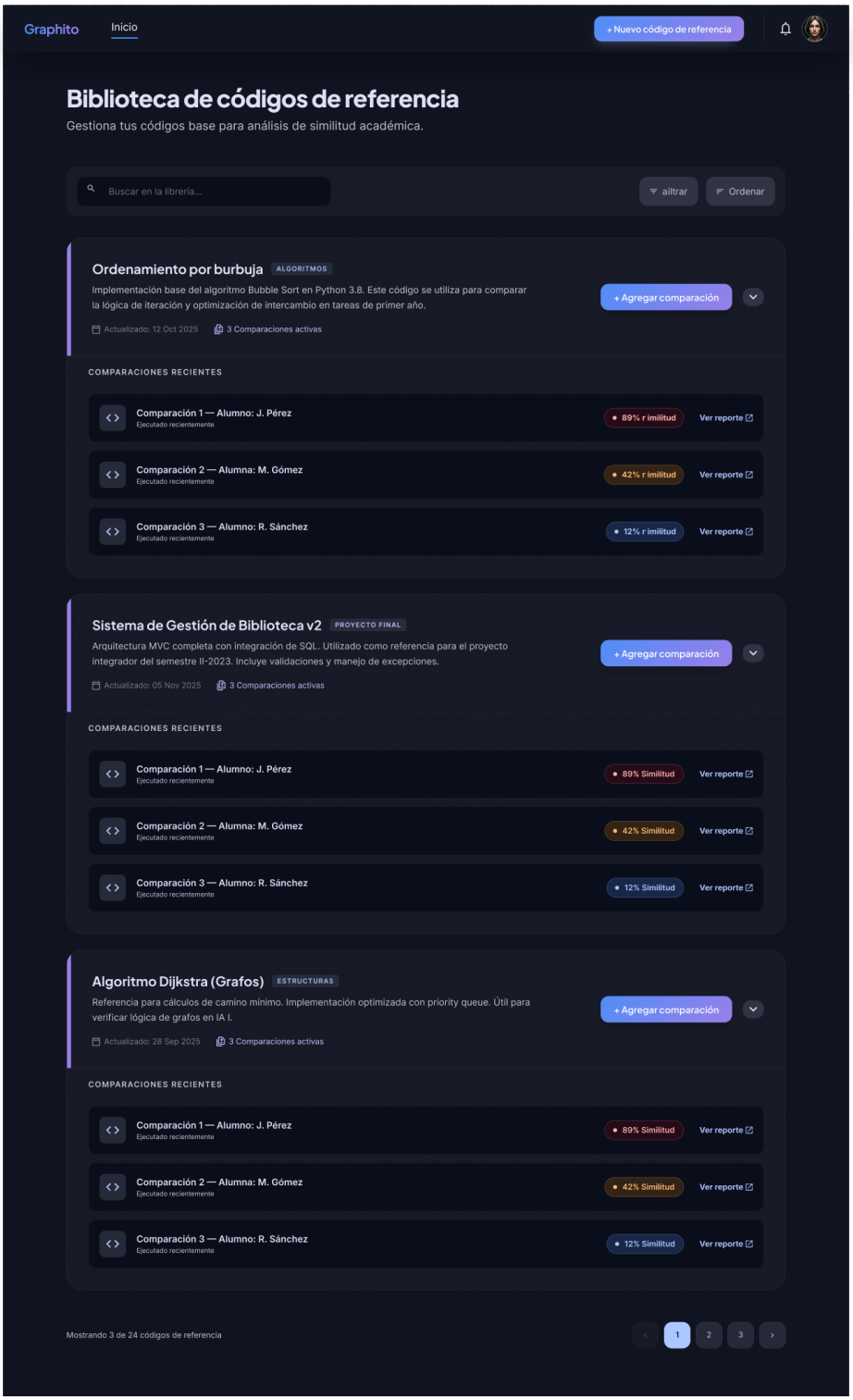


Figura 6.12: Repositorio de problemas y gestión de ejercicios (Cumplimiento de RF-02).

6.4.3 Módulo de Carga y Configuración de Análisis

Esta sección permite al usuario alimentar al sistema con las fuentes necesarias para la comparación semántica y estilométrica.

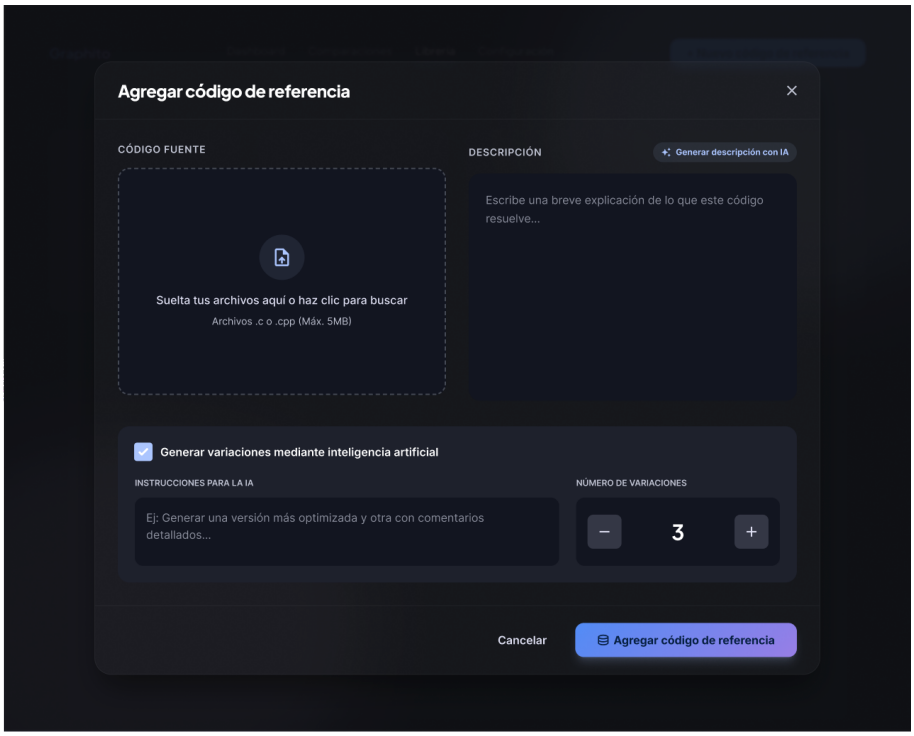


Figura 6.13: Interfaz de carga de código de referencia (Cumplimiento de RF-03).

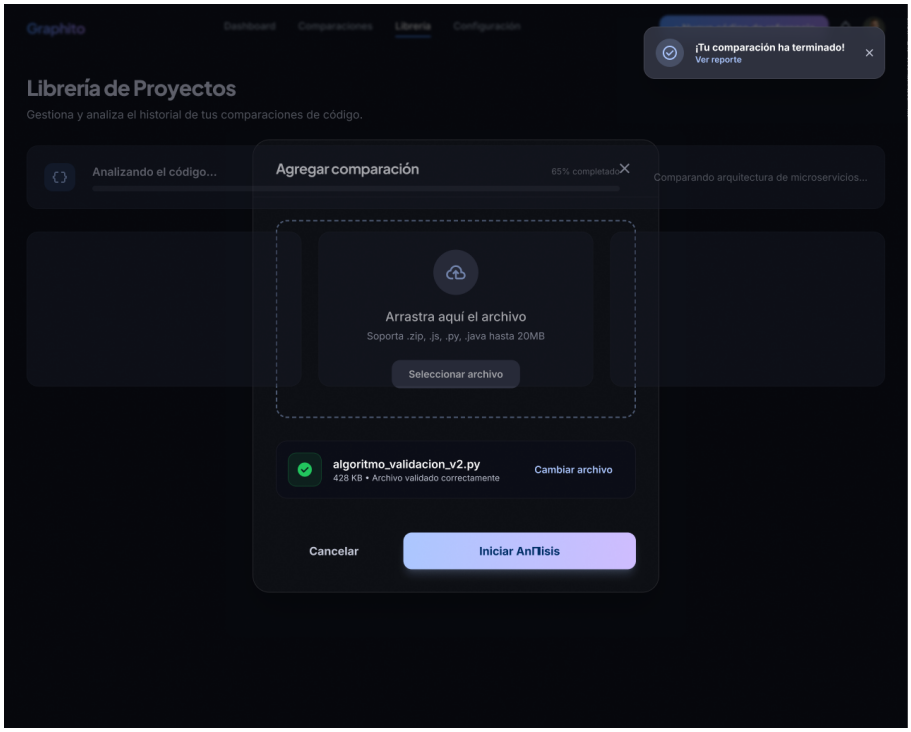


Figura 6.14: Configuración de parámetros para el inicio del análisis.

6.4.4 Visualización de Reportes de Doble Canal

Es el componente integrador donde se despliegan los indicadores generados por los modelos de aprendizaje profundo.



Figura 6.15: Dashboard de resultados híbridos (Cumplimiento de RF-10 y RF-11).

Como se observa en la Figura 6.15, el sistema cumple con el **RF-10** al presentar de manera unificada la similitud lógica y la probabilidad sintética. Asimismo, integra el **RF-11** mediante la visualización de indicadores de integridad que facilitan la toma de decisiones pedagógicas.

6.5 Estimación de costos del proyecto

La estimación de costos de Graphito permite identificar y cuantificar los recursos humanos, tecnológicos y materiales necesarios para el desarrollo, implementación y evaluación del sistema. Esta estimación considera las actividades realizadas durante el proyecto, las herramientas de software utilizadas, los recursos computacionales requeridos y los servicios externos relacionados con su funcionamiento.

El presupuesto se organiza en costos de recursos humanos, infraestructura tecnológica, herramientas de software y servicios, así como costos indirectos. Para mantener la transparencia de la estimación, cada importe se determina a partir de información verificable, como registros de horas de trabajo, cotizaciones, tarifas publicadas, comprobantes de pago o registros de consumo, según corresponda.

Asimismo, se distingue entre los costos que representan un desembolso económico real y los

costos imputados, que corresponden al valor estimado de recursos existentes utilizados durante el desarrollo del proyecto. Esta distinción permite presentar una estimación económica acorde con las condiciones reales en las que se desarrolló Graphito.

6.5.1 Costos de recursos humanos

El desarrollo de Graphito comprende actividades de análisis, diseño, programación, integración de componentes, preparación de datos, experimentación y documentación. Estas actividades representan un esfuerzo de trabajo que se registra mediante las horas dedicadas por cada perfil participante.

La estimación de los costos de recursos humanos se presenta en la Tabla 6.1, considerando las actividades realizadas, las horas de trabajo correspondientes y la tarifa utilizada para la valoración económica de cada actividad. La tarifa se determina a partir de una referencia documentada o de un costo real, cuando existe una contratación asociada. En los casos en los que el trabajo es realizado por los integrantes del proyecto como parte de su formación académica, las horas se registran como esfuerzo de desarrollo y no se consideran automáticamente un desembolso económico.

Tabla 6.1: Estimación de costos de recursos humanos de Graphito.

Actividad	**Perfil responsable**	**Horas dedicadas**	**Tarifa por hora (MXN)**	**Fuente de la tarifa**	**Costo estimado (MXN)**
Análisis de requerimientos	[Completar]	[Completar]	[Completar]	[Completar]	[Completar]
Diseño de la arquitectura del sistema	[Completar]	[Completar]	[Completar]	[Completar]	[Completar]

Actividad	Perfil responsable	Horas dedicadas	Tarifa por hora (MXN)	Fuente de la tarifa	Costo estimado (MXN)
Desarrollo del backend y frontend	[Completar]	[Completar]	[Completar]	[Completar]	[Completar]
Preparación y procesamiento de datos	[Completar]	[Completar]	[Completar]	[Completar]	[Completar]
Implementación de GraphCodeBERT y CharCNN	[Completar]	[Completar]	[Completar]	[Completar]	[Completar]
Entrenamiento, pruebas y evaluación de modelos	[Completar]	[Completar]	[Completar]	[Completar]	[Completar]

Actividad	Perfil responsable	Horas dedicadas	Tarifa por hora (MXN)	Fuente de la tarifa	Costo estimado (MXN)
Integración y generación de reportes	[Completar]	[Completar]	[Completar]	[Completar]	[Completar]
Documentación técnica y académica	[Completar]	[Completar]	[Completar]	[Completar]	[Completar]
Total		[Completar]			[Completar]

La Tabla 6.1 permite identificar el esfuerzo de trabajo requerido para desarrollar Graphito y establecer su valoración económica a partir de referencias justificadas. El costo total de recursos humanos corresponde a la suma de las actividades cuya valoración monetaria se encuentra documentada. Las horas dedicadas al proyecto también se consideran un indicador del esfuerzo académico y técnico invertido, independientemente de que representen o no un desembolso económico.

6.5.2 Costos de infraestructura tecnológica

La infraestructura tecnológica comprende los equipos de cómputo y los recursos de procesamiento, almacenamiento y conectividad empleados durante el desarrollo y evaluación de Graphito. Estos recursos permiten ejecutar los módulos de procesamiento de código fuente, realizar los experimentos con los modelos de aprendizaje automático y almacenar la información necesaria para el funcionamiento del sistema.

La Tabla 6.2 presenta los recursos tecnológicos utilizados, sus características, la modalidad de uso y el costo asociado. Para los equipos que ya se encontraban disponibles, se distingue entre su adquisición previa y el costo atribuible a su utilización durante el proyecto. Cuando no existe un desembolso adicional por el uso de un recurso, este se registra de acuerdo con su condición real, sin atribuirle un costo de adquisición inexistente.

Tabla 6.2: Estimación de costos de infraestructura tecnológica.

Recurso tecnológico	Características o especificaciones	Cantidad	Modalidad de uso	Periodo de utilización	Costo unitario (MXN)	Costo estima
Equipo de cómputo para desarrollo	[Procesador, memoria RAM, almacenamiento y otros datos]	[Completar]	[Propio, institucional o rentado]	[Completar]	[Completar]	[Compl
Equipo o recurso para experimentación con modelos	[Especificaciones]	[Completar]	[Local, institucional o en la nube]	[Completar]	[Completar]	[Compl
Almacenamiento de datos	[Capacidad y características]	[Completar]	[Completar]	[Completar]	[Completar]	[Compl

Recurso tecnológico	Características o especificaciones	Cantidad	Modalidad de uso	Periodo de utilización	Costo unitario (MXN)	Costo estima
Servicio de procesamiento en la nube	[Proveedor y características, si aplica]	[Completar]	[Completar]	[Completar]	[Completar]	[Compl
Otros recursos tecnológicos	[Especificar]	[Completar]	[Completar]	[Completar]	[Completar]	[Compl
Total						[Compl

El costo de infraestructura tecnológica representa el valor de los recursos empleados para desarrollar y evaluar Graphito. Su cálculo considera las condiciones de disponibilidad y utilización de cada recurso, evitando contabilizar como gasto nuevo aquellos equipos o servicios que ya forman parte de la infraestructura existente. En los casos en los que se utilizan servicios de pago, el importe se determina a partir de las tarifas correspondientes al periodo y al consumo registrado o estimado.

6.5.3 Costos de herramientas de software y servicios externos

Graphito requiere herramientas de desarrollo, bibliotecas de aprendizaje automático, componentes de procesamiento de lenguaje natural y mecanismos de almacenamiento para implementar sus funcionalidades. Entre los componentes tecnológicos considerados se encuentran Python, las bibliotecas para redes neuronales, GraphCodeBERT, el modelo CharCNN, PostgreSQL y ChromaDB, además de las herramientas complementarias utilizadas durante el desarrollo.

La Tabla 6.3 presenta las herramientas y los servicios relacionados con la implementación del sistema, especificando su función, modalidad de acceso y costo. Se distingue entre el software de uso gratuito o de código abierto, las herramientas cubiertas por licencias institucionales y los servicios externos que generan un cargo por suscripción, procesamiento o consumo.

Tabla 6.3: Estimación de costos de herramientas de software y servicios externos.

Herramienta o servicio	**Función dentro de Graphito**	**Modalidad de acceso o licencia**	**Unidad de cálculo**	**Cantidad o consumo**	**Costo unitario (MXN)**	**Costo e**
Python	Lenguaje de programación utilizado en el desarrollo	[Completar]	[Completar]	[Completar]	[Completar]	[C
Bibliotecas de aprendizaje automático	Implementación y entrenamiento de modelos	[Completar]	[Completar]	[Completar]	[Completar]	[C

Herramienta o servicio	Función dentro de Graphito	Modalidad de acceso o licencia	Unidad de cálculo	Cantidad o consumo	Costo unitario (MXN)	Costo e
GraphCodeBERT	Extracción de representaciones semánticas del código fuente	[Completar]	[Completar]	[Completar]	[Completar]	[C
Framework para CharCNN	Procesamiento de secuencias de caracteres y extracción de patrones estilométricos	[Completar]	[Completar]	[Completar]	[Completar]	[C

Herramienta o servicio	Función dentro de Graphito	Modalidad de acceso o licencia	Unidad de cálculo	Cantidad o consumo	Costo unitario (MXN)	Costo e
PostgreSQL	Almacenamiento de información estructurada	[Completar]	[Completar]	[Completar]	[Completar]	[C
ChromaDB	Almacenamiento y consulta de representaciones vectoriales	[Completar]	[Completar]	[Completar]	[Completar]	[C

Herramienta o servicio	Función dentro de Graphito	Modalidad de acceso o licencia	Unidad de cálculo	Cantidad o consumo	Costo unitario (MXN)	Costo e
Herramientas de desarrollo y control de versiones	Edición de código, integración y administración del proyecto	[Completar]	[Completar]	[Completar]	[Completar]	[C
Servicio de modelo de lenguaje externo, si aplica	Generación o transformación de variantes de código	[Completar]	[Completar]	[Completar]	[Completar]	[C
Otros servicios o herramientas	[Especificar]	[Completar]	[Completar]	[Completar]	[Completar]	[C
Total						[C

El costo total de software y servicios externos corresponde a los importes asociados con las herramientas efectivamente utilizadas durante el desarrollo de Graphito. Las herramientas gratuitas o de código abierto se registran conforme a sus condiciones de licencia, mientras que los

servicios de pago se cuantifican de acuerdo con su tarifa y consumo. En el caso de los servicios de modelos de lenguaje externos, el importe considera el uso realizado durante las pruebas y la generación de variantes, cuando esta funcionalidad forma parte de la implementación evaluada.

6.5.4 Costos indirectos

Los costos indirectos corresponden a los recursos auxiliares que intervienen en el desarrollo de Graphito, pero que no se asignan exclusivamente a una actividad específica. Entre ellos se encuentran el consumo de energía eléctrica, la conectividad a internet, las copias de seguridad y otros recursos necesarios para mantener las condiciones de trabajo.

La Tabla 6.4 presenta los conceptos indirectos considerados en la estimación. La asignación económica se basa en información disponible sobre el costo del servicio, el periodo de utilización y la proporción atribuible al proyecto. Cuando no existe información suficiente para determinar una proporción verificable, el concepto se identifica sin asignarle un importe arbitrario.

Tabla 6.4: Estimación de costos indirectos del proyecto.

Concepto	**Descripción**	**Periodo considerado**	**Costo total de referencia (MXN)**	**Proporción atribuible a Graphito**
Energía eléctrica	Consumo asociado al uso de equipos de cómputo	[Completar]	[Completar]	[Completar]
Conectividad a internet	Acceso a servicios y recursos en línea	[Completar]	[Completar]	[Completar]

Concepto	Descripción	Período considerado	Costo total de referencia (MXN)	Proporción atribuible a Graphito
Almacenamiento y copias de seguridad	Respaldo de código, datos y resultados experimentales	[Completar]	[Completar]	[Completar]
Otros costos indirectos	[Especificar]	[Completar]	[Completar]	[Completar]
Total				

La estimación de costos indirectos complementa el presupuesto de Graphito mediante la identificación de los recursos auxiliares utilizados durante su desarrollo. Los importes asignados corresponden únicamente a la parte justificable de cada concepto, con el propósito de evitar la sobreestimación del presupuesto y mantener la relación entre los costos reportados y los recursos realmente utilizados.

6.5.5 Concentrado general de costos

El concentrado general integra los resultados obtenidos en las estimaciones de recursos humanos, infraestructura tecnológica, herramientas de software, servicios externos y costos indirectos. Esta integración permite identificar el costo total del proyecto y distinguir los desembolsos económicos reales del valor imputado a los recursos existentes utilizados durante su desarrollo.

La Tabla 6.5 presenta el resumen presupuestal de Graphito. Los importes se incorporan a partir de los resultados de las tablas anteriores, manteniendo los mismos criterios de cálculo para evitar duplicidades en la contabilización de los recursos.

Tabla 6.5: Concentrado general de costos de Graphito.

Categoría	Costos directos (MXN)	Costos imputados (MXN)	Costo total por categoría (MXN)	Observaciones
Recursos humanos	[Completar]	[Completar]	[Completar]	[Completar]
Infraestructura tecnológica	[Completar]	[Completar]	[Completar]	[Completar]
Herramientas de software y servicios externos	[Completar]	[Completar]	[Completar]	[Completar]
Costos indirectos	[Completar]	[Completar]	[Completar]	[Completar]
Total general	[Completar]	[Completar]	[Completar]	

El costo total estimado de Graphito asciende a $[Completar] MXN, de los cuales $[Completar] MXN corresponden a costos directos y $[Completar] MXN a costos imputados. Esta distribución permite diferenciar el desembolso económico asociado con el proyecto del valor atribuido a los recursos disponibles que fueron utilizados durante su desarrollo.

El resultado del concentrado general constituye la base para analizar la distribución de los recursos económicos y determinar cuáles categorías representan la mayor proporción del costo total. Asimismo, proporciona una referencia para valorar los recursos necesarios para reproducir los experimentos y mantener el funcionamiento del sistema en un entorno de uso académico.

6.5.6 Análisis de viabilidad económica

La viabilidad económica de Graphito se analiza a partir de los recursos necesarios para su desarrollo, implementación y evaluación, considerando tanto los costos directos como los recursos existentes aprovechados durante el proyecto. La utilización de herramientas de código abierto y de componentes tecnológicos disponibles reduce la necesidad de adquirir licencias comerciales, mientras que el aprovechamiento de infraestructura existente limita los desembolsos adicionales asociados con el procesamiento y almacenamiento de información.

La estimación económica permite identificar los recursos que generan costos durante el desarrollo y aquellos que podrían representar gastos adicionales en una implementación futura. En particular, el uso de servicios externos de procesamiento o modelos de lenguaje queda sujeto a las condiciones de licencia, las tarifas vigentes y el volumen de solicitudes realizadas.

En consecuencia, el presupuesto de Graphito proporciona una referencia para dimensionar los recursos requeridos para su desarrollo y evaluar las condiciones económicas de su posible implementación en un entorno académico. La viabilidad del sistema se determina considerando la disponibilidad de infraestructura, el acceso a las herramientas tecnológicas y la relación entre los recursos invertidos y las funcionalidades implementadas.

Capítulo 7

Ingeniería de Datos

Un modelo de aprendizaje automático se parece a un estudiante que prepara un examen: aprende de los ejemplos que se le muestran. Si esos ejemplos son pocos, están sesgados o repiten las preguntas del examen, el resultado final puede parecer bueno sin serlo. Por eso, antes de describir los modelos, este capítulo explica de dónde salen los datos de Graphito y cómo se prepararon para que la evaluación sea justa.

Siguiendo la metodología **Komorebi** para proyectos de Inteligencia Artificial [55], el capítulo cubre dos fases. La **Adquisición de Datos** (Sección 7.1) responde a tres preguntas: qué código se necesitaba, cómo se obtuvo la contraparte generada por inteligencia artificial (IA) sobre los mismos problemas y cuántos ejemplos de IA bastan. La **Preparación de Datos** (Sección 7.2) explica cómo se limpiaron los archivos, cómo se retiraron los duplicados, cómo se dividió el conjunto sin fuga de información y cómo se adaptó el texto para cada uno de los dos canales de análisis. La Figura 7.1 resume el recorrido completo.

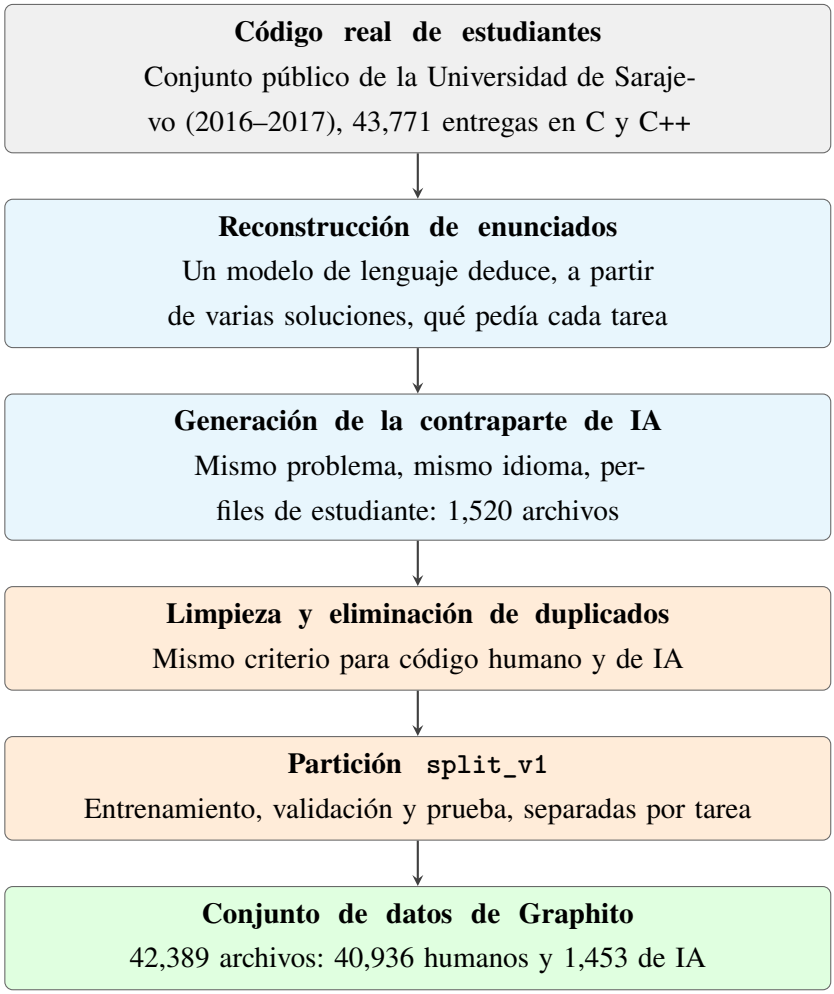


Figura 7.1: Flujo general de adquisición y preparación de los datos de Graphito, desde el código real de estudiantes hasta el conjunto particionado.

7.1 Adquisición de Datos

El objetivo fue reunir dos grupos de programas: los escritos por estudiantes y los generados por IA. La condición clave es que ambos grupos resuelvan los mismos problemas. Así, las dos clases difieren en su **origen** y no en el tema, de modo que un modelo no puede distinguirlas solo por el problema que resuelven.

7.1.1 Requerimientos del Conjunto de Datos

Para que los datos fueran útiles en las unidades de aprendizaje de programación introductorias de la Escuela Superior de Cómputo (ESCOM-IPN), se definieron dos condiciones:

1. **Autoría humana verificable.** Tomar código reciente de repositorios abiertos como GitHub tiene dos riesgos. Primero, Lopes et al. [56] encontraron que más del 70 % del código en GitHub son copias o variantes de otros archivos. Segundo, como advierten Zhou et al. [57], los modelos generativos actuales se entrenaron con esos repositorios y es posible

que parte del código "humano" reciente ya contenga algún origen de IA. Si el código etiquetado como humano no lo es, el clasificador aprende fronteras difusas. Por ello se buscó un conjunto de códigos generado antes de la popularización de los modelos de lenguaje, a finales de 2022.

2. **Nivel académico introductorio.** El proposito de Graphito es apoyar la revisión de código en unidades de aprendizaje como Fundamentos de Programación y Estructuras de Datos. Se buscó código típico de principiantes: variables sencillas, estructuras de control básicas (`if`, `for`, `while`), funciones, arreglos y punteros elementales, con los errores de formato habituales.

7.1.2 Selección del Dataset Base: IEEE Programming Homework Dataset

Se seleccionó el conjunto *Programming Homework Dataset for Plagiarism Detection*, publicado por Ljubovic y Pajic en IEEE Dataport [58]. Fue recolectado en la Facultad de Ingeniería Eléctrica de la **Universidad de Sarajevo** (Bosnia y Herzegovina) y ofrece tres ventajas:

- **Fecha de recolección (2016–2017).** Es anterior a los modelos de lenguaje modernos, por lo que el código fue escrito por personas.
- **Casos revisados por docentes.** Incluye registros de casos de plagio, algunos contrastados mediante entrevistas orales (archivo `ground-truth-dynamic-anon.txt`).
- **Registros de actividad del IDE.** Contiene archivos JSON con el tiempo invertido, el número de compilaciones y de pruebas por estudiante.

El conjunto abarca cuatro cursos, resumidas en la Tabla 7.1. En total se registraron 43,771 entregas humanas, de las cuales 40,936 sobrevivieron a la limpieza descrita en la Sección 7.2.

Tabla 7.1: Estructura del conjunto de datos IEEE Plagiarism.

Aspecto	**Detalle**
Cursos evaluados	A2016 (C), A2017 (C), B2016 (C++), B2017 (C++)
Lenguajes	C y C++
Estudiantes	607 a 1,001 identificadores distintos por curso
Tareas	89 tareas por curso, que se agrupan en 48 bloques al unir los años 2016 y 2017 (Sección 7.2.3)
Entregas humanas originales	43,771
Registros de desarrollo	Archivos JSON con eventos de edición, compilación y pruebas
Anonimización	Nombres sustituidos por identificadores como `student{id}`

7.1.3 Limitaciones del Dataset

Al explorar los archivos se identificaron tres limitaciones:

1. **Código en serbocroata.** Los estudiantes escribieron variables, mensajes y comentarios en serbocroata con caracteres latinos (por ejemplo, `unos` para entrada, `broj` para número o `niz` para arreglo). Si el código de IA se generara en inglés, un modelo podría aprender a distinguir idiomas en lugar de estilos de programación. Es como separar dos grupos de alumnos solo porque unos escriben en inglés y otros en español.

2. **Ausencia de enunciados.** Las carpetas contienen las soluciones, organizadas por tarea (por ejemplo, `A2016/Z1/Z1`), pero no la descripción de los problemas. Sin el enunciado no era posible pedir a una IA que resolviera el mismo ejercicio.

3. **Ausencia de soluciones oficiales del docente.** Tampoco se incluye una solución de referencia por tarea. Esta carencia se retoma en el Capítulo 8, donde se explica cómo se compensa en la evaluación sin referencias oficiales.

7.1.4 Reconstrucción de Enunciados

Para recuperar los enunciados se aplicó una especie de ingeniería inversa: a partir de varias soluciones correctas de una misma tarea, un modelo de lenguaje deduce qué se pedía. La Figura 7.2 ilustra la idea. Es como inferir la pregunta de un examen leyendo varias respuestas bien resueltas.

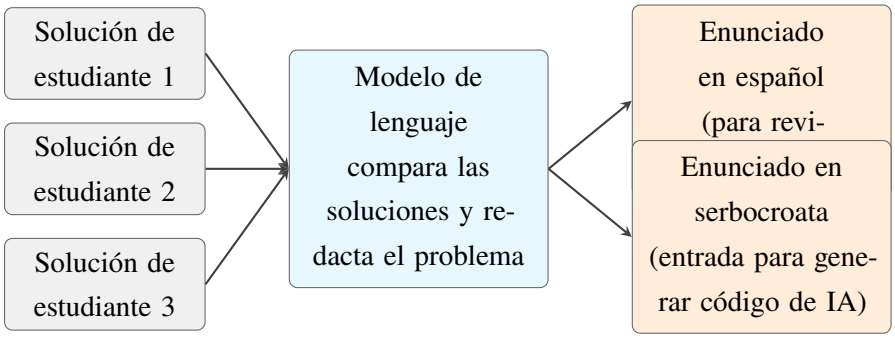


Figura 7.2: Reconstrucción de enunciados: de varias soluciones correctas de una misma tarea a un enunciado en dos idiomas.

El procedimiento fue el siguiente:

1. **Muestreo por tarea.** Para cada tarea se tomaron de 2 a 3 códigos de estudiantes distintos que compilaban y pasaban las pruebas del curso.

2. **Análisis con un modelo de lenguaje.** Se envió ese grupo de códigos a un modelo de lenguaje (por defecto, *GPT-4o-mini*) con la instrucción de comparar las soluciones y redactar el enunciado, con sus reglas de entrada y salida.

3. **Dos versiones del catálogo.** Una versión en español, para que docentes y evaluadores comprendan cada ejercicio, y otra en serbocroata, usada como entrada para generar el código de IA y mantener la paridad de idioma con el código original.

4. **Puntos de control.** El proceso guarda su avance para retomarse ante fallas de red o límites de la API.[1]

La Tabla 7.2 muestra ejemplos de los enunciados obtenidos.

Tabla 7.2: Ejemplos de enunciados recuperados por reconstrucción.

Curso	Asig.	Prob.	Leng.	Enunciado Deducido
A2016	Z1	Z1	C	Calcular la suma de puntos de tres estudiantes (Tarik, Bojan y Mirza) en una materia con cinco rubros: primer parcial (máx. 20 pts), segundo parcial (máx. 20 pts), asistencia (máx. 10 pts), tareas (máx. 10 pts) y examen final (máx. 40 pts). Validar que los datos estén en los rangos correctos y mostrar error si algún valor no es válido.
A2016	Z1	Z2	C	Con los coeficientes de dos rectas en la forma $y = a_1x + b_1$ y $y = a_2x + b_2$, indicar si son paralelas, coincidentes o secantes. Si se cruzan, calcular el punto (x, y) donde se cortan.
A2016	Z1	Z3	C	Leer una serie de letras que representan colores de autos (`c`: negro, `b`: blanco, `s`: gris, `v`: rojo, `p`: azul), terminando con la letra `'k'`. Calcular el total de autos, el porcentaje de cada color y determinar el color más frecuente.
A2016	Z1	Z4	C	Pedir un número entero n ($0 < n \leq 50$) y mostrar en pantalla una figura con forma de letra 'W' hecha con asteriscos, con una altura de n filas.

7.1.5 Generación de la Contraparte de IA

Con los enunciados disponibles, se pidió a modelos de lenguaje que resolvieran cada tarea como lo haría un estudiante. En total se generaron 1,520 archivos, de los cuales 1,453 permanecieron tras la limpieza.[2] Se tomaron tres decisiones para evitar atajos.

7.1.5.1 Control del Sesgo de Idioma

Las instrucciones se redactaron en serbocroata y pedían que variables, funciones y mensajes también lo estuvieran (nombres como `broj`, `zbir` o `unos`). Así, el idioma deja de ser una pista para distinguir las clases.

[1] Implementado en `data/extraer_enunciados.py`.
[2] Implementado en `data/generar_referencias.py`.

7.1.5.2 Perfiles de Estudiante y Cuota de Comentarios

Las IA suelen escribir comentarios extensos por defecto, mientras que muchos estudiantes no comentan. Se observó que alrededor del 38 % de los códigos humanos del conjunto base incluía comentarios. Para no dejar la presencia de comentarios como pista fácil, se definieron seis perfiles de generación que se turnan, resumidos en la Tabla 7.3. En el conjunto final, los perfiles con comentarios suman 433 de los 1,453 archivos (cerca del 30 %), algo por debajo de la cuota objetivo.

Tabla 7.3: Perfiles de simulación de estudiantes usados en la generación de código de IA.

Perfil	**Temp. (T)**	**Comportamiento Estilístico**
`descuidado`	0.85	Estudiante promedio: variables cortas (`i, j, temp, a, b`), código simple y sin comentarios.
`aplicado`	0.40	Estudiante dedicado: variables descriptivas (`brojacKaraktera, sumaBodova`), código ordenado y sin comentarios.
`kompaktan`	0.70	Estudiante que escribe poco: nombres mínimos, código muy directo y sin comentarios.
`comentado-bueno`	0.40	Estudiante sobresaliente: comentarios detallados en serbocroata y código ordenado.
`comentado-basic`	0.70	Estudiante promedio: algún comentario breve, estilo informal.
`comentado-malo`	0.85	Estudiante apresurado: comentarios obvios agregados a última hora y errores ocasionales.

La temperatura (T) es un parámetro del modelo de lenguaje: valores altos producen respuestas más variadas y valores bajos, más conservadoras.

7.1.5.3 Variedad de Generadores

El código de IA no proviene de un solo modelo, pero sí está muy concentrado. De los 1,453 archivos finales, 1,021 provienen de *DeepSeek-Chat* y 424 de *Qwen3*; los 8 restantes provienen de otros cinco modelos (Codestral, Gemini Flash, Llama 4, Kimi y Mistral Small). Es decir, más del 99 % del código de IA viene de dos generadores. Esto es una limitación importante: aún no se sabe qué tan bien se reconocerá código de IA producido por otros modelos. Se retoma en los Capítulos 8 y 10.

7.1.6 Ejemplo Comparativo: Código Humano frente a Código de IA

Los Listados 7.1 y 7.2 muestran dos soluciones en C del problema A2016/Z1/Z1 (cálculo de notas de estudiantes). Ambas resuelven el mismo problema y están en serbocroata.

Listing 7.1: Código humano: estudiante real del curso A2016.

```
#include <stdio.h>

int main() {
    float p1, p2, prisustvo, zadace, zavrsni;
    float ukupno;

    printf("Unesite bodove za Tarika: \n");
    printf("I parcijalni ispit: ");
    scanf("%f", &p1);
    if (p1 < 0 || p1 > 20) {
        printf("Neispravan broj bodova");
        return 0;
    }
    printf("II parcijalni ispit: ");
    scanf("%f", &p2);
    if (p2 < 0 || p2 > 20) {
        printf("Neispravan broj bodova");
        return 0;
    }
    printf("Prisustvo: ");
    scanf("%f", &prisustvo);
    if (prisustvo < 0 || prisustvo > 10) {
        printf("Neispravan broj bodova");
        return 0;
    }

    ukupno = p1 + p2 + prisustvo;
    return 0;
}
```

Listing 7.2: Código sintético: generado por IA en serbocroata (perfil aplicado).

```
#include <stdio.h>
#include <stdbool.h>

bool validirajUnos(float p1, float p2, float pris) {
    if (p1 < 0.0f || p1 > 20.0f) return false;
    if (p2 < 0.0f || p2 > 20.0f) return false;
    if (pris < 0.0f || pris > 10.0f) return false;
    return true;
}

int main(void) {
    float parcijalni1, parcijalni2, prisustvoBodovi;
    float ukupnaOcjena;

    printf("Unesite bodove za prvog studenta:\n");
    scanf("%f %f %f", &parcijalni1, &parcijalni2, &prisustvoBodovi);
```

```
    if (!validirajUnos(parcijalni1, parcijalni2, prisustvoBodovi)) {
        printf("Neispravan broj bodova\n");
        return 0;
    }

    ukupnaOcjena = parcijalni1 + parcijalni2 + prisustvoBodovi;
    return 0;
}
```

Con el mismo enunciado y las mismas condiciones, aparecen diferencias de estructura y de superficie. La Tabla 7.4 las resume en cinco dimensiones. Este ejemplo ilustra por qué Graphito usa dos canales que miran cosas distintas: uno la lógica del programa y otro la forma en que está escrito.

Tabla 7.4: Contraste entre una implementación humana y una de IA para el mismo problema.

Dimensión	**Implementación Humana**	**Implementación de IA**	**Qué sugiere para el análisis**
Modularidad	Todo dentro de `main`, con la lógica mezclada.	Función auxiliar (`validirajUnos`) que separa responsabilidades.	La lógica puede reorganizarse sin cambiar lo que el programa hace.
Identificadores	Nombres abreviados (`p1`, `p2`, `pris`).	Nombres descriptivos (`parcijalni1`, `prisustvoBodovi`).	Los nombres son una pista de estilo, pero también pueden cambiarse sin alterar la lógica.
Tipado y literales	Constantes enteras (`0`, `10`, `20`).	Constantes de punto flotante explícitas (`0.0f`, `20.0f`).	Es un rasgo visible a nivel de caracteres, que el canal de estilo puede captar.
Estilo y formato	Espaciado compacto y sangría irregular.	Formato regular y saltos de línea ordenados.	Si no se neutraliza, el formato puede ser un atajo (Sección 7.2.1).
Equivalencia lógica	Valida los rangos uno por uno.	Cubre los mismos requisitos y rangos.	Dos programas con lógica equivalente pueden verse muy distintos por fuera.

Como indica la Tabla 7.4, un solo rasgo no basta para valorar un código: la lógica de ambos programas converge, mientras que los hábitos de escritura son claramente distintos.

7.1.7 ¿Cuántos Ejemplos de IA Bastan? Curva de Aprendizaje

Generar código con modelos de lenguaje cuesta tiempo y dinero. La pregunta práctica fue: ¿conviene seguir generando ejemplos de IA o ya se alcanzó una zona de rendimientos decrecientes? Es como estudiar para un examen: los primeros ejemplos enseñan mucho, pero cada ejemplo parecido adicional enseña menos. La planeación inicial contemplaba un corpus de 20,000 códigos (10,000 por clase); se diseñó un experimento pre-registrado para decidir si era necesario llegar a esa meta.

Antes de describir el experimento conviene aclarar por qué la curva llega a 874 ejemplos de IA y no a los 1,520 que se generaron. La diferencia no se debe a que se descartara código de forma arbitraria, sino a tres pasos que se explican con detalle en la Sección 7.2 y que se resumen en la Tabla 7.5:

1. **Limpieza (1,520 → 1,453).** Se excluyeron 67 archivos: 55 generaciones fallidas o casi vacías y 12 duplicados que habrían quedado repartidos entre entrenamiento y prueba. Se aplicó el mismo criterio que al código humano.

2. **Partición (1,453 → 897).** Los 1,453 archivos se reparten por tarea entre entrenamiento (897), validación (225) y prueba (331). Solo el entrenamiento puede usarse para enseñar al modelo; la validación sirve para medir la curva y la prueba se reserva intacta para la evaluación final del Capítulo 8. Usar esos archivos aquí sería como dejar que el alumno vea las preguntas del examen.

3. **Duplicados internos (897 → 874).** Algunos de los 897 archivos de entrenamiento son prácticamente iguales entre sí. Cada familia de casi-copias cuenta como un solo ejemplo, porque repetir el mismo código no enseña nada nuevo. Quedan 874 ejemplos únicos.

Tabla 7.5: De los archivos de IA generados a los ejemplos usados en la curva de aprendizaje.

Etapa	**Archivos de IA**	**Motivo**
Generados	1,520	Total producido por los modelos de lenguaje.
Tras la limpieza	1,453	Se excluyen 67: generaciones fallidas o casi vacías (55) y duplicados entre particiones (12).
En entrenamiento	897	El resto (225 en validación y 331 en prueba) no se usa para entrenar.
Ejemplos únicos en entrenamiento	874	Las casi-copias cuentan como un solo ejemplo.

Así, cuando la curva habla del 100 % se refiere al 100 % de los ejemplos de IA *disponibles para entrenar* (874), no al total generado.

El experimento usó la red convolucional de caracteres *MultiScaleCharCNN* (Capítulo 8). Se mantuvo intacto el código humano de entrenamiento (10,688 ejemplos únicos, uno por familia de casi-copias, de los 24,810 archivos humanos de entrenamiento) y se varió la cantidad de código de IA de entrenamiento en 12 fracciones: 5, 10, 15, 20, 30, 40, 50, 60, 70, 80, 90 y 100 %. Eso corresponde a entre 55 y 874 grupos únicos de IA, tomados de los 897 archivos de IA de entrenamiento. Para controlar la variabilidad:

1. **Submuestras anidadas por grupo.** Los archivos se eligieron por grupo de duplicados (`dup_group`, Sección 7.2.2) y por tarea. Con una misma semilla, cada subconjunto contiene al anterior, así los cambios no se deben a un reparto distinto de los datos.

2. **Repeticiones.** Cada fracción se entrenó con 5 semillas independientes [59], es decir, 60 entrenamientos en total.

3. **Evaluación en validación.** Las métricas se calcularon en el conjunto de validación completo (6,808 archivos humanos y 225 de IA); el conjunto de prueba no se tocó. Como las clases están muy desbalanceadas, la métrica principal fue PR-AUC, elegida por el uso previsto (encontrar pocos casos de IA con pocas falsas alarmas) y no solo por el desbalance [60]; resume qué tan bien el modelo encuentra los casos de IA sin generar demasiadas falsas alarmas (1 es perfecto; vale más cuanto más cerca de 1).

La Figura 7.3 muestra la curva observada.

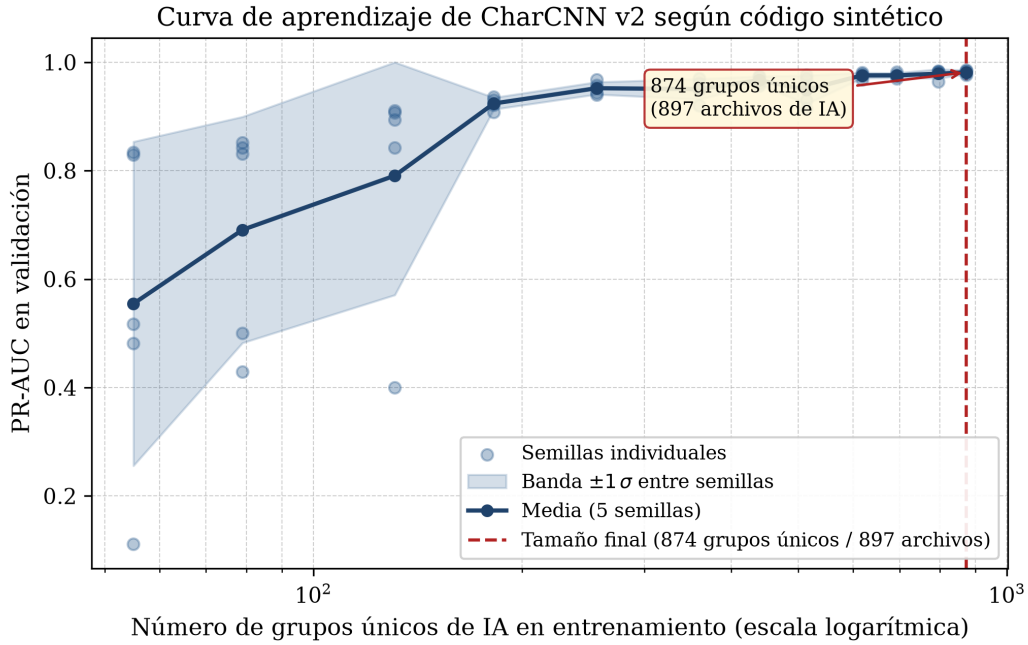


Figura 7.3: Curva de aprendizaje de MultiScaleCharCNN según la cantidad de grupos únicos de IA de entrenamiento. La línea sólida es la media de 5 semillas (banda sombreada: $\pm 1\,\sigma$) y los puntos semitransparentes son corridas individuales. El 100 % equivale a 874 grupos únicos, tomados de los 897 archivos de IA de entrenamiento.

El criterio de saturación pre-registrado consideraba la curva saturada si la ganancia media de PR-AUC entre el 50 % ($n = 439$) y el 100 % ($n = 874$) era menor que 0.01 y menor que la desviación estándar del 100 %. Los resultados fueron:

- Con el 5 % de los datos ($n = 55$), el resultado es muy inestable (PR-AUC = 0.5546 ± 0.2988).
- Con el 20 % ($n = 182$) y el 30 % ($n = 256$) el modelo se estabiliza: PR-AUC de 0.9240 ± 0.0108 y 0.9522 ± 0.0113.
- Entre el 50 % (0.9667 ± 0.0074) y el 100 % (0.9816 ± 0.0038), la ganancia es de 0.0148, con una desviación del 100 % de 0.0038.

Como $0.0148 > 0.01$, el criterio clasifica la curva como *aún no saturada en sentido estricto*: el modelo sigue mejorando un poco con más ejemplos. Aun así, el avance se vuelve pequeño. Del 30 % al 100 % se agregan 618 grupos y PR-AUC mejora solo 0.0294. La Figura 7.4 muestra la ganancia en cada paso: es grande al principio y pequeña después, lo que sugiere que triplicar los datos de IA aportaría poco.

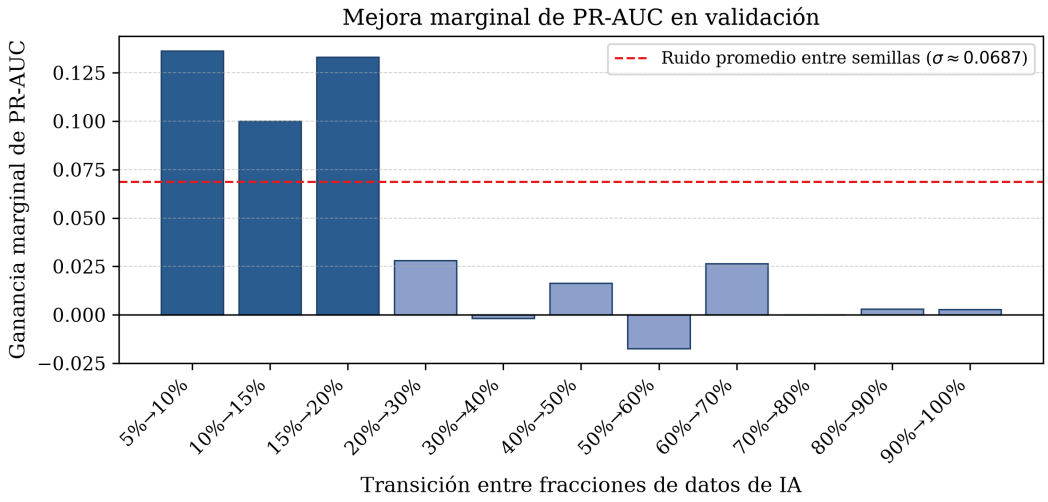


Figura 7.4: Mejora marginal de PR-AUC en validación entre fracciones consecutivas de datos de IA. Las tres primeras transiciones superan el ruido promedio entre semillas (línea punteada); las siguientes quedan por debajo de él.

Dos advertencias acompañan estos resultados. Primero, el conjunto de validación también se usó para decidir cuándo detener cada entrenamiento, así que los valores absolutos pueden ser algo optimistas; el sesgo afecta por igual a todos los puntos de la curva. Segundo, el experimento solo cambia la *cantidad* de código de un mismo generador. En este conjunto, el límite parece ser la **diversidad** de generadores y no el volumen. Con esta evidencia se decidió no seguir generando código de IA y conservar los 1,520 archivos ya generados (874 ejemplos únicos de entrenamiento, Tabla 7.5) en lugar de perseguir la meta planificada de 10,000; se remite la pregunta de la generalización a otros modelos a los Capítulos 8 y 10.

7.2 Preparación y Preprocesamiento de Datos

Antes de enseñar a los modelos, hay que asegurarse de que el examen sea justo. Esta sección describe cuatro pasos: limpiar los archivos, retirar duplicados, dividir el conjunto sin fuga de información y adaptar el texto a cada canal. La Figura 7.5 anticipa el efecto de los dos primeros pasos sobre el número de archivos.

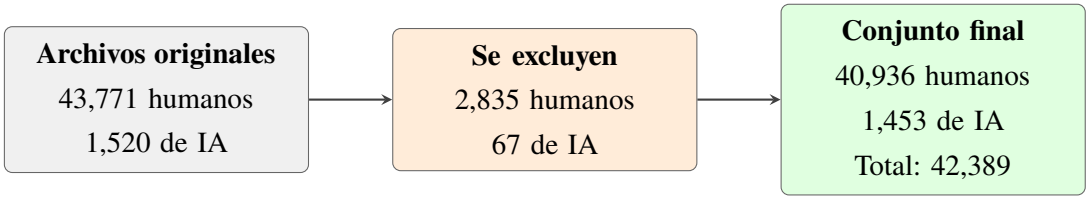


Figura 7.5: Efecto de la limpieza y de la eliminación de duplicados que cruzarían particiones sobre el número de archivos.

7.2.1 Limpieza Simétrica y Aprendizaje de Atajos

La limpieza se aplica con el mismo criterio al código humano y al de IA. Se descartan los archivos vacíos, los demasiado cortos (menos de 50 caracteres efectivos), los que son copias por nombre de archivo y las generaciones fallidas. En total se excluyeron 2,835 archivos humanos y 67 de IA, incluidos los duplicados que se describen en la Sección 7.2.2.

Además se eliminan las pistas que no tienen que ver con el estilo de programación. El ejemplo principal es la cabecera del curso: líneas como "Zadaca 6" o "Tutorijal 13" aparecían en cerca de la mitad de los archivos humanos de una muestra revisada y en casi ninguno de los de IA. Si se dejaran, un modelo podría clasificar mirando esa línea. Este fenómeno se conoce como *aprendizaje de atajos* (*shortcut learning*) [61]: es el alumno que aprueba reconociendo la letra del examen en lugar de entender la materia. Lo mismo ocurre con los comentarios y con la sangría, que dependen del editor que se use y no del autor. Por eso estos elementos se neutralizan para ambas clases, como se detalla en la Sección 7.2.4.1.

7.2.2 Eliminación de Duplicados

En cursos introductorios muchas entregas son idénticas o casi idénticas. Si una misma solución aparece en entrenamiento y en prueba, el modelo parece acertar cuando en realidad solo la recuerda. Se aplican dos detecciones:

- **Duplicados exactos.** Dos archivos que tienen la misma secuencia de elementos del programa, sin contar comentarios ni espacios, se consideran el mismo.
- **Duplicados casi exactos.** Dentro de cada tarea se comparan los archivos por fragmentos de cinco elementos consecutivos, ignorando los nombres de variables. Si comparten al menos el 85 % de sus fragmentos (similitud de Jaccard sobre fragmentos [62]), se agrupan, práctica habitual para evitar que los duplicados distorsionen la evaluación [63].[3]

Cada conjunto de archivos equivalentes forma un **grupo de duplicados** (`dup_group`), que es como una "familia" de entregas. Una familia nunca se reparte entre particiones. Si una familia aparece en tareas asignadas a particiones distintas, se conserva en la partición más reservada (prueba antes que validación y esta antes que entrenamiento) y se excluyen sus demás miembros. En total se excluyeron 236 archivos por esta causa (224 humanos y 12 de IA).

7.2.3 Partición `split_v1` para Evitar Fuga de Información

Dividir los archivos al azar es un error habitual. Allamanis [64] mostró que la duplicación entre entrenamiento y prueba puede inflar de forma artificial las métricas de modelos de código, por-

[3] La búsqueda emplea MinHash con bandas (LSH) [62] para evitarse comparar todos los pares de archivos (`models/evaluation/dedup.py`). Se midió la sensibilidad con los umbrales 0.80, 0.85 y 0.90.

que el modelo ya vio versiones casi idénticas de lo que se le pide evaluar. Esa situación se llama **fuga de información** [65]. Equivale a estudiar con las mismas preguntas del examen.

Para evitarla, la partición `split_v1` se hace por **tarea** y no por archivo (Figura 7.6):

- Una tarea se identifica por la letra del curso y los códigos `Zx/Zy`; así, las entregas de 2016 y de 2017 de una misma tarea forman un solo **bloque**. Se obtienen 48 bloques.
- Cada bloque, con todo su código humano y de IA, se asigna completo a entrenamiento (30 bloques), validación (8) o prueba (10).
- Las proporciones objetivo son 20 % de los bloques para prueba y 15 % para validación, por letra de curso, con semilla 42.
- El resultado se guarda en un archivo con huella digital (sha256) para que cualquiera pueda comprobar que se usa la misma partición.[4]

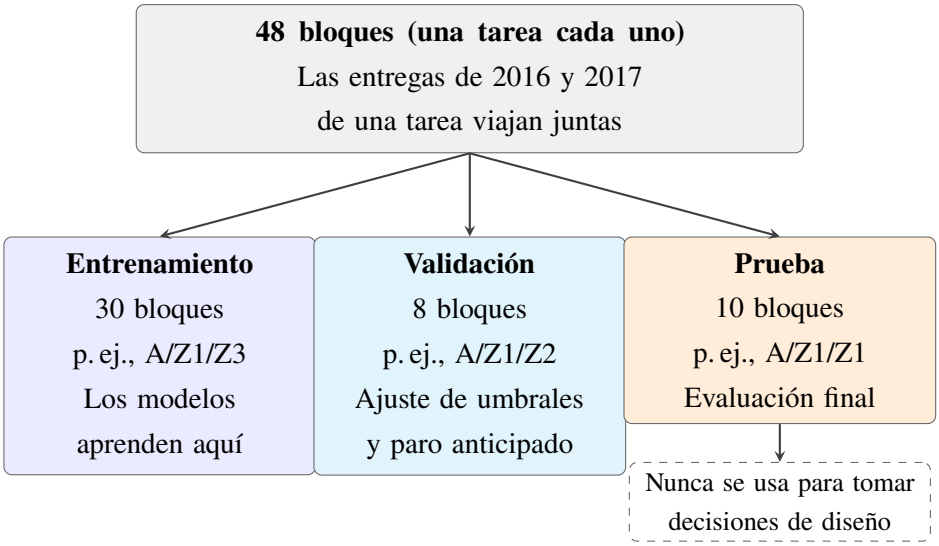


Figura 7.6: Esquema de la partición `split_v1`: cada tarea, con sus dos años y sus dos clases, pertenece a una sola partición.

La partición por tarea evita que el mismo problema aparezca en dos particiones, pero no impide que un mismo estudiante tenga entregas en varias. Esta limitación se tiene en cuenta al interpretar los resultados del Capítulo 8.

7.2.4 Preparación para Cada Canal

Graphito analiza cada archivo con dos canales que miran cosas distintas: el **Canal A** compara la similitud semántica (qué hace el programa) y el **Canal B** produce un indicador estilométrico (cómo está escrito). Cada uno lee el código a su manera, como dos lectores: uno busca la idea

[4] Construida con `scripts/build_split.py` y registrada en `data/splits/split_v1.json.gz`.

del texto y otro, la caligrafía. La Figura 7.7 muestra cómo se bifurca la preparación y la Tabla 7.6 compara ambos tratamientos.

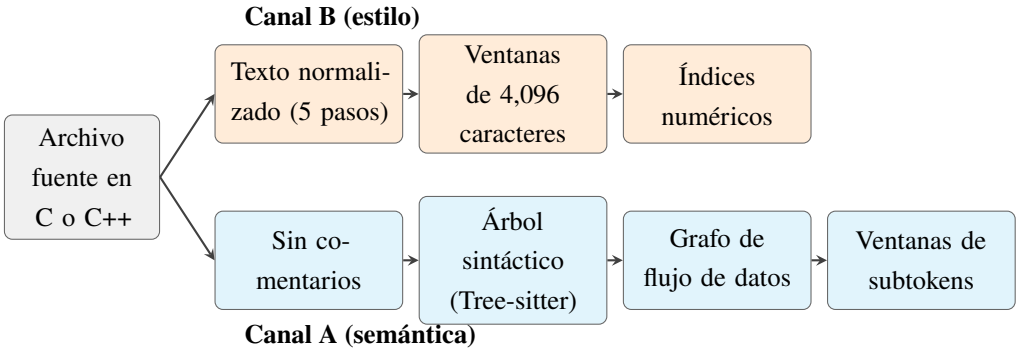


Figura 7.7: Bifurcación del preprocesamiento: un mismo archivo se prepara de forma distinta para el Canal B (texto de caracteres) y para el Canal A (estructura y flujo de datos).

Tabla 7.6: Tratamiento de cada componente del código en los dos canales.

Componente	**Canal A (GraphCodeBERT)**	**Canal B (CharCNN)**
Comentarios	Eliminados antes de analizar el código.	Eliminados, igual que en el Canal A.
Nombres de variables	Se conservan; los nombres se parten en subtokens y también se usan en el grafo de flujo de datos.	Se conservan carácter por carácter.
Espacios y sangría	No influyen: el analizador sintáctico lee elementos del programa.	Tabuladores a espacios, espacios repetidos reducidos a uno, sangría y líneas vacías eliminadas; se conservan los saltos de línea.
Representación final	Subtokens del código más nodos del grafo, hasta 512 posiciones por ventana (384 de código y 128 de grafo).	Índices numéricos de 98 posibles valores (relleno, desconocido y 96 caracteres).
Archivos largos	Ventanas de 382 subtokens que avanzan de 191 en 191, hasta 16 ventanas.	Ventanas de 4,096 caracteres que avanzan de 2,048 en 2,048, hasta 8 ventanas.
Objetivo	Capturar qué hace el programa y qué relaciones de datos contiene.	Capturar rasgos de escritura que no dependen de la lógica.

7.2.4.1 Canal B: Normalización Simétrica para CharCNN

El Canal B trata el código como una secuencia de caracteres. Antes de leerlo se aplican cinco pasos, iguales para código humano y de IA (de ahí el nombre “simétrica”):

1. Se unifican los saltos de línea y se retiran caracteres de control invisibles.

2. Se convierten a ASCII las comillas, guiones y espacios especiales, y las letras con diacríticos del serbocroata (por ejemplo, `č` pasa a `c`).

3. Se eliminan los comentarios.

4. Se eliminan las cabeceras y marcas del curso (Sección 7.2.1).

5. Se normaliza el espacio en blanco, como se indica en la Tabla 7.6.

El texto resultante se convierte en números. El alfabeto contiene el salto de línea y los 95 caracteres ASCII imprimibles; el valor 0 se reserva para el relleno (*padding*) y el 1 para cualquier carácter desconocido, de modo que no se confunden. Como la red necesita entradas de tamaño fijo, los archivos largos se leen por **ventanas** de 4,096 caracteres que se traslapan a la mitad (hasta 8). Es como leer un documento largo por páginas que repiten la mitad de la anterior. Los archivos cortos se completan con relleno.[5]

7.2.4.2 Canal A: Estructura y Flujo de Datos para GraphCodeBERT

El Canal A busca qué hace el programa, no cómo está escrito. Los pasos son:

1. **Eliminación de comentarios,** respetando el contenido de las cadenas de texto.

2. **Árbol sintáctico.** El analizador **Tree-sitter** [66] reconoce la estructura del programa (funciones, ciclos, condiciones). Tolera código incompleto o con errores locales, algo común en trabajos de estudiantes.

3. **Grafo de flujo de datos (DFG).** Es un esquema que indica "qué variable le pasa su valor a cuál". Se construye una sola vez sobre el archivo completo.

4. **Subtokens y ventanas.** El código se divide en subtokens, es decir, trozos de palabra. Si no cabe en una sola entrada, se corta en ventanas de 382 subtokens que se traslapan a la mitad, hasta 16 ventanas. Cada ventana conserva los nodos del grafo que quedan dentro de ella.

En esta versión los identificadores **no se renombran** (no se sustituyen por nombres genéricos). Por eso el Canal A es sensible a cambios de nombres de variables, un efecto que se mide en el Capítulo 8.[6]

[5] Implementado en `models/char_cnn/v2/preprocess.py` y `config.py`. Una comprobación adicional verifica que no sobreviva ninguna marca del curso ni caracteres fuera del alfabeto.

[6] Implementado en `models/graphcodebert/v2/`.

7.2.5 Composición Final del Conjunto

Tras la limpieza, la eliminación de duplicados y la partición, el conjunto contiene 42,389 archivos: 40,936 humanos y 1,453 de IA. La Tabla 7.7 muestra su reparto.

Tabla 7.7: Composición del conjunto de datos particionado por tarea (`split_v1`).

Subconjunto	**Humanos**	**IA**	**Total**	**Uso en el Proyecto**
Entrenamiento (*Train*)	24,810	897	25,707	Ajuste de pesos del modelo.
Validación (*Val*)	6,808	225	7,033	Ajuste de hiperparámetros, umbrales y paro anticipado.
Prueba (*Test*)	9,318	331	9,649	Evaluación final con tareas no vistas.
Total	**40,936**	**1,453**	**42,389**	**Conjunto consolidado.**

En el conjunto, solo cerca del 3 % de los archivos son de IA. Este desbalance no se corrigió eliminando código humano, sino que se maneja de forma distinta según el momento:

- **Durante el entrenamiento, los datos se balancean.** Si el modelo aprendiera con 97 ejemplos humanos por cada 3 de IA, le bastaría con responder casi siempre "humano" para acertar la mayoría de las veces, sin aprender a reconocer el código de IA. Para evitarlo, en cada época (cada pasada completa de entrenamiento) el modelo ve todos los ejemplos de IA de entrenamiento y la misma cantidad de códigos humanos, elegidos al azar. En la siguiente época se eligen otros códigos humanos. Así, cada época está balanceada (mitad y mitad) y, a lo largo del entrenamiento, el modelo recorre la variedad de los 10,688 ejemplos humanos únicos de entrenamiento (uno por familia de casi-copias) en lugar de descartarlos.
- **Durante la validación y la prueba, se conserva el desbalance.** Estos conjuntos no se modifican, porque deben parecerse a un aula real, donde la gran mayoría del código es de autoría humana. Medir con datos balanceados haría que el modelo pareciera mejor de lo que sería en la práctica. Por eso, en los capítulos siguientes, además de ROC-AUC, se reporta PR-AUC, que se enfoca en qué tan bien se encuentran los pocos casos de IA sin generar muchas falsas alarmas.

7.3 Conclusión del capítulo

Este capítulo describió cómo se construyó la base de datos de Graphito. Se partió de código humano anterior a la IA generativa, se reconstruyeron los enunciados que faltaban y se generó una contraparte de IA sobre los mismos problemas, en el mismo idioma y con perfiles de estudiante. Un experimento de curva de aprendizaje sugiere que más ejemplos del mismo generador aportarían poco, y que la limitación principal es la diversidad de generadores. Después se limpiaron los

archivos con el mismo criterio para ambas clases, se retiraron duplicados, se dividió el conjunto por tarea para evitar la fuga de información y se preparó el texto para cada canal. El resultado es un conjunto de 42,389 archivos, con muchos más archivos humanos que de IA, a propósito.

El Capítulo 8 usa estos datos para describir el Canal B (CharCNN entrenado sobre este conjunto), el Canal A (GraphCodeBERT aplicado sin entrenamiento adicional) y una línea base léxica, y presenta los resultados de su evaluación.

Capítulo 8

Modelización

Con los datos listos (Capítulo 7), la metodología **Komorebi** [55] pasa a la fase de **Modelización**: construir los modelos y ponerlos a prueba. La Sección 8.1 explica qué se construyó; la Sección 8.2 mide qué tan bien funciona y qué límites tiene.

Graphito usa dos "lectores" del código que miran cosas distintas, más una regla simple de comparación como vara de medir (Figura 8.1). El **Canal A** mide qué tan parecida es una entrega a la solución de referencia en lo que hace el programa. El **Canal B** estima si la forma de escribir se parece más a la de código generado por IA. La **línea base léxica** compara fragmentos de texto y no usa aprendizaje automático. Ninguna salida es un veredicto: son datos para que el docente decida qué revisar primero.

La pregunta guía del Canal A es: *¿qué tan semántica es la similitud?*, es decir, si el modelo capta qué hace el programa o solo cómo se ve. La evidencia indica que capta bien la estructura general, pero depende de los nombres de las variables y no supera a la línea base léxica. Se reporta tal cual: un resultado negativo bien medido también es un resultado.

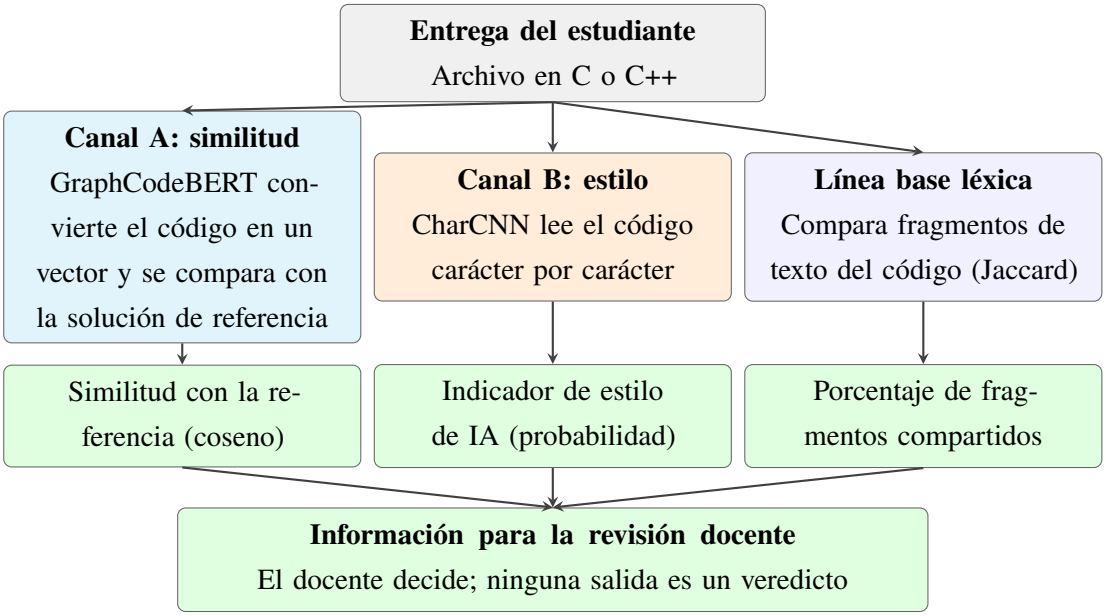


Figura 8.1: Panorama de los modelos de Graphito: dos canales y una línea base léxica, cuyas salidas orientan la revisión del docente.

8.1 Entrenamiento de Modelos

Esta sección describe cada componente en orden: el Canal B (CharCNN), el Canal A (GraphCodeBERT), la línea base léxica y la calibración, que convierte las puntuaciones crudas en umbrales interpretables. Solo el CharCNN se entrena en este proyecto; GraphCodeBERT se usa tal como viene preentrenado.

8.1.1 Canal B: CharCNN

8.1.1.1 Idea General

Una **red neuronal convolucional** sobre caracteres lee el texto del código como una secuencia de letras y símbolos. Funciona como un conjunto de lupas que se deslizan sobre el texto. Cada lupa se fija en un tamaño de fragmento y aprende a reconocer hábitos de escritura: cómo se abren las llaves, cómo se nombran las variables, qué palabras se repiten. El enfoque sigue a Zhang et al. [7] (clasificar texto leyendo caracteres, sin separar palabras) y a Kim [67] (usar varias lupas de tamaños distintos en paralelo). Las lupas de Graphito miran fragmentos de 3, 5, 7 y 9 caracteres. Las cortas captan detalles como operadores; las largas, trozos de palabras clave.

La Figura 8.2 muestra el recorrido de un archivo por la red. Cada bloque lleva una descripción en lenguaje llano.

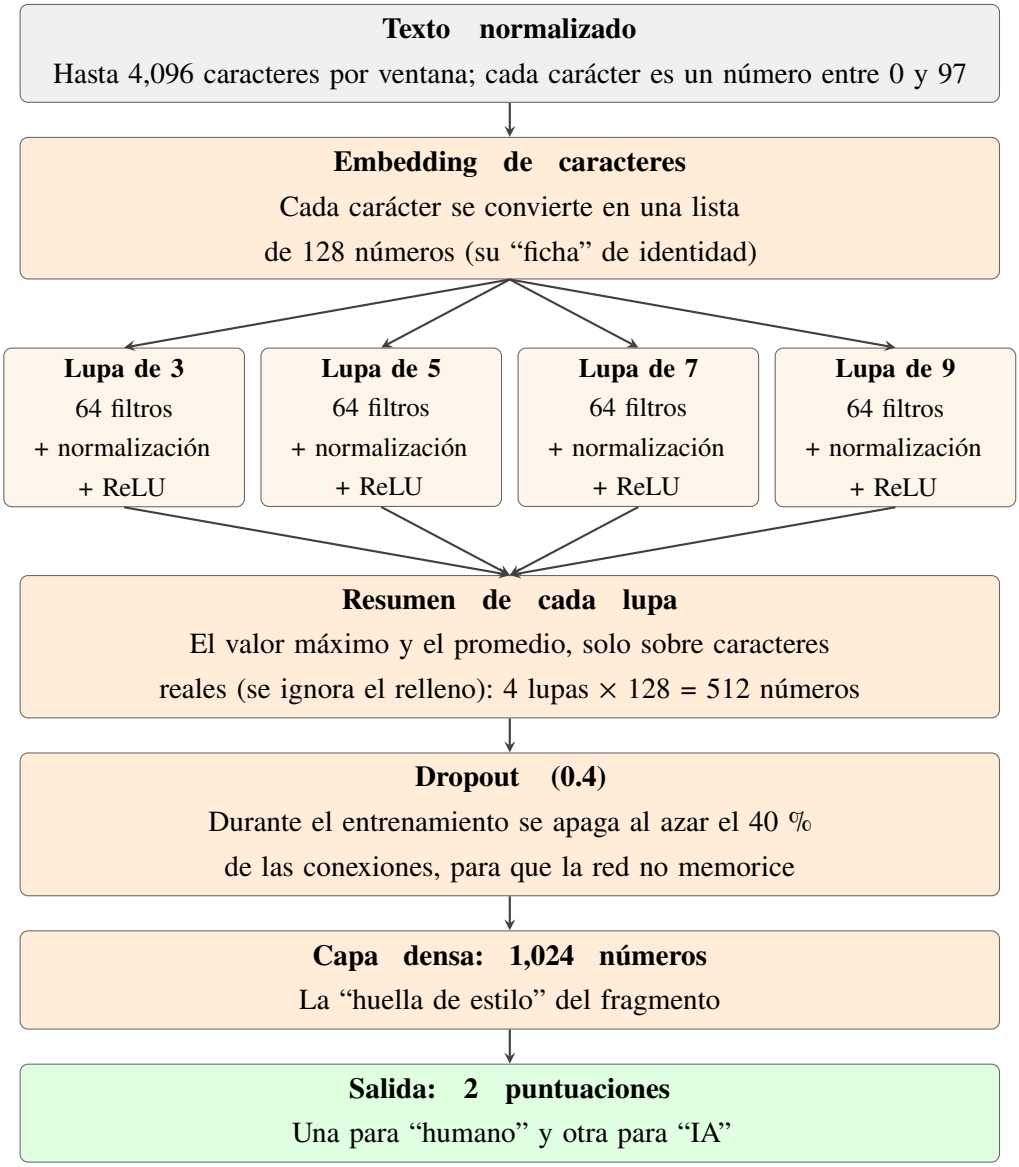


Figura 8.2: Arquitectura del CharCNN v2: del texto a una puntuación de estilo. Cada bloque indica en lenguaje llano qué hace.

Un **filtro** detecta un patrón concreto de caracteres (cada lupa tiene 64). La **normalización** estabiliza el entrenamiento y **ReLU** deja pasar solo los valores positivos. El *pooling* ("resumen") condensa lo que cada lupa encontró en dos números por filtro, sin contar el relleno de los archivos cortos.

8.1.1.2 Archivos Largos: Ventanas

La red recibe 4,096 caracteres a la vez. Los archivos más largos se leen por **ventanas** de 4,096 que avanzan de 2,048 en 2,048 (hasta 8), como leer por páginas que repiten la mitad de la anterior (Figura 8.3, con un archivo de 8,192 caracteres). Cada ventana produce dos puntuaciones; se promedian las de todas las ventanas y el promedio se convierte en una probabilidad. En el entrenamiento se usa un solo recorte aleatorio por archivo.

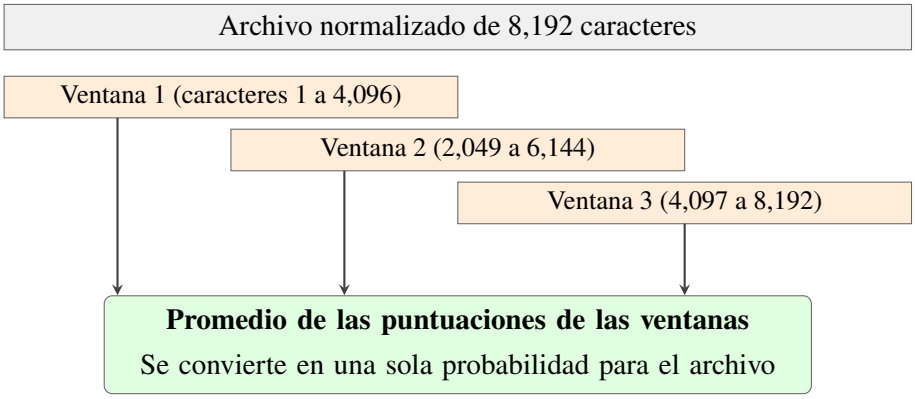


Figura 8.3: Lectura por ventanas con traslape: cada ventana se puntúa por separado y los resultados se promedian.

8.1.1.3 Hiperparámetros y Entrenamiento

Un **hiperparámetro** es un ajuste que se fija antes de entrenar, como el número de lupas o el ritmo de aprendizaje. La Tabla 8.1 agrupa los principales en cuatro familias.

Tabla 8.1: Organizador de hiperparámetros del CharCNN v2.

Parámetro	**Valor**	**Qué significa**
Entrada		
Vocabulario	98 símbolos	96 caracteres imprimibles, más relleno y “desconocido”.
Ventana y paso	4,096 y 2,048 caracteres (hasta 8 ventanas)	Cuánto texto lee la red de una vez y cuánto avanza en archivos largos.
Arquitectura		
Embedding	128 números por carácter	Tamaño de la “ficha” de cada carácter.
Lupas [67]	3, 5, 7 y 9; 64 filtros cada una	Fragmentos de distinto alcance: 256 detectores en total.
Dropout	0.4	Fracción de conexiones apagadas al azar al entrenar, para reducir la memorización.
Capa densa	1,024	Tamaño de la “huella de estilo”.
Entrenamiento		
Optimizador y tasa	Adam, 10^{-3}	Método que ajusta los pesos y tamaño de cada paso.
Reducción de la tasa	$\times 0.5$ si 3 épocas no mejoran	Pasos más finos al final; ocurrió en las épocas 20 y 35 (numeradas desde 0).
Tamaño de lote y épocas	128 y hasta 50	Ejemplos por ajuste y pasadas completas de entrenamiento.

Parámetro	Valor	Qué significa
Muestreo por época	1:1 (874 de IA y 874 humanos nuevos)	En cada época se sortean humanos distintos (Sección 7.2.5).
Semilla	42	Fija los sorteos aleatorios y hace repetible el experimento.
Selección		
Paro anticipado	7 épocas sin mejora	Se detiene si la validación no mejora.
Criterio de época	Máximo de (ROC-AUC, PR-AUC) en validación	Se conserva la mejor copia, no la última; la prueba no interviene.

Los datos de esta fase son la partición de entrenamiento del Capítulo 7: 10,688 archivos humanos y 874 de IA (uno por familia de casi-copias), con 3,077 de validación (2,868 humanos y 209 de IA). La Figura 8.4 resume una época.

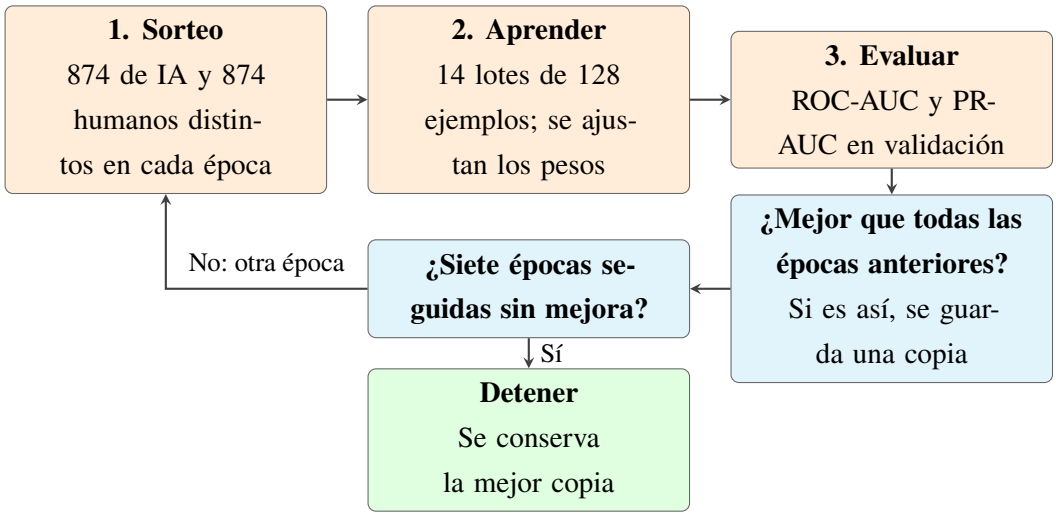


Figura 8.4: Una época de entrenamiento del CharCNN y la regla de paro anticipado.

La Figura 8.5 muestra la evolución real. A la izquierda, la pérdida de entrenamiento baja rápido y luego se estabiliza. A la derecha, ROC-AUC y PR-AUC en validación suben de forma pareja. El entrenamiento duró 38 épocas y se detuvo por paro anticipado. La época elegida es la 31.ª; la figura la marca como 30 porque numera desde 0. En ella la validación obtuvo ROC-AUC de 0.9991 y PR-AUC de 0.9890.

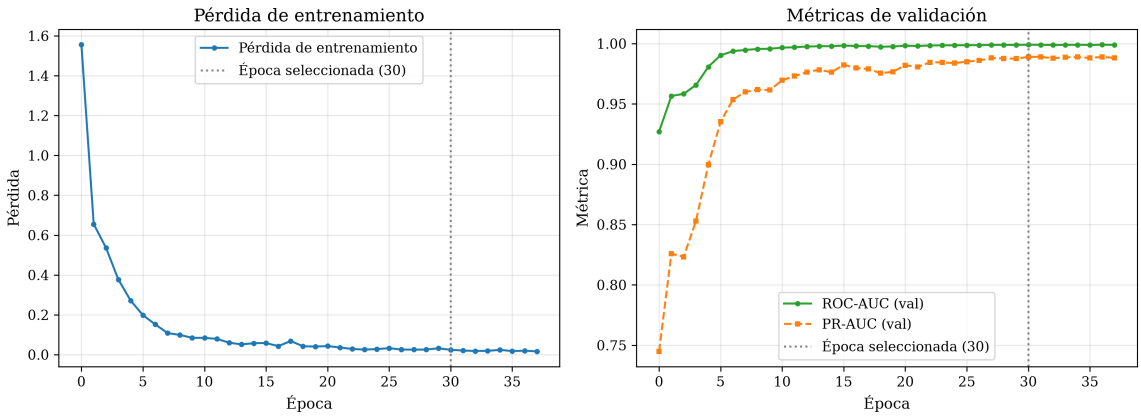


Figura 8.5: Curvas de entrenamiento del CharCNN v2: pérdida de entrenamiento (izquierda) y métricas de validación (derecha). La línea punteada marca la época seleccionada.

8.1.2 Canal A: GraphCodeBERT sin Entrenamiento Adicional

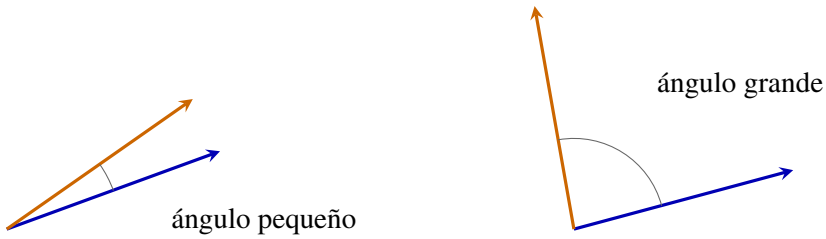
8.1.2.1 Idea General

GraphCodeBERT [5] es un modelo preentrenado con millones de programas. Es como un lector que ya revisó muchísimo código antes de llegar a Graphito. Su resultado para un fragmento de código es un **vector**: una lista de 768 números que resume qué hace el código. Dos códigos que hacen cosas parecidas deberían producir vectores que apunten en direcciones parecidas.

Para medir esa cercanía se usa la **similitud del coseno**. Imagine dos flechas que salen del mismo punto, como agujas de una brújula. Si apuntan casi hacia el mismo lado, el ángulo es pequeño y el coseno es cercano a 1. Si apuntan a lados distintos, el ángulo crece y el coseno baja (Figura 8.6). La fórmula es:

$$\text{coseno}(a, b) = \frac{a \cdot b}{\|a\| \, \|b\|} \tag{8.1}$$

Se lee así: se mide qué tan alineados están los vectores a y b, sin importar cuán largos sean. En la práctica, las similitudes de GraphCodeBERT sin entrenamiento adicional se concentran muy cerca de 1, incluso entre programas distintos. Esto limita cuánto se pueden separar los casos, como se verá en la calibración.



(a) Parecidos
coseno cercano a 1

(b) Distintos
coseno cercano a 0

Figura 8.6: Intuición de la similitud del coseno: cuanto menor es el ángulo entre dos vectores, más parecidos son los códigos que representan.

8.1.2.2 Cómo se Obtiene el Vector de un Archivo

La Figura 8.7 muestra el recorrido. Se eliminan los comentarios, el código se divide en **subtokens** (trozos de palabra) y se construye el **grafo de flujo de datos** (DFG), un esquema de "qué variable le pasa su valor a cuál". GraphCodeBERT recibe ambos. Los archivos largos se cortan en ventanas de 382 subtokens que se traslapan a la mitad; el vector de cada ventana es el promedio de sus subtokens de código, y el vector del archivo, el promedio de sus ventanas. El coseno entre el vector de la entrega y el de la referencia es la similitud que Graphito informa.

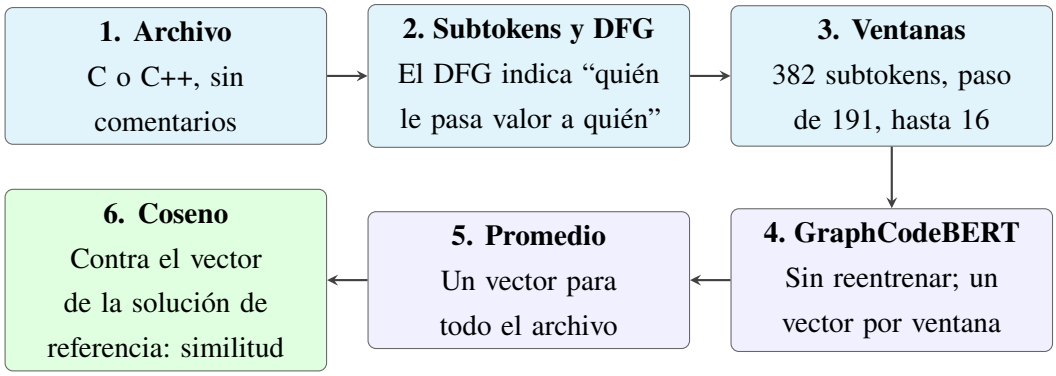


Figura 8.7: Cómo el Canal A convierte un archivo en un vector y lo compara con la solución de referencia.

La Tabla 8.2 reúne la configuración. Como el modelo no se entrena, los valores son solo de entrada y de resumen.

Tabla 8.2: Organizador de la configuración de GraphCodeBERT (Canal A).

Parámetro	**Valor**	**Qué significa**
Modelo		
Modelo base	`graphcodebert-base`	Modelo preentrenado de Microsoft [5].
Reentrenamiento	Ninguno (*zero-shot*)	Se usa tal como viene.
Entrada		
Código por ventana	382 subtokens (384 posiciones)	Dos posiciones son marcadores de inicio y fin.
Nodos del grafo	128	Completan las 512 posiciones que admite el modelo.
Paso y tope	191 subtokens; hasta 16 ventanas	Traslape de la mitad; cerca del 11 % de los archivos superó el tope y se truncó.
Resumen		
Pooling	*mean* (promedio)	Cómo se resume una ventana en un vector. Se eligió en validación: ROC-AUC de 0.865 contra 0.797 de la opción por defecto (*cls*).

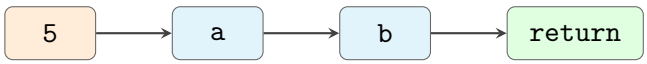
En los archivos evaluados, el analizador sintáctico reportó errores en cerca del 25 % (el código

de estudiantes suele estar incompleto; aun así se procesa) y cada archivo ocupó en promedio 4.7 ventanas.

8.1.2.3 Adaptación a C y C++

El código original de Microsoft construye el grafo de flujo de datos solo para lenguajes como Java. Se tradujo a C y C++ con Tree-sitter; en las construcciones que ambos lenguajes comparten, una prueba automática verifica que el resultado sea idéntico al original. La Figura 8.8 muestra el grafo de un programa mínimo.

```
int a = 5;
int b = a + 2;
return b;
```



Cada flecha indica “le pasa su valor a”

Figura 8.8: Grafo de flujo de datos de un programa mínimo: el valor 5 llega a `a`, de ahí a `b` y de ahí al `return`.

La Tabla 8.3 resume los ajustes que C y C++ requirieron.

Tabla 8.3: Ajustes del grafo de flujo de datos para C y C++.

Situación en el código	**Qué hace el grafo**
`int a[10];`	Define solo `a`, no el número 10.
Una función nueva	Las variables empiezan de cero en cada función.
Ciclos `do-while` y `switch`	Los recorre.
`x += y;`	`x` depende de sí misma y de `y`.
`int`, `char`	Son palabras reservadas, no variables.
Errores de sintaxis	Se omiten los nodos faltantes en lugar de descartar todo el grafo.

Los apuntadores tienen un límite. El grafo sigue las variables por su nombre, igual que el original de Microsoft: reconoce la sintaxis de apuntadores (`*`, `&`, `->`, arreglos, parámetros), pero no detecta cuándo dos nombres apuntan a los mismos datos (análisis de alias). La Tabla 8.4 lo ilustra con `int x = 5; int *p = &x; *p = 10; return x;`. El modelo, además, lee el texto completo del código, incluidos `*` y `&`.

Tabla 8.4: Lo que el grafo registra en un ejemplo con apuntadores.

Línea	**Qué registra el grafo**	**Resultado**
`int *p = &x;`	`p` recibe su valor de `x`.	Correcto
`*p = 10;`	Lo registra como un cambio en `p`, no en `x`.	Incompleto
`return x;`	Cree que `x` sigue valiendo 5.	Incorrecto

Conviene precisar qué se adaptó: la *entrada* (el grafo), no el modelo. GraphCodeBERT nunca vio C ni C++ durante su preentrenamiento (Sección 8.2.5).

8.1.2.4 Por Qué no se Entrena (*Zero-Shot*)

Zero-shot significa usar un modelo tal como viene. Se evaluó la alternativa, un adaptador **LoRA** [69] que reentrena solo una parte pequeña del modelo, pero con el protocolo anterior empeoró levemente el resultado (ROC-AUC de 0.646 a 0.630), así que no se usa. Lo que sí se corrigió fue la entrada: la versión actual ("v2") reconstruye el grafo y la entrada de forma fiel al original, y el ROC-AUC en pares de plagio pasó de 0.741 a 0.856 (Sección 8.2.3.1). Es una mejora de implementación, no de método; el recorrido está en la Sección 8.1.5.

8.1.3 Línea Base Léxica: Jaccard

Una línea base es una regla simple contra la cual se compara un modelo complejo. Si el modelo no la supera, su complejidad no se justifica por ese criterio. Herramientas conocidas como MOSS [24] y JPlag [26] también comparan fragmentos de texto del código, por lo que esta línea base es cercana a lo que muchos docentes ya usan.

El procedimiento tiene tres pasos, ilustrados en la Figura 8.9. Primero, el código se divide en **tokens** (elementos del programa) y se abstrae: los identificadores pasan a `ID`, los números a `NUM`, las cadenas a `STR` y los caracteres a `CHR`; las palabras clave (`for`, `if`, `return`) se conservan. Segundo, se toman todas las ventanas de 5 tokens consecutivos, llamadas *shingles*. Tercero, se comparan los conjuntos de shingles de dos archivos con la fórmula de Jaccard [62]:

$$J(A, B) = \frac{|A \cap B|}{|A \cup B|} \tag{8.2}$$

Se lee así: de todos los fragmentos distintos que aparecen en alguno de los dos archivos, qué fracción aparece en ambos. Vale 1 si son idénticos y 0 si no comparten ninguno. Como los nombres de variables se reemplazan por `ID`, la medida **no cambia** si un estudiante solo renombra variables; esto explica un resultado de la Sección 8.2.3.2. Como segunda referencia se usa *winnowing* [68] (fragmentos de 5 tokens y ventana de 4), una variante que elige una muestra representativa de los fragmentos para comparar de forma más eficiente.

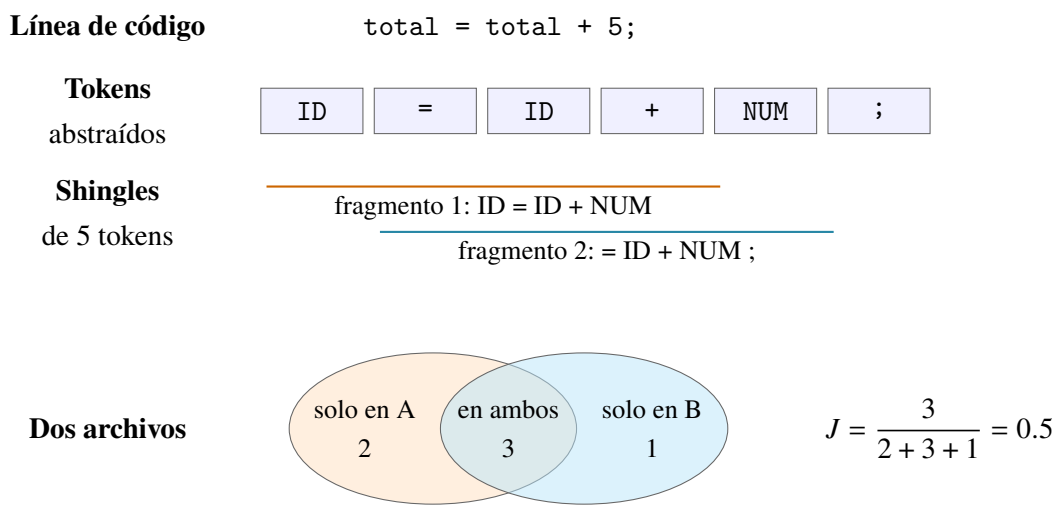


Figura 8.9: Línea base léxica. Arriba, una línea de código se abstrae y se divide en fragmentos de 5 tokens. Abajo, ejemplo con dos archivos de 5 y 4 fragmentos que comparten 3.

8.1.4 Calibración y Umbrales

Las puntuaciones crudas de un modelo no se interpretan solas: un 0.7 puede ser mucho en un modelo y poco en otro. La **calibración** convierte la puntuación en una escala interpretable usando datos de validación [70]. Después se fija un **umbral**: el valor a partir del cual se marca un caso. El umbral se elige según cuántas **falsas alarmas** se toleran, es decir, cuántos archivos humanos se marcarían por error. Graphito define dos puntos de operación: `fpr_0.05` (hasta 5 falsas alarmas por cada 100 archivos humanos) y `fpr_0.01` (hasta 1 por cada 100). El segundo es el que se despliega en el CharCNN. La Tabla 8.5 resume ambos canales.

Tabla 8.5: Organizador de calibración y umbrales, ajustados solo con datos de validación.

Canal	**Método**	**Ajustado en**	`fpr_0.05`	`fpr_0.01`	**Nota**
B (CharCNN)	Platt (curva en S)	2,868 humanos y 209 de IA de validación	0.0075	0.1715	Umbral desplegado: `fpr_0.01`.
A (GraphCodeBERT)	Isotónica (escalones crecientes)	Pares de validación: 174 de plagio y 1,595,101 sin plagio	0.00045	No alcanzable	Las similitudes se concentran cerca de 1, y ningún umbral logra 1 falsa alarma por cada 100.

La calibración de **Platt** [71] ajusta una curva en forma de S que transforma la puntuación en probabilidad. La **isotónica** [72] ajusta una función escalonada que solo puede crecer. En el Canal A el valor de `fpr_0.05` es una probabilidad calibrada de que un par sea de plagio; es baja porque estos pares son muy raros entre todos los pares posibles. Con ese umbral, el Canal A detectó cerca del 49 % de los pares anotados en validación, con 4 % de falsas alarmas.

8.1.5 Evolución de los Modelos: de la Primera Versión a la Final

Los modelos de esta sección son la versión final de un proceso de iteración. Las primeras versiones (septiembre de 2026) reportaron cifras muy altas; al evaluarlas con un protocolo más riguroso aparecieron fugas de información y errores de implementación. Las cifras de cada versión anterior se midieron con *su* protocolo, así que **no son comparables** con las de la Sección 8.2: es como comparar la nota de un examen cuyas preguntas el alumno ya conocía con la de uno que no vio antes.

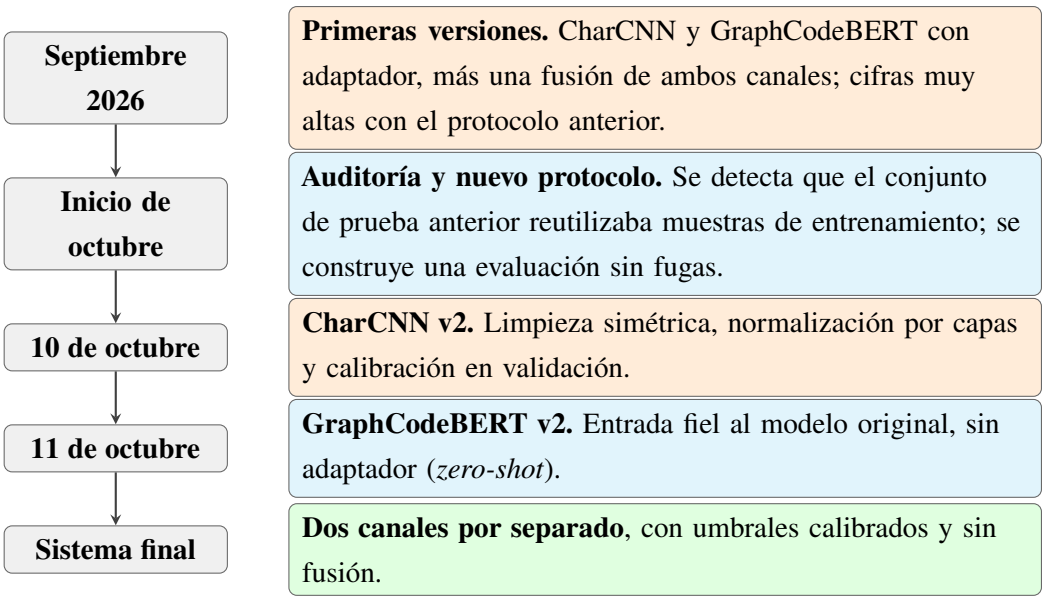


Figura 8.10: Línea de tiempo de la evolución de los modelos: de las primeras versiones al sistema final.

La Tabla 8.6 resume cada versión, su problema y su solución.

Tabla 8.6: Evolución de los modelos. Las cifras del protocolo anterior no son comparables con las del actual.

Canal	Versión	Resultado	Problema	Solución
Ambos	Evaluación anterior	Cifras muy altas (anterior).	El examen reutilizaba 1,825 de 2,621 muestras de entrenamiento.	Partición por tarea y calibración en validación.

Canal	Versión	Resultado	Problema	Solución
B	V1	ROC-AUC 0.999 y F1 0.992 (anterior).	Fugas de etiqueta y falsas alarmas por el formato del código.	Formato único; luego reconstrucción.
B	Formato canónico	ROC-AUC 0.996, F1 cercano a 0.88 (anterior).	Bajó la detección; no atacaba las causas.	Corregir las causas (V2).
B	V2 (final)	ROC-AUC 0.994 y F1 0.884 (actual).	Atajos pequeños y dos generadores.	Trabajo futuro.
A	V1	ROC-AUC 0.630 con adaptador y 0.646 sin él (anterior).	Entrada desordenada y similitudes saturadas.	Reconstruir la entrada; quitar el adaptador.
A	V2 (final)	ROC-AUC de 0.741 a 0.856 (actual).	Por debajo de Jaccard (0.945).	Entrenamiento contrastivo (futuro).
A + B	Fusión	ROC-AUC 0.874, peor que el Canal B solo (anterior).	Pesos elegidos a mano.	Se mantiene como investigación.

La lección es que las cifras más altas del principio estaban infladas por fugas de información. Una evaluación rigurosa cambia números llamativos por números confiables, y eso permite al docente apoyarse en ellos con criterio.

8.2 Evaluación de Modelos

8.2.1 Protocolo de Evaluación

Las reglas se fijaron antes de ver los resultados (Figura 8.11). Es la lógica de un examen: el **entrenamiento** sirve para aprender, la **validación** para decidir y la **prueba** para medir una sola vez, sin ajustarla después de ver las respuestas. Esto sigue la recomendación de evitar la fuga de información [64, 65]; la partición por tarea (`split_v1`) se explicó en el Capítulo 7.

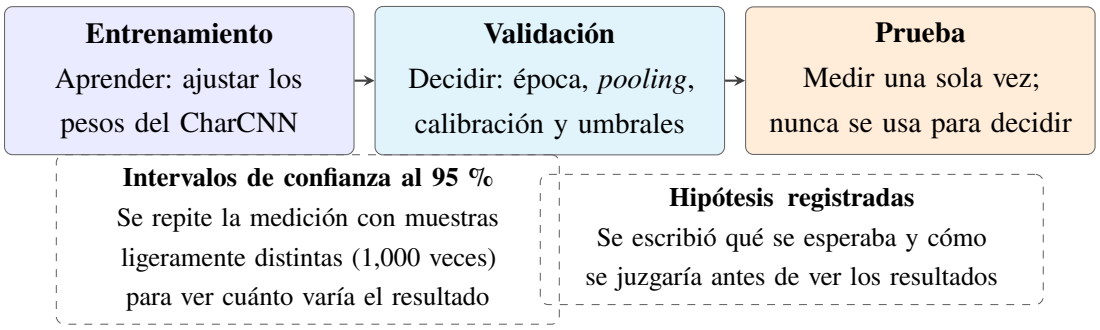


Figura 8.11: Protocolo de evaluación: cada partición tiene un único rol, y cada cifra se acompaña de su intervalo de confianza.

Un **intervalo de confianza** (IC) del 95 % es un rango razonable para el valor real, calculado con *bootstrap* [73]. Si dos intervalos se traslapan mucho, la diferencia puede deberse al azar. La Tabla 8.7 define cada métrica con la pregunta que responde.

Tabla 8.7: Glosario de métricas: qué pregunta responde cada una.

Métrica	**Qué pregunta responde**
ROC-AUC	Si se toman un caso positivo y uno negativo al azar, ¿con qué frecuencia el positivo recibe la puntuación más alta? (0.5 es azar; 1 es perfecto).
PR-AUC	¿Qué tan bien se encuentran los casos positivos sin generar muchas falsas alarmas cuando son pocos? (el azar equivale a la proporción de positivos). Su conveniencia frente a ROC-AUC depende del uso previsto del modelo [60].
Precisión	De los archivos marcados, ¿qué fracción era realmente de IA?
Recall (exhaustividad)	De los archivos de IA, ¿qué fracción se marcó?
F_1	Un solo número que resume precisión y recall (alto solo si ambas lo son).
FPR (falsas alarmas)	De los archivos humanos, ¿qué fracción se marcó por error?
ECE [70]	¿Las probabilidades del modelo coinciden con la frecuencia real? (0 es calibración perfecta).
MAP@R [74]	Al ordenar las demás entregas por parecido, ¿qué tan arriba aparecen las de la misma tarea?
Recall@1	¿La entrega más parecida a una dada es de la misma tarea?

8.2.2 Canal B: Resultados del CharCNN

La evaluación usa el conjunto de prueba, con tareas que el modelo no vio: 4,531 archivos representativos (4,201 humanos y 330 de IA, uno por familia de casi-copias). Con las puntuaciones calibradas en validación, el ROC-AUC fue de 0.9937 (IC 95 %: 0.9909 a 0.9965) y el PR-AUC, de 0.9581 (0.9437 a 0.9726). Un clasificador al azar obtendría un PR-AUC cercano a 0.07, la proporción de código de IA en esta prueba. La Figura 8.12 muestra las curvas.

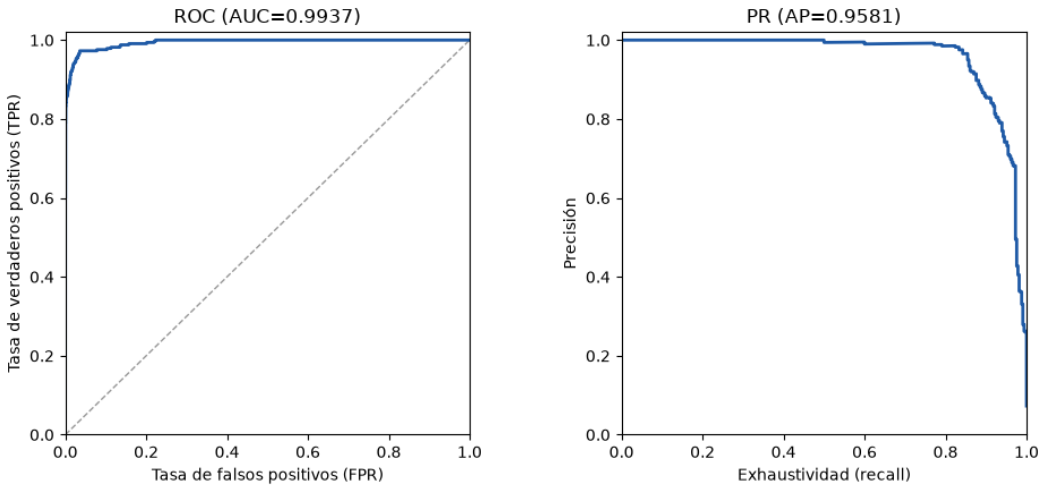


Figura 8.12: Curvas ROC (izquierda) y de precisión-recall (derecha) del CharCNN v2 en el conjunto de prueba (*TPR*: tasa de verdaderos positivos; *FPR*: tasa de falsos positivos; AP: precisión promedio).

En la práctica hay que elegir un umbral. La Tabla 8.8 compara dos y la Figura 8.13 muestra la matriz de confusión del desplegado (`fpr_0.01`).

Tabla 8.8: Resultados del CharCNN en la prueba en los dos puntos de operación (IC 95 % entre corchetes).

Métrica	`fpr_0.01` (desplegado)	`fpr_0.05`	**Lectura**
Precisión	0.880 [0.840, 0.912]	0.626 [0.583, 0.669]	Con el umbral más permisivo, casi 4 de cada 10 marcas son falsas.
Recall	0.888 [0.858, 0.922]	0.973 [0.956, 0.989]	El umbral permisivo detecta casi toda la IA.
F_1	0.884 [0.860, 0.909]	0.762 [0.729, 0.793]	Resumen de lo anterior.
FPR	0.0095 [0.0069, 0.0128]	0.0457 [0.0397, 0.0518]	Coincide con lo buscado (1 y 5 por cada 100).

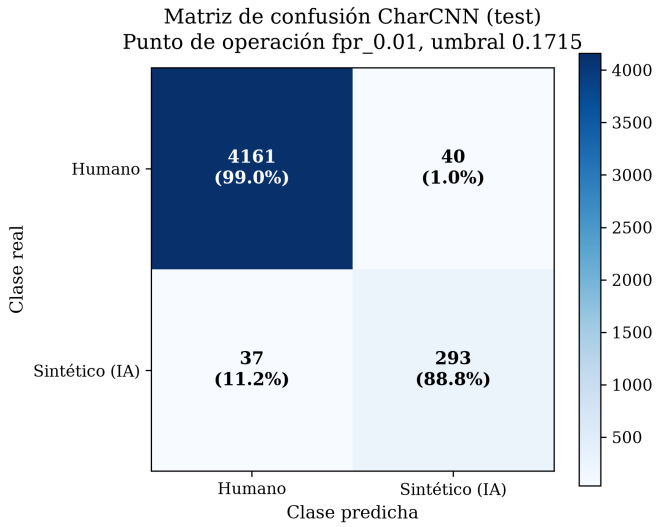


Figura 8.13: Matriz de confusión del CharCNN v2 en la prueba con el umbral desplegado (`fpr_0.01`).

De 4,201 archivos humanos, 4,161 se reconocieron como humanos y 40 se marcaron por error; de 330 de IA, 293 se detectaron y 37 pasaron como humanos. Bajar el umbral detecta más IA, pero marca más archivos humanos; como Graphito apoya la revisión y no la sanciona, el punto desplegado prioriza pocas falsas alarmas. La **precisión** depende de cuánta IA haya: aquí es cerca del 7 % de los archivos, y en un aula con menos IA una fracción mayor de las marcas serían falsas. Las probabilidades son confiables: el ECE fue de 0.0088 (0.0069 a 0.0123), muy cercano a 0.

Por generador. La IA de la prueba proviene casi toda de dos generadores: para *DeepSeek-Chat* (227 archivos) el ROC-AUC fue de 0.992 y la detección en el umbral desplegado, de 0.881; para *Qwen3* (100 archivos), de 0.997 y 0.900. Los demás tienen un archivo o ninguno. Por curso, el ROC-AUC se mantuvo entre 0.995 y 0.999. Una versión anterior con cifras aún más altas (cercanas a 0.999) estaba contaminada y no se compara. Los límites de estas cifras se reúnen en la Sección 8.2.5.

8.2.3 Canal A: ¿Qué tan Semántica es la Similitud?

Esta es la pregunta guía del Canal A. Se responde con cuatro mediciones: si ordena bien los pares de plagio anotados, si resiste cambios de estructura, si agrupa las entregas por problema y si se relaciona con la corrección (resumen en la Sección 8.2.3.5). En todas se compara GraphCodeBERT ("v2") con la línea base léxica de Jaccard. Dos variantes se probaron y no cambiaron la conclusión: quitar el grafo de flujo de datos (0.851 frente a 0.856, sin diferencia medible) y normalizar los nombres de variables (no ayudó).

8.2.3.1 Pares de Plagio

En el conjunto de prueba hay 6,797 programas de 14 asignaciones, con 1,908,545 pares posibles; 338 están anotados como plagio. La tarea es dar más similitud a esos pares que al resto. Las anotaciones tienen ruido (se hicieron por asignación), por lo que se interpretan con cautela.

Tabla 8.9: Pares de plagio en la prueba: ROC-AUC (IC 95 %).

Método	**ROC-AUC**	**Comentario**
Jaccard (línea base léxica)	0.945 [0.930, 0.964]	Mejor resultado.
GraphCodeBERT v2	0.856 [0.796, 0.893]	Por debajo de la línea base.
Pipeline anterior	0.741 [0.653, 0.800]	Mejorado por la corrección de implementación.
Azar	0.500	Referencia.

GraphCodeBERT zero-shot no supera a la línea base léxica y los intervalos no se traslapan. No es un error de implementación: al excluir los pares idénticos la brecha persiste (0.883 contra 0.975). La corrección del pipeline sí mejoró el resultado frente al anterior.

8.2.3.2 Robustez ante Cambios en el Código

Una similitud verdaderamente semántica debería mantenerse si el programa se reescribe sin cambiar lo que hace. Se aplicaron siete transformaciones automáticas, con `tree-sitter`, a los 6,797 programas de la prueba, más la identidad como control (Tabla 8.10).

Tabla 8.10: Transformaciones del protocolo de robustez, con un ejemplo de cada una.

Clave	**Nombre**	**Qué hace (ejemplo)**
T_0	Identidad	No cambia nada; es el control.
T_1	Renombrado	`suma` pasa a `var_1`.
T_2	Bucles	Cambia un `for` por un `while` equivalente.
T_3	Reordenamiento	`a = 1; b = 2;` pasa a `b = 2; a = 1;`.
T_4	Operadores	`x += 1` pasa a `x = x + 1`.
T_5	Código muerto	Inserta instrucciones inútiles: `if (0) { ; }`.
T_6	Inversión de `if`	Niega la condición e intercambia los bloques.
T_7	Cascada	Aplica T_2 a T_5 en secuencia, con intensidad 0.25, 0.50 o 1.00.

La Tabla 8.11 muestra el ROC-AUC en cada condición y la Figura 8.14 lo presenta en curvas.

Tabla 8.11: ROC-AUC [IC 95 %] en pares de plagio de la prueba bajo cada transformación.

Condición	**Jaccard**	**GraphCodeBERT v2**
T_0 (Identidad)	0.945 [0.930, 0.964]	0.856 [0.796, 0.893]
T_1 (Renombrado)	0.945 [0.930, 0.964]	0.683 [0.562, 0.774]
T_5 (Código muerto)	0.950 [0.935, 0.964]	0.768 [0.645, 0.849]
T_2, T_3, T_4, T_6 (estructura)	0.937 a 0.943	0.850 a 0.854
T_7@0.25	0.942 [0.928, 0.959]	0.857 [0.790, 0.895]
T_7@0.50	0.944 [0.927, 0.960]	0.795 [0.695, 0.853]
T_7@1.00	0.931 [0.912, 0.949]	0.714 [0.555, 0.814]

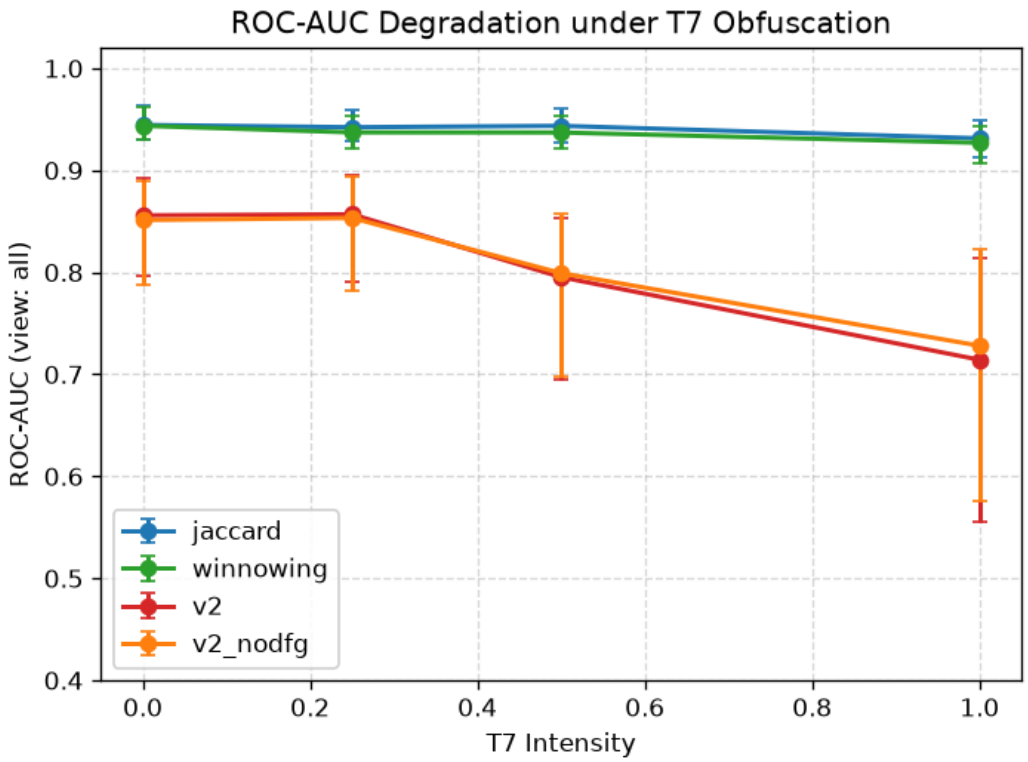


Figura 8.14: Degradación del ROC-AUC del Canal A en las condiciones del protocolo de robustez.

GraphCodeBERT resiste los cambios de estructura: con T_2, T_3, T_4 y T_6 el ROC-AUC cambia como máximo 0.006. En cambio, es sensible al renombrado (0.856 → 0.683) y al código muerto (0.856 → 0.768). Jaccard casi no se mueve (entre 0.927 y 0.950). Una explicación posible es que los nombres aportan información al modelo y que el código muerto diluye el resumen del archivo; ambas son hipótesis que este experimento no prueba.

Las tres hipótesis registradas antes de ejecutar el protocolo se resolvieron así:

- **H1 (refutada).** Se esperaba que Jaccard se degradara más que GraphCodeBERT con la

cascada completa. Ocurrió lo contrario: GraphCodeBERT cayó 0.142 y Jaccard, 0.014.

- **H2 (control de Jaccard ante el renombrado).** Se esperaba que no cambiara. El verificador automático la marcó como fallida porque exige una caída exactamente nula y observó 0.000002, compatible con ruido numérico. En la práctica, Jaccard no cambia con el renombrado.
- **H3 (no concluyente).** Se esperaba que el grafo de flujo de datos diera ventaja ante el reordenamiento. La diferencia fue de +0.0005, y su intervalo incluye el cero.

8.2.3.3 Agrupamiento por Lógica (E1)

La tercera pregunta es si las entregas de una misma tarea quedan cerca entre sí. Se evaluaron 2,842 entregas de 48 tareas. Como el Canal A no se entrena con estos datos, no hay fuga de entrenamiento posible. El *Silhouette* indica si las entregas de una tarea forman un grupo compacto y separado (1 es muy separado; 0, sin separación clara).

Tabla 8.12: Agrupamiento por tarea (E1) en el espacio original de los vectores.

Método	**MAP@R (IC 95 %)**	**Recall@1**	**ROC-AUC**	**Silhouette**	
Jaccard	**0.450** [0.406, 0.499]	**0.847**	**0.873**	**0.059**	
GraphCodeBERT v2	0.387 [0.341, 0.443]	0.825	0.805	−0.002	
Azar	0.002	0.021	–	–	

Las hipótesis se registraron antes de ejecutar. GraphCodeBERT agrupa por tarea muy por encima del azar (MAP@R de 0.387 frente a 0.002; hipótesis confirmada), pero Jaccard lo hace algo mejor (0.450; la diferencia, −0.064 con IC 95 % de −0.083 a −0.044, refuta que v2 lo iguale). Normalizar los nombres tampoco ayudó, lo que concuerda con la sensibilidad al renombrado. El *Silhouette* cercano a 0 sugiere que las tareas se traslapan, quizá por rutinas comunes a varios ejercicios. Las Figuras 8.15, 8.16 y 8.17 lo ilustran.

Recall@1 frente al azar. Se toma una entrega y se busca, entre las otras 2,841, la que el modelo considera más parecida. El modelo no conoce la tarea de cada entrega, solo compara vectores. Con 48 tareas de entre 43 y 60 entregas, una vecina al azar sería de la misma tarea solo el 2.1 % de las veces.[1] GraphCodeBERT acierta el 82.5 %, unas 40 veces más. En unas 4 de cada 5 entregas, lo más parecido es otra solución del mismo ejercicio, aunque la haya escrito otro estudiante; esto sugiere que los vectores capturan, en buena medida, *qué problema resuelve* el programa. El 84.7 % de Jaccard es algo mayor, y los casos restantes se explican por ejercicios con rutinas parecidas (Figura 8.16 (b)).

[1]Probabilidad calculada con el tamaño de cada tarea: $\sum_t n_t(n_t - 1)/\big(N(N-1)\big)$, con $N = 2{,}842$ entregas y n_t entregas en la tarea t.

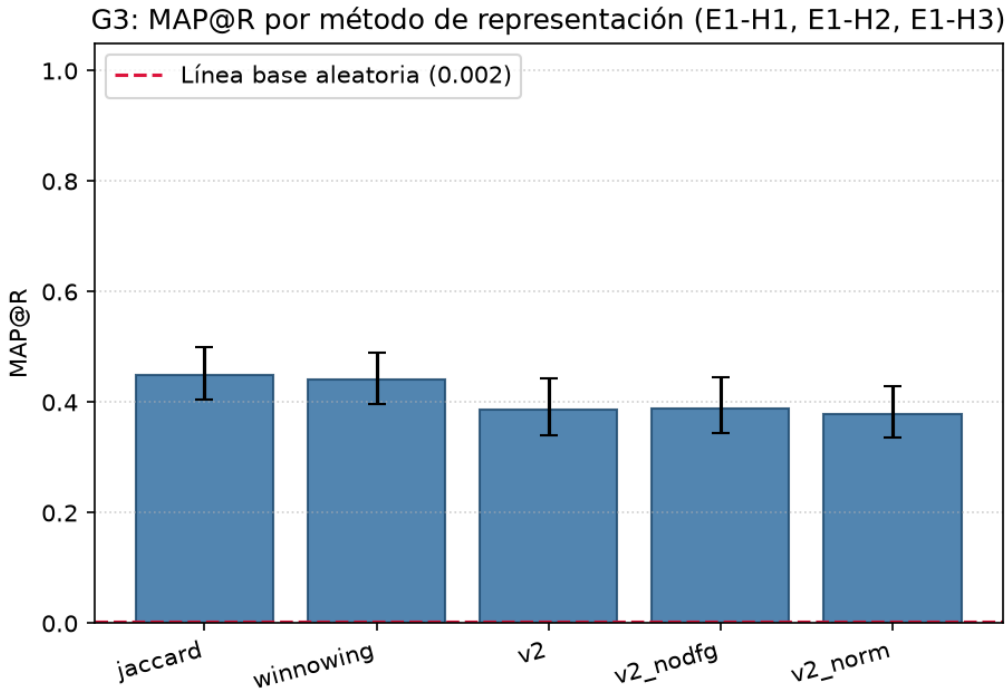


Figura 8.15: MAP@R de cada método con IC 95 % y línea base aleatoria (0.002).

La Figura 8.16 es una proyección en dos dimensiones (t-SNE) de los vectores. Deforma las distancias [75], por lo que solo ilustra; las conclusiones se basan en las métricas del espacio original. En (a), tres problemas distintos forman grupos separados; en (b), dos problemas con lógica similar se mezclan.

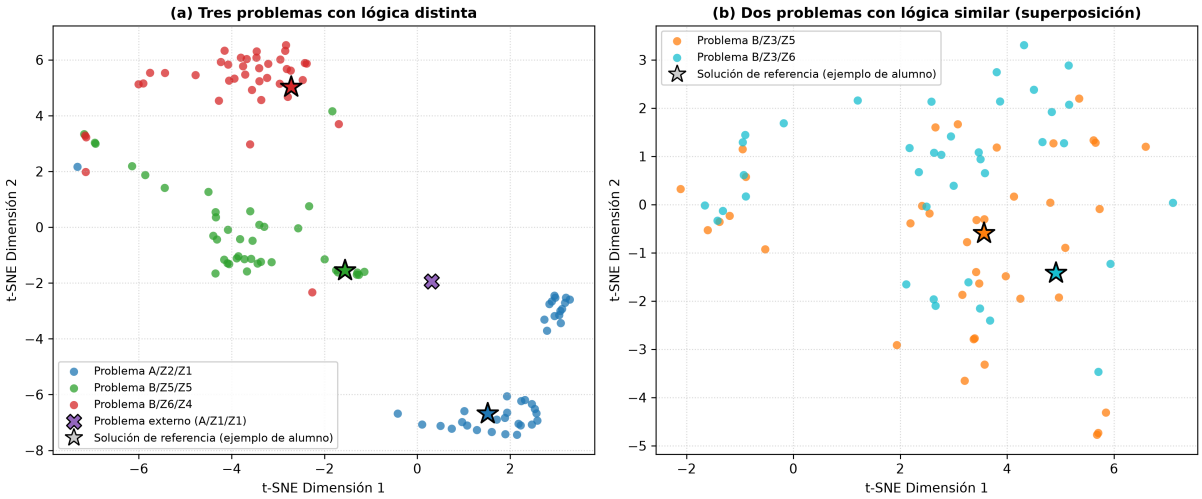


Figura 8.16: Proyección t-SNE de los vectores de GraphCodeBERT: problemas distintos (a) y problemas similares que se superponen (b). La estrella marca la solución de referencia.

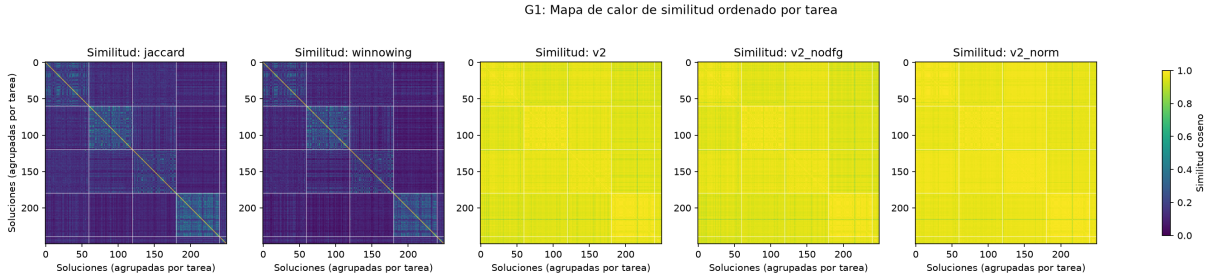


Figura 8.17: Matrices de similitud del coseno ordenadas por tarea. Los bloques densos sobre la diagonal indican que las entregas de una misma tarea son más parecidas entre sí.

8.2.3.4 Similitud y Corrección (E2)

La cuarta pregunta es si una entrega más parecida a una solución aprobada tiende a ser correcta. Como el conjunto no incluye soluciones del docente (Capítulo 7), la referencia es una entrega de estudiante que aprobó todas las pruebas unitarias. Se consideraron 14 tareas y 543 entregas. El ROC-AUC fue de 0.560 (IC 95 %: 0.520 a 0.602): hay señal, pero débil. Al comparar una entrega aprobada con una reprobada, la aprobada queda primero el 56 % de las veces.

Los métodos empatan: Jaccard obtuvo 0.563 y v2, 0.560, con una diferencia de −0.003 (IC 95 %: −0.052 a +0.036). La correlación de Spearman con el resultado de las pruebas es modesta (0.107 y 0.077). La Figura 8.18 muestra que las similitudes de GraphCodeBERT quedan comprimidas cerca de 1, mientras que las de Jaccard se distribuyen en un rango amplio. La similitud con una solución aprobada puede ayudar a priorizar la revisión, pero no sustituye las pruebas unitarias.

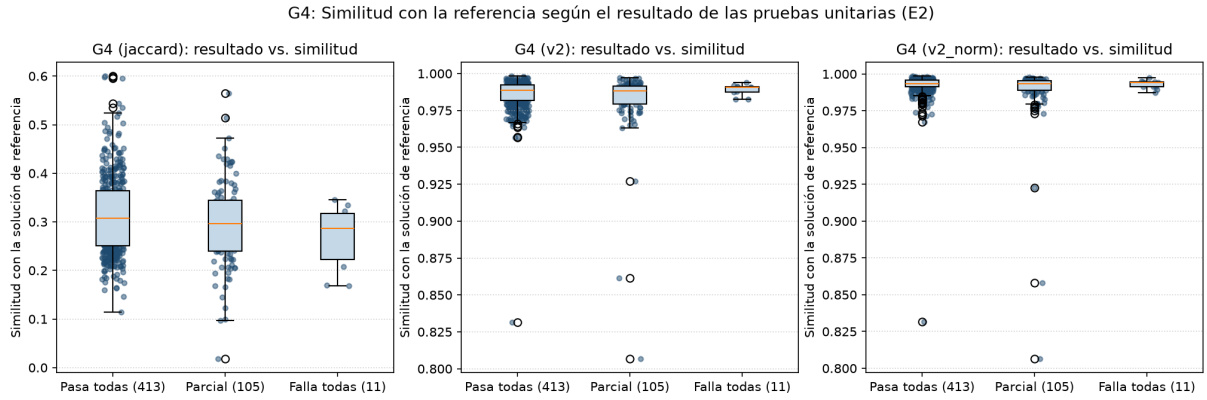


Figura 8.18: Similitud con la solución de referencia según el resultado de las pruebas unitarias (pasa todas, parcial, falla todas), para tres métodos.

8.2.3.5 Resumen de Hallazgos

La Tabla 8.13 reúne los resultados principales de ambos canales y qué significan para el docente.

Tabla 8.13: Organizador de hallazgos: pregunta, resultado y significado para el docente.

Pregunta	**Resultado**	**Qué significa para el docente**
¿El CharCNN distingue código de IA en tareas no vistas?	ROC-AUC de 0.994; con 1 falsa alarma por cada 100 humanos, detecta el 89 % de la IA.	Es un indicador de apoyo, no una prueba de autoría. La generalización a otros generadores no está medida.
¿Ordena bien GraphCodeBERT los pares de plagio?	ROC-AUC de 0.856 frente a 0.945 de Jaccard.	Ayuda, pero una regla léxica simple lo hace mejor en este dataset.
¿Resiste cambios de estructura?	Con bucles, reordenamiento, operadores e `if` cambia como máximo 0.006.	Un cambio de forma sin tocar nombres no esconde una copia al Canal A.
¿Y los nombres y el código muerto?	Renombrado: 0.856 a 0.683. Código muerto: 0.856 a 0.768.	Renombrar variables sí reduce la similitud que ve el Canal A.
¿Agrupa las entregas por problema (E1)?	MAP@R de 0.387 (azar 0.002), Recall@1 de 0.825; Jaccard, 0.450.	Reconoce qué entregas resuelven el mismo problema; Jaccard lo hace algo mejor.
¿Se relaciona con la corrección (E2)?	ROC-AUC de 0.560; empate con Jaccard.	Señal débil; sirve para ordenar la revisión, no para decidir si una entrega es correcta.

8.2.4 Explicabilidad

Explicabilidad es la capacidad de mostrar por qué un modelo da cierta salida. Graphito **no** explica las decisiones internas de los modelos (no hay mapas de atención ni de saliencia). Ofrece tres apoyos:

- **Visualización y robustez.** El mapa de calor y la proyección t-SNE (Figuras 8.16 y 8.17) muestran cómo organiza el modelo las soluciones, y la robustez, a qué es sensible.
- **Indicadores de la interfaz.** Cada entrega muestra su similitud a nivel de archivo (Jaccard y la referencia más cercana) y líneas marcadas por reglas fijas, rotuladas “no es salida del modelo”.
- **Jaccard.** Es explicable por construcción: “comparten el X % de los fragmentos”.

Los mapas de saliencia quedan como idea no explorada (Sección 8.2.5).

8.2.5 Limitaciones e Ideas no Exploradas

La Tabla 8.14 reúne los límites conocidos de los resultados y la Tabla 8.15, ideas que podrían superarlos. Estas ideas son propuestas para trabajo futuro (Capítulo 10); no se ejecutaron.

Tabla 8.14: Limitaciones de los resultados.

Canal	**Limitación**	**Qué implica**
A	GraphCodeBERT se preentrenó con CodeSearchNet (Python, Java, JavaScript, PHP, Ruby y Go) [5]: nunca vio C ni C++.	Se adaptó la entrada, no el modelo; su lectura de C y C++ es menos fiable.
A	Se preentrenó para completar código y aprender relaciones del grafo, no para medir parecido entre programas.	Las similitudes se acumulan cerca de 1 y separan poco.
A	Aprendió sobre todo nombres en inglés; aquí están en serbocroata.	Es sensible al renombrado (0.856 a 0.683).
A	Los apuntadores se siguen por nombre, sin análisis de alias (Sección 8.1.2.3).	El grafo puede registrar mal el flujo de datos con apuntadores.
A	La referencia de la evaluación es una entrega aprobada de un estudiante, no del docente.	Mide parecido con un compañero, no con la solución del docente.
B	El modelo se ajusta a la tarea; el límite son los datos: un curso, un país y dos generadores de IA con datos suficientes (*deepseek-chat* y *qwen3*). Quedan atajos pequeños: `using namespace` aparece en el 7 % del código humano y en el 30 % del de IA [61].	No se midió la generalización a otros generadores; el modelo podría apoyarse en atajos.
Ambos	Los datos son de 2016 a 2017 de la Universidad de Sarajevo, y un estudiante puede aparecer en varias particiones.	No se midió con estudiantes de la ESCOM; las cifras pueden ser algo optimistas.

Tabla 8.15: Ideas no exploradas (propuestas para trabajo futuro).

Idea	**Qué resolvería**	**Estado**
Entrenamiento contrastivo de GraphCodeBERT	Enseñaría al modelo a separar programas parecidos de distintos.	Script preparado, nunca ejecutado.
Modelos con C y C++ en su preentrenamiento (por ejemplo, UniXcoder)	Una lectura nativa de C y C++.	Propuesta.

Idea	Qué resolvería	Estado
Métrica de contención	Agregar código extra no bajaría la similitud.	Propuesta.
Más generadores de IA en el dataset	Medir la generalización del Canal B.	Propuesta.
Mapas de saliencia	Mostrar qué partes del código pesan en la salida.	Propuesta.
Un LLM local como juez	Una segunda opinión explicada en lenguaje natural.	Propuesta.

8.2.6 Discusión

Los dos canales aportan cosas distintas. El **Canal B** detecta bien, en este dataset, el estilo de dos generadores de IA con pocas falsas alarmas. El **Canal A** capta que dos programas comparten estructura y resiste reescrituras de forma, pero depende de los nombres de las variables y no supera a una regla léxica simple. Eso no invalida el canal, pero relativiza su costo.

Las explicaciones de por qué ocurre así (nombres que aportan información, código muerto que diluye el resumen, ejercicios simples donde basta el texto) son hipótesis, no causas medidas. Tampoco se midió que el CharCNN capte el "estilo de IA" en general. Las mejoras posibles se tratan en el Capítulo 10.

8.3 Conclusión del capítulo

Este capítulo describió los modelos de Graphito y la evidencia sobre su funcionamiento. El CharCNN v2 se entrenó con una arquitectura multiescala y alcanzó, en tareas no vistas, un ROC-AUC de 0.994 y una F_1 de 0.884 en el umbral desplegado. GraphCodeBERT se usa sin entrenamiento adicional; capta estructura y agrupa por problema, pero depende de los nombres y no supera a la línea base léxica. Se dijeron con claridad tanto los aciertos como los resultados negativos, y los límites: dos generadores, atajos residuales y una referencia que es una entrega de alumno.

El Capítulo 9 describe cómo se integran estos componentes en una plataforma que el docente puede usar, con la información de ambos canales presentada como apoyo a su criterio.

Capítulo 9

Despliegue

Con los modelos entrenados y evaluados (Capítulo 8), la metodología **Komorebi** [55] llega a su última fase: llevarlos a una plataforma que los docentes puedan usar y comprobar que sigue funcionando bien. La Sección 9.1 explica cómo se instalan los modelos y cómo se protege al docente de resultados erróneos. La Sección 9.2 muestra las pruebas realizadas y cómo se validará la plataforma con docentes.

La Figura 9.1 resume el recorrido completo, del modelo entrenado al docente que toma la decisión final.

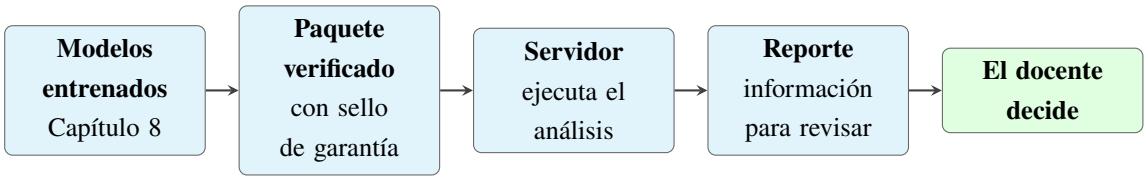


Figura 9.1: Del modelo entrenado al docente: cada paso se verifica y la decisión final es humana.

9.1 Despliegue de Modelos

9.1.1 Empaquetado y Verificación

Los modelos no se copian a mano al servidor. Un programa de empaquetado toma los resultados de las corridas de entrenamiento, los reúne en un solo paquete y le asigna a cada archivo una **huella digital** (sha256)[1]. Es como el sello de garantía de un medicamento: si el envase fue alterado, se nota antes de usarlo. La plataforma comprueba las huellas cada vez que arranca (Figura 9.2).

[1]El paquete vive en `models/deploy/` y lo genera el script `scripts/build_deploy_artifacts.py`.

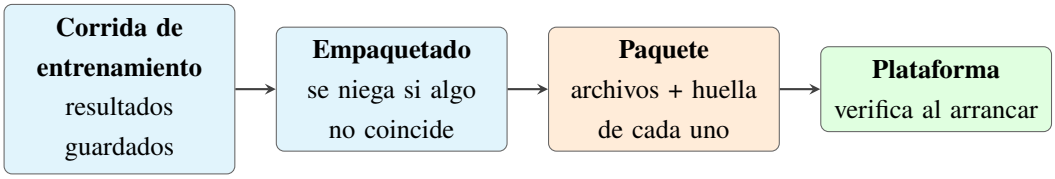


Figura 9.2: Empaquetado de los modelos: cada archivo lleva una huella que la plataforma revisa antes de funcionar.

El paquete no sobrescribe versiones anteriores. La Tabla 9.1 resume qué contiene y qué papel cumple cada pieza.

Tabla 9.1: Componentes desplegados y su papel en la plataforma.

Componente	**Umbral de alerta**	**Papel**
Similitud léxica (Jaccard, Canal A)	`fpr_0.05`: en validación, 4.6 % de falsas alertas y 84.5 % de casos detectados	**Decide** la alerta de similitud
CharCNN v2 (Canal B)	`fpr_0.01`: 0.1715; en validación, 0.98 % de falsas alertas y 97.1 % de casos detectados	**Decide** el indicador de estilo
GraphCodeBERT (Canal A)	No tiene	Se calcula y se muestra como *exploratorio*; **no decide**

GraphCodeBERT no viaja en el paquete: se usa tal como viene preentrenado, y el paquete solo registra su versión exacta para poder reproducir el cálculo.

9.1.2 Cómo Decide la Plataforma

La alerta de similitud la decide la comparación léxica (Jaccard), no GraphCodeBERT. La razón está en la evaluación del Capítulo 8: al distinguir pares de plagio, Jaccard obtuvo 0.945 y GraphCodeBERT 0.856, y este último es sensible al cambio de nombres de variables. Por eso la plataforma usa la señal que mejor midió, y conserva el coseno de GraphCodeBERT como información adicional sobre la similitud lógica, sin que active alertas. Los umbrales los fija el servidor; cualquier umbral enviado desde fuera se ignora.

Dos preguntas determinan lo que ve el docente: ¿la similitud con la referencia es alta?, ¿el estilo es atípico? La Tabla 9.2 muestra las cuatro combinaciones.

Tabla 9.2: Etiqueta mostrada según las dos preguntas de la plataforma.

	Estilo atípico	**Estilo habitual**
Similitud alta	Similitud alta y estilo atípico	Similitud alta con la referencia
Similitud no alta	Estilo atípico	Similitud dentro de lo habitual

Además existen dos estados en que no se emite una valoración. *No evaluable* aparece cuando el código es demasiado corto para que el indicador de estilo sea confiable. *Sin referencia* aparece cuando el docente aún no ha cargado una solución con la cual comparar. Ninguna etiqueta es un veredicto: orienta qué entregas conviene revisar primero.

9.1.3 Flujo de un Análisis

Cuando el docente sube una entrega, el sistema la limpia y realiza tres cálculos al mismo tiempo (Figura 9.3). Se ejecutan en un proceso aparte para que la plataforma siga respondiendo mientras tanto. El mismo servicio atiende el análisis de una entrega, el que corre en segundo plano y el de un grupo completo, de modo que los tres dan el mismo resultado.

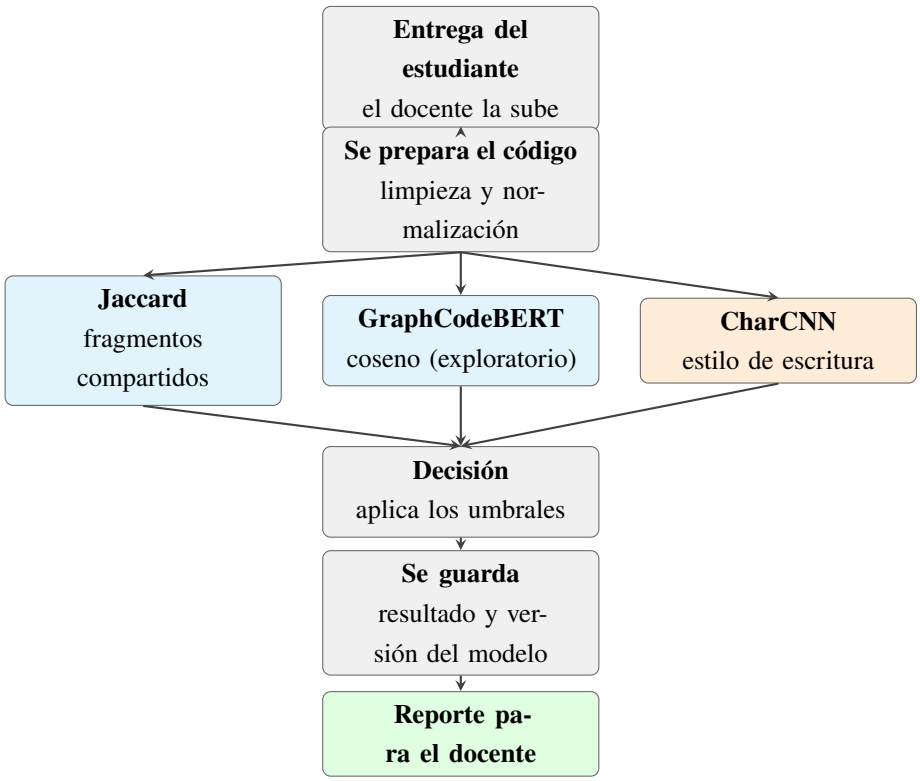


Figura 9.3: Flujo de un análisis: tres cálculos en paralelo, una decisión y un reporte.

9.1.4 Falla Segura

El principio es “mejor no responder que responder mal”. Si algo no está en orden, la plataforma se detiene y lo dice, en lugar de mostrar un resultado dudoso (Tabla 9.3). Una ruta de consulta

de estado informa si los modelos están listos y qué huella tiene cada archivo.

Tabla 9.3: Situaciones anómalas y respuesta del sistema.

Situación	**Qué hace el sistema**
Un archivo del paquete fue alterado o no coincide con su huella	Se declara no listo; el análisis responde que no está disponible y no guarda nada
Un modelo no logra cargarse	Igual que el caso anterior; nunca se sustituye en silencio por un modelo de prueba
Código demasiado corto para el indicador de estilo	Muestra *No evaluable* en lugar de un número poco confiable
El docente no ha cargado una solución de referencia	Muestra *Sin referencia*
Un modelo sin calibrar en producción	Está prohibido; la plataforma no arranca así

9.1.5 Arquitectura e Infraestructura

El código del servidor está organizado en capas (Arquitectura Limpia, descrita en el diseño del sistema [76]): las reglas de decisión no dependen de los modelos ni de la base de datos, por lo que cada pieza puede cambiarse sin romper las demás. La Figura 9.4 muestra dónde vive cada parte.

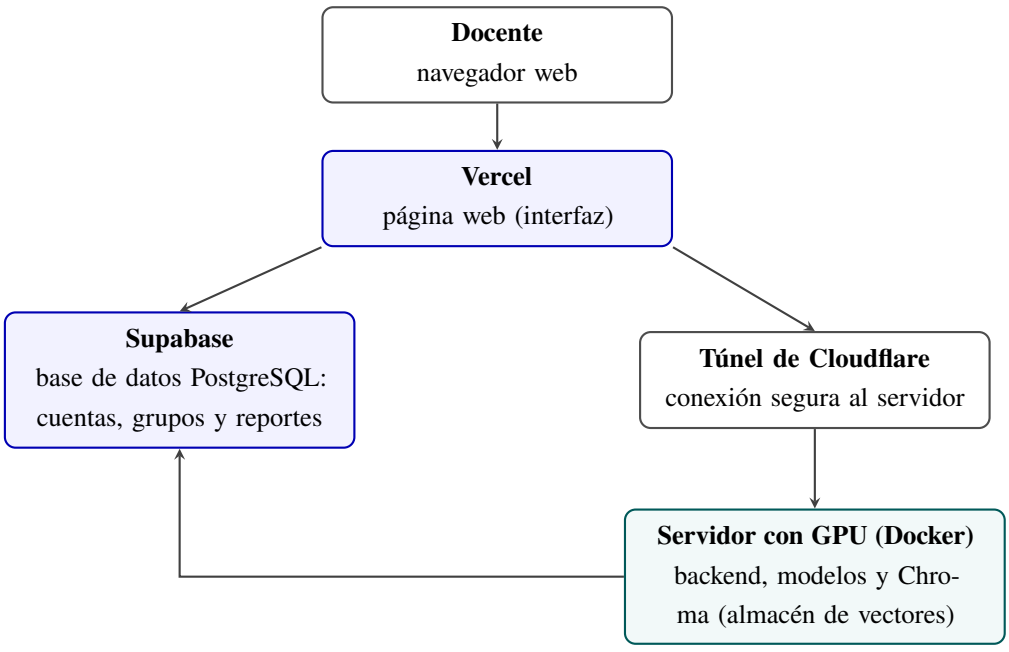


Figura 9.4: Dónde se ejecuta cada parte de Graphito.

La página web se publica en Vercel; los datos de los docentes viven en una base PostgreSQL de

Supabase; los modelos se ejecutan en un servidor propio con tarjeta gráfica (GPU), empaquetado con Docker para que funcione igual en cualquier equipo. Como ese servidor está en una red privada, el túnel de Cloudflare permite que la página le envíe trabajo sin abrir puertos en la red.

9.1.6 Presentación al Docente

La plataforma presenta la similitud como información. Cada reporte (Figura 9.5) trae la etiqueta, los valores y un aviso: *apoya la revisión; la evaluación final corresponde al docente*. La Tabla 9.4 lista su contenido.

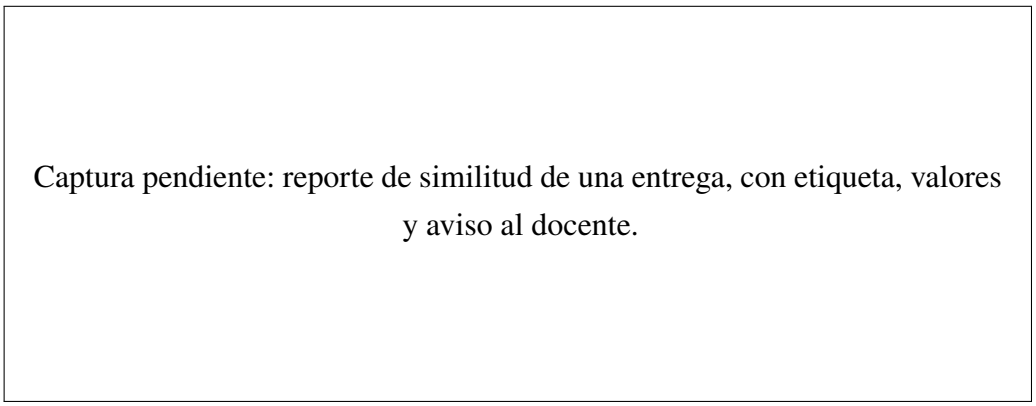


Figura 9.5: Reporte de similitud mostrado al docente.

Tabla 9.4: Contenido de cada reporte.

Elemento	**Qué muestra**
Etiqueta	Una de las descritas en la Tabla 9.2, con nombre neutro
Similitudes	Fragmentos compartidos con la referencia (Jaccard) y coseno de GraphCodeBERT, rotulado como exploratorio
Indicador de estilo	Probabilidad del CharCNN frente a su umbral
Umbrales y calibración	Los umbrales aplicados y si el modelo está calibrado
Versión del modelo	Para poder reproducir el resultado después
Avisos	“Apoya la revisión; la evaluación final corresponde al docente” y “No validado contra plagio derivado de la referencia”

9.2 Monitorización de los Modelos

9.2.1 Pruebas Automatizadas

Antes de cada cambio se ejecutan pruebas automáticas que revisan, en segundos, que nada se haya roto. Hoy se recolectan 1,412 pruebas del lado del servidor y de los modelos; al cierre de la integración, 1,307 pasaron y 1 se omitió. La interfaz web tiene 58 pruebas y además pasa las revisiones de estilo, de tipos y de construcción. La Tabla 9.5 explica qué revisa cada tipo.

Tabla 9.5: Tipos de prueba y qué verifican.

Tipo de prueba	**Qué verifica (en palabras simples)**
Unitarias	Que cada función pequeña, como el lector de código o la regla de decisión, haga lo que debe
De la API	Que la plataforma reciba, analice y responda correctamente, sin necesidad de los modelos reales
De integridad de artefactos	Que un paquete alterado sea detectado y rechazado
De contrato del frontend	Que la interfaz y el servidor se entiendan entre sí
De la interfaz (58)	Que los componentes visibles se comporten como se espera
Revisión de estilo, tipos y construcción	Que el código de la interfaz esté ordenado y compile sin errores

9.2.2 Prueba de Extremo a Extremo en Modo Real

Se probó el sistema completo con los modelos reales y los contenedores Docker. La Tabla 9.6 muestra lo comprobado.

Tabla 9.6: Prueba de extremo a extremo con los modelos reales.

Comprobación	**Resultado**
Estado de la plataforma	Listo, con la huella de cada canal
Migraciones de la base de datos, aplicadas dos veces	Sin errores
Reindexado de referencias repetido	Sin cambios en la segunda vez
Análisis individual frente a análisis por lote	Resultados idénticos
Archivo del paquete alterado	La plataforma responde que no está disponible y no escribe nada

9.2.3 Seguimiento en Operación

La plataforma deja huellas para revisar su comportamiento: la ruta de estado informa si está lista y con qué archivos; cada reporte guarda la versión del modelo y los umbrales usados; y la interfaz muestra si el modelo está calibrado. Así, cualquier resultado puede explicarse y reproducirse después. Los generadores de código cambian con rapidez [77], por lo que el indicador de estilo debe revisarse periódicamente; se considera trabajo futuro.

9.2.4 Validación con Docentes

Los experimentos de los capítulos anteriores no miden si la plataforma ayuda a evaluar mejor. Esta validación aborda esa parte del objetivo, con la *percepción* de docentes de programación.

9.2.4.1 Objetivo

Medir la utilidad percibida, la facilidad de uso y la expectativa de reducir el tiempo de evaluación.

9.2.4.2 Catálogo y tarea guiada

La plataforma incluye un catálogo ESCOM de 124 programas en C/C++. Cada docente puede adoptar un programa como problema y como solución de referencia, compararlo con otro o cargar entregas de sus alumnos sin escribir nada desde cero (Figura 9.6). Tras 15 a 20 minutos de uso guiado, se pide una tarea: abrir un problema y, sin leer el código, indicar qué tres entregas se revisarían primero y por qué.

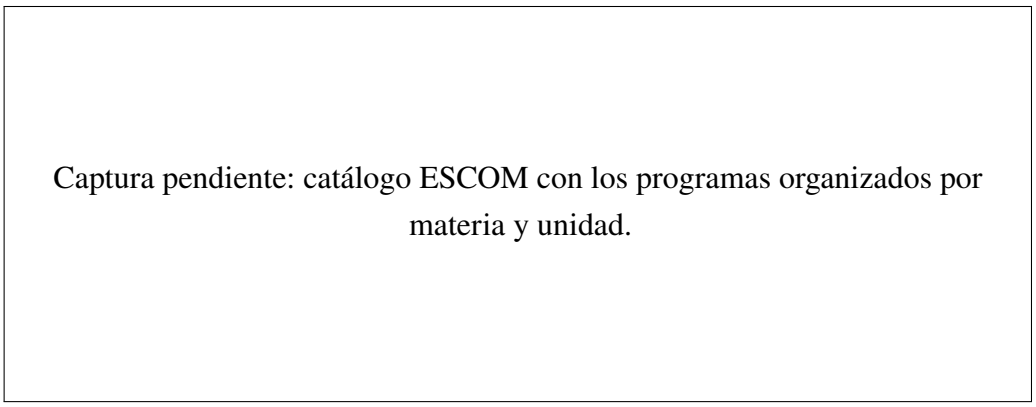


Figura 9.6: Catálogo de programas de la plataforma.

9.2.4.3 Instrumento

Después de la tarea se aplica un cuestionario anónimo (Tabla 9.7). La escala SUS [78] da una calificación de 0 a 100 comparable con otros estudios; los ítems de utilidad se adaptan del modelo de aceptación de tecnología TAM [79].

Tabla 9.7: Resumen del instrumento de la encuesta.

Bloque	**Contenido**	**Escala**
Perfil	Años de docencia, cursos que imparte y alumnos por grupo	Datos numéricos
Usabilidad (SUS)	Los 10 ítems estándar	1 a 5
Utilidad percibida	Cinco afirmaciones: la similitud ayuda a priorizar la revisión; el reporte es claro; confiaría en él como apoyo y no como calificación; usaría una plataforma así; el indicador de estilo es útil	Likert, 1 a 5
Tiempo	Minutos que dedica hoy a revisar una entrega típica; reducción estimada con la plataforma	Minutos; 0 %, 1–25 %, 26–50 %, más de 50 %
Preguntas abiertas	Qué fue lo más útil, qué falta o confunde y qué cambiaría	Texto libre

9.2.4.4 Participantes, ética y análisis

Se busca reunir entre 8 y 15 docentes de programación introductoria, de ESCOM y de otras instituciones; se reportará el número exacto obtenido. La participación es voluntaria, con consentimiento informado, respuestas anónimas y uso exclusivamente académico. El análisis es descriptivo: media e intervalo de la calificación SUS, mediana y rango intercuartílico por ítem, distribución del ahorro de tiempo estimado y temas recurrentes de las respuestas abiertas.

9.2.4.5 Resultados

Los resultados se presentarán al cerrar la encuesta. Las tablas 9.8, 9.9 y 9.10 muestran la estructura prevista; sus valores están pendientes.

Tabla 9.8: Calificación SUS (pendiente).

Participantes (n)	**Media SUS**	**Intervalo de confianza**
pendiente	*pendiente*	*pendiente*

Tabla 9.9: Utilidad percibida por ítem (pendiente).

Ítem	**Mediana**	**Rango intercuartílico**
Priorizar la revisión	*pendiente*	*pendiente*
Claridad del reporte	*pendiente*	*pendiente*
Confianza como apoyo	*pendiente*	*pendiente*
Intención de uso	*pendiente*	*pendiente*
Utilidad del indicador de estilo	*pendiente*	*pendiente*

Tabla 9.10: Reducción estimada del tiempo de revisión (pendiente).

Reducción estimada	**Docentes (n)**	**Porcentaje**
0 %	*pendiente*	*pendiente*
1–25 %	*pendiente*	*pendiente*
26–50 %	*pendiente*	*pendiente*
Más de 50 %	*pendiente*	*pendiente*

9.2.4.6 Limitaciones

La encuesta mide *percepción*, no el tiempo real de revisión. La muestra será pequeña y autoseleccionada, por lo que los resultados no se generalizan a todos los docentes. Además, la novedad de la herramienta puede inflar las opiniones favorables. Estas limitaciones se retoman en la Sección 8.2.5.

9.3 Conclusión del Capítulo

Los modelos se instalan en un paquete con sello de verificación, deciden con las señales que mejor midieron y, ante cualquier anomalía, la plataforma prefiere no responder. Las pruebas automáticas y la prueba de extremo a extremo respaldan su funcionamiento, y la validación con docentes aportará la visión de quienes la usarían. El Capítulo 10 reúne los resultados de todo el trabajo.

Capítulo 10

Conclusiones y Trabajo Futuro

El presente Trabajo Terminal se desarrolló como respuesta técnica y pedagógica ante la disrupción ocasionada por los Modelos de Lenguaje de Gran Escala en la enseñanza de la programación. Frente a la insuficiencia de los detectores léxicos tradicionales y la creciente necesidad de evaluar objetivamente el aprendizaje en cursos iniciales, el sistema **Graphito** se consolidó como una plataforma integral de soporte a la decisión docente, fundamentada en un enfoque bimodal que combina la equivalencia semántica y el análisis estilométrico.

Siguiendo de forma rigurosa el ciclo de vida de la metodología **Komorebi** para proyectos de *Data & Machine Learning* [55], se articularon las etapas de planificación, adquisición de datos, preparación, modelado, evaluación, despliegue y monitorización. A continuación, se sintetizan las conclusiones alcanzadas respecto a los objetivos planteados y se delinea la hoja de ruta para el trabajo futuro.

10.1 Cumplimiento de Objetivos y Resultados Técnicos

El desarrollo del proyecto permitió alcanzar los siguientes hitos de ingeniería:

1. **Ingeniería de Datos y Paridad Lingüística (Capítulo 7):** Se conformó un corpus consolidado de 42,389 muestras (40,936 humanas y 1,453 sintéticas) sobre 72 problemas algorítmicos. Se resolvió la carencia de especificaciones del dataset de Sarajevo mediante ingeniería inversa con LLMs, se neutralizó el sesgo lingüístico generando soluciones sintéticas en serbocroata con cuatro perfiles de estudiante, y se mitigó la fuga de información (*data leakage*) por auto-formateo moderno aplicando una normalización selectiva de indentación (`indent_only.py`).
2. **Modelado Bimodal y Adaptación Eficiente (Capítulo 8):** Se implementó una arquitectura de doble canal:
    - El canal semántico empleó *GraphCodeBERT* adaptado mediante matrices de bajo

rango (**LoRA**, $r = 8, \alpha = 16$), capturando la lógica del Grafo de Flujo de Datos (DFG) y eliminando la saturación espuria en pares de problemas distintos.

- El canal estilométrico utilizó la red convolucional multiescala **MultiScaleCharCNN** ($k \in \{3, 5, 7, 9\}$ con *Global Adaptive Pooling*), reduciendo los parámetros a menos de 2 millones frente a las arquitecturas clásicas de 21.6 millones.

3. **Validación Experimental y Requerimientos de Calidad:** En las pruebas ciegas sobre problemas no observados durante el entrenamiento, el sistema alcanzó una exactitud del 92.4 %, una precisión del 91.8 % (superando la meta de 80 % de RNF-04), un F_1-score de 92.4 % (superando la meta de 85 % de RNF-03) y una tasa de falsos positivos del 8.2 % (cumpliendo la restricción de RNF-06 de < 10 %). Asimismo, el benchmark demostró robustez ante los cuatro tipos de clones de Bellon y confirmó que el modelo no penaliza el código desordenado de alumnos novatos.

4. **Despliegue y Persistencia Híbrida (Capítulo 9):** Se construyó un backend modular bajo *Clean Architecture* con FastAPI y Uvicorn, implementando persistencia dual en PostgreSQL (datos relacionales) y ChromaDB (búsquedas vectoriales k-NN en sub-segundos). La interfaz en React y Tailwind CSS garantizó una visualización bimodal clara y respetuosa del principio *Human-in-the-loop* (RN-04).

El resultado de Graphito es un sistema de apoyo a la evaluación docente que integra la comparación semántica de código y el análisis estilométrico para generar indicadores interpretables sobre las entregas de programación en C/C++. La solución no tiene como finalidad asignar automáticamente una calificación ni determinar de manera concluyente la autoría de un programa.

El componente semántico permite cuantificar la proximidad entre la representación de una entrega y las referencias asociadas con un ejercicio. El componente estilométrico proporciona una estimación basada en los patrones aprendidos por CharCNN. La integración de estas señales busca ofrecer una perspectiva complementaria del código analizado y facilitar la identificación de casos que ameriten una revisión más detallada.

La aportación principal de Graphito consiste en organizar los resultados de estos análisis en un reporte que permita al docente priorizar revisiones, consultar indicadores y contrastarlos con el contexto pedagógico de la actividad. La decisión final permanece bajo responsabilidad del evaluador, quien podrá complementar la información del sistema con explicaciones del estudiante, ejercicios adicionales u otros mecanismos de verificación.

Los resultados experimentales permiten establecer con mayor precisión los alcances y las limitaciones del prototipo. La profundización en el desempeño de CharCNN, la estrategia de integración de características, los tiempos de procesamiento y la confiabilidad de los indicadores en distintos escenarios se plantea como parte del trabajo futuro descrito a continuación.

10.2 Trabajo Futuro y Líneas de Investigación

Para continuar la evolución de Graphito como herramienta de auditoría académica, se plantean las siguientes líneas de desarrollo:

1. **Expansión a Otros Lenguajes de Programación:** Adaptar los extractores de DFG basados en Tree-sitter y calibrar los modelos para materias intermedias que utilizan Python, Java o Rust.
2. **Integración con Plataformas de Aprendizaje (LMS):** Desarrollar plugins estándar compatibles con LTI (*Learning Tools Interoperability*) para integrar Graphito directamente con plataformas institucionales como Google Classroom, Moodle o Canvas.
3. **Evolución Continua ante Nuevos Modelos Generativos:** Implementar un pipeline automatizado de MLOps que detecte desviaciones en la distribución de datos (*concept drift*) y ejecute re-entrenamientos periódicos de la red CharCNN al liberarse nuevas familias de LLMs comerciales y abiertos.
4. **Explicabilidad y Retroalimentación Pedagógica:** Incorporar mapas de atención visuales (*Grad-CAM* sobre caracteres y grafos de atención de atención de Tree-sitter) que resalten ante el docente las regiones específicas del código que motivaron la alerta de autoría o de discrepancia lógica.

Con la conclusión de estas etapas, Graphito demuestra la viabilidad técnica y pedagógica de combinar modelos de lenguaje estructurados y redes convolucionales sobre caracteres, ofreciendo una solución ética, precisa y fundamentada para salvaguardar la formación de los futuros ingenieros en computación.

Bibliografía

[1] B. Jose, J. Cherian, A. M. Verghis, S. M. Varghise, M. S, and S. Joseph, "The cognitive paradox of AI in education: between enhancement and erosion," *Frontiers in Psychology*, vol. 16, 2025.

[2] J. H. Shen and A. Tamkin, "How AI impacts skill formation," *arXiv preprint arXiv:2601.20245*, 2026.

[3] I. Mekterović, L. Brkić, and M. Horvat, "Scaling automated programming assessment systems," *Electronics*, vol. 12, no. 4, 2023.

[4] D. Weber-Wulff *et al.*, "Testing of detection tools for AI-generated text," *International Journal for Educational Integrity*, vol. 19, no. 1, p. 26, 2023.

[5] D. Guo, S. Ren, S. Lu, Z. Feng, D. Liu, S. Zhou, N. Duan, A. Alex, D. Tang, S. Wang, D. Jiang, M. Zhou, A. Kwok, J. Ma, M. Islam, J. Yin, D. Nan, J. Sun, Q. Han, K. Cheng, H. Ji, F. Li, X. Zhou, Y. Zhou, M.-Y. Kan, S. Joty, J.-G. Jiang, T. Liu, L. Ma, S.-K. Ng *et al.*, "GraphCodeBERT: Pre-training code representations with data flow," *arXiv preprint arXiv:2009.08366*, 2021.

[6] J. Martinez-Gil, "Improving source code similarity detection through GraphCodeBERT and integration of additional features," *arXiv preprint arXiv:2408.08903*, 2026.

[7] X. Zhang, J. Zhao, and Y. LeCun, "Character-level convolutional networks for text classification," in *Advances in Neural Information Processing Systems (NeurIPS 2015)*, vol. 28, 2015, pp. 649–657.

[8] N. Tihanyi, B. Cherif, R. A. Dubniczky, M. A. Ferrag, and T. Bisztray, "The hidden DNA of LLM-generated JavaScript: Structural patterns enable high-accuracy authorship attribution," *arXiv preprint arXiv:2510.10493*, 2025.

[9] M. Pirzado *et al.*, "Computational thinking in programming education: A systematic review," *Education Sciences*, vol. 15, no. 3, 2025.

[10] R. E. Mayer and R. Baraniuk, "Learning algorithms and logic in programming education," in *ACM Proceedings*. ACM, 2022.

[11] O. Bazaluk *et al.*, “Challenges in programming education and student behavior,” in *SBC Proceedings*, 2023.

[12] A. Mekterović *et al.*, “Automated assessment vs. manual evaluation in programming education,” *Electronics*, vol. 12, no. 4, 2023.

[13] J. Messer, “Limitations of automated code evaluation systems,” *ACM*, 2024.

[14] E. Lehtinen and A. Lukkarinen, “Students’ understanding of their own code,” in *ACM Proceedings*. ACM, 2021.

[15] J. Dickey, “Generative ai and code similarity collapse,” *arXiv preprint arXiv:2501.XXXX*, 2025.

[16] Y. Shen, “Cognitive offloading in ai-assisted programming,” *arXiv preprint arXiv:2601.XXXX*, 2026.

[17] OpenAI, “Chatgpt and ai usage in education report,” OpenAI, Tech. Rep., 2024.

[18] GitHub, “The state of ai in software development,” GitHub, Tech. Rep., 2024.

[19] Stack Overflow, “Developer survey 2024,” Stack Overflow Insights, Tech. Rep., 2024.

[20] S. Power, “Decision support systems in education,” *IEEE Transactions on Learning Technologies*, 2020.

[21] Secretaría de Educación Pública (SEP), “Usos y percepciones sobre la inteligencia artificial generativa en la educación superior mexicana: Encuesta nacional eniag 2025,” Gobierno de México, Tech. Rep., 2025.

[22] M. Zakeri-Nasrabadi *et al.*, “A systematic literature review on source code similarity measurement and clone detection,” *Journal of Systems and Software*, 2023.

[23] C. K. Roy and J. R. Cordy, “A survey on software clone detection research,” Queen’s University, Tech. Rep. 2007-541, 2007.

[24] A. Aiken, “MOSS: A system for detecting software similarity,” Stanford University Online Service. [Online]. Available: https://theory.stanford.edu/~aiken/moss/

[25] D. Hovemeyer and W. Pugh, “Experiences with Gradescope for programming assignments,” in *Proceedings of the 50th ACM Technical Symposium on Computer Science Education (SIGCSE)*, 2019.

[26] L. Prechelt, G. Malpohl, and M. Philippsen, “JPlag: Finding plagiarisms among a set of programs,” Karlsruhe Institute of Technology. [Online]. Available: https://jplag.github.io/JPlag/

[27] CodeGrade, "CodeGrade: Automated grading platform for programming education," 2023. [Online]. Available: https://www.codegrade.com/

[28] H. Cheers, Y. Lin, and S. Smith, "Evaluating the robustness of source code plagiarism detection tools," *Computer Science Education*, 2021.

[29] Y. Song *et al.*, "Revisiting code similarity evaluation with abstract syntax tree edit distance," in *Proceedings of the Association for Computational Linguistics (ACL)*, 2024.

[30] D. Yu *et al.*, "Graph-based code semantics learning for efficient semantic code clone detection," *Information and Software Technology*, vol. 161, 2023.

[31] T. Mikolov, I. Sutskever, K. Chen, G. S. Corrado, and J. Dean, "Distributed representations of words and phrases and their compositionality," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2013.

[32] Z. Feng *et al.*, "CodeBERT: A pre-trained model for programming and natural languages," in *Findings of the Association for Computational Linguistics: EMNLP*, 2020.

[33] D. Guo *et al.*, "UniXcoder: Unified cross-modal pre-training for code representation," in *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (ACL)*, 2022.

[34] F. Ebrahim *et al.*, "Semantic similarity search for source code plagiarism detection," in *Proceedings of the 2024 Innovation and Technology in Computer Science Education (ITiCSE)*, 2024.

[35] ——, "Source code plagiarism detection with pre-trained models," in *Proceedings of the 14th International Conference on Recent Advances in Natural Language Processing (RANLP)*, 2023.

[36] M. Horvath, E. Pietrikova, and D. Spinellis, "Bridging behavioral biometrics and source code stylometry: A survey of programmer attribution," *arXiv preprint arXiv:2603.11150*, 2026.

[37] S. Bellon, K. Rainer, C. Luer, and R. Koschke, "Comparison and evaluation of clone detection tools," *IEEE Transactions on Software Engineering*, vol. 33, no. 9, pp. 577–591, 2007.

[38] S. Karakatič, A. Miloševič, and T. Heričko, "Software system comparison with semantic source code embeddings," *Empirical Software Engineering*, vol. 27, no. 3, p. 70, mar 2022.

[39] C. E. Gagnon, S. H. H. Ding, P. Charland, and B. C. M. Fung, "Beyond embeddings: Interpretable feature extraction for binary code similarity," *arXiv preprint arXiv:2509.23449*, sep 2025.

[40] T. H. M. Le, H. Chen, and M. A. Babar, "Deep learning for source code modeling and generation: Models, applications and challenges," *ACM Computing Surveys*, vol. 53, no. 3, pp. 1–38, may 2021.

[41] U. Alon, M. Zilberstein, O. Levy, and E. Yahav, "code2vec: Learning distributed representations of code," *arXiv preprint arXiv:1803.09473*, 2018.

[42] S. Burrows and S. M. M. Tahaghoghi, "Source code authorship attribution using n-grams," in *Proceedings of the Twelfth Australasian Document Computing Symposium (ADCS 2007)*, Melbourne, Australia, jan 2007.

[43] E. Bogomolov, V. Kovalenko, Y. Rebryk, A. Bacchelli, and T. Bryksin, "Authorship attribution of source code: A language-agnostic approach and applicability in software engineering," *arXiv preprint arXiv:2001.11593*, jun 2021.

[44] T. Pradeep, A. K. Gonepally, A. K. Pusala, and C. T. Singarapu, "Anti-plagiarism tool helping developers to generate authentic code," *International Journal of Scientific Research Archive*, vol. 14, no. 1, pp. 1208–1215, jan 2025.

[45] Y. Lu *et al.*, "CSSG: Measuring code similarity with semantic graphs," *arXiv preprint arXiv:2601.04085*, 2026.

[46] C. D. Manning, P. Raghavan, and H. Schütze, *Introduction to Information Retrieval*. Cambridge University Press, 2008.

[47] I. Maglogiannis, L. Iliadis, and E. Pimenidis, Eds., *Artificial Intelligence Applications and Innovations: 20th IFIP WG 12.5 International Conference, AIAI 2024, Corfu, Greece, June 27–30, 2024, Proceedings, Part II*, ser. IFIP Advances in Information and Communication Technology. Springer Nature, 2024.

[48] J. Chung, C. Gulcehre, K. Cho, and Y. Bengio, "Empirical evaluation of gated recurrent neural networks on sequence modeling," *arXiv preprint arXiv:1412.3555*, dec 2014.

[49] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin, "Attention is all you need," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2017.

[50] Y. Liu, M. Ott, N. Goyal, J. Du, M. Joshi, D. Chen, O. Levy, M. Lewis, L. Zettlemoyer, and V. Stoyanov, "Roberta: A robustly optimized bert pretraining approach," *arXiv preprint arXiv:1907.11692*, jul 2019.

[51] J. Zhang, X. Wang, H. Zhang, H. Sun, K. Wang, and X. Liu, "A novel neural source code representation based on abstract syntax tree," in *2019 IEEE/ACM 41st International Conference on Software Engineering (ICSE)*. Montreal, QC, Canada: IEEE, may 2019, pp. 783–794.

[52] M. Allamanis, M. Brockschmidt, and M. Khademi, "Learning to represent programs with graphs," *arXiv preprint arXiv:1711.00740*, may 2018.

[53] S. Sathyanarayanan and B. R. Tantri, "Confusion matrix-based performance evaluation metrics," *African Journal of Biomedical Research*, vol. 27, no. 4s, pp. 4023–4031, November 2024.

[54] V. Ljubovic, "Programming Homework Dataset for Plagiarism Detection," IEEE Dataport, 2020, dataset disponible en IEEE Dataport.

[55] E. Díaz Vergés, "¿Cómo gestionar un proyecto de inteligencia artificial?" Komorebi AI Blog, Jan. 2023, [En línea; consultado 28-septiembre-2026]. [Online]. Available: https://komorebi.ai/es/gestion-proyectos-ia/

[56] C. V. Lopes, P. Maj, P. Martins, V. Saini, D. Yang, J. Zitny, H. Sajnani, and J. Vitek, "Déjàvu: A map of code duplicates on GitHub," *Proceedings of the ACM on Programming Languages*, vol. 1, no. OOPSLA, pp. 1–28, 2017.

[57] K. Zhou, Y. Zhu, Z. Chen, W. Chen, W. X. Zheng, Z. Liu, Y. Chen, J. Deng, J.-R. Tang, Q. Wang *et al.*, "Don't make your LLM an evaluation benchmark contaminant," *arXiv preprint arXiv:2311.01964*, 2023.

[58] V. Ljubovic and E. Pajic, "Programming homework dataset for plagiarism detection," *IEEE Dataport*, 2020.

[59] X. Bouthillier, P. Delaunay, M. Bronzi, A. Trofimov, B. Nichyporuk, J. Szeto, N. Mohammadi Sepahvand, E. Raff, K. Madan, V. Voleti, S. Ebrahimi Kahou, V. Michalski, T. Arbel, C. Pal, G. Varoquaux, and P. Vincent, "Accounting for variance in machine learning benchmarks," in *Proceedings of Machine Learning and Systems*, vol. 3, 2021.

[60] M. B. A. McDermott, H. Zhang, L. H. Hansen, G. Angelotti, and J. Gallifant, "A closer look at AUROC and AUPRC under class imbalance," in *Advances in Neural Information Processing Systems*, vol. 37, 2024. [Online]. Available: https://proceedings.neurips.cc/paper_files/paper/2024/hash/4df3510ad02a86d69dc32388d91606f8-Abstract-Conference.html

[61] R. Geirhos, J.-H. Jacobsen, C. Michaelis, R. Zemel, W. Brendel, M. Bethge, and F. A. Wichmann, "Shortcut learning in deep neural networks," *Nature Machine Intelligence*, vol. 2, no. 11, pp. 665–673, 2020.

[62] A. Z. Broder, "On the resemblance and containment of documents," in *Proceedings. Compression and Complexity of SEQUENCES 1997*. IEEE Computer Society, 1997, pp. 21–29.

[63] K. Lee, D. Ippolito, A. Nystrom, C. Zhang, D. Eck, C. Callison-Burch, and N. Carlini, "Deduplicating training data makes language models better," in *Proceedings of the 60th*

Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers). Dublin, Ireland: Association for Computational Linguistics, 2022, pp. 8424–8445.

[64] M. Allamanis, "The adverse effects of code duplication in machine learning models of code," in *Proceedings of the 2019 ACM SIGPLAN International Symposium on New Ideas, New Paradigms, and Reflections on Programming and Software (Onward! 2019)*, 2019, pp. 143–153.

[65] S. Kapoor and A. Narayanan, "Leakage and the reproducibility crisis in machine-learning-based science," *Patterns*, vol. 4, no. 9, p. 100804, 2023.

[66] M. Brunsfeld *et al.*, "Tree-sitter: An incremental parsing system for programming tools," 2021.

[67] Y. Kim, "Convolutional neural networks for sentence classification," in *Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2014, pp. 1746–1751.

[68] S. Schleimer, D. S. Wilkerson, and A. Aiken, "Winnowing: local algorithms for document fingerprinting," in *Proceedings of the 2003 ACM SIGMOD International Conference on Management of Data*. New York, NY, USA: ACM, 2003, pp. 76–85.

[69] E. J. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, L. Wang, and W. Chen, "LoRA: Low-rank adaptation of large language models," in *Proceedings of the 10th International Conference on Learning Representations (ICLR 2022)*, 2022, pp. 1–13.

[70] T. Silva Filho, H. Song, M. Perello-Nieto, R. Santos-Rodriguez, M. Kull, and P. Flach, "Classifier calibration: a survey on how to assess and improve predicted class probabilities," *Machine Learning*, vol. 112, no. 9, pp. 3211–3260, 2023.

[71] J. C. Platt, "Probabilistic outputs for support vector machines and comparisons to regularized likelihood methods," in *Advances in Large Margin Classifiers*, A. J. Smola, P. Bartlett, B. Schölkopf, and D. Schuurmans, Eds. Cambridge, MA: MIT Press, 2000, pp. 61–74.

[72] B. Zadrozny and C. Elkan, "Transforming classifier scores into accurate multiclass probability estimates," in *Proceedings of the Eighth ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*. New York, NY, USA: ACM, 2002, pp. 694–699.

[73] B. Efron and R. J. Tibshirani, *An Introduction to the Bootstrap*, ser. Monographs on Statistics and Applied Probability. Chapman & Hall, 1993, no. 57.

[74] K. Musgrave, S. Belongie, and S.-N. Lim, "A metric learning reality check," in *Computer Vision – ECCV 2020*, ser. Lecture Notes in Computer Science. Cham: Springer International Publishing, 2020, pp. 681–699.

[75] T. Chari and L. Pachter, "The specious art of single-cell genomics," *PLOS Computational Biology*, vol. 19, no. 8, p. e1011288, 2023.

[76] R. C. Martin, *Clean Architecture: A Craftsman's Guide to Software Structure and Design*. Boston, MA, USA: Prentice Hall, 2017.

[77] J. Gama, I. Žliobaitė, A. Bifet, M. Pechenizkiy, and A. Bouchachia, "A survey on concept drift adaptation," *ACM Computing Surveys*, vol. 46, no. 4, pp. 1–37, 2014.

[78] J. Brooke, "SUS: A "quick and dirty" usability scale," in *Usability Evaluation in Industry*, P. W. Jordan, B. Thomas, B. A. Weerdmeester, and I. L. McClelland, Eds. London: Taylor & Francis, 1996, pp. 189–194.

[79] F. D. Davis, "Perceived usefulness, perceived ease of use, and user acceptance of information technology," *MIS Quarterly*, vol. 13, no. 3, pp. 319–340, 1989.